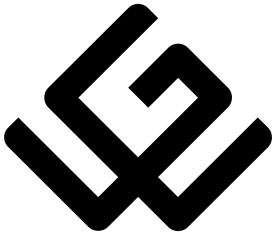


# GATEWIZARD

## MANUAL

API 1.0.56
GUI 1.0.15

API repository
GUI repository
Documentation

Generated 2026-10-25

# Contents


Contributors ..... 3
Home ..... 4
Installation ..... 7
User Guide ..... 14
Analysis ..... 31
Study cases ..... 41
- Overview ..... 41
- STP10 desktop app ..... 41
- STP10 API ..... 56
- Prepare systems ..... 59

Roadmap ..... 70
Troubleshooting ..... 71
API ..... 79
- Overview ..... 79
- Quick Reference ..... 82
- Method dictionary ..... 87
- Structure Manager ..... 93
- Structure checker ..... 118
- Biological assembly ..... 119
- Preparation Module ..... 119
- Builder Module ..... 156
- MemPrO Module ..... 198
- Hydration Module ..... 205
- Equilibration ..... 210
- Analysis ..... 258

# Contributors

## API

- Constanza González
- Mauricio Bedoya

### GitHub

- maurobedoya
- franciscoadasme
- ConstanzaGV12

## GUI

- Constanza González
- Francisco Adasme
- Mauricio González
- Janin Riedelsberger
- Mauricio Bedoya

### GitHub

- maurobedoya
- franciscoadasme
- orordnz

# Home

## GateWizard Documentation

Welcome to GateWizard, a tool for membrane protein preparation and molecular dynamics analysis.

### Overview

GateWizard is a Python-based GUI application designed to prepare protein membrane structures for molecular dynamics simulations. It integrates various computational tools to provide a unified interface for structure preparation, analysis, and plot customization.

### Key Features

- **Protein Structure Preparation**: Clean PDB files, and add missing atoms.
- **Propka Integration**: Automatic pKa calculations with protonation state assignment
- **Protein Capping**: Add ACE/NME caps to protein termini for proper MD simulations
- **Force Field Support**: Compatible with Amber force fields (ff14SB, ff19SB, etc.)
- **Membrane System Building**: Automated membrane protein insertion and equilibration protocols
- **Trajectory Analysis**: RMSD, RMSF, distances, and radius of gyration calculations
- **Energy Analysis**: NAMD log file parsing and visualization
- **Python API**: `pip install gatewizard` for scripts and notebooks
- **Desktop app**: Electron + Svelte in gatewizard-gui (separate install)

### Quick Start

#### Installation

```
# Create conda environment with dependencies
conda create -n gatewizard -c conda-forge python sqlite ambertools=26 parmed=4.3.0 openmm cudatoolkit
gromacs -y

# Activate environment
conda activate gatewizard

# AmberTools 26 does not ship reduce
conda install -c conda-forge -c bioconda reduce -y

# OpenMM, ParmEd, PDBFixer (short loops), and CPU MemProO.
pip install "gatewizard[full]"
```

#### Desktop app

The graphical interface is a separate project: **gatewizard-gui**. Install it from Releases — do not expect a `gatewizard` terminal command after `pip install gatewizard`.

Use the library from Python:

```
import gatewizard
print(gatewizard.__version__)
```

### Upgrade

```
pip install --upgrade gatewizard
```

For more detailed installation instructions, see the Installation Guide.

## Documentation Sections

- Installation Guide - Detailed installation instructions for all platforms
- User Guide - Complete guide to using GateWizard features
- Analysis Features - Trajectory and energy analysis capabilities
- Study cases - STP10 (PDB 7AAQ) in the desktop app and the API, including MemPro and titration, and the prepare-systems timings
- Roadmap - Work saved for later
- Quick Reference - Download a PDB and run the first API calls
- API Reference - Developer documentation
- Troubleshooting - Common issues and solutions

## System Requirements

- **Operating System**: Linux, macOS, or Windows (with WSL recommended)
- **Python**: 3.8 or higher
- **Conda**: For managing dependencies (Miniconda or Anaconda)
- **RAM**: Minimum 4GB (8GB+ recommended for large systems)
- **Disk Space**: ~2GB for installation plus space for your projects

## Getting Help

If you encounter issues or have questions:

1. Check the Troubleshooting Guide
2. Review the User Guide for usage instructions
3. Ask in GitHub Discussions — community forum for the API and GUI
4. Report bugs via API issues or GUI issues, depending on where the problem is

## Contributing

GateWizard is open-source software. Contributions are welcome. See CONTRIBUTING.md.

## License

GateWizard is licensed under the MIT License.

Copyright (c) 2025 Constanza González, Francisco Adasme, Mauricio González, Janin Riedelsberger, and Mauricio Bedoya

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

## Authors


- **Constanza González**
- **Francisco Adasme**
- **Mauricio González**
- **Janin Riedelsberger**
- **Mauricio Bedoya**


## Citation

If you use GateWizard in your research, please cite:

```
[Citation information to be added]
```

---

# Installation

## Installation Guide

This guide provides detailed instructions for installing GateWizard on different platforms.

### Prerequisites

Before installing GateWizard, ensure you have:

- **Conda** (Miniconda or Anaconda): Download here
- **Git** (for cloning the repository): Download here
- **Python 3.8+** (will be installed via conda)

### Full API installation (Linux or WSL)

Use this when the new machine should build membranes, orient with MemPrO, and run OpenMM or GROMACS. NAMD stays a separate download.

```
conda create -n gatewizard -c conda-forge python sqlite ambertools=26 parmed=4.3.0 openmm cudatoolkit
gromacs -y
conda activate gatewizard
conda install -c conda-forge -c bioconda reduce -y
pip install "gatewizard[full]"
python -c "import gatewizard; print(gatewizard.__version__)"
which pdb4amber
which mempro
```

On macOS, omit `cudatoolkit`. Prefer the CPU `gromacs` package. A CUDA GROMACS build in this same environment often stalls in the solver; see GROMACS: CPU vs CUDA.

AmberTools 26 does not ship `reduce`. `conda install -c bioconda reduce` puts that binary in the same environment so `pdb4amber --reduce` can add Amber hydrogens.

The same stack from a git checkout:

```
git clone https://github.com/maurobedoya/gatewizard.git
cd gatewizard
conda env create -f environment.yml
conda activate gatewizard
```

`environment.yml` already installs the package with `pip install -e ".[full]"`, including CPU MemPrO (`mempro` ≥ 0.3.1). `pip install "gatewizard[full]"` installs that same CPU package. A GPU build is optional and goes into this conda env only; see MemPrO on a GPU.

For the **desktop app**, install gatewizard-gui separately. On first launch the GUI embeds its own micromamba runtime. That runtime is not this conda env. See Desktop GUI runtime (GROMACS / CUDA) below.

### API package only

This installs the Python library without OpenMM or MemPrO:

```
conda create -n gatewizard -c conda-forge python sqlite ambertools=26 parmed=4.3.0 -y
conda activate gatewizard
conda install -c conda-forge -c bioconda reduce -y
pip install gatewizard
python -c "import gatewizard; print(gatewizard.__version__)"
```

## Platform-Specific Instructions

### Linux

GateWizard works natively on Linux. Follow the Quick Installation steps above.

**Additional Notes:** - Ensure you have `tkinter` support: `sudo apt-get install python3-tk` (Ubuntu/Debian) - For display issues, ensure X11 is properly configured

### macOS

GateWizard works on macOS with some considerations:

```
# Follow the Quick Installation steps, then:

# If you encounter display issues:
conda install -c conda-forge python.app
```

**Known Issues:** - On Apple Silicon (M1/M2), some dependencies may require Rosetta 2

### Windows

GateWizard is best used on Windows with WSL (Windows Subsystem for Linux):

#### Option 1: WSL (Recommended)

```
# 1. Install WSL2 with Ubuntu
wsl --install

# 2. Inside WSL, follow the Linux installation instructions
# 3. Install X server on Windows (e.g., VcXsrv, X410)
# 4. Set DISPLAY environment variable in WSL
export DISPLAY=:0
```

#### Option 2: Native Windows (Experimental)

```
# Use Anaconda Prompt
conda create -n gatewizard -c conda-forge python sqlite ambertools=26 parmed=4.3.0 -y
conda activate gatewizard
conda install -c conda-forge -c bioconda reduce -y
pip install gatewizard
python -c "import gatewizard; print(gatewizard.__version__)"
```

For the desktop app on Windows, use the gatewizard-gui Windows installer or the Linux/WSL build for the full MD workflow.

If Windows Task Manager shows `VmmemWSL` using several GB after you close the app, cap the VM — see Troubleshooting: VmmemWSL RAM.

## Dependencies

### Core Python Dependencies

Installed by `pip install gatewizard`:

- **Python** ≥ 3.8
- **NumPy** ≥ 1.21.0
- **Matplotlib** ≥ 3.5.0
- **requests** ≥ 2.25.0

- **MDAnalysis** ≥ 2.0.0
- **lipyphilic** ≥ 0.12.0
- **freud-analysis** ≥ 2.5.0
- **Propka** ≥ 3.2.0
- **Pillow** ≥ 9.0.0
- **RDKit** ≥ 2023.3.1
- **psique** ≥ 1.2.1
- **gemmi** ≥ 0.6.5

`pip install "gatewizard[full]"` also installs ParmEd, OpenMM, PDBFixer, and the CPU build of MemPrO. AmberTools, `reduce`, the conda ParmEd build, OpenMM's CUDA toolkit, and CPU GROMACS come from conda, not from that pip extra.

**Scientific Computing Dependencies**

Must be installed via conda:

- **AmberTools 26** - Molecular dynamics preparation and analysis
- **reduce** (bioconda) - Amber hydrogens for `pdb4amber --reduce`. AmberTools 26 does not ship this binary.
- **Parmed 4.3.0** - Parameter/topology manipulation (must be from conda-forge)
- **PDBFixer** ≥ 1.9 - short loops of 1–6 residues. Installed by `gatewizard[full]`, not by `pip install gatewizard` alone.

**External Requirements**

- **NAMD** - Required for NAMD equilibration (separate install; not on conda-forge)
- Recommended: **NAMD 3.0.1 or later** (`namd3`)
- Download from: NAMD Official Website
- Must be on PATH or selectable via the GUI engine picker
- **OpenMM** (optional for API users) - Python MD engine via conda-forge
- `conda install -c conda-forge openmm cudatoolkit` on Linux/WSL with an NVIDIA driver for the CUDA platform
- macOS uses Metal/OpenCL (no `cudatoolkit`)
- Auto-selects **CUDA** → **OpenCL** → **CPU**; override with `PLATFORM=CUDA` when running equilibration scripts
- **GROMACS** (optional) - Recommended via conda-forge; the GUI also installs a build into its embedded runtime
- **CPU (recommended default):** `conda install -c conda-forge gromacs`
- **CUDA (Linux only, advanced):** `conda install -c conda-forge 'gromacs=*=nompi_cuda*'`
- System installs under `/usr/local/gromacs` (with `GMXRC`) are auto-detected
- See the notes below before choosing the CUDA conda build

**GROMACS: CPU vs CUDA (conda-forge)**

GateWizard equilibration works with either a **CPU** or **CUDA** `gmx`. Prefer the CPU package unless you specifically need a conda CUDA GROMACS binary.

| Goal | Command / behavior |
|---|---|
| Reliable conda install | `conda install -c conda-forge gromacs` |
| CUDA GROMACS (opt-in) | `conda install -c conda-forge 'gromacs=*=nompi_cuda*'` |
| GPU OpenMM (separate) | `conda install -c conda-forge openmm cudatoolkit` |

**Why CUDA GROMACS often fails or hangs**

- Installing `gromacs=*=nompi_cuda*` into an environment that already has **OpenMM + `cudatoolkit`** can stall for a very long time in the dependency **solver** (little or no log output after `Pinned packages:`).

- Log lines about the CUDA Toolkit EULA or Anaconda Terms of Service are **informational**. Non-interactive installs (`conda`/`micromamba` with `-y`) do **not** wait for you to type "yes".
- If the CUDA solve is cancelled or times out, installing the **CPU** `gromacs` package normally succeeds quickly.
- For GPU MD with GateWizard, **OpenMM + `cudatoolkit`** is usually enough. Use a **system** CUDA GROMACS (or GMXRC) if you need `gmx` on the GPU, or keep conda GROMACS on CPU.

#### Desktop GUI runtime (GROMACS / CUDA)

gatewizard-gui bootstraps an embedded micromamba env on first launch (see that repo's README and `runtime-install.log`).

- **OpenMM:** on Linux/WSL, if `nvidia-smi` sees a GPU, the GUI installs `openmm` + `cudatoolkit` into the embedded env (unless `GATEWIZARD_SKIP_CONDA_CUDA=1`).
- **GROMACS (default):** the GUI installs the **CPU** conda-forge build. This avoids multi-hour CUDA solver hangs next to OpenMM's toolkit.
- **GROMACS CUDA (opt-in):** set `GATEWIZARD_CONDA_GROMACS_CUDA=1` before launching the GUI. The bootstrap tries a frozen CUDA install with a short timeout, then falls back to CPU GROMACS if the solve stalls.
- Quitting mid-install kills leftover `micromamba` processes for that env so a relaunch is not blocked by an orphaned solver.
- Linux/WSL log: `~/.config/gatewizard-gui/runtime-install.log` (look for `[gromacs] starting…` / `failed` / `installed`).
- The embedded env is `~/.config/gatewizard-gui/mamba-env`. `sudo apt remove gatewizard-gui-linux` leaves it. `sudo apt purge gatewizard-gui-linux` deletes that directory for the user who ran sudo. It does not remove a separate conda env such as `~/miniconda3/envs/gatewizard`.

Manual API environments should follow the same preference: install **CPU** `gromacs` by default; only add the CUDA matchspec if you accept a long or fragile solve.

#### MemPrO on a GPU

`gatewizard[full]` and the desktop app both install the CPU package (`mempro` with `jax[cpu]`). Neither installs `mempro[gpu]`. The API conda env and the GUI micromamba env are separate Pythons. Installing the GPU extra in one does not change the other. The extra is `jax[cuda12]`, for Linux with CUDA 12 drivers. macOS, Windows, and a machine with no NVIDIA driver stay on CPU. Asking for a GPU still fails when the card is not visible, including WSL without GPU passthrough.

**API.** Use the conda env from this page, not a `pip` that belongs to another env:

```
conda activate gatewizard
python -m pip install "mempro[gpu]"
```

`MemPrO.run(..., device="gpu")` then uses that env. `device="cpu"` and `device="auto"` still run on CPU when JAX sees no GPU.

**Desktop app.** Quit GateWizard first. The GPU choice in Visualize reads JAX inside the embedded env, not the conda env above. On Linux or WSL:

```
~/.config/gatewizard-gui/mamba-env/bin/python -m pip install "mempro[gpu]"
```

On macOS:

```
~/Library/gatewizard-gui/mamba-env/bin/python -m pip install "mempro[gpu]"
```

Start the app again. The GPU choice stays off until that env's JAX can see a GPU. There is no native Windows installer; on a Windows PC this is the WSL command above, inside the Linux distro, not Windows Python and not `%APPDATA%\gatewizard-gui\mamba-env`.

### Verifying Installation

After installation, verify that everything works:

```
# Activate environment
conda activate gatewizard

# Check Python version
python --version  # Should be 3.8+

# Check key dependencies
python -c "import MDAnalysis; print('MDAnalysis:', MDAnalysis.__version__)"
python -c "import parmed; print('ParmEd:', parmed.__version__)"
python -c "import openmm; print([openmm.Platform.getPlatform(i).getName() for i in
range(openmm.Platform.getNumPlatforms())])"

# Check AmberTools
which pdb4amber  # Should show path in conda environment

# Verify GateWizard API
python -c "import gatewizard; print(gatewizard.__version__)"
```

### Upgrading GateWizard

To upgrade to the latest version from PyPI:

```
# Activate environment
conda activate gatewizard

# Upgrade GateWizard
pip install --upgrade gatewizard

# Verify new version
python -c "import gatewizard; print(gatewizard.__version__)"
```

**Check for updates:** You can check if a newer version is available:

```
pip list --outdated | grep gatewizard
```

**Force reinstall (if needed):** If you encounter issues after an update:

```
pip install --force-reinstall gatewizard
```

### Troubleshooting Installation

#### Issue: conda GROMACS CUDA install hangs after "Pinned packages"

**Cause:** The `gromacs=*=nompi_cuda*` solve often conflicts with an environment that already has OpenMM/`cudatoolkit`. The solver can run for a very long time with almost no new log lines. EULA / Terms-of-Service messages in the log are not interactive prompts.

**Solution:**

```
# Cancel the stuck install (Ctrl+C), then install CPU GROMACS:
conda install -c conda-forge gromacs -y

# Optional: use a system CUDA GROMACS / GMXRC instead of the conda CUDA build
```

For **gatewizard-gui**, leave the default (CPU GROMACS). Only set `GATEWIZARD_CONDA_GROMACS_CUDA=1` if you explicitly want the timed CUDA attempt. See Desktop GUI runtime (GROMACS / CUDA).

**Issue: ImportError with numpy.compat**

**Cause:** Version conflict between NumPy and Parmed.

**Solution:**

```
# Reinstall Parmed from conda-forge
conda install -c conda-forge parmed=4.3.0 --force-reinstall
```

**Issue: pdb4amber command not found**

**Cause:** AmberTools not properly installed or environment not activated.

**Solution:**

```
# Ensure environment is activated
conda activate gatewizard

# Reinstall AmberTools
conda install -c conda-forge ambertools=26
conda install -c conda-forge -c bioconda reduce
```

**Issue: Module not found errors**

**Cause:** Installation incomplete or wrong environment.

**Solution:**

```
# Ensure you're in the correct environment
conda activate gatewizard

# Reinstall the API
cd /path/to/gatewizard
pip install -e ".[full]" --force-reinstall
```

**Tip:** GateWizard follows semantic versioning. Check the releases page for changelog and new features.

**For Development Installation**

If you installed from source:

```
# Update repository
cd /path/to/gatewizard
git pull origin main

# Update dependencies if needed
conda env update -f environment.yml

# Reinstall
pip install -e ".[full]" --force-reinstall

# Re-verify import
python -c "import gatewizard; print(gatewizard.__version__)"
```

## Uninstallation

To completely remove GateWizard:

```
# Remove conda environment
conda deactivate
conda env remove -n gatewizard

# Remove repository (if desired)
rm -rf /path/to/gatewizard
```

### Next Steps

After successful installation:

1. Read the User Guide to learn about features
2. Try the example workflows
3. Check the Troubleshooting Guide if you encounter issues

---

*For additional help, please contact the developers or check the GitHub repository.*

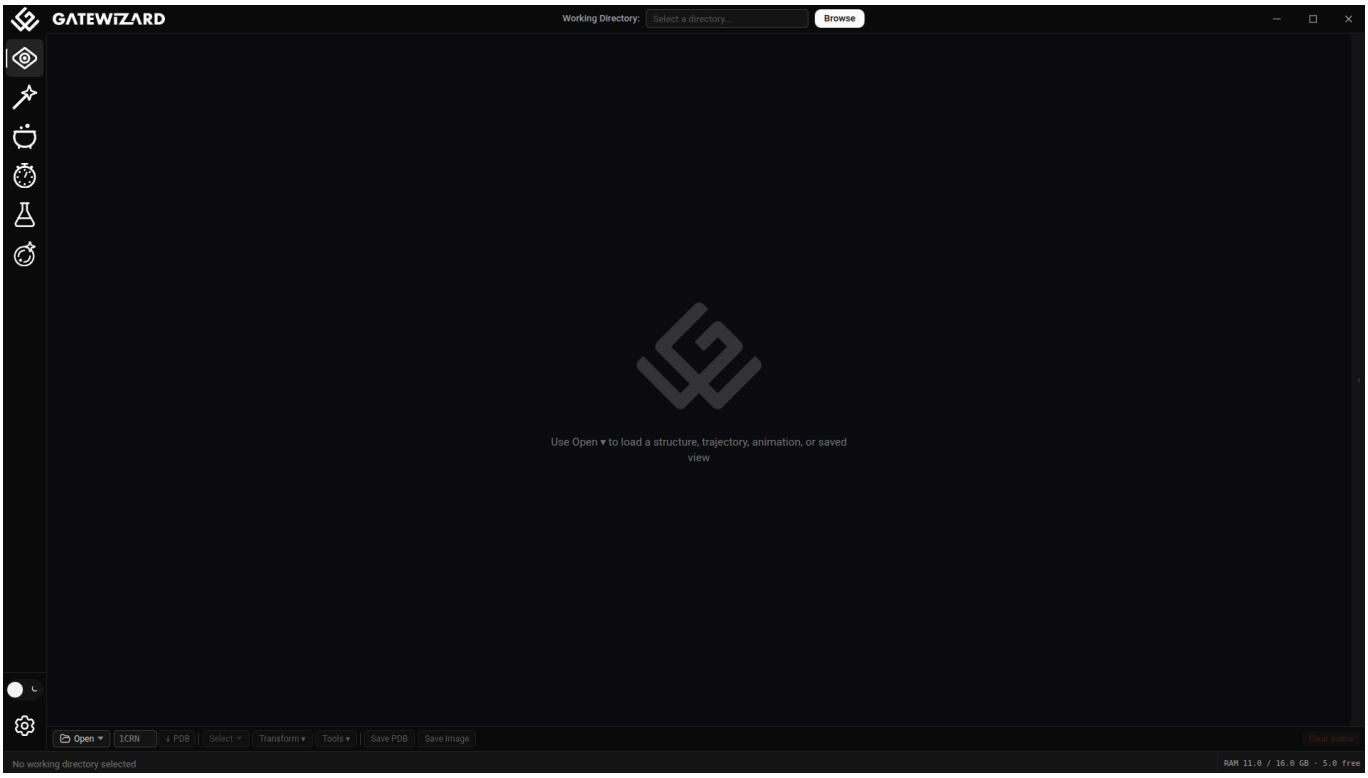
# User Guide

This guide matches the **gatewizard-gui** desktop app (Electron). The `gatewizard` pip package is the Python API only — it does not open this window.

## Pages

The left **activity bar** switches pages:

| Page | What it does |
|---|---|
| **Visualize** | 3D structures, trajectories, representations, sequence and short loops, structure checker, minimize, mutate / split / superimpose, figures and movies |
| **Preparation** | PropKa pKa table, protonation, disulfide bonds, ACE/NME caps, short loops, alternate conformations, inline 3D |
| **Builder** | packmol-memgen leaflets, salt, ligand force field and charges, tleap, job cards |
| **Equilibration** | NAMD / GROMACS / OpenMM / Amber schemes, Generate, Watch / Pull, cluster |
| **Tools** | Fix PBC and other trajectory utilities |
| **Analysis** | Structural (RMSD/RMSF/APL) and energetic log plots |




*Visualize workspace*

## Visualization

The bottom toolbar opens on a click. Open, Select, Transform, and Tools each stay closed until you click that button. Clicking another one closes the menu that was open. Click outside, press Escape, or scroll the toolbar to close it.

**Open** loads a structure (PDB or mmCIF), a topology plus a coordinate PDB, a trajectory, a saved animation, or a saved view. **Clear scene** removes the structure and the representations. The four-letter box next to Open downloads an RCSB entry (PDB, or mmCIF when RCSB does not publish a PDB).

**Select** turns on interactive selection for the structure named on the button. The level is atom, residue, chain, or molecule. Hovering the structure highlights that group. A left click keeps the selection. **Custom (MDAnalysis…)** types a selection instead. **Exit edit mode** turns selection off.

**Transform** rotates, translates, or aligns the selection, and can rename a chain, rename residues, renumber residues, or delete atoms. The originals stay until you confirm the dialog.

Orbit with the left mouse button. **Ctrl+scroll** zooms more slowly.

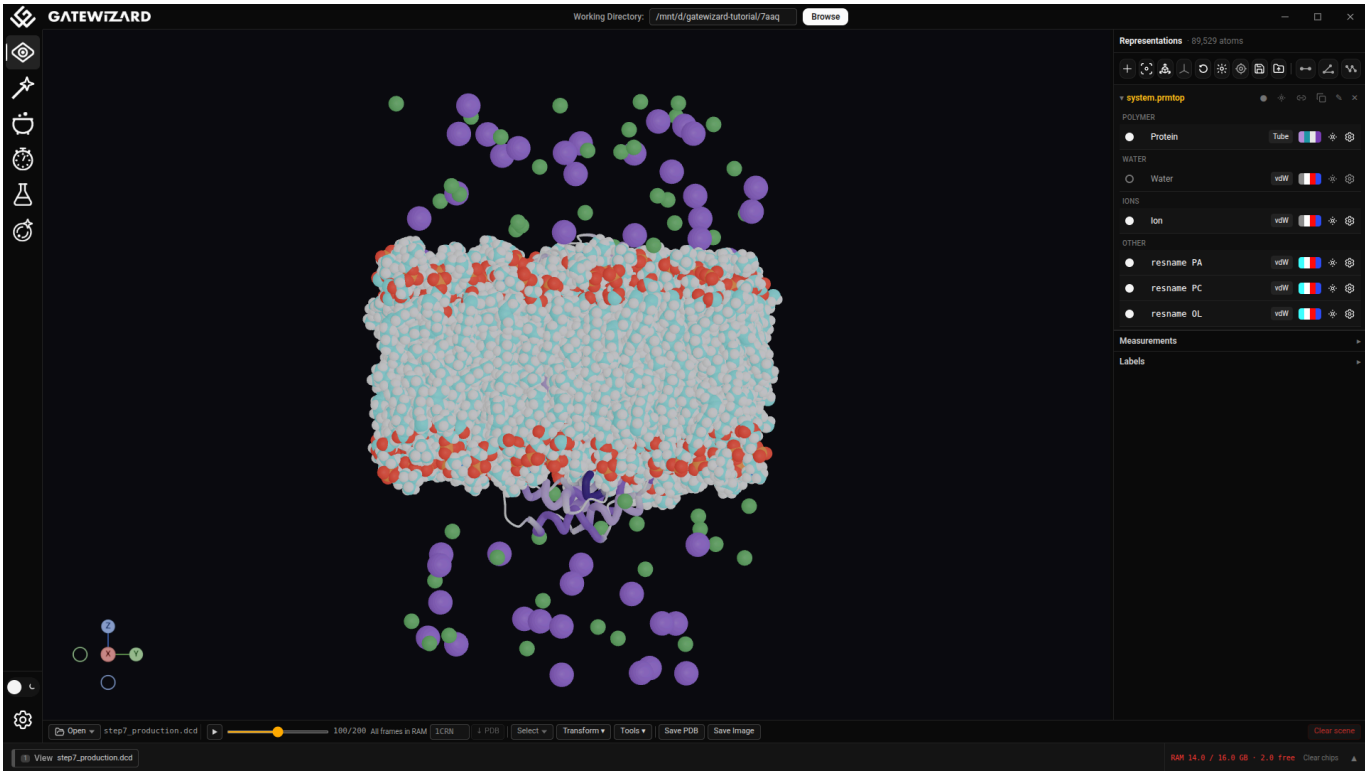
The Representations dock adds a view of the loaded atoms. The styles are points, van der Waals, ball-and-stick, licorice, cartoon, and tube. Color, material, and the selection live on that card. The header shows how many atoms are selected. Hide a card, or delete it. A right-click on a representation can copy it onto another loaded structure.

**Save Image** writes the viewport as a PNG. The dialog sets the width, height, and quality. Quality 2× draws the picture larger and saves it at the size you set. A transparent PNG and a yellow crop overlay are available there.

## Trajectories

**Open** → **Trajectory…** takes a topology plus one or more DCD, XTC, TRR, or NetCDF files. A companion `.prmtop` or `.psf` in the same folder is used when you open a structure that has one. Each file can keep every frame or a stride.

Play stays off until the frame cache is ready. The status line says Indexing, then Loading frames into RAM. The slider and the play button then move through the frames. **Traj-smooth** on a representation card follows the movie instead of jumping each frame. **Tools** → **Align / RMSD…** aligns that trajectory or reports the RMSD.




## Animation

**Tools** → **Animate** opens the timeline under the viewer. The book on that bar opens this section. Play and Stop run the animation. Time and Frame jump the playhead. The playhead snaps to the timeline frame rate, which is not the MD frame.

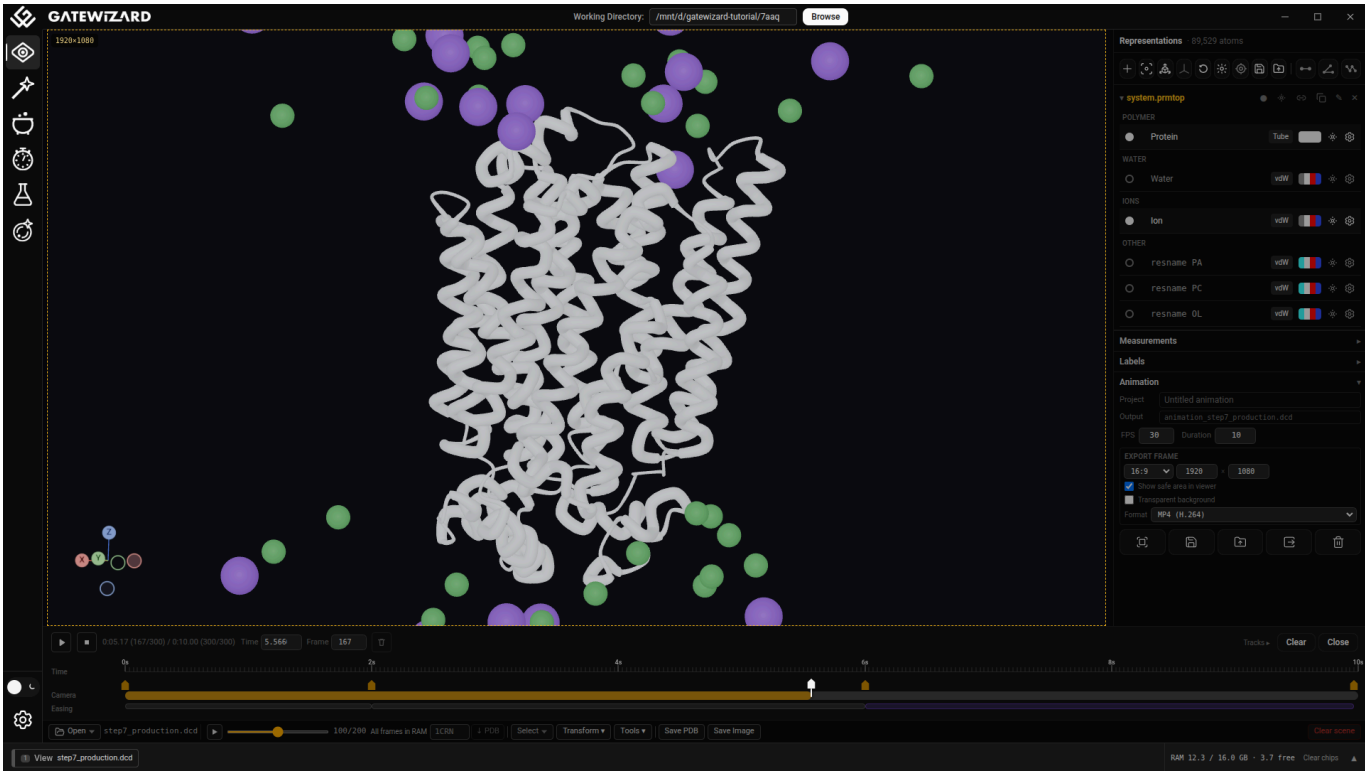
Capture writes a key on the track that is active. There are three kinds of track:

- **Camera** stores the view, and also the background, lights, shadows, depth of field, and depth cueing. Those follow the camera key's easing.
- **Structure** stores how that structure moves.
- **Representation** stores color, material, size, and a crossfade from one representation type to another.

Click a track to make Capture write keys there. Drag a key to move it. Right-click a key to edit its easing, or the fade on a representation. The easing and fade cards open on the right and can be dragged by the title bar. Ctrl+click selects several keys so they move or delete together. Delete and Backspace remove the selected keys. Clear removes every key and leaves the duration and the frame rate.

The **−** and **+** buttons change the timeline width. Drag the top edge to shorten the track list; a vertical scrollbar reaches the rows that no longer fit. Each easing bar has its own color.

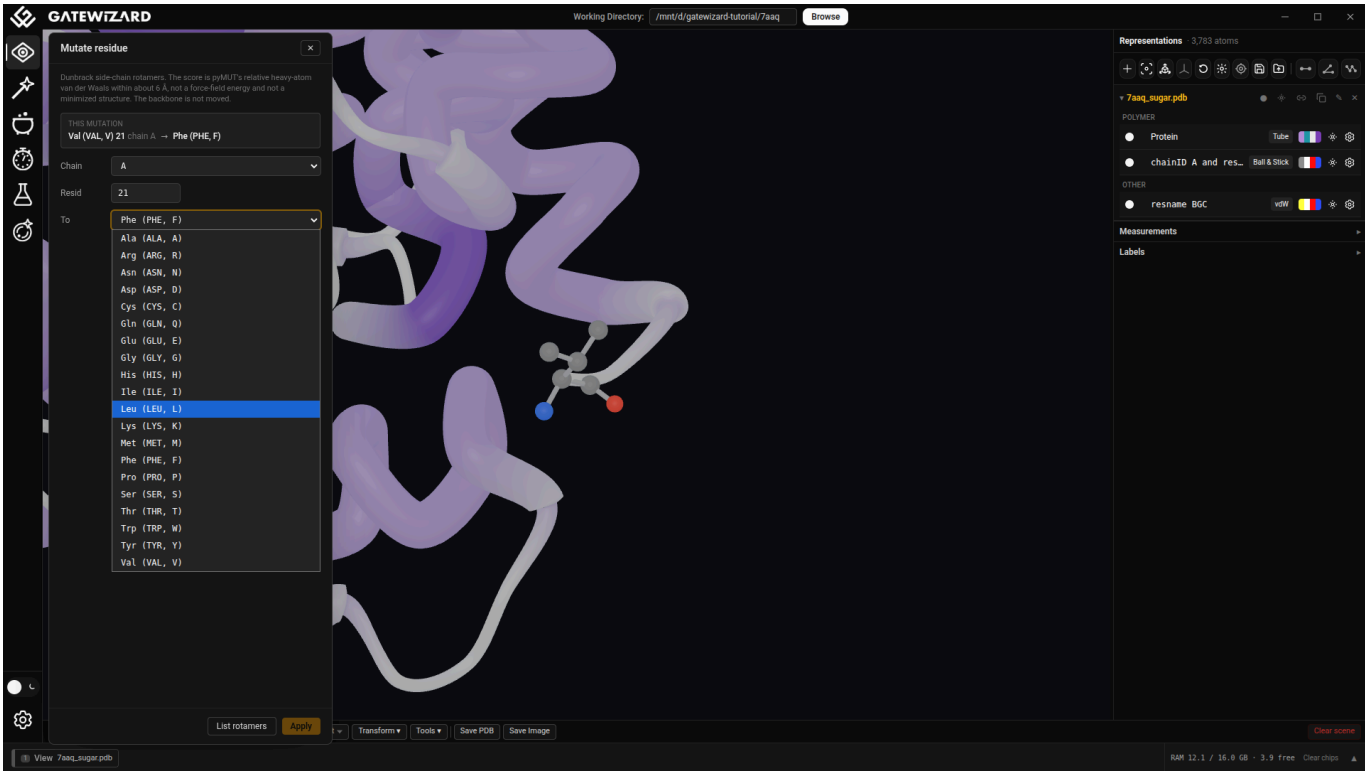
Save and Load keep the project. Open → Animation… loads a saved project and the structure it needs. Export writes the movie. GIF, WebM, and PNG frames can be transparent. MP4 and MOV stay opaque. The export row sets the aspect ratio, width, height, and format.




## Tools on Visualize

**Tools** holds the structure edits that are not Transform:

- **MemPro orientation** places a membrane protein. The panel lists each run and which structure it belonged to.
- **Packmol hydration** fills a cavity with water.
- **Align / RMSD…** needs a trajectory.
- **Animate** opens the timeline above.
- **Mutate residue…** previews Dunbrack rotamers and writes a new structure.
- **Structure checker…** is the report in the next section.
- **Split chain…** copies one chain into a new structure.
- **Build assembly…** is described with the structure checker.
- **Superimpose…** needs two loaded structures. **Merge structures** writes one PDB from the structures selected in the panel. The originals stay loaded.




**Save Image** is the button to the right of Tools. It opens the figure dialog (size, quality, transparent PNG, yellow crop overlay).

## Sequence bar and short loops

**Sequence** opens the protein sequence by chain. Click a residue to add it as licorice; click it again to remove only that residue. Drag to add a range. Right-click adds another representation.

A numbering hole of **1 to 6** residues is marked on that bar. **Fill short gaps** builds it with PDBFixer from SEQRES, or from a sequence you type, and saves a new structure. A longer hole is listed and left open. A residue that is not a standard amino acid, such as HSK, stays open. The same checkbox is on Preparation.

## Structure checker and alternate conformations

**Structure checker** lists missing atoms, gaps, alternate conformations, and chain differences. An alternate conformation is the PDB altLoc letter. The more likely letter is the higher occupancy. A tie keeps the earlier letter.

A ligand, water, or other molecule is its own group, even when it reuses a protein residue number. Atoms that overlap and have no altLoc letter are listed too. Tick one letter to draw only that pose. Tick both to draw both, with the more likely pose in yellow.

**Build assembly** expands the biological assembly already stored in the file. Crystal waters can be included. Copies that land on the same point can be dropped.

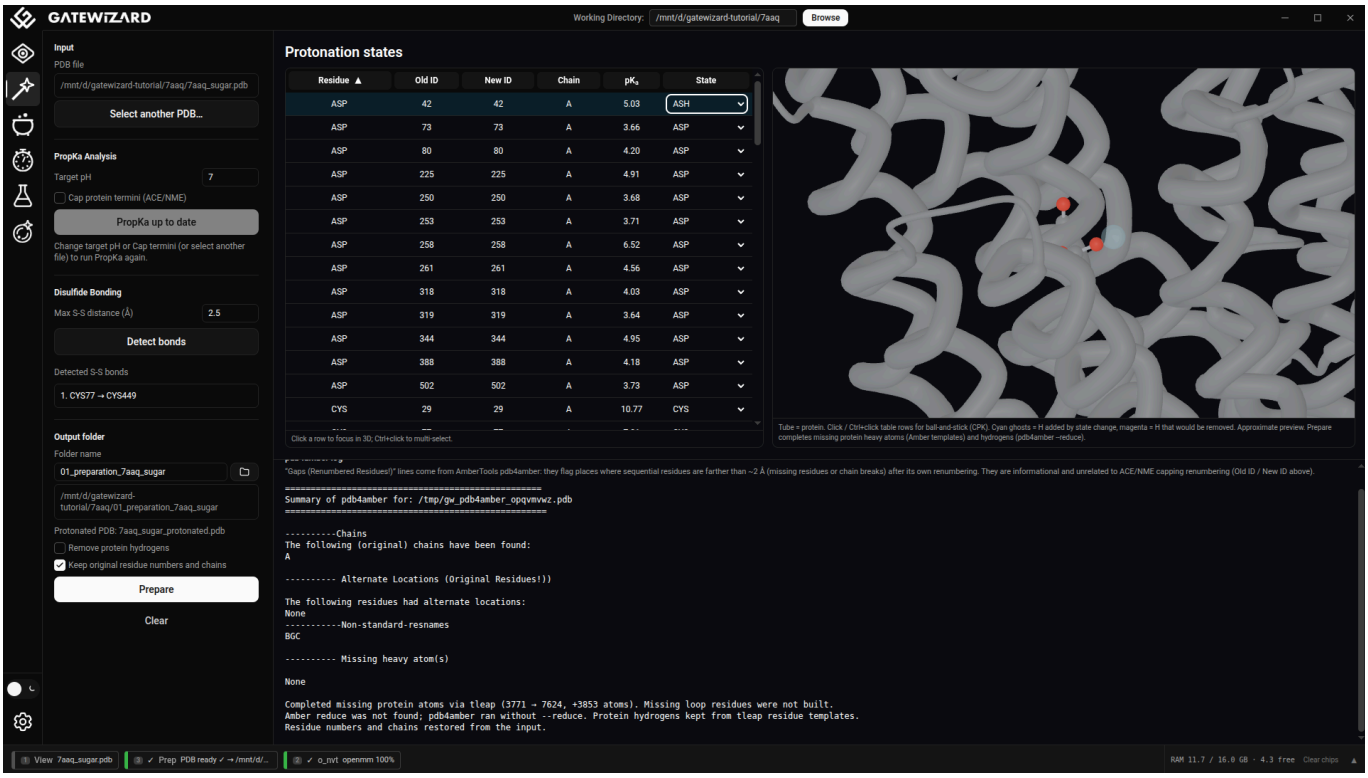
## Minimize a selection

The minimize control on the Representations toolbar relaxes the residues, atoms, or molecule already selected. The icon uses the last settings. A new session starts at 50 steps. The corner mark sets the step count, the convergence, and how far the atoms may move, without starting a run.

This is a short clash fix, not a finished energy minimum. Atoms Amber can build stay fixed except the selection. A water or another molecule Amber cannot build is relaxed without breaking its bonds. Potassium and the other Amber monoatomic ions can be minimized. The result is a new PDB, `name_min_001.pdb` and then `name_min_002.pdb`, in a `minimized` folder. The structure you started from stays.

## Preparation

Preparation turns a crystal or model PDB into a file Builder can pack. The page runs PropKa, can cap termini, detects disulfides, and writes a prepared PDB. The right-hand view is that working file: the tube is the protein, a table click draws ball-and-stick, cyan ghosts are hydrogens a state change would add, and magenta ghosts are hydrogens it would remove.




1. Set the working directory in the top bar and load a PDB. Output is written under that directory.
2. Choose the target pH and, when the chain should not stay charged, cap the termini.
3. Run PropKa. The table lists residue, Old ID, New ID, chain, pKa, and the state at that pH.
4. Detect disulfide bonds if the structure has them, then **Prepare**.

**Prepare** writes missing protein heavy atoms from Amber residue templates, applies the PropKa residue names (ASH, GLH, HIP, CYX, and the histidine names), and adds hydrogens. Missing loop residues are not invented here. Stale CONECT and LINK records are dropped because atom serials change. If the Amber `reduce` binary is missing, Prepare keeps the tleap-template hydrogens and still writes the PDB. The same steps in the API are `complete_missing_heavy_atoms`, `apply_protonation_states`, and `run_pdb4amber_with_cap_fix`.

The 3D header and the pdb4amber log use the same page. “Gaps (Renumbered Residues!)” lines in that log come from pdb4amber: sequential residues farther than about 2 Å after its own renumbering. They are informational. They are separate from ACE/NME capping renumbering (Old ID / New ID in the table).

**Cap protein termini (ACE/NME)** is for fragments and chain ends that should not stay charged in MD. The capper writes ACE as residue 1 in its own file and shifts the chain; Prepare, with residue numbers preserved, puts the protein numbers back. New ACE then sits at N-terminus − 1 (residue **0** when the chain starts at 1) and NME at C-terminus + 1. The mapping file is kept for later Builder and PropKa steps. API: `ProteinCapper` and `detect_terminal_caps`. A chain that is already capped is reported on the page and is not capped again.

**Remove protein hydrogens** writes a prepared PDB with no protein hydrogens. Ligands and other heteroatoms keep theirs. The protonation choice stays in the residue name (ASH, GLH, HIP, CYX, and the histidine names). Leaving the box on is safe for a membrane build: packmol-memgen adds the protein hydrogens again from those names. Uncheck it when the prepared PDB itself should already contain the Amber hydrogens from pdb4amber `--reduce`. `count_protein_hydrogens` is what Builder later uses to warn about non-Amber hydrogens.

**Keep original residue numbers and chains** is on by default. PropKa and AmberTools still add hydrogens and missing atoms. Residue numbers and chain IDs stay as in the input, so a missing loop that jumps from 200 to 205 stays that gap. Uncheck it for the sequential Amber numbering tleap and MD expect. API: `restore_original_residue_numbers`.

**Fill short gaps (1–6 residues)** builds those holes with PDBFixer when SEQRES has the sequence, or from a sequence you type. A longer gap is listed and left open. A residue that is not a standard amino acid stays open. Visualize marks the same holes on the sequence bar and can run the same fill as a new structure. API: `classify_short_gaps` and `fill_short_gaps`.

**Keep the most likely alternate conformations** is on by default. Each residue that has alternate locations keeps the higher-occupancy letter, including ligands, waters, and other molecules. The other copies are dropped before protonation. A tie keeps the earlier letter. Turn it off to leave the choice to pdb4amber, which keeps the first copy of each atom. Choose the letters on Visualize when the assignment matters. API: `keep_most_likely_altlocs` and `diagnose_structure`.

## PropKa

PropKa predicts a pKa for each titratable residue and the protonation state at the pH you set. The table is that result. Change the pH or the termini cap and run the analysis again; a finished run at the same pH and cap is not repeated until one of those changes.

Columns:

| Column | Meaning |
|---|---|
| Residue | Amino acid and its protonation name at this pH |
| Old ID | Residue number in the file you loaded |
| New ID | Number after capping or Amber renumbering, when that shift applies |
| Chain | Chain ID |
| pKa | Predicted pKa |
| State | Protonation state PropKa assigns at the target pH |

Click a row to focus that residue in the 3D view. Ctrl+click adds it to the ball-and-stick selection. Changing a state in the table previews the hydrogens that would be added (cyan) or removed (magenta). That preview is an estimate, and the hydrogen positions can differ slightly from the ones Amber writes later. Prepare records the residue names. Those names are what the final system uses for protonation.

Amber names used downstream: ASP/ASH, GLU/GLH, CYS/CYX, and HIS as HID, HIE, or HIP. Builder's **Skip protonation** must stay on when this file already carries those names, or packmol-memgen will protonate again and fight them.

API: `PreparationManager.run_analysis` writes the `.pka` file, `parse_summary` fills the table, and `apply_protonation_states` writes the PDB. `get_ph_titration_curve` and `build_titration_figure` are the Analysis titration figure, not this table.

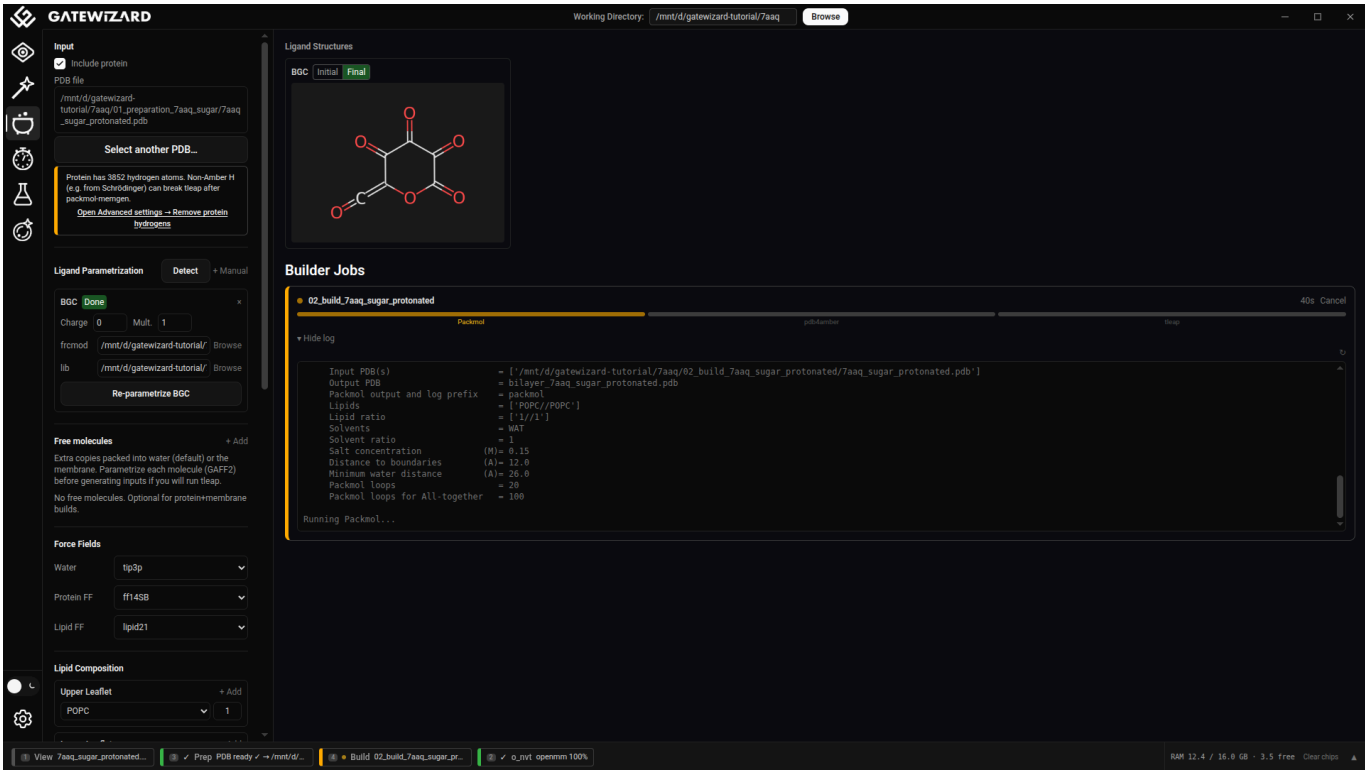
## Disulfide bonds

**Detect bonds** finds cysteine pairs whose SG atoms lie within the max S–S distance (default 2.5 Å). Each pair is listed as `CYS33 → CYS90`. After PropKa has loaded the working structure, the pair is drawn on the 3D view: the cysteines are ball-and-stick and the sulfur–sulfur bond is included. **Focus** frames that one pair. If the list is full but the view is empty, run PropKa first, or detect again after the working file changes, so the residue numbers match the structure on screen.

Prepare renames bonded cysteines to CYX so tleap builds a disulfide instead of two thiols. The distance cutoff is the same value `detect_disulfide_bonds` uses. `apply_disulfide_bonds` is the rename step.

## Builder

Builder packs the bilayer, water, ions, and any extra molecules, then can parametrize with tleap. Each run is a job card under the project folder. **Generate Input Files** writes the job. **Build system** launches `run_preparation.sh`. **Cancel** stops a running job.




Load the prepared protein, or uncheck **Include protein** for a bilayer only. A bilayer without a protein needs a membrane XY size under System Options (`--distxy_fix`). Packmol-Memgen still builds the bilayer. The page checks the PDB when you load it: missing heavy atoms, a gap longer than six residues, or no protein hydrogens asks you to go back to Preparation. Alternate conformations can be reduced there to the most likely letter. Ligands and peptide caps are detected with the file, so **Detect** is only a refresh.

**Generate Input Files** stays disabled while a blocking problem is on screen: no PDB, no lipid, no membrane size, a negative salt concentration, an unknown force-field name, or a ligand, cap, or free molecule that still needs parameters while **Parametrize with tleap** is on. An untested force-field combination is a warning and still allows Generate. A folder that already has a build is not renamed: Generate stays off until you re-run that folder or pick the next number (`02_build_name_2`). **Build system** runs `validate_system_inputs` once more and does not start packmol if that check fails. Turning parametrization off allows a pack-only run; tleap is then your later step. API: `prepare_system` and `run_preparation`.

**Skip protonation** stays on when the PDB already has PropKa names (ASH, GLH, HIP, CYX). **Pre-oriented in membrane** skips MEMEMBED. **Remove protein hydrogens** is the escape hatch when the protein still carries non-Amber hydrogens that would break tleap after packing.

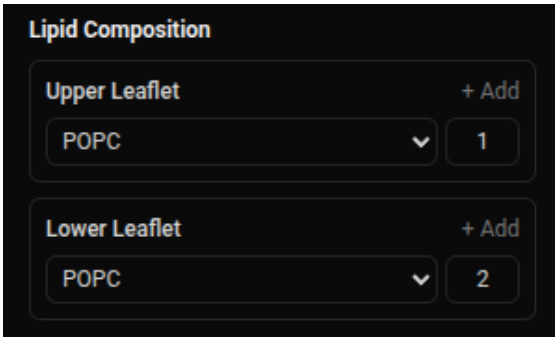
### Lipid catalog

Upper and lower leaflets are edited separately. Each row is a lipid name and a ratio. Ratios on a leaflet are relative to each other (`1:1`, or `3:1` for a 75:25 mix). An empty leaflet or a ratio that does not match the count of lipids fails at Start.

GateWizard supports **4028 lipids** from the packmol-memgen catalog (`--available_lipids_all`, version 2025.1.29). 258 of them are Amber lipid21 or lipid17. The other 3770 use PACKMOL-Memgen's Lipid_ext library on top of that Amber leaprc. Those parameters are not in Amber's `lipid21.lib`. The picker marks those rows **Lipid_ext**. The catalog lists

4055 names; the 27 SIRAH lipids are coarse-grained and are not offered. Opening the picker shows a spinner, then a window of the list you can search by name, charge, or chemical name.

A familiar subset and the packing call are in the builder reference, including `get_lipid_catalog` and `get_available_lipids`. Composition examples: lipid selection.




## Force fields

Water, protein, and lipid force fields are chosen together. The menus are the names `ForceFieldManager` knows. An unknown name is marked in red and blocks Generate. A combination that has not been tested in the literature is a warning: `validate_combination` still returns valid, and Generate stays available. The stable default in the API notes is TIP3P, ff14SB, and lipid21. OPC with ff19SB and lipid21 is the newer set.

Salt, when **Add bulk salt** is on, is a concentration in mol/L. Packmol-memgen takes that value as `--saltcon` and has no upper limit; 0.15 M is its default and the physiological starting point. A negative value is rejected. The cation and anion neutralize the system in tleap and, when bulk salt is on, are also the packmol ions. Details: salt concentration.

A **4-site water** model (OPC, TIP4P-D, TIP4P-Ew) asks which MD engine will run the system, because the water parameters tleap writes depend on that engine:

- **NAMD** makes tleap use FlexibleWater. Equilibration can then set `waterModel tip4`.
- **GROMACS, OpenMM, and Amber** keep the standard Amber water parameters, the same path as a 3-site model.

Water thickness and the solute-to-box distance are the packmol-memgen `--distwat` and `--dist` values. Guidance: water layer thickness.

## Ligand charges

Free ligands and polymer caps (FVA, FOR, ETA, and the other peptide termini) go through antechamber, parmchk2, and tleap before the system tleap. The default is **GAFF2** with **ABCG2**. Amber's recommended pairings are GAFF with AM1-BCC, and GAFF2 with ABCG2. The other two pairings are marked on the two menus. The sentence is a warning; it does not by itself block Generate. Leaving a row on **Pending** while **Parametrize with tleap** is on does block it: packmol would finish and tleap would fail hours later. **Timeout** is unchecked, so antechamber runs until it finishes. Check it and set the minutes to stop that ligand.

**Detect** reads the PDB again. A completed parametrization is reused when the `.frcmod`, `.lib`, and mol2 are already next to the job. The Final drawing uses the bonds in the parametrized mol2. A ring bond stored as a triple bond is drawn from its length, so a piperazine and an aromatic ring stay ordinary rings.

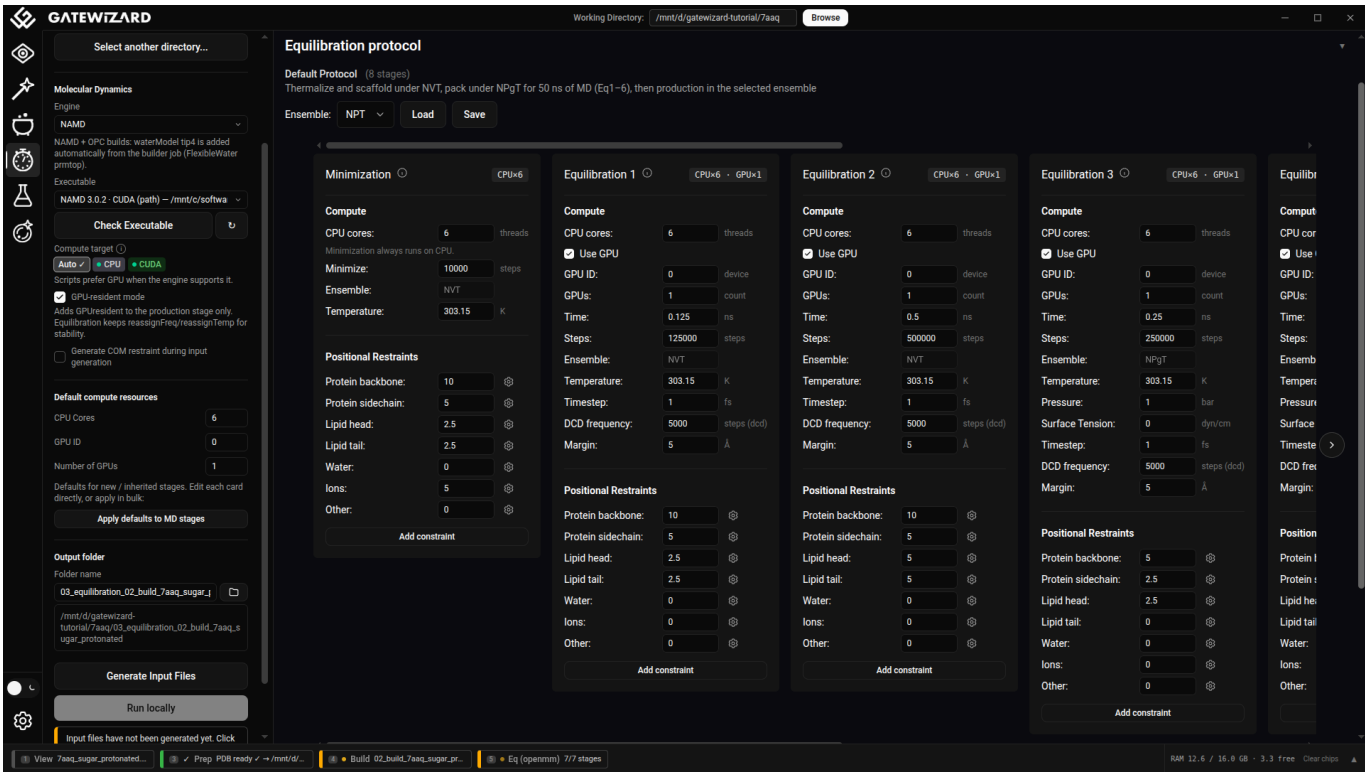
API: `detect_ligands`, `parametrize_ligand`, `parametrize_ligand_from_system_pdb`, `parametrize_all_ligands`, and the recommended pairs in atom type selection. Constants live in `gatewizard.tools.ligand_parametrization` (`CHARGE_METHODS`, `ATOM_TYPES`, `RECOMMENDED_COMBOS`). `timeout` on those methods limits antechamber. Leave it unset to wait until antechamber exits. parmchk2 and tleap each stop after 120 seconds. Polymer caps use the same antechamber wait through `parametrize_peptide_cap`.

### Free molecules

A free molecule is an extra copy packed with packmol-memgen `--solute`, on top of the protein and the lipids. Each row has a residue name that matches the molecule PDB, a count or concentration (`4` or `0.1M`), and a choice of water or membrane. Parametrize it with the same GAFF / GAFF2 and AM1-BCC / ABCG2 menus before tleap. Its **Timeout** box works the same way as a ligand: unchecked waits until antechamber finishes. The API passes these as the `solutes` list on `prepare_system`.

## Equilibration

Equilibration reads the Builder folder (`.prmtop` and `.inpcrd`), builds a multi-stage protocol, and writes run scripts. Pick the engine, the executable, the compute target, and the scheme, then **Generate Input Files**.




The input directory must contain the topology and coordinates from Builder. The page reports the atom count. If no protein is found, protein restraints are left out of the protocol; lipid restraints stay. That detection is `pdb_has_protein` in the equilibration module.

**Engine** is NAMD, GROMACS, OpenMM, or Amber. Changing it reloads that engine's resource defaults and the stage fields. **Executable** lists installs found on PATH, in conda, and, for GROMACS, via GMXRC. **Custom path** is a binary you type. **Check Executable** runs it. The refresh button scans again.

NAMD with the OPC water model adds `waterModel tip4` from the Builder job, because tleap wrote a FlexibleWater prmtop for that choice. The other engines keep the Amber water parameters Builder wrote for them.

Each engine has a manager with the same idea and different files: `NAMDEquilibrationManager`, and the OpenMM, GROMACS, and Amber managers beside it. NAMD's full setup is `setup_namd_equilibration`. File discovery is `find_system_files`.

**Generate COM restraint** writes a center-of-mass restraint during input generation. The selection is MDAnalysis syntax (`name CA` is the usual protein reference). Turning it on also offers a rotation restraint. Force constants are in kcal/mol/Å$^2$. A system with no protein skips these protein restraints.

### Compute target

Compute target is the platform written into the generated run scripts. It is the GPU flag, or the OpenMM `PLATFORM`. It is separate from the CPU-core and GPU-count fields under Default compute resources.

| Target | What the script does |
|---|---|
| **Auto** | OpenMM picks the fastest platform when the job starts. The other engines prefer GPU when that engine supports it. |
| **CPU** | The script stays on CPU. GPU id and GPU count are hidden, because this run is not asking for a device. |
| **CUDA** | The script requests CUDA. NAMD, GROMACS, and Amber use a CUDA build. OpenMM sets `PLATFORM=CUDA`. |
| **OpenCL** | OpenMM only. The script sets `PLATFORM=OpenCL`. |
| **Metal** | OpenMM only, for a Mac GPU. |

A **green dot** means the executable currently selected supports that target. You can still choose CUDA or OpenCL when this binary does not. The script is written for that target anyway, so you can pick a matching build from **Executable** or run the folder later on another machine. The amber line under the buttons names a matching install when one was found.

NAMD **GPU-resident** is available when the target is not CPU. It adds `GPUresident` to the production stage only. Equilibration keeps `reassignFreq` and `reassignTemp` so the heating stage stays stable.

Engine defaults, applied when you change the engine:

| Engine | Sidebar start | Minimization | Later MD |
|---|---|---|---|
| NAMD | 6 CPU, 1 GPU, Auto | CPU | GPU |
| GROMACS | 6 CPU, 1 GPU, Auto | CPU | GPU |
| Amber | 1 CPU, 1 GPU, Auto | CPU | GPU (`pmemd.cuda`). The first packing barostat stays on CPU. |
| OpenMM | 1 CPU, 1 GPU, Auto | GPU | GPU, including the first packing stage |

### Default compute resources

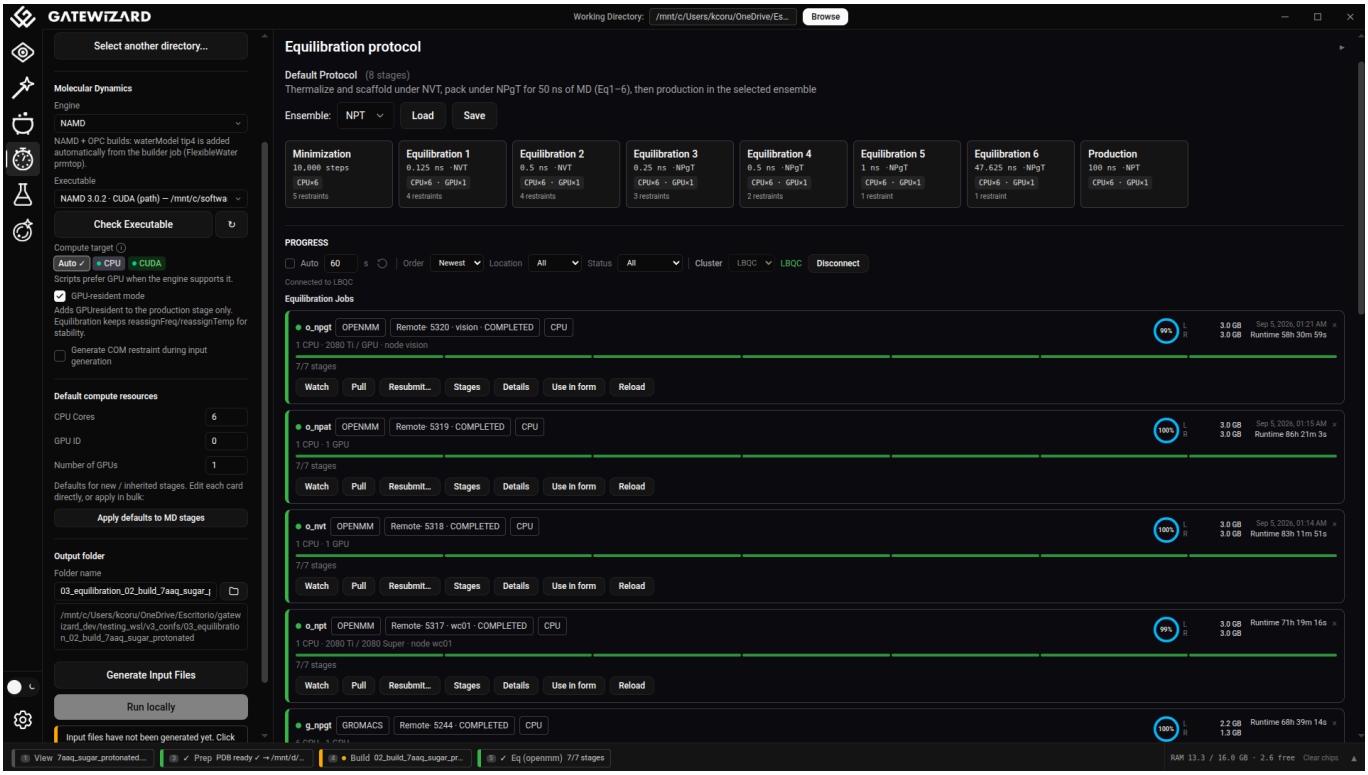
These numbers are the local defaults for new stages and for stages that inherit the sidebar. **CPU cores**, **GPU ID**, and **Number of GPUs** are copied when you press **Apply defaults to MD stages**. Until you press it, edits stay on the sidebar and each stage card keeps its own values. Edit a card directly when one stage should differ (a minimization on CPU, production on a second GPU).

GPU ID is the device index the engine receives (`0` is the first GPU). Number of GPUs is how many devices that stage asks for. Both fields appear when the compute target is not CPU.

This block is the machine you are sitting at, or the counts you want written into a local run script. A remote machine is Cluster: Settings → Clusters, then **Run on cluster** on the job.

### Jobs

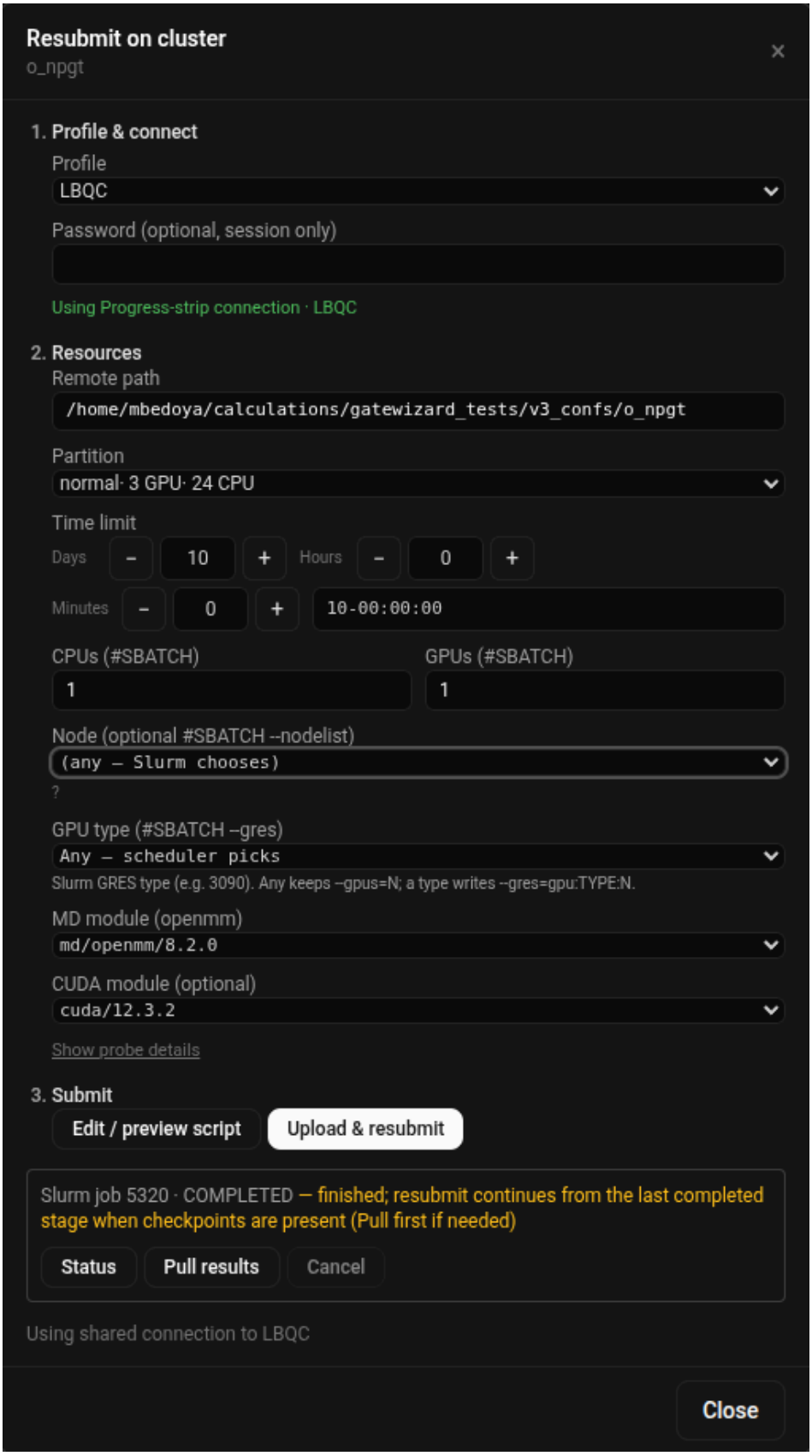
Each card is one generated equilibration folder.




**Watch** follows local logs and, after you connect, the remote stage state. **Pull** downloads results from the cluster. Watching does not download the full trajectory; Pull does. Per-stage lines show which step is waiting, running, or finished. **Cancel** stops a running stage. The same folder layout is what `JobMonitor` scans for Builder jobs; equilibration progress on the cluster is the `execution` metadata from `gatewizard.utils.cluster`.

## Cluster

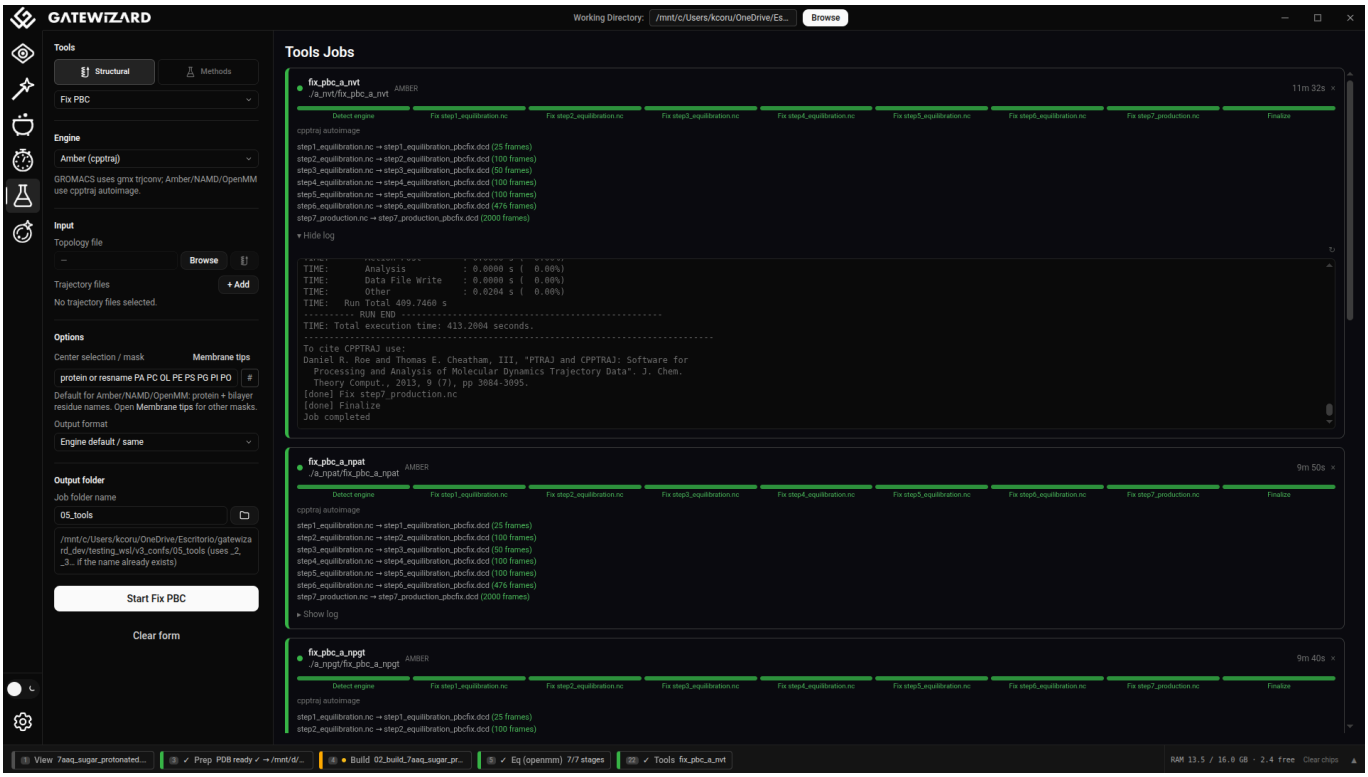
**Run on cluster** uses a profile from Settings → Clusters: host, user, SSH key, remote work directory, and an optional scratch directory. The dialog writes a Slurm batch script next to the local equilibration folder. Locally you already have `run_equilibration.sh`. The cluster path adds `run_equilibration_cluster.sh` and wraps it with the batch script.




The batch script is `render_batch_script` (`BatchScriptRequest`: job name, CPU and GPU counts, modules, and a workdir strategy such as `scratch_job_id`). Generate the inputs on this machine first, then submit that script on the cluster. GPU safe mode in Settings is session-only unless you change it there.

## Tools

**Fix PBC** unwraps and centers a trajectory so Analysis and Visualize see one continuous molecule instead of jumps across the box.




**Engine.** Auto-detect reads the topology and trajectory and picks a method. The line under the menu is that detection. You can force an engine:

- **GROMACS** uses `gmx trjconv`. It needs the topology that holds the molecule definitions.
- **Amber, NAMD, and OpenMM** use `cpptraj autoimage`.
- **MDAnalysis** is the fallback when those binaries are not the right tool for the files.

**Input** is the topology plus the trajectory list. Stride `1` keeps every frame. `2`, `5`, or `10` writes a smaller file. Drag a row to reorder. A PDB or GRO in the list is ignored for a tool that needs a trajectory with a periodic box.

**Options** are the center group, the output name, and the image choice passed to that engine. A missing working directory and a failed run stay on the page. Other tools on this page stay marked coming soon; finished tools keep a status JSON in the project folder. Folder-wide job scans pause while you are on Visualize so playback stays smooth.

## Analysis

Analysis has two modes on one page. **Structural** reads a topology and trajectories. **Energetic** reads an MD log. The engine you pick selects the parser. Empty states and errors stay on the page. Formats, plots, and install notes: Analysis features. The Python calls are the analysis module.

### Structural

The active set is the topology and the trajectories for this run. Supported topologies are PSF, PDB, and GRO. Trajectories are DCD, XTC, TRR, and NetCDF. A PDB or GRO in the trajectory list has no periodic box, so thickness and area-per-lipid skip it. For RMSD against a starting structure, use **Reference PDB** rather than listing that file as a trajectory. Amber `.rst7`, `.restrt`, and `.inpcrd` files are restarts, not trajectories, and are skipped.

| Analysis | What it measures | API |
|---|---|---|
| RMSD | Deviation from a reference frame or PDB, with optional alignment | `calculate_rmsd` |
| RMSF | Per-residue fluctuation | `calculate_rmsf` |
| Distances | One or more MDAnalysis pairs over time | `calculate_distances` |
| Radius of gyration | Compactness of the selection | `calculate_radius_of_gyration` |
| Area per lipid | Bilayer area, via FATSLiM, GridMAT, or lipyphilic | `calculate_area_per_lipid`, `run_fatslim_apl`, `run_gridmat_md_apl` |
| Thickness | Membrane thickness with the same tools | `calculate_membrane_thickness` |

Selections use MDAnalysis syntax (`protein and name CA`). How to read the plots: structural analysis and atom selections.

Several trajectory files need a time for each file. Otherwise the x-axis is concatenated steps, not nanoseconds. See time management.

### RMSD

Deviation of the selection from a reference frame or a reference PDB. Alignment can be turned on. A protein trace uses `protein and name CA`. Reading the plot: RMSD. The call is `calculate_rmsd`.

### RMSF

Fluctuation of each residue in the selection across the trajectory. Peaks are the flexible parts. Reading the plot: RMSF. The call is `calculate_rmsf`.

### Distance

Distance between MDAnalysis selections over time. Each pair is its own trace. Reading the plot: distances. The call is `calculate_distances`.

### Radius of gyration

Compactness of the selection over time. A steady trace means the selection keeps its size. Reading the plot: radius of gyration. The call is `calculate_radius_of_gyration`.

### Area per lipid

Mean area of each lipid, from FATSLiM, GridMAT, or lipyphilic. A PDB or GRO in the trajectory list is skipped because that file has no periodic box. Reading the plot: area per lipid. The call is `calculate_area_per_lipid`.

### Membrane thickness

Distance between the two leaflet headgroups. A PDB or GRO in the trajectory list is skipped because that file has no periodic box. Reading the plot: membrane thickness. The call is `calculate_membrane_thickness`.

### Titration

Titration is a structural figure built from a PropKa run on one PDB. The longer notes are titration.

- **Panel A** is the pKa comparison.
- **Panel B** is the protonated fraction.
- **Panel C** is the titration curve.

**A/B** includes that residue in the pKa and fraction plots. **C** draws its curve. Cysteines that are in a disulfide are left out of A and B. Uncheck A/B on any other residue to leave it out of those panels. Uncheck a residue type to hide that type from A and B. The figure builder is `build_titration_figure`.

### Energetic

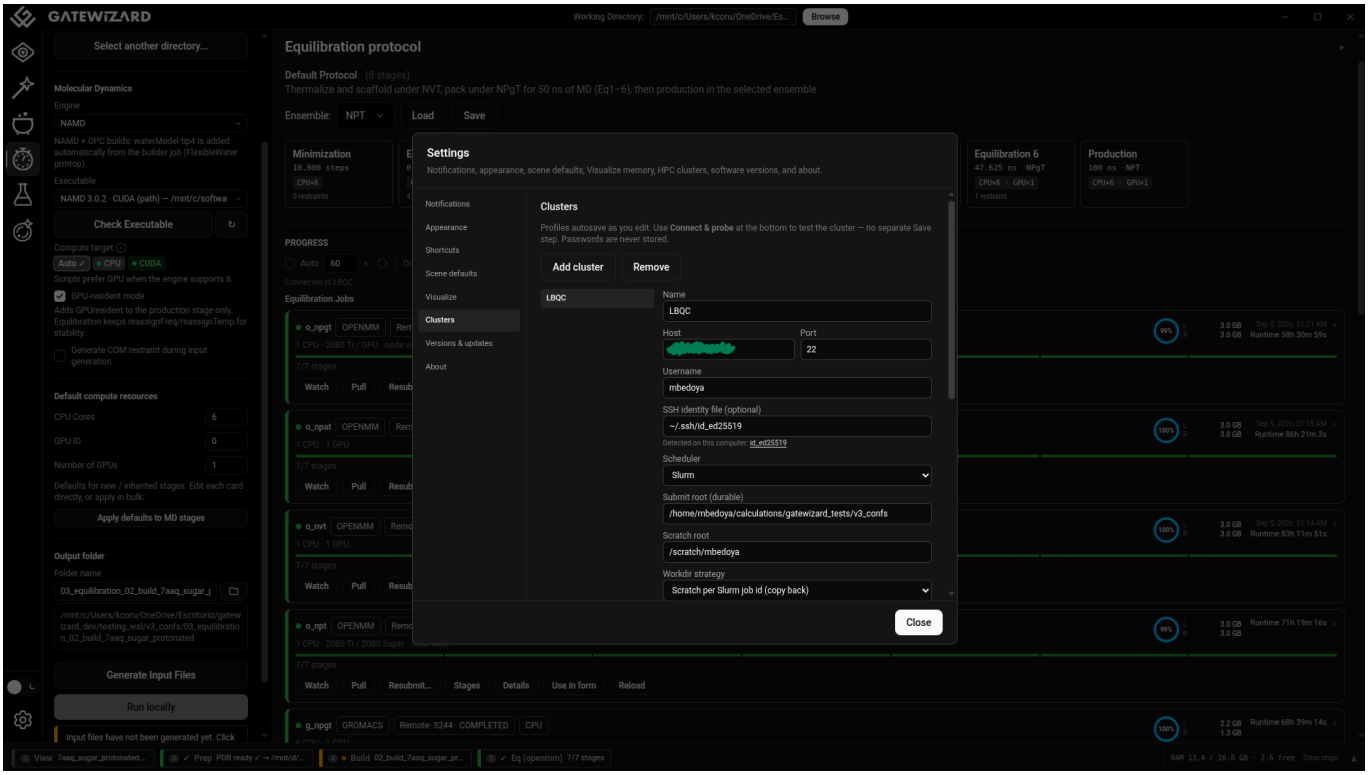
Energetic analysis parses a log and plots the terms you tick. The **MD engine** menu chooses the parser:

| Engine | Parser |
|---|---|
| NAMD | `EnergyAnalyzer` / `parse_namd_log` |
| OpenMM | `OpenMMLogAnalyzer` |
| GROMACS | `GROMACSLogEnergyAnalyzer` |
| Amber | `AmberLogEnergyAnalyzer` |

Time, which energy terms are drawn, and the plotted window apply to the active log. Multi-file logs need a time for each file, same as trajectories. Units and the common “energy looks wrong” cases: energetic analysis.

## Settings and clusters

Gear → **Clusters**: host, user, key, remote workdir, optional scratch. GPU safe mode is session-only unless you change it here.




## Status bar

The footer shows backend connectivity and **system RAM** (not polled while you stay on Visualize).

## Python API

For scripts, use the API reference and method dictionary. Worked examples are the `tests/<module>_examples/` scripts in the API repository.

# Analysis

## Analysis Features

Desktop steps live in the User Guide → Analysis. This page is the feature reference (formats, methods, FATSLiM / GridMAT install).

### Overview

GateWizard provides three main types of analysis:

1. **Structural Analysis**: For trajectory files (.dcd, .xtc, .trr)
2. **Bilayer Analysis**: Area per lipid and membrane thickness (via lipyphilic, installed with gatewizard)
3. **Energetic Analysis**: For NAMD log files

Both types support multiple files, custom time assignment, and flexible unit selection.

### Structural Analysis

#### Supported Formats

- **Topology**: PSF, PDB, GRO
- **Trajectories**: DCD, XTC, TRR, NetCDF

#### Analysis Types

##### RMSD (Root Mean Square Deviation)

**Purpose**: Measure structural deviation from a reference structure.

**Usage**:

```
1. Select RMSD from Analysis Type
2. Set reference frame (default: first frame)
3. Choose atom selection (e.g., "protein and name CA")
4. Enable/disable alignment
5. Run analysis
```

**Output**: - Plot: Time vs RMSD - Data: Time points and RMSD values - Units: Ångstroms (Å) or nanometers (nm)

**Interpretation**: - Low RMSD (~1-3 Å): Structure remains close to reference - High RMSD (>5 Å): Significant structural changes - Plateau: System has equilibrated - Increasing trend: System still evolving

##### RMSF (Root Mean Square Fluctuation)

**Purpose**: Identify flexible and rigid regions of the protein.

**Usage**:

```
1. Select RMSF from Analysis Type
2. Choose atom selection (typically C-alpha atoms)
3. Run analysis
```

**Output**: - Plot: Residue number vs RMSF - Data: Per-residue fluctuation values - Units: Ångstroms (Å) or nanometers (nm)

**Interpretation**: - Low RMSF (<1 Å): Rigid regions (core, secondary structures) - High RMSF (>2 Å): Flexible regions (loops, termini) - Peaks indicate highly mobile residues

#### Distance Analysis

**Purpose**: Track distances between specific atoms or groups.

**Usage**:

```
1. Select Distances from Analysis Type
2. Define atom pairs:
   - Selection 1: "resid 10 and name CA"
   - Selection 2: "resid 50 and name CA"
3. Run analysis
```

**Output**: - Plot: Time vs Distance - Data: Time points and distance values - Units: Ångstroms (Å) or nanometers (nm)

**Use Cases**: - Salt bridge formation - Domain movements - Ligand-protein interactions - Conformational changes

#### Radius of Gyration

**Purpose**: Measure overall compactness of the structure.

**Usage**:

```
1. Select Radius of Gyration
2. Choose atom selection (typically all protein atoms)
3. Run analysis
```

**Output**: - Plot: Time vs Rg - Data: Time points and Rg values - Units: Ångstroms (Å) or nanometers (nm)

**Interpretation**: - Constant Rg: Stable, compact structure - Increasing Rg: Unfolding or expansion - Decreasing Rg: Compaction or folding

#### Titration

**Purpose**: PropKa pKa values, protonated fractions, and titration curves for one PDB. A trajectory is not used.

**Usage**:

```
1. Select Titration from Analysis Type
2. Browse a PDB (for STP10, 7aaq_ligand_oriented.pdb)
3. Set the target pH, the pH step, and the pH range
4. Run analysis
```

**Output**:

- **Panel A** is the pKa comparison.
- **Panel B** is the protonated fraction.
- **Panel C** is the titration curve.

**A/B** includes that residue in the pKa and fraction plots. **C** draws its curve. Cysteines that are in a disulfide are left out of A and B. The figure builder is `build_titration_figure`.

### Bilayer Analysis (lipyphilic + FATSLiM)

Lipid bilayer properties are calculated via the `BilayerTrajectoryAnalyzer` class. Leaflet assignment and membrane thickness always use lipyphilic. Area per lipid methods:

| apl_method | Backend |
|---|---|
| `fatslim` / `auto` (**default**) | Original archived FATSLiM CLI (`fatslim apl`) via subprocess |
| `evapl` | **EVAPL** (experimental / **not yet validated**): freud Voronoi + half-plane clips for occupants |
| `lipyphilic` | Official `lipyphilic.AreaPerLipid` |
| `gridmat` | **GridMAT (GW, experimental)** — in-process reimplementation closer to GridMAT-MD.pl (not bit-identical) |
| `gridmat_md` | Original GridMAT-MD.pl via Perl (external; literature-faithful) |
| `vtmc` | **VTMC (GW, experimental)** — GateWizard reimplementation of Voronoi + Monte Carlo protein disks (Mori et al.) |

#### Where the install scripts live (read this first)

Two different "GateWizard" installs are easy to mix up:

| What you have | What it is | Has `scripts/install_*.sh`? |
|---|---|---|
| **gatewizard** Python package (`pip install gatewizard` / `pip install -e .`) | The library the GUI and CLI import | **No** — only the importable `gatewizard/` package |
| **gatewizard** GitHub source tree | maurobedoya/gatewizard clone with `pyproject.toml` + top-level `scripts/` | **Yes** |
| **gatewizard-gui** (`.deb`, AppImage, `npm run dev`, `/usr/bin/gatewizard-gui`, …) | Desktop app | **No** — it depends on the Python package; it does **not** ship these helper scripts |

So: installing or launching the **GUI** does **not** put `scripts/install_fatslim_env.sh` next to the executable. A normal `pip install gatewizard` also does **not** install those scripts into `site-packages`. You need a **checkout of the API repository** (or download the script file itself) only to *run the installer*. After that, FATSLiM / GridMAT live in their own locations (`fatslim-py38` conda env, `~/gridmat-md/`, …) and GateWizard finds them via env vars / `PATH` — they never have to sit beside the GUI binary.

**If you only use the GUI**, still run the steps below in the same **WSL / Linux** environment that runs the analysis backend (not Windows PowerShell, and not the GUI's embedded Python).

**Get the scripts (pick one):**

```
# A) Shallow clone of the API repo (recommended — also has docs and both installers)
git clone --depth 1 https://github.com/maurobedoya/gatewizard.git
cd gatewizard

# B) Or download one script without a full clone, e.g.:
# curl -fsSL -o install_fatslim_env.sh \
#   https://raw.githubusercontent.com/maurobedoya/gatewizard/main/scripts/install_fatslim_env.sh
# bash install_fatslim_env.sh
```

If you already develop against a local clone (e.g. `~/gatewizard`, `/mnt/d/github/gatewizard`), use that folder — it is the directory that contains **both** `pyproject.toml` and `scripts/`.

#### Installing FATSLiM (required for the default APL method)

FATSLiM is **GPLv3** and needs **Python ≤3.8**. GateWizard is MIT and never imports FATSLiM — it only calls the `fatslim` **binary**. Install a **companion** conda env (not the GateWizard analysis env, not the GUI mamba-env).

```
# from the API repo checkout (see above)
bash scripts/install_fatslim_env.sh

# point GateWizard at the binary (put this in ~/.bashrc on the WSL/Linux side)
export GATEWIZARD_FATSLIM="$(conda run -n fatslim-py38 which fatslim)"
```

The installer creates/updates conda env `fatslim-py38` and prints the full path to `fatslim`. You only needed the repo (or the script file) to run the installer; you can delete the clone afterward if you want.

Discovery order: `GATEWIZARD_FATSLIM` → `PATH` → conda env `fatslim-py38`.

| Surface | Notes |
|---|---|
| **API / scripts** | Linux, WSL, or macOS (best-effort). Set `GATEWIZARD_FATSLIM` in the shell that runs GateWizard. |
| **GUI** | Analysis runs under **WSL / Linux**. Export `GATEWIZARD_FATSLIM` there **before** starting the backend (or add it permanently to that user's shell profile). Do **not** `pip install fatslim` into the embedded GUI Python. The Analysis sidebar only warns when FATSLiM is selected and the CLI is missing. |
| **Missing binary** | `apl_method='fatslim'` fails with an install hint — it does **not** silently fall back to EVAPL. |
| **Parallelism** | `fatslim_nthreads` (per-process `--nthreads`; `-1` = all CPUs) and `fatslim_jobs` (frame chunks via `--begin-frame` / `--end-frame`). Prefer **small nthreads (1–4) + more jobs** for better wall-clock time than one job with all CPUs. GUI exposes both when FATSLiM is selected. |

Upstream FATSLiM is archived/unmaintained; GateWizard centers the bilayer before export to reduce leaflet-ID failures. Prefer orthorhombic boxes and a correct `lipid_sel` / NDX headgroups group.

#### Installing GridMAT-MD.pl (optional external APL)

Same idea as FATSLiM: the helper script lives in the **API source tree**, not in the GUI install. It clones jalemkul/gridmat-md (default `~/gridmat-md`) and prints an export for the `.pl` file.

```
# from the API repo checkout (see "Where the install scripts live")
bash scripts/install_gridmat_md.sh

# default clone path — adjust if you set GRIDMAT_MD_DIR
export GATEWIZARD_GRIDMAT_MD="$HOME/gridmat-md/GridMAT-MD.pl"
```

Requires **perl** on PATH when you run analysis (or set `GATEWIZARD_PERL`). Discovery: `GATEWIZARD_GRIDMAT_MD` → `PATH` → common clone locations. GateWizard stays MIT (subprocess only). Use `apl_method='gridmat'` for the experimental in-process path, or `gridmat_md` for the original `.pl`. Parallelism: `gridmat_md_jobs` (default `min(8, cpu_count)`).

With `apl_method="lipyphilic"` and a non-empty `exclude_sel`, the installed lipyphilic must accept `exclude_sel` (upstream PR #164). PyPI 0.12.x does not; from an API checkout run:

```
pip install -r requirements-lipyphilic-git.txt
```

(That requirements file is also only in the API source tree / sdist layout — not inside the GUI package.)

Otherwise use `apl_method="fatslim"` (default) or `apl_method="evapl"` (experimental). Other dependencies install with the Python package (`pip install gatewizard` or `pip install -e .` from a clone).

Working examples: **Example 14** (area per lipid) and **Example 15** (membrane thickness) in `tests/analysis_examples/`, using `equilibration_folder/system.pdb` and equilibration DCD trajectories. See Analysis Module API.

#### Area per Lipid

**Purpose**: Measure the lateral area occupied by each lipid. Default **FATSLiM** maps `exclude_sel` to FATSLiM's interacting group (default `protein`). `apl_method="evapl"` is GateWizard's experimental freud path (not yet validated). `apl_method="lipyphilic"` runs upstream `AreaPerLipid`.

**Usage**:

```
1. Install companion FATSLiM env (scripts/install_fatslim_env.sh) and set GATEWIZARD_FATSLIM
2. Load bilayer topology and trajectories from equilibration_folder (Example 14 / 15)
3. Set lipid headgroup selection (e.g. "resname PC and name P31" for AMBER POPC)
4. Optionally set exclude_sel="protein" (default)
5. Run area-per-lipid analysis (default apl_method → fatslim)
```

**Output**: - Time series of mean area per lipid (Å$^2$) - Per-lipid areas and per-leaflet means - Residue IDs for each lipid

**Interpretation**: - ~40–70 Å$^2$: Typical phospholipid areas (force-field dependent) - Lower area: Higher packing / Lo phase, or protein footprint excluded - Higher area: Looser packing / Ld phase; without exclusion, protein–membrane systems can approach box / n_lipids

##### Membrane Thickness

**Purpose**: Measure bilayer thickness as the mean interleaflet headgroup distance.

**Usage**:

```
1. Load bilayer topology and trajectories from equilibration_folder (Example 14 / 15)
2. Set headgroup selection (e.g. "resname PC and name P31")
3. Optionally exclude species via leaflet_filter_sel
4. Run membrane thickness analysis
```

**Output**: - Time series of membrane thickness (Å) - Summary statistics (mean, std, min, max)

**Interpretation**: - ~35–45 Å: Typical hydrated phospholipid bilayers (model dependent) - Stable plateau: Equilibrated membrane - Drift: Possible incomplete equilibration or phase change

### Time Management for Trajectories

#### Without Time Assignment

Uses frame numbers × time step: - Automatically calculated from trajectory metadata - Suitable for single, continuous trajectories

#### With Time Assignment

Distributes frames evenly across assigned time:

**Example**:

```
File1.dcd: 1000 frames, assigned 10 ns
  → Frame 0: 0.0 ns
  → Frame 500: 5.0 ns
  → Frame 999: 10.0 ns

File2.dcd: 2000 frames, assigned 50 ns
  → Frame 0: 10.0 ns (continues from File1)
  → Frame 1000: 35.0 ns
  → Frame 1999: 60.0 ns
```

**Benefits**: - Accurate time representation for concatenated runs - Proper handling of different sampling rates - Sequential placement of multiple simulations

### Atom Selections

GateWizard uses MDAnalysis selection syntax:

**Common Selections**:

```
protein                     # All protein atoms
protein and name CA         # C-alpha atoms only
protein and backbone        # Backbone atoms (N, CA, C, O)
resid 1:50                  # Residues 1 to 50
resname ALA                 # All alanine residues
name CA and resid 10:20     # C-alpha of residues 10-20
segid PROA                  # Atoms in segment PROA
```

**Advanced Selections**:

```
protein and not name H*             # Protein without hydrogens
(resid 10:20) or (resid 50:60)     # Multiple regions
around 5 resname LIG                # Within 5Å of ligand
```

#### Units and Conversions

**Distance Units**: - Ångstroms (Å): Default in structural biology - Nanometers (nm): Common in MD simulations - Conversion: 1 nm = 10 Å

**Time Units**: - Picoseconds (ps): Fine-grained time scale - Nanoseconds (ns): Common for simulation lengths - Microseconds (μs): Long simulations - Conversions: 1 ns = 1000 ps, 1 μs = 1000 ns

### Energetic Analysis

#### NAMD Log File Analysis

**Supported Data**: - **ETITLE**: Column headers from log file - **ENERGY**: Data lines with timesteps and values

**Available Properties**: - TEMP: Temperature (K) - TOTAL: Total energy (kcal/mol) - KINETIC: Kinetic energy (kcal/mol) - POTENTIAL: Potential energy (kcal/mol) - ELECT: Electrostatic energy (kcal/mol) - VDW: van der Waals energy (kcal/mol) - PRESSURE: Pressure (bar) - VOLUME: Volume ($Å^2$)

#### Time Distribution for Log Files

##### Without Time Assignment

Uses timestep (TS) column:

```
TS = 0:     0 ps
TS = 1000:  2 ps (if timestep=2fs, output every 1000 steps)
TS = 50000: 100 ps
```

##### With Time Assignment

Distributes data points evenly:

**Example**:

```
equilibration.log: 500 points, assigned 5 ns
  → Point 0: 0.0 ns
  → Point 250: 2.5 ns
  → Point 499: 5.0 ns

production.log: 5000 points, assigned 100 ns
  → Point 0: 5.0 ns (continues from equilibration)
  → Point 2500: 55.0 ns
  → Point 4999: 105.0 ns
```

**Benefits**: - Correct timeline for multi-stage simulations - Proper spacing for different output frequencies - Sequential representation of simulation phases

#### Energy Units

**kcal/mol** (default in NAMD): - Standard in US molecular modeling - 1 kcal/mol = 4.184 kJ/mol

**kJ/mol** (SI unit): - Standard in international publications - Conversion applied automatically when selected

**Use Cases**: - Plot TOTAL energy to check stability - Monitor TEMP for thermostat performance - Check PRESSURE for barostat performance - Compare KINETIC and POTENTIAL for energy distribution

#### Multiple Property Plotting

Plot multiple properties simultaneously:

1. Select multiple columns (Ctrl+Click)
2. Each property shown in different color
3. Legend automatically added
4. Y-axis label adjusted for multiple properties

**Example**:

```
Plot: TOTAL, TEMP, PRESSURE
Result: Three lines showing:
  - Energy evolution (left Y-axis)
  - Temperature stability
  - Pressure fluctuations
```

### Plot Customization

#### Available Options

**Title and Labels**: - Plot Title: Custom description - X-Axis Label: Time (with units) - Y-Axis Label: Property (with units)

**Units**: - X-Axis: ps, ns, μs - Y-Axis (Distance): Å, nm - Y-Axis (Energy): kcal/mol, kJ/mol

**Appearance**: - Background Color: Dark Gray, Light Gray, White, Black - Line Color: Blue, Red, Green, etc. - Grid: Automatic with transparency

**Ranges**: - X-Min, X-Max: Limit time range - Y-Min, Y-Max: Limit value range - Auto-scaling: Enabled by default

#### Export Options

**Plot Export**: - PNG: Raster image (good for presentations) - SVG: Vector image (good for publications) - High resolution available

**Data Export**: - **CSV**: Spreadsheet format

```
Time(ns),RMSD(Å)
0.0,0.0
0.1,0.5
0.2,0.8
```

- **JSON**: Structured format

```
{
  "time": [0.0, 0.1, 0.2],
  "rmsd": [0.0, 0.5, 0.8],
  "units": {"time": "ns", "rmsd": "Å"}
}
```

- **NumPy**: Python analysis

```
import numpy as np
data = np.load('results.npz')
time = data['time']
rmsd = data['rmsd']
```

## Advanced Features

### Draggable File List

Reorder files by dragging: 1. Click and hold on file bar handle (☰) 2. Drag to new position 3. Release to reorder 4. Time distribution updates automatically

### Real-Time Progress

During analysis: - Progress bar shows completion - Status messages update - Can cancel long-running analyses

### Batch Processing

Analyze multiple trajectories: 1. Add all files at once 2. Assign times for each 3. Run analysis once 4. Data combined automatically

## Troubleshooting Analysis

### Issue: RMSD values too high

**Possible causes**: - Misaligned structures - Wrong reference frame - Including flexible loops

**Solutions**: - Enable alignment option - Try different reference frames - Use more specific atom selection (e.g., secondary structure only)

### Issue: Choppy or discontinuous plots

**Possible causes**: - Files not in correct order - Time assignment incorrect - Missing frames

**Solutions**: - Reorder files using drag-and-drop - Verify time assignments - Check trajectory files for completeness

### Issue: Energy values seem wrong

**Possible causes**: - Wrong units selected - Log files not from same simulation - Parsing errors

**Solutions**: - Check unit selection (kcal/mol vs kJ/mol) - Verify log files are sequential - Check status messages for warnings

### Issue: Memory errors with large trajectories

**Solutions**: - Analyze in smaller chunks - Use more specific atom selections - Increase system RAM - Close other applications

## Best Practices

### For Structural Analysis

1. **Always check alignment**: Enable for RMSD calculations
2. **Use appropriate selections**: Backbone for overall structure, C-alpha for simplicity
3. **Verify time assignments**: Especially for concatenated trajectories
4. **Export data regularly**: Don't lose analysis results
5. **Check convergence**: RMSD should plateau

### For Energetic Analysis

1. **Sequential file ordering**: Equilibration → Production phases
2. **Assign times accurately**: Match actual simulation times
3. **Plot multiple properties**: Check system stability
4. **Monitor temperature**: Should be stable (±1-2 K)
5. **Check energy drift**: Total energy should be conserved (NVE) or stable (NVT/NPT)

### For Publication

1. **Use consistent units**: Pick one set and stick to it
2. **Export high-resolution plots**: SVG for vector graphics
3. **Save raw data**: CSV or NumPy for later analysis
4. **Document parameters**: Note selections, reference frames, etc.
5. **Check error bars**: Consider statistical analysis of multiple runs

## Example Workflows

### Workflow 1: Basic RMSD Analysis

```
1. Load topology: system.psf
2. Add trajectory: production.dcd
3. Assign time: 100 ns
4. Select: RMSD analysis
5. Atom selection: protein and name CA
6. Reference frame: 0
7. Enable alignment: Yes
8. Run analysis
9. Export: plot as PNG, data as CSV
```

### Workflow 2: Multi-File Energy Analysis

```
1. Add files:
   - min.log → 0.1 ns
   - equil.log → 1.0 ns
   - prod1.log → 50 ns
   - prod2.log → 50 ns
2. Total timeline: 0-101.1 ns
3. Select properties: TOTAL, TEMP, PRESSURE
4. Units: X=ns, Y=kcal/mol
5. Run analysis
6. Export: data as JSON
```

**Workflow 3: Residue Flexibility**

```
1. Load topology and trajectory
2. Select: RMSF analysis
3. Atom selection: protein and name CA
4. Run analysis
5. Identify flexible regions (RMSF > 2 Å)
6. Cross-reference with structure
7. Export: plot and data
```

---

*For more examples and detailed tutorials, see the User Guide.*

# Study cases

## Overview

### Study cases

End-to-end walkthroughs live here. API method pages still include the numbered scripts under `tests/<module>_examples/`.

- STP10 in the desktop app — PDB 7AAQ from Visualize through Analysis: MemPro orientation, a titration figure on the oriented PDB, pH 6, HID histidines, a pre-oriented POPC build, 0.15 M KCl, and 100 ns of NPT production.
- STP10 with the API — the same system from `tests/study_cases/7aaq/`: MemPro, preparation, the pre-oriented membrane build, and the OpenMM inputs.
- Prepare systems — POPC membranes for the systems in `prepare_systems.py`: what each PDB keeps, what the structure checker found, how long the finished builds took, and what to change before a new run.

## STP10 desktop app

### STP10 in the desktop app

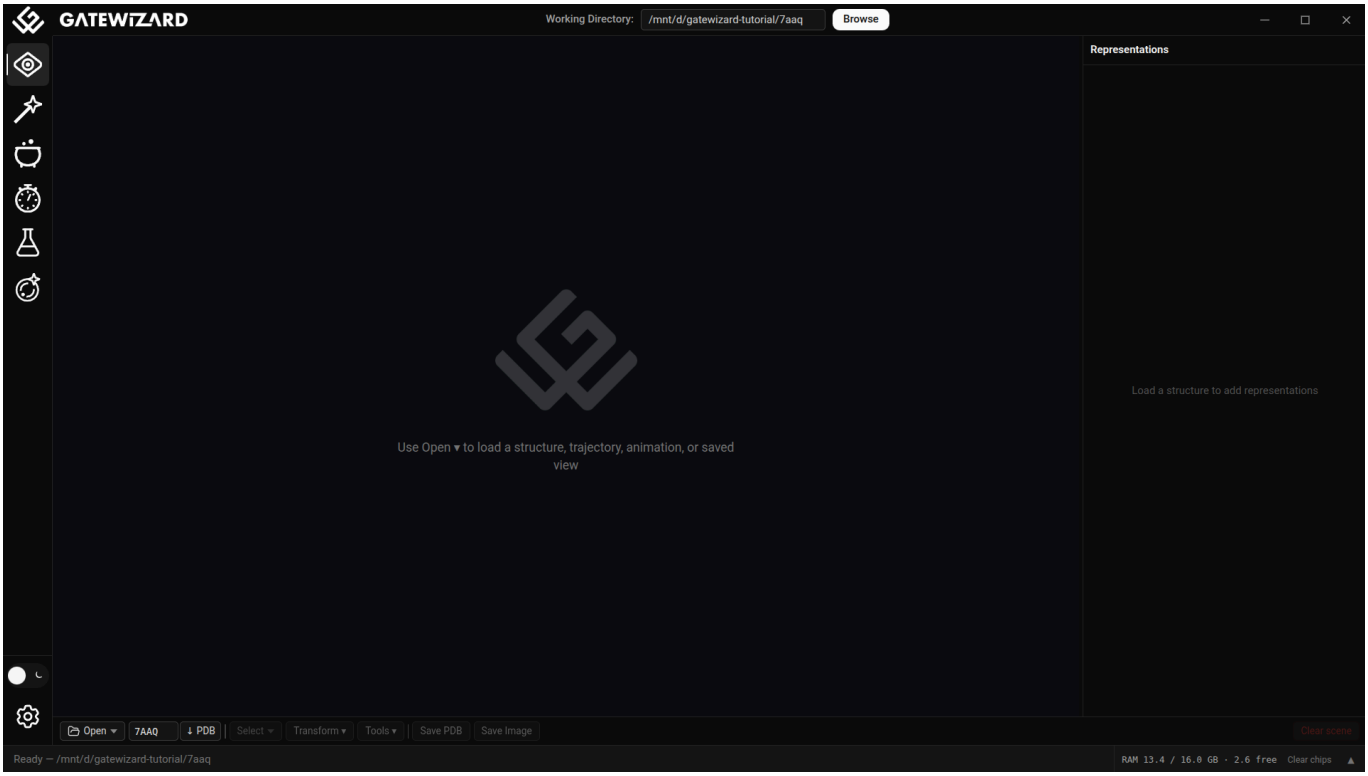
This walkthrough prepares *Arabidopsis thaliana* STP10 (PDB 7AAQ) in the desktop app: Visualize, MemPro, Preparation, Builder, Equilibration, Tools, and Analysis. The same system from Python is STP10 with the API.

Set a writable working directory in the top bar before MemPro, Preparation, Builder, or Equilibration.

#### 1. Visualize

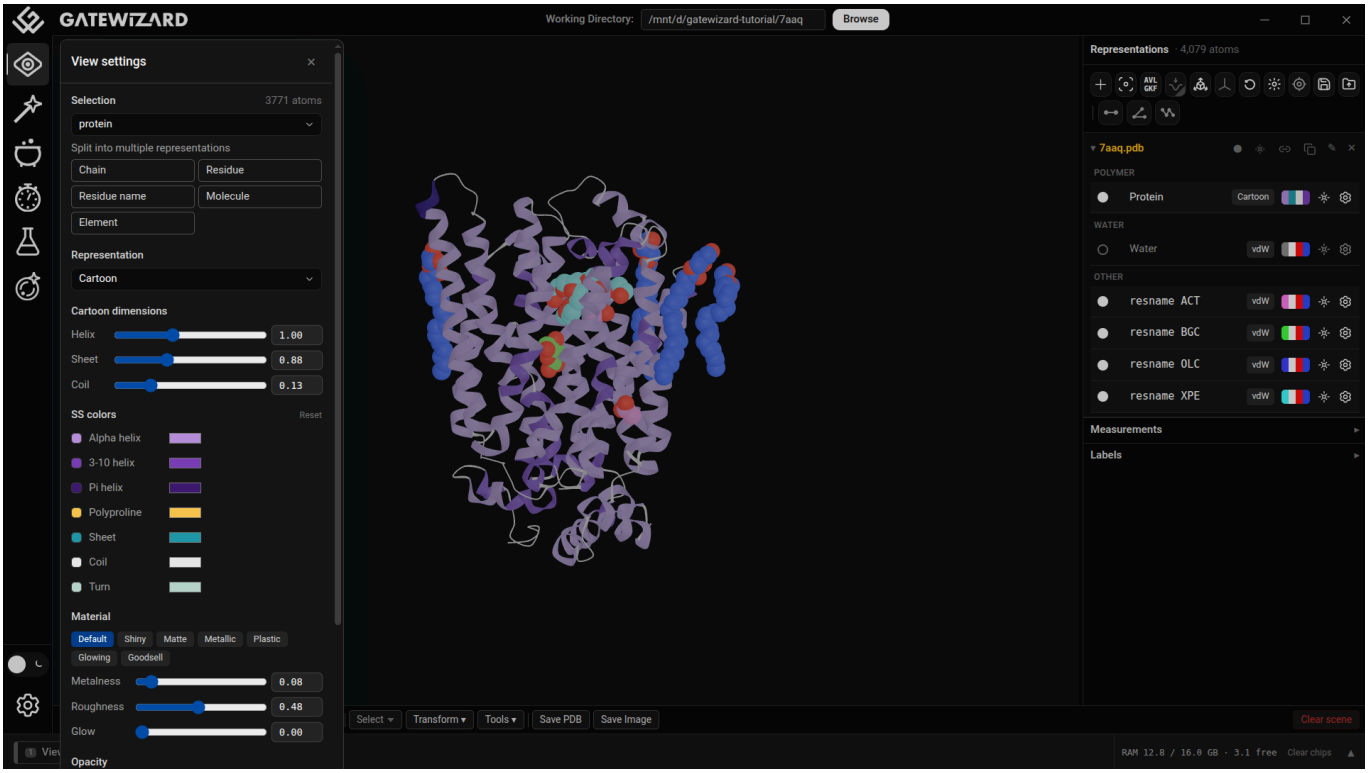
The app opens on Visualize.

In the PDB box at the bottom of the window, type `7AAQ` and press Enter, or click ↓ **PDB**. A file already on disk is **Open** → **Structure**.



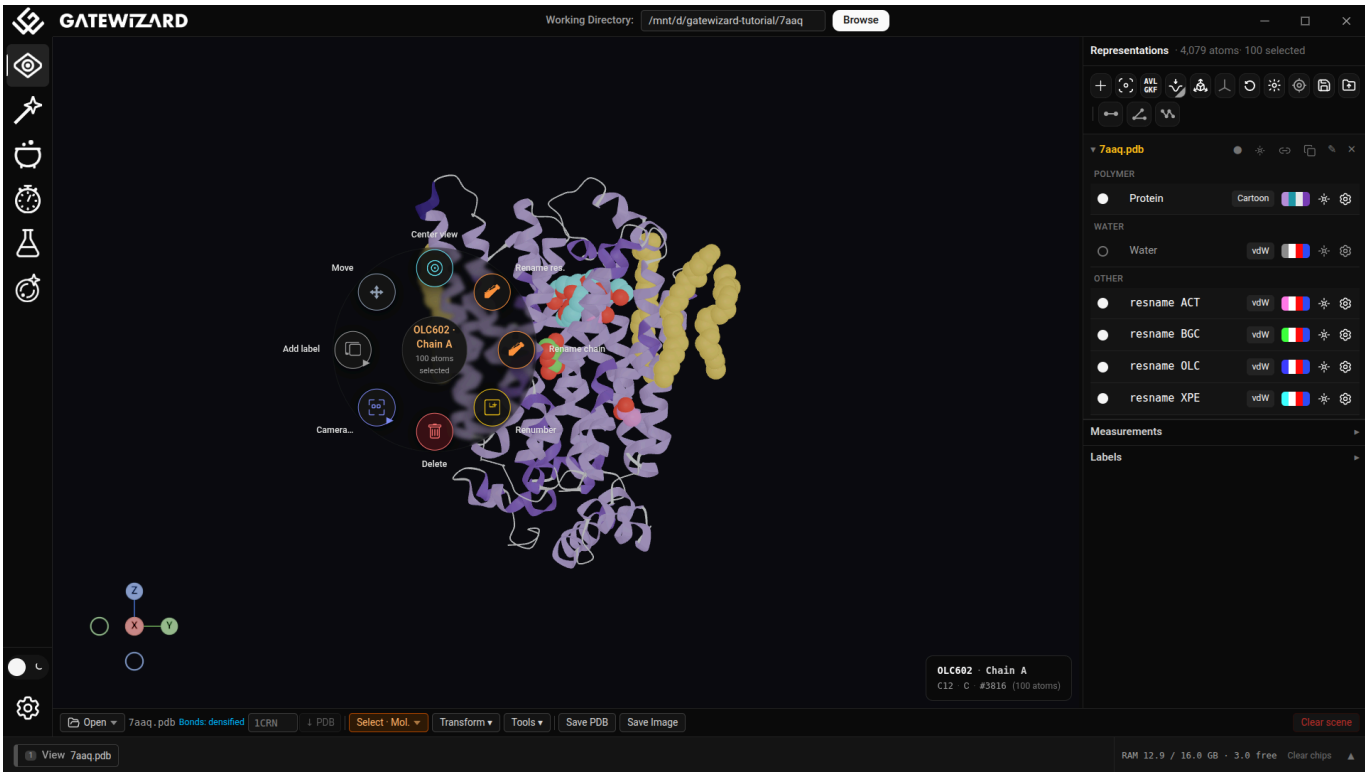
Rotate, pan, and zoom. The reference axes sit in the bottom-left corner.

Open **View Settings**, or choose **Auto Generate Representation** in the Representations panel.




Show or hide protein, water, ligands, and lipids, and change the representation and color of each.

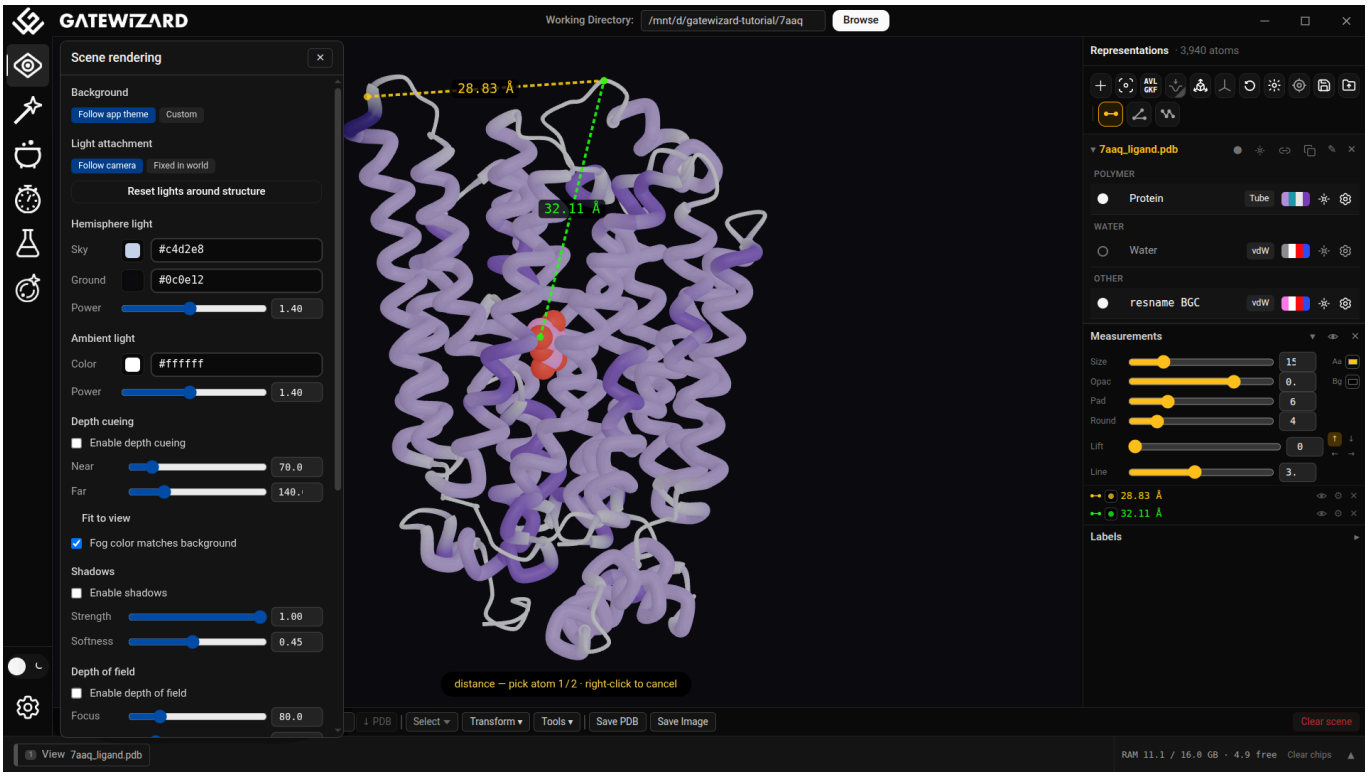
Selection covers atoms, residues, chains, and molecules. Chain names, residue names, and residue numbers can be edited from that same panel.



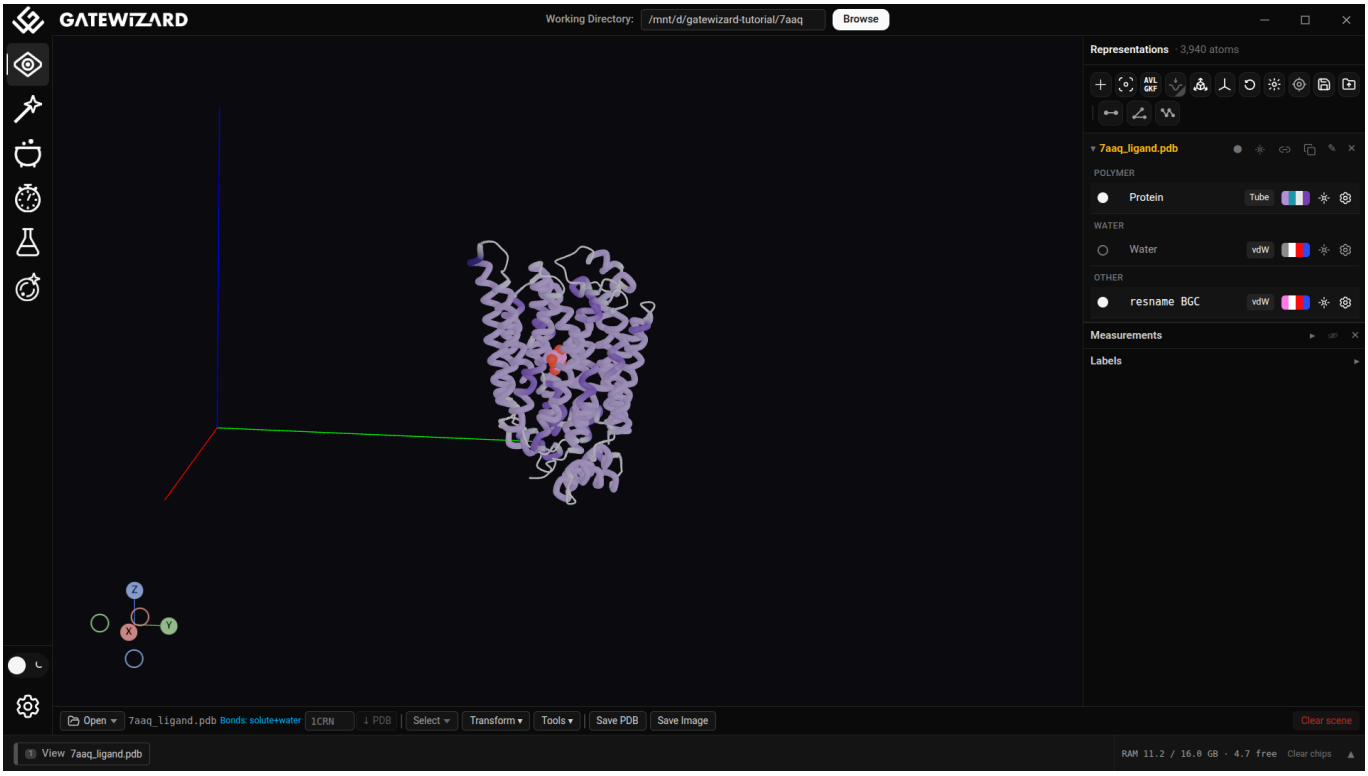

Keep the protein and the bound sugar BGC. Delete the monoolein (OLC), the water, the PEG detergent (XPE), and the acetate (ACT). Hiding a component first makes the next selection easier.

Save the structure as `7aaq_ligand.pdb`.

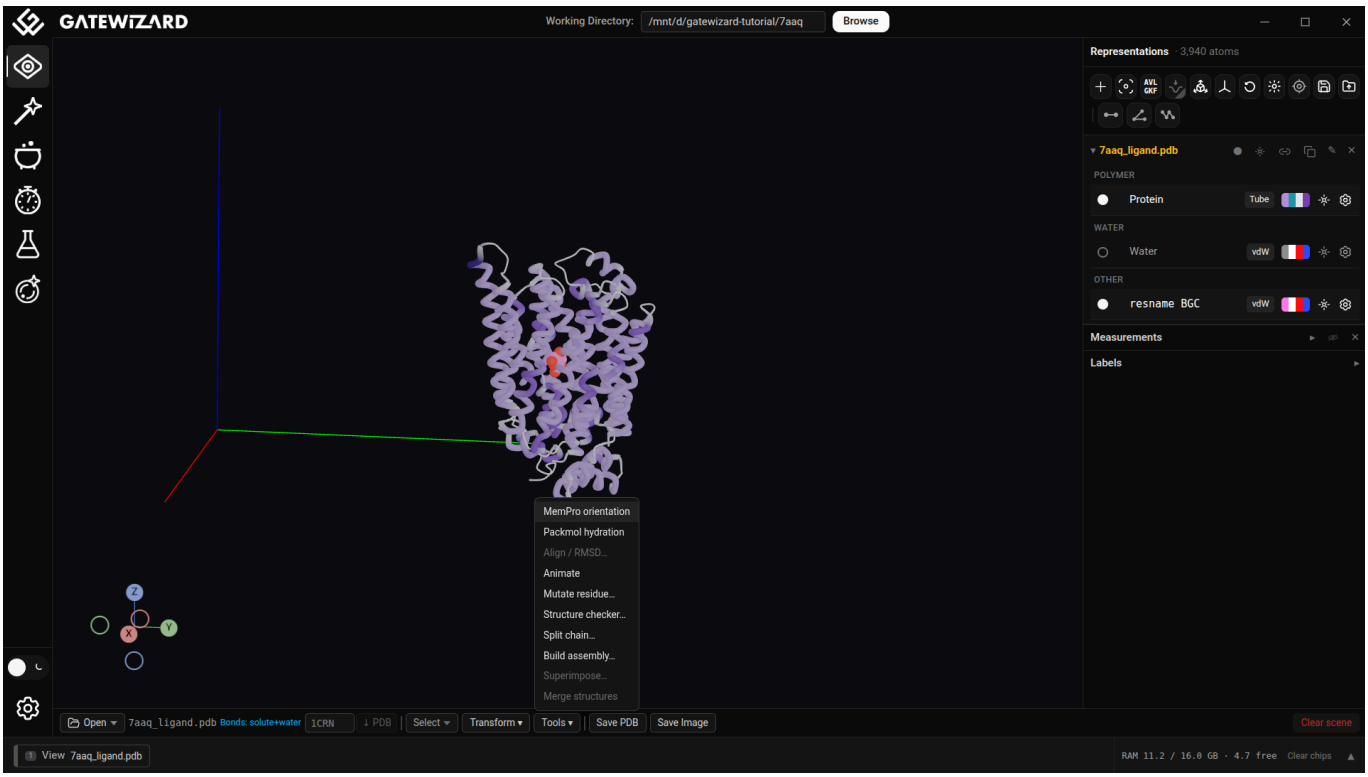
The measurement tools report distances and angles between selected atoms. Mutate Residue, Split Chain, and the animation tools are on this page when you need them. A rendered image uses the representation already on screen.



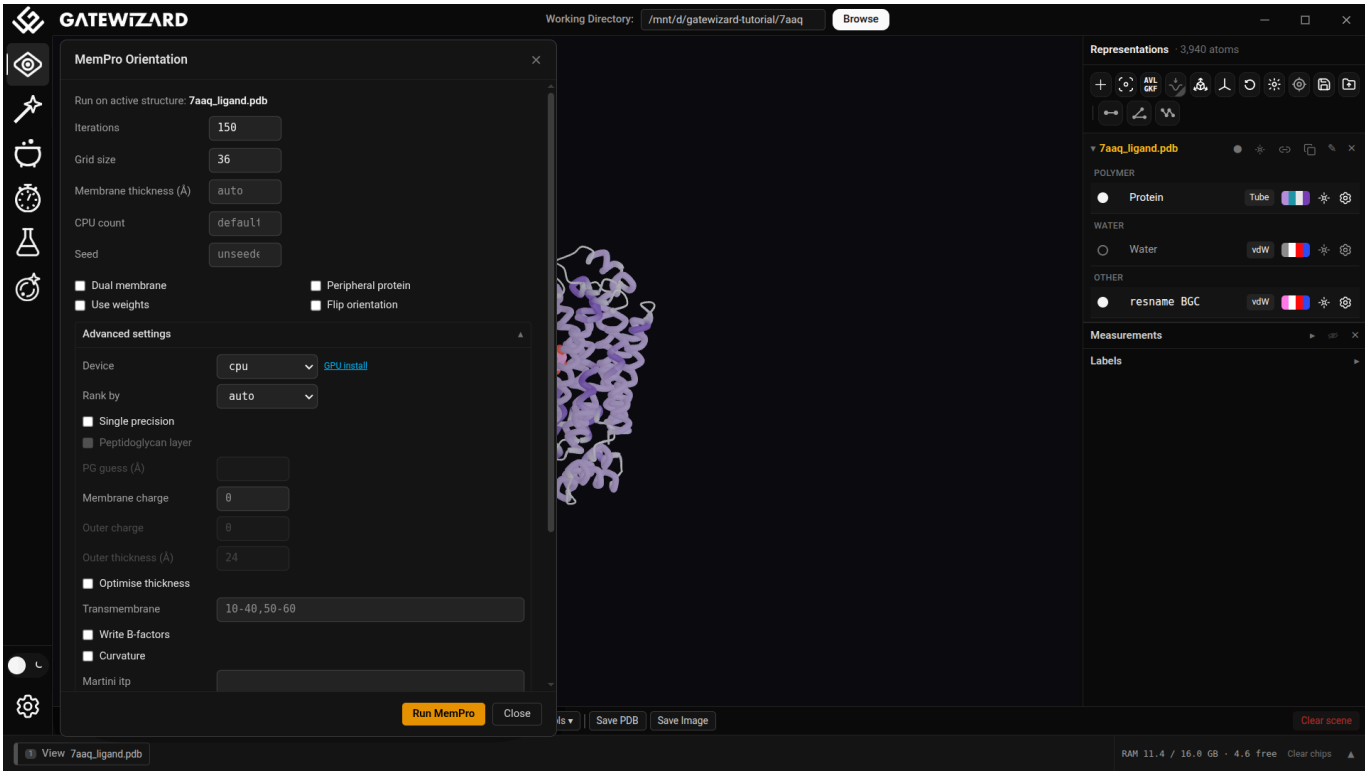

The helix bundle is still tilted. For a membrane protein those helices should run along the membrane normal, which here is Z. On the right-hand toolbar, click **Show axes lines**. The small axes in the corner and the long lines through the protein make the tilt easy to see. Packmol-memgen can estimate an orientation when it builds the bilayer. This walkthrough sets the pose with MemPro first, so you can look at it and keep the one you want before the membrane is added.



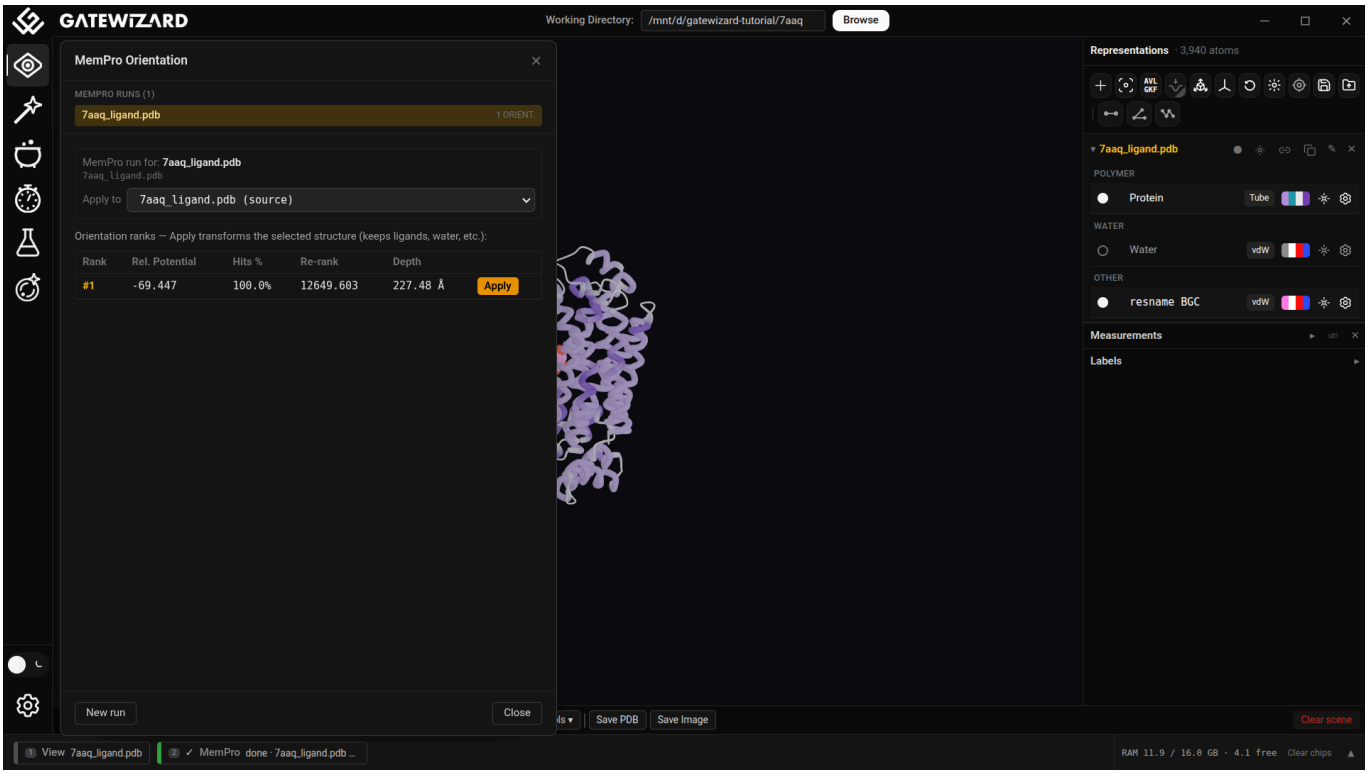

Open **Tools → MemPro orientation**.



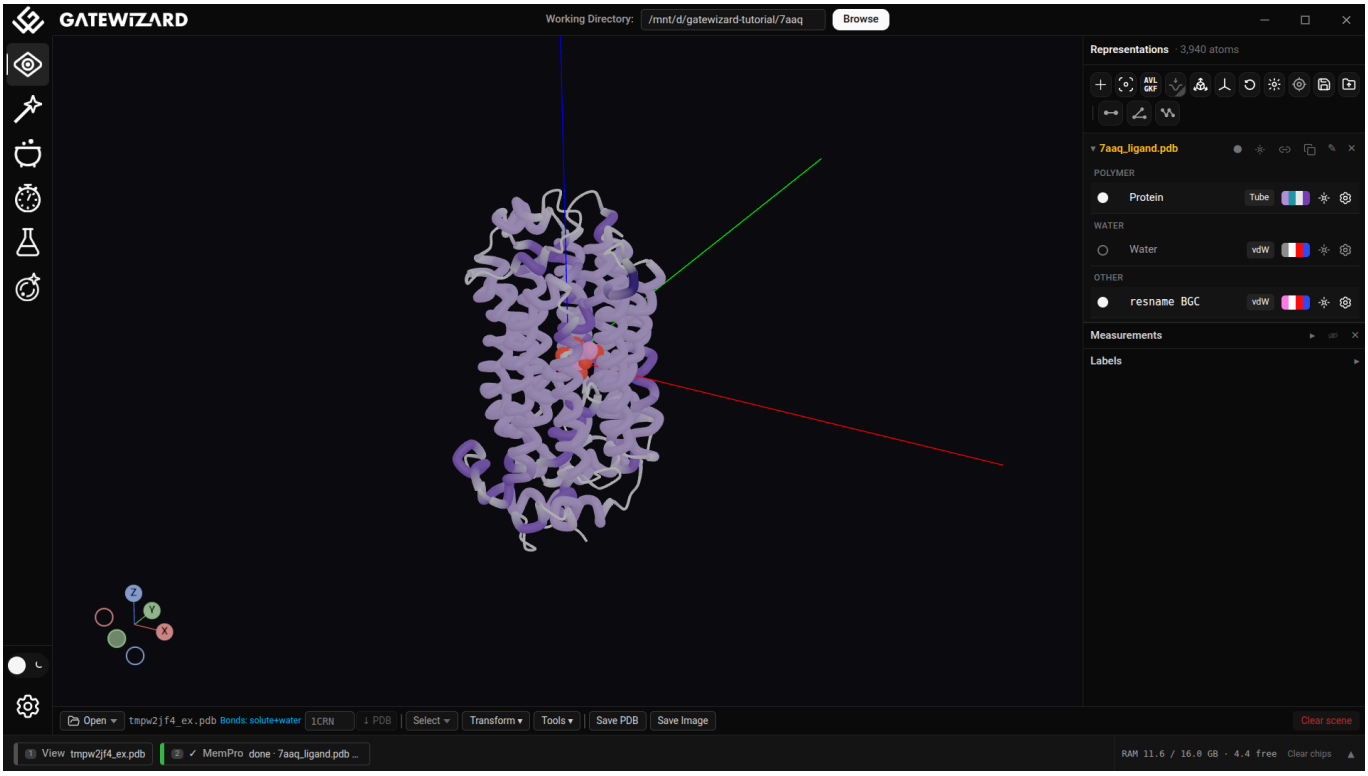

The dialog runs on `7aaq_ligand.pdb`. Leave the starting values: 150 iterations, grid size 36, membrane thickness **auto**, CPU count at the MemPro default, and Seed empty. Click **Run MemPro**.



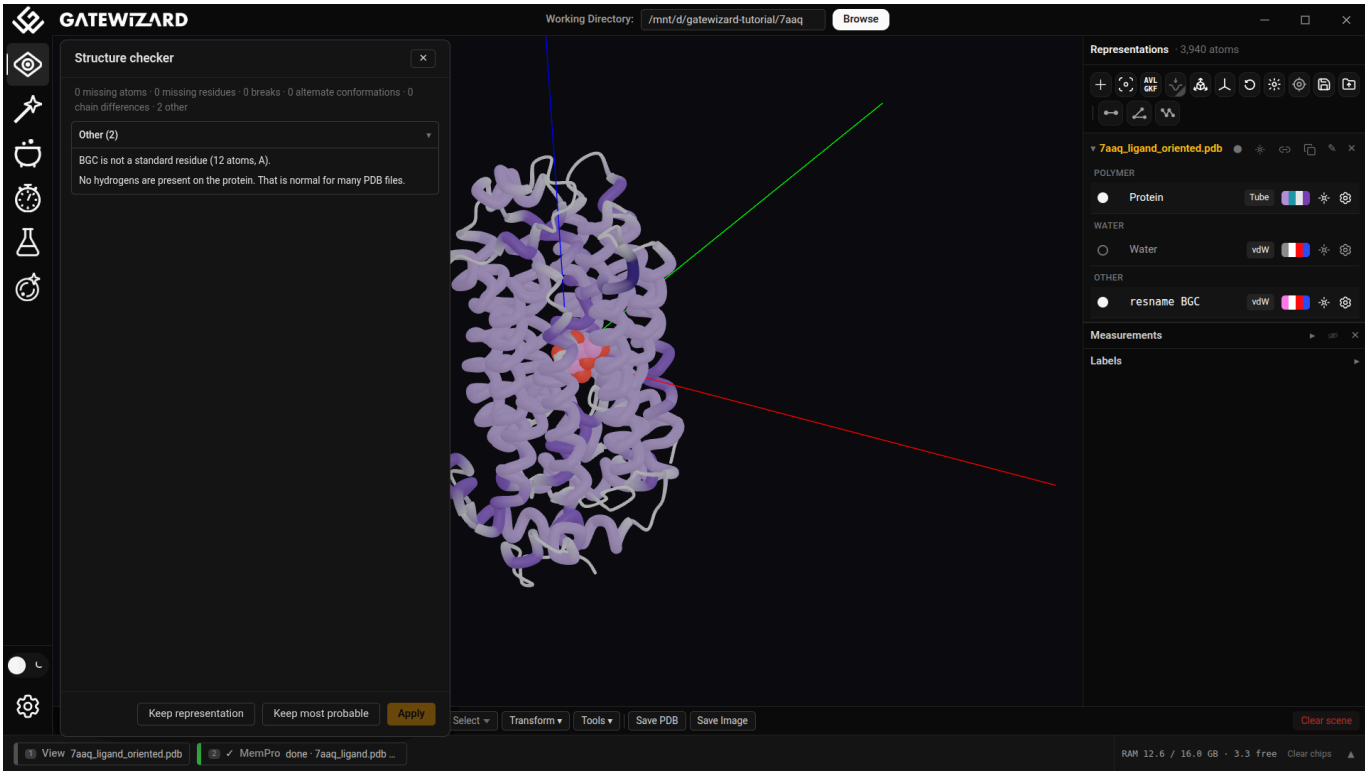

When the run finishes, rank 1 is the pose to keep. **Apply to** stays on `7aaq_ligand.pdb` (source), so BGC moves with the protein. Click **Apply**.



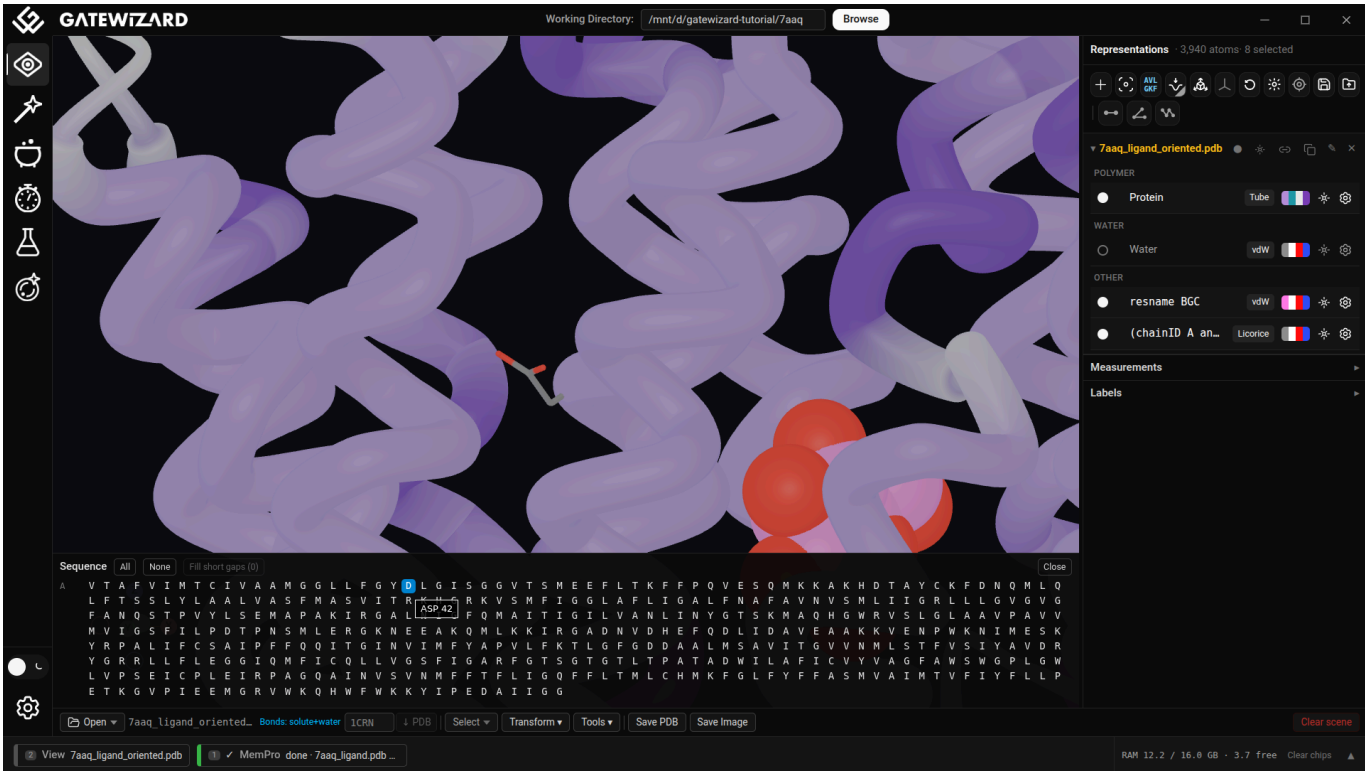

The bundle now lies along Z. Save the structure as `7aaq_ligand_oriented.pdb`.




Open the structure checker on that file. The summary is 0 missing atoms, 0 missing residues, 0 breaks, and 0 alternate conformations. The two Other lines are expected here: BGC is the bound sugar, and the protein has no hydrogens yet. Preparation adds those hydrogens.

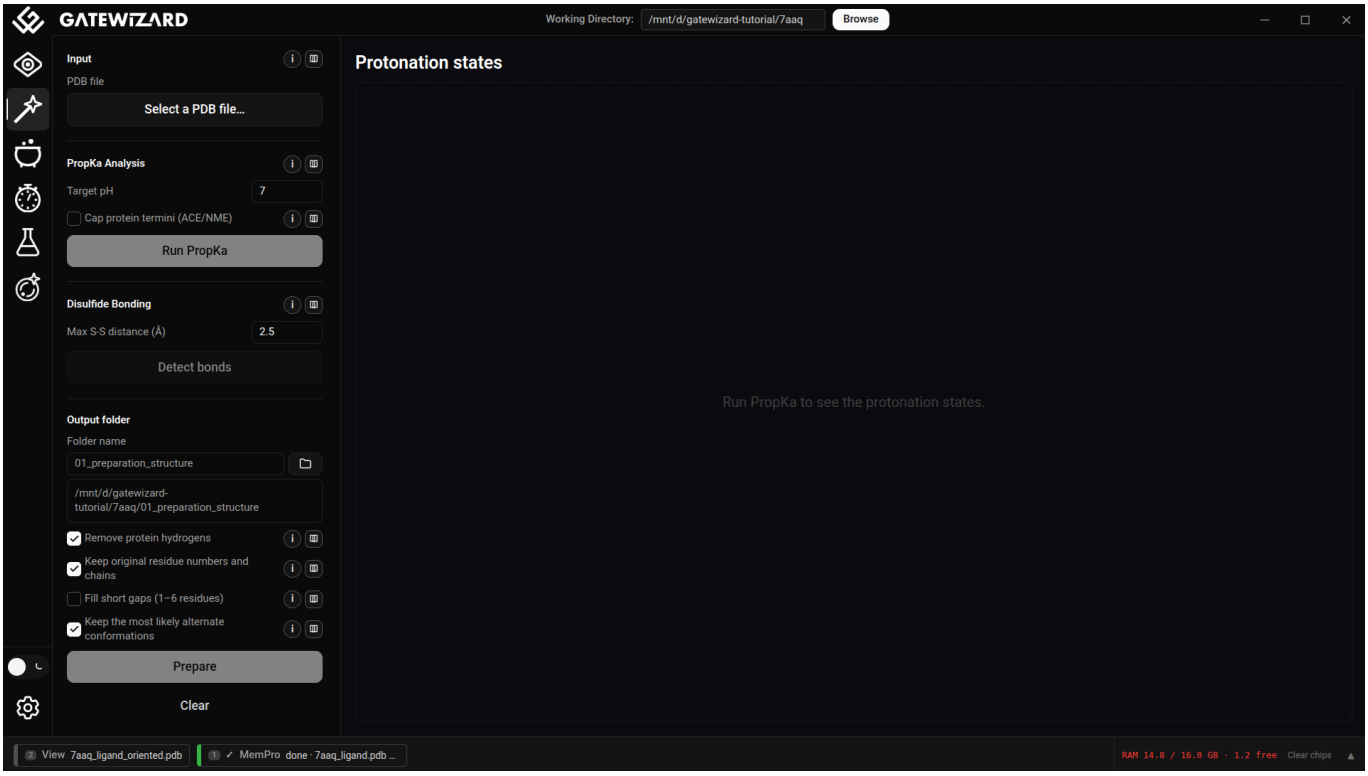



Open the sequence viewer. Chain A is one continuous sequence. ASP42 is a residue that is present. Amber protonation names appear only after Preparation.



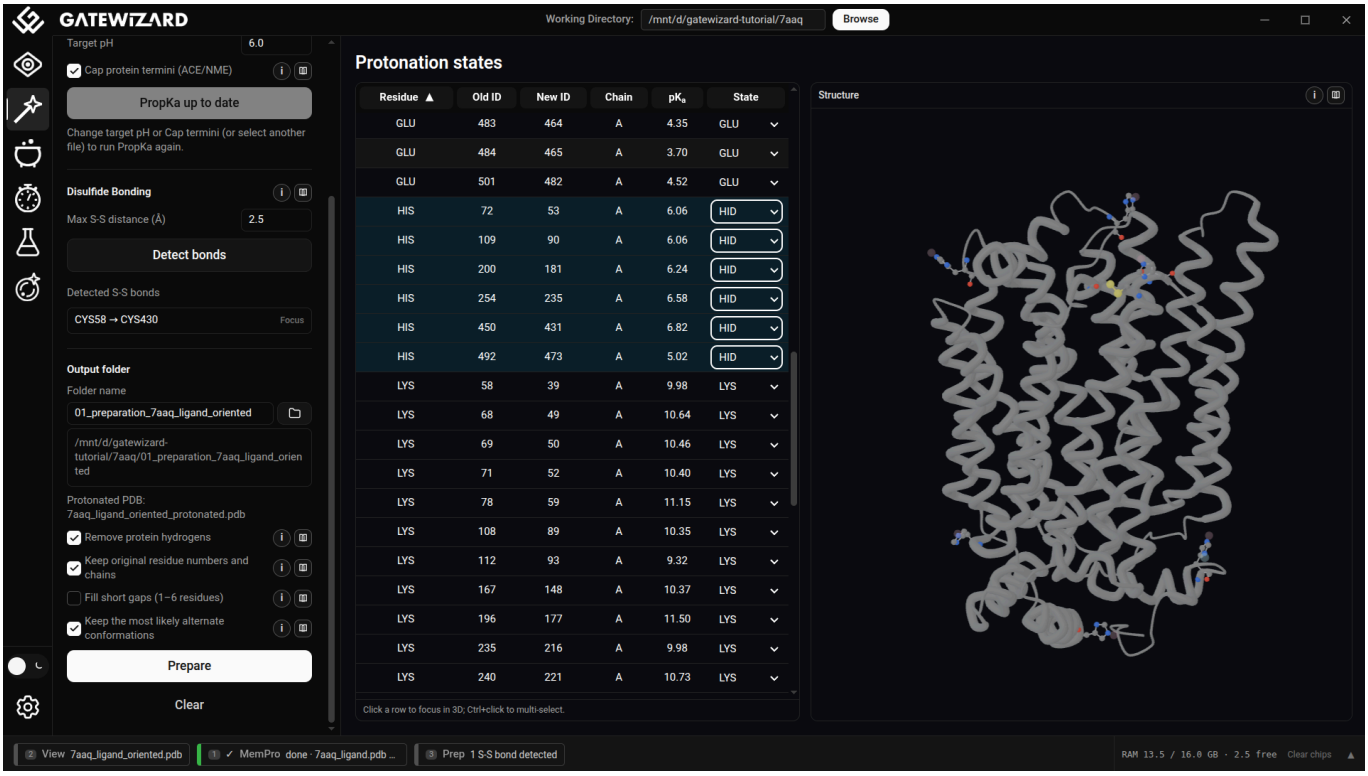

## 2. Preparation

Open Preparation and load `7aaq_ligand_oriented.pdb`.




Set the target pH to 6. Turn on **Cap protein termini (ACE/NME)**. That box starts off. Click **Run PropKa**.

The table lists the ionizable residues at that pH. A histidine is HIP when the pH is below its pKa, and HIE otherwise. For this tutorial, change every histidine listed as HIP to HID. Clicking a row focuses that residue, and a changed state previews the hydrogens Prepare will add or remove. That 3D picture is an estimate. The hydrogen positions can shift a little once Amber rebuilds them.




Leave **Remove protein hydrogens** on. The box starts on. The prepared PDB then keeps the heavy atoms and the residue names, and the protein hydrogens are left out. Ligands such as BGC keep theirs. This is safe for the membrane build because packmol-memgen adds the protein hydrogens again. What has to be kept is the name PropKa assigned, such as HID, ASH, GLH, or CYX. The final system follows those names when it places the hydrogens.

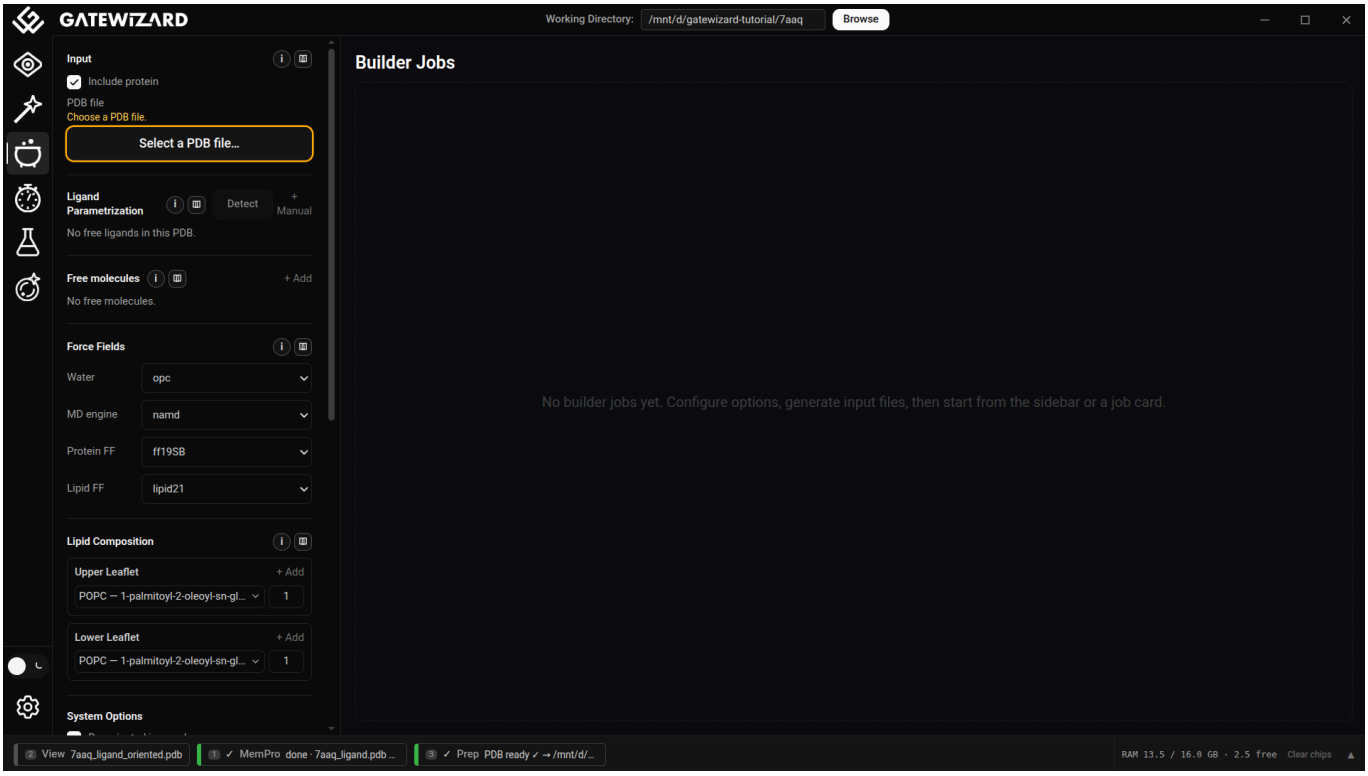
Click **Detect bonds** and confirm the disulfide between CYS58 and CYS430. After PropKa, that pair is drawn on the structure.

Name the output folder, or keep the default `01_preparation_7aaq_ligand_oriented`. Click **Prepare**. The prepared file is named from the input stem, so this save writes `7aaq_ligand_oriented_protonated.pdb` inside that folder.

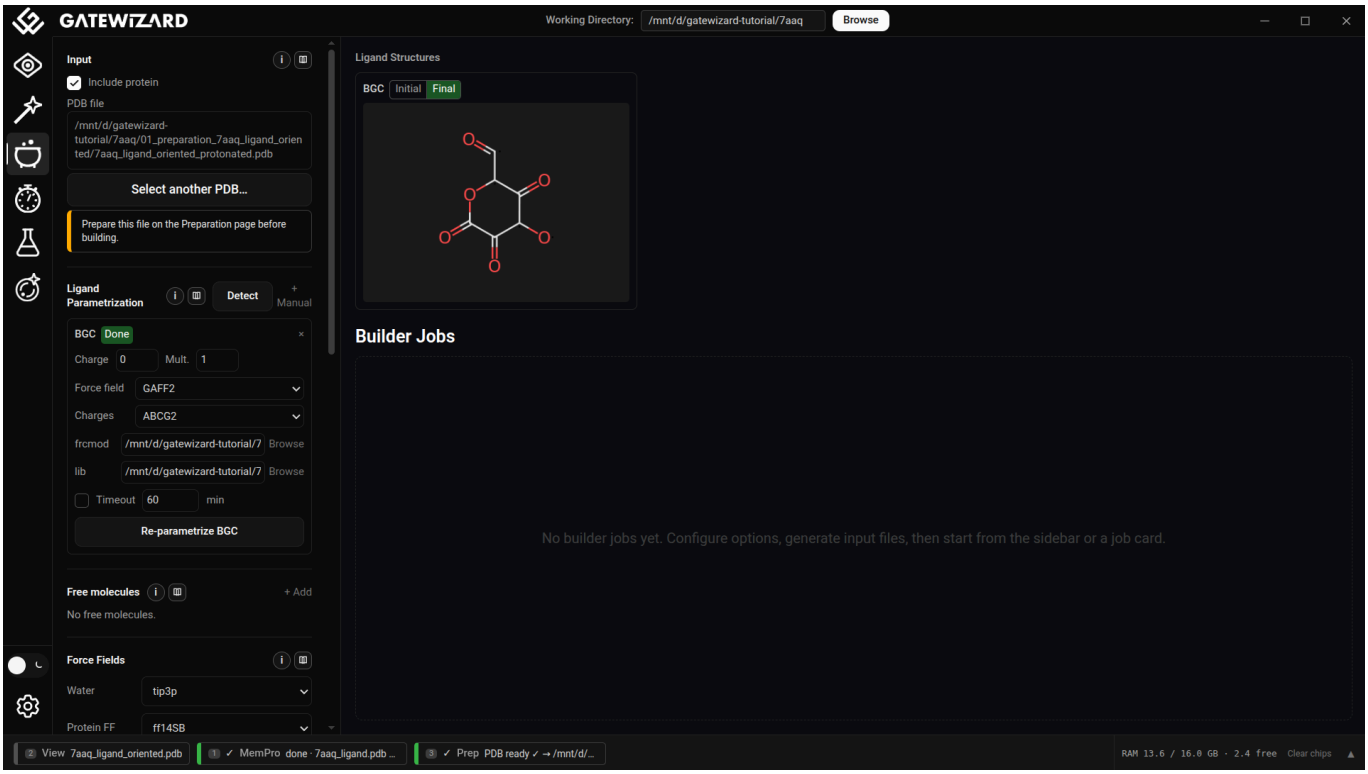
Leave **Keep original residue numbers and chains** on. That is the desktop default. The API study case turns it off and uses sequential Amber numbers.

### 3. Builder

Open Builder. Click **Select a PDB file...** and load `7aaq_ligand_oriented_protonated.pdb`.




Parametrize BGC with the values already on the ligand card: GAFF2, ABCG2, and charge 0. Leave **Timeout** unchecked so antechamber runs until it finishes. The button reads **Parametrize BGC**. After it finishes, the card says **Done**.

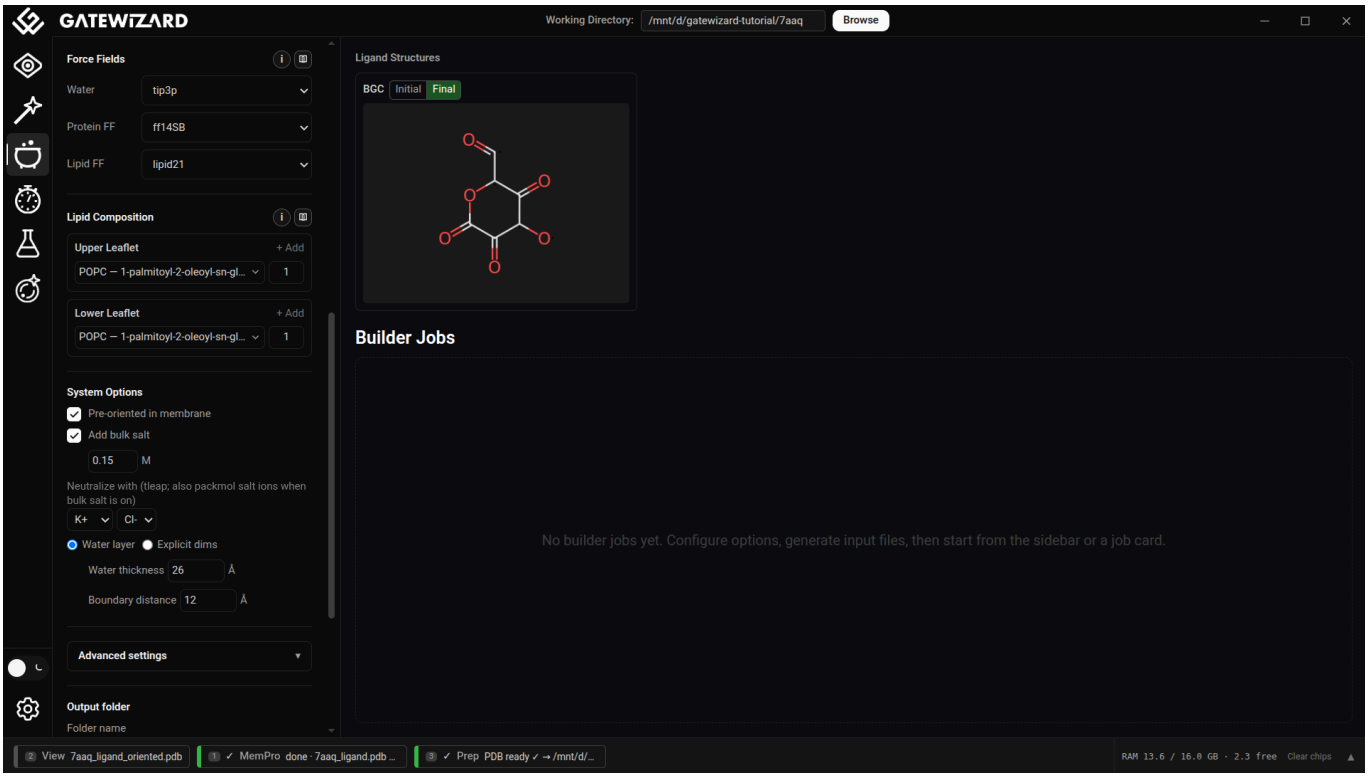



In Force fields, set:

- Water model: TIP3P
- Protein force field: ff14SB
- Lipid force field: Lipid21

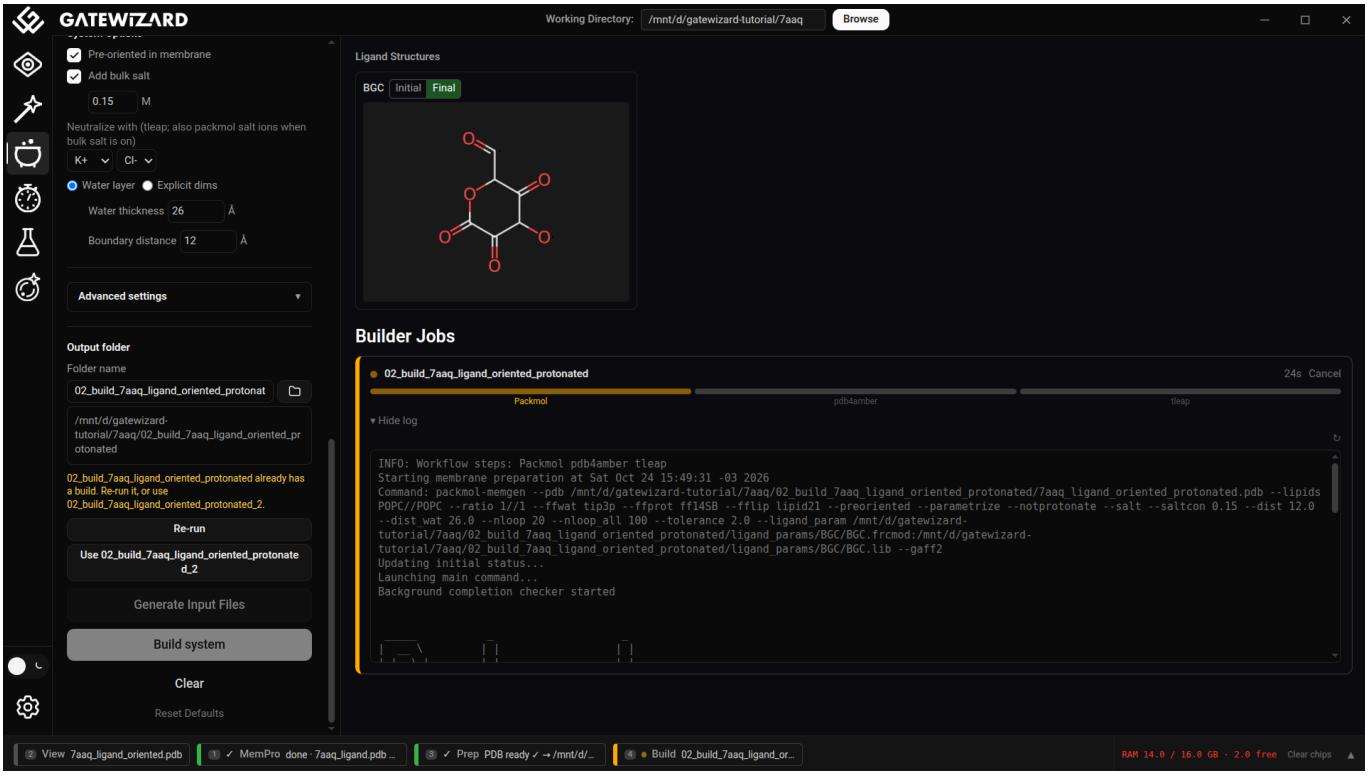
Use POPC in both leaflets. Turn on **Add bulk salt**, set 0.15 M, and keep K+ and Cl−. Leave **Pre-oriented in membrane** on. MemPro already placed the protein, so the build skips MEMEMBED. That box starts on. Water thickness stays 26 Å and the boundary distance stays 12 Å.

Open **Advanced settings** and leave both boxes on. They start that way. **Parametrize with tleap** makes this job write `system.prmtop` and `system.inpcrd` after the membrane is packed, which is what Equilibration loads. **Skip protonation** keeps the names Preparation already chose, such as HID, ASH, GLH, and CYX. Those names are the protonation the final system follows. Packmol-memgen writes the protein hydrogens from the Amber template for each name, which is why leaving **Remove protein hydrogens** on during Preparation is safe.

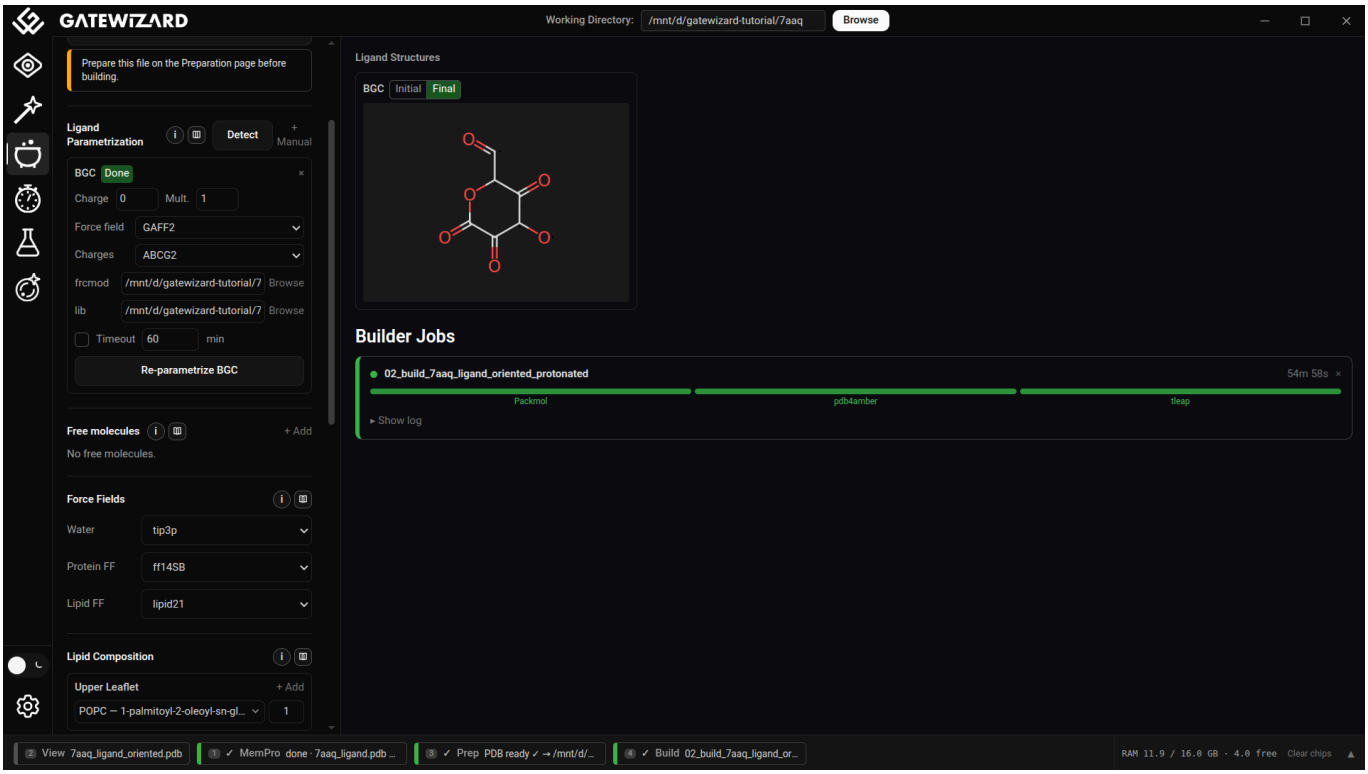


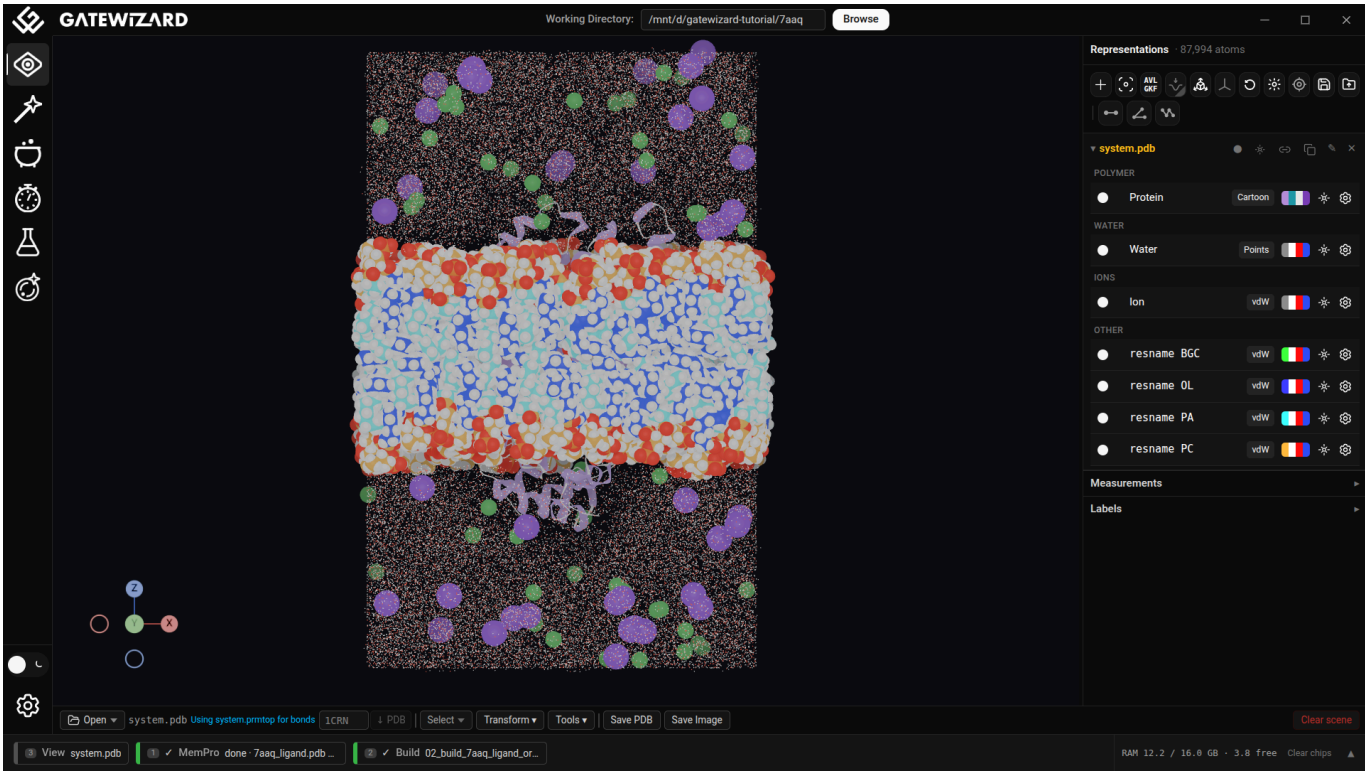

Click **Generate Input Files**, then **Build system**. The card for a job that is already generated still says **Start**. A folder that already contains a build keeps **Generate Input Files** off until you choose **Re-run** or the next number, such as `02_build_7aaq_ligand_oriented_protonated_2`. A new name generates immediately.

The folder is `02_build_7aaq_ligand_oriented_protonated`.



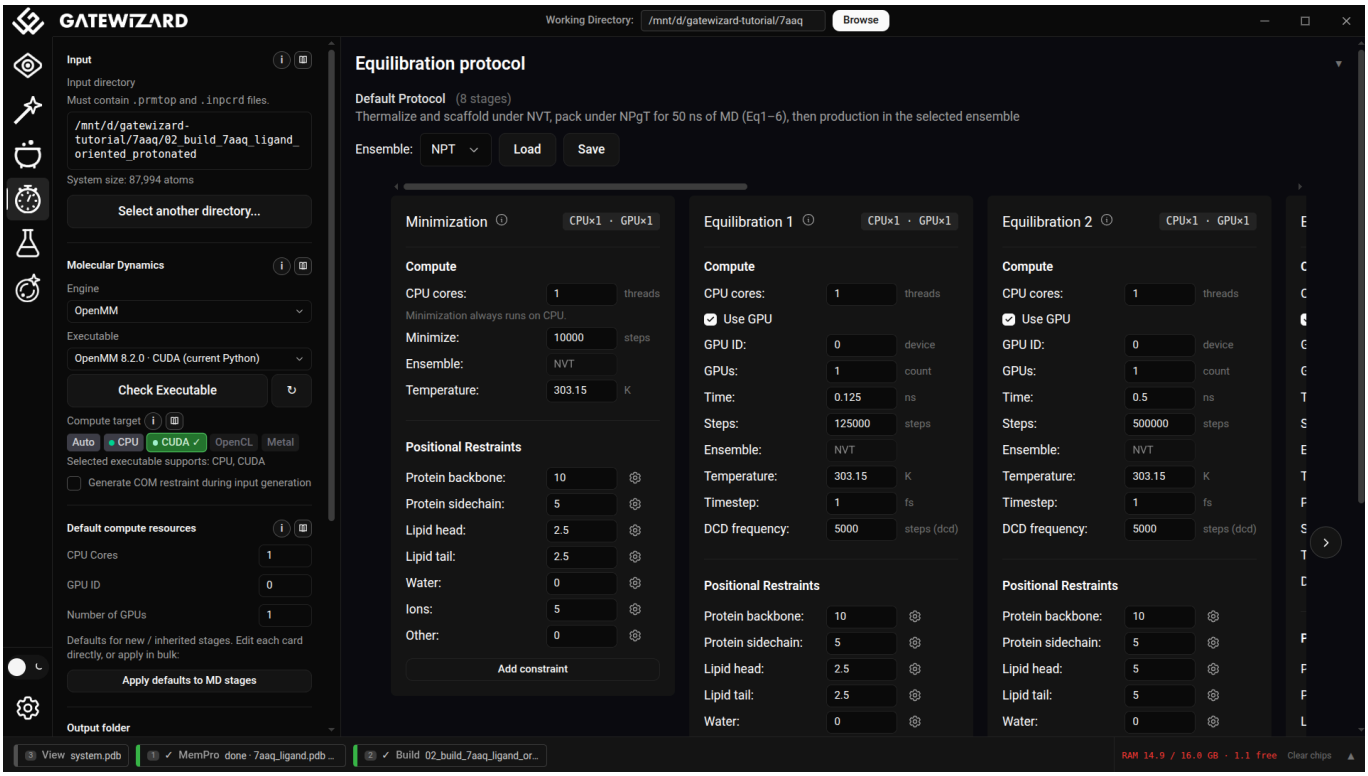

Packmol, pdb4amber, and tleap finish on the same card. A finished job contains `system.pdb`, `system.inpcrd`, and `system.prmtop`. Open `system.pdb` in Visualize.



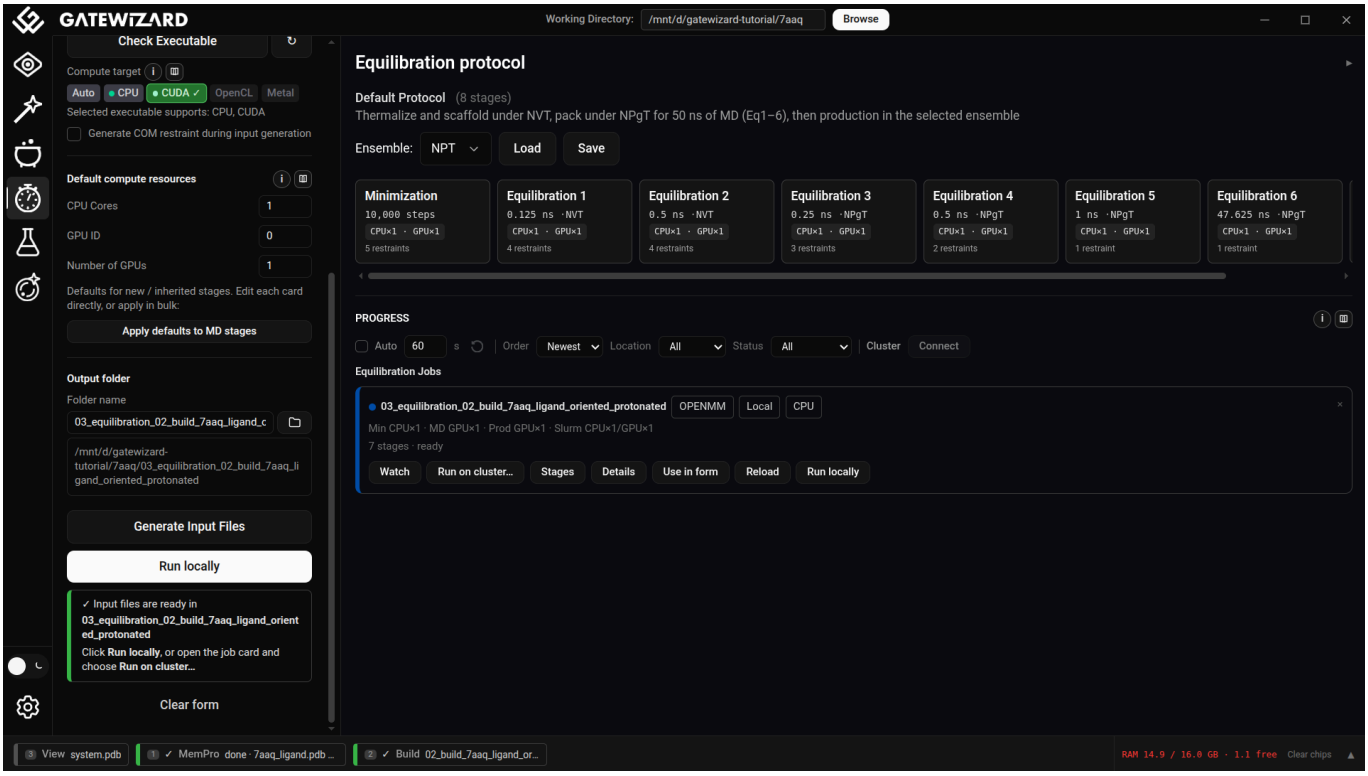



### 4. Equilibration

Open Equilibration and select the build folder. The page reads `system.prmtop` and `system.inpcrd` and shows the atom count.

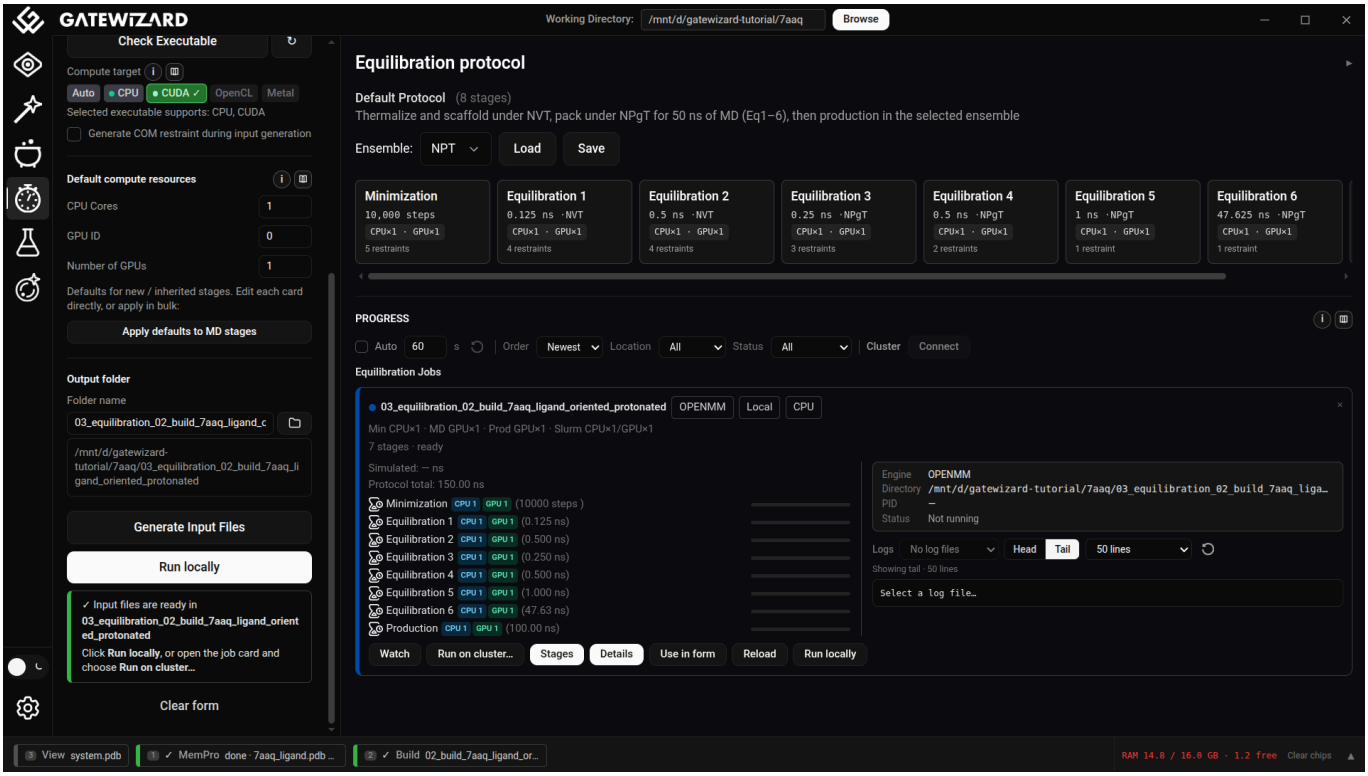



Set the engine to OpenMM. Its executable is Python. Leave the ensemble on **NPT**, where that menu starts, so production follows NPT. The protocol cards are Minimization, Equilibration 1–6, and Production. Minimization is 10,000 steps. Equilibration 1 and 2 stay NVT. Equilibration 3 through 6 pack the membrane under NPgT. The Production card starts at 100 ns. For this tutorial, set it to 200 ns.




Leave the other stage settings as they are. Choosing OpenMM sets the sidebar to 1 CPU and 1 GPU. Under **Compute target**, a small green dot means this local executable can run that platform. Turn on a GPU target that shows the dot, usually **CUDA**, so the molecular-dynamics stages use the GPU. OpenMM writes the minimization steps into `step1_equilibration.inp`. Leave **Generate COM restraint** off, so no center-of-mass or rotation restraint is written.

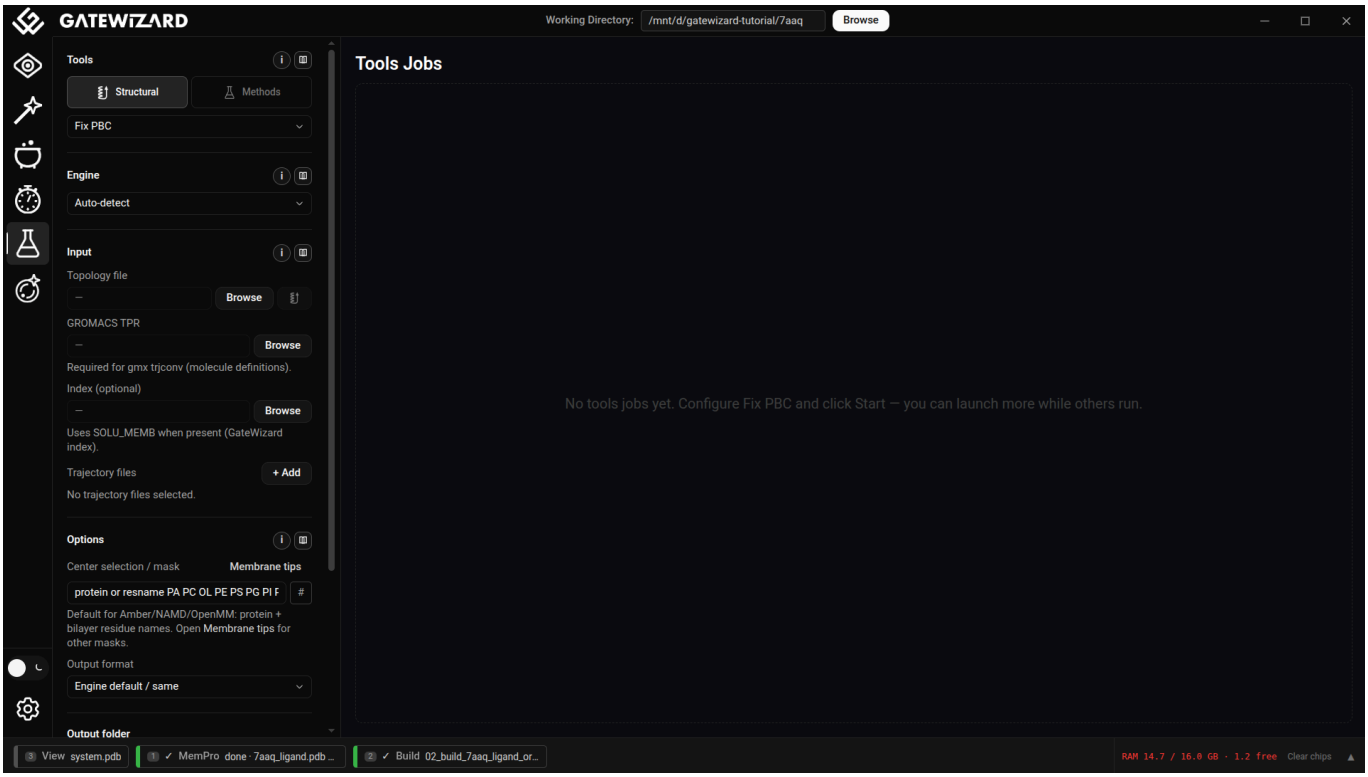
Click **Generate Input Files**. The folder name is `03_equilibration_` plus the build folder name. It contains `step1_equilibration.inp` through `step6_equilibration.inp`, `step7_production.inp`, `run_equilibration.sh`, and `run_equilibration_cluster.sh`.



Run the job on this machine, or open Settings → Clusters and choose **Run on cluster**. The job card shows the log and the status of each stage.

### 5. Tools

Open Tools. Under **Fix PBC**, load the topology and the OpenMM trajectory. Set the periodic-boundary options so the system is rebuilt and centered. OpenMM trajectories use `cpptraj autoimage`.




Choose the output directory and click **Start Fix PBC**. Use that trajectory in Visualize or Analysis.

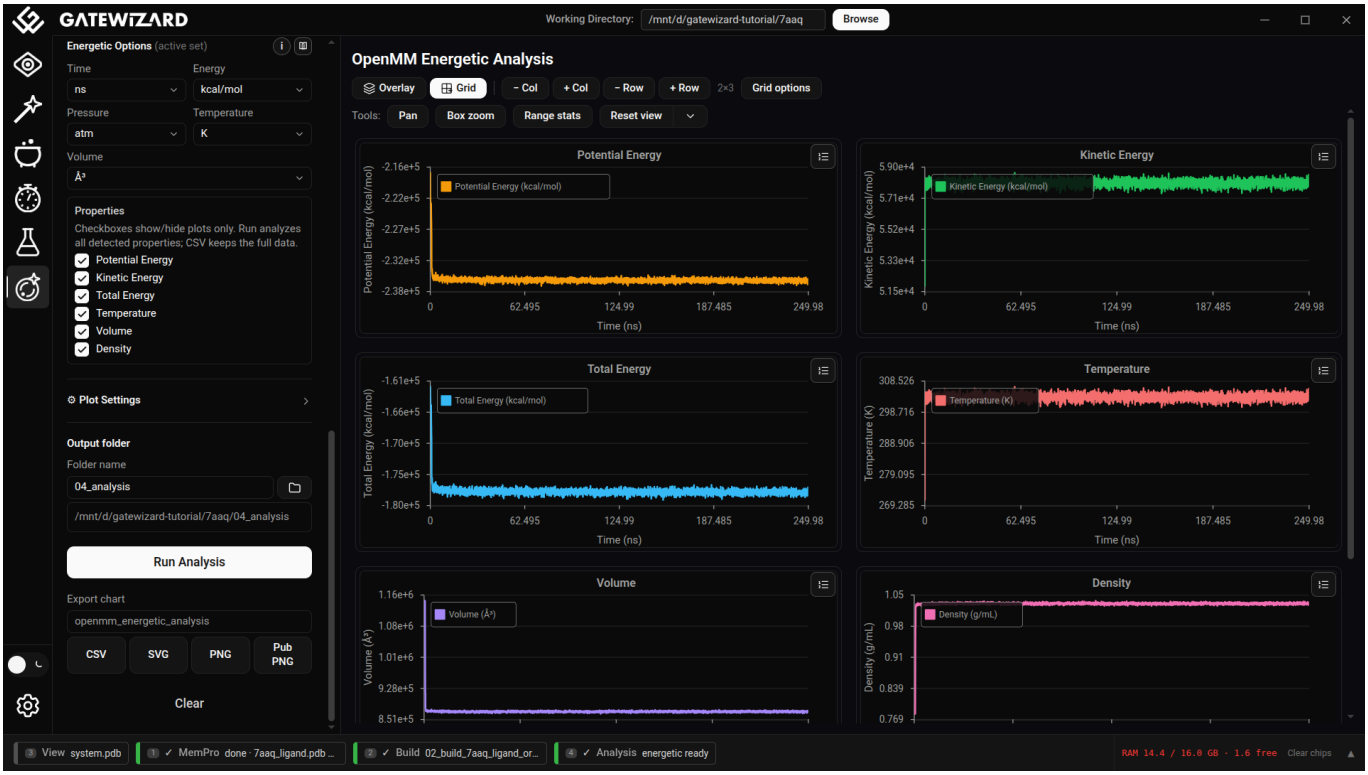
### 6. Analysis

Open Analysis and choose **Structural**. Load the topology and the trajectory files. Each file needs the simulation time for that stage. Click **Protocol times** to fill those boxes from the default protocol, then change any value that does not match the run you did. Set the stride above 1 to keep every Nth frame and finish the analysis sooner.

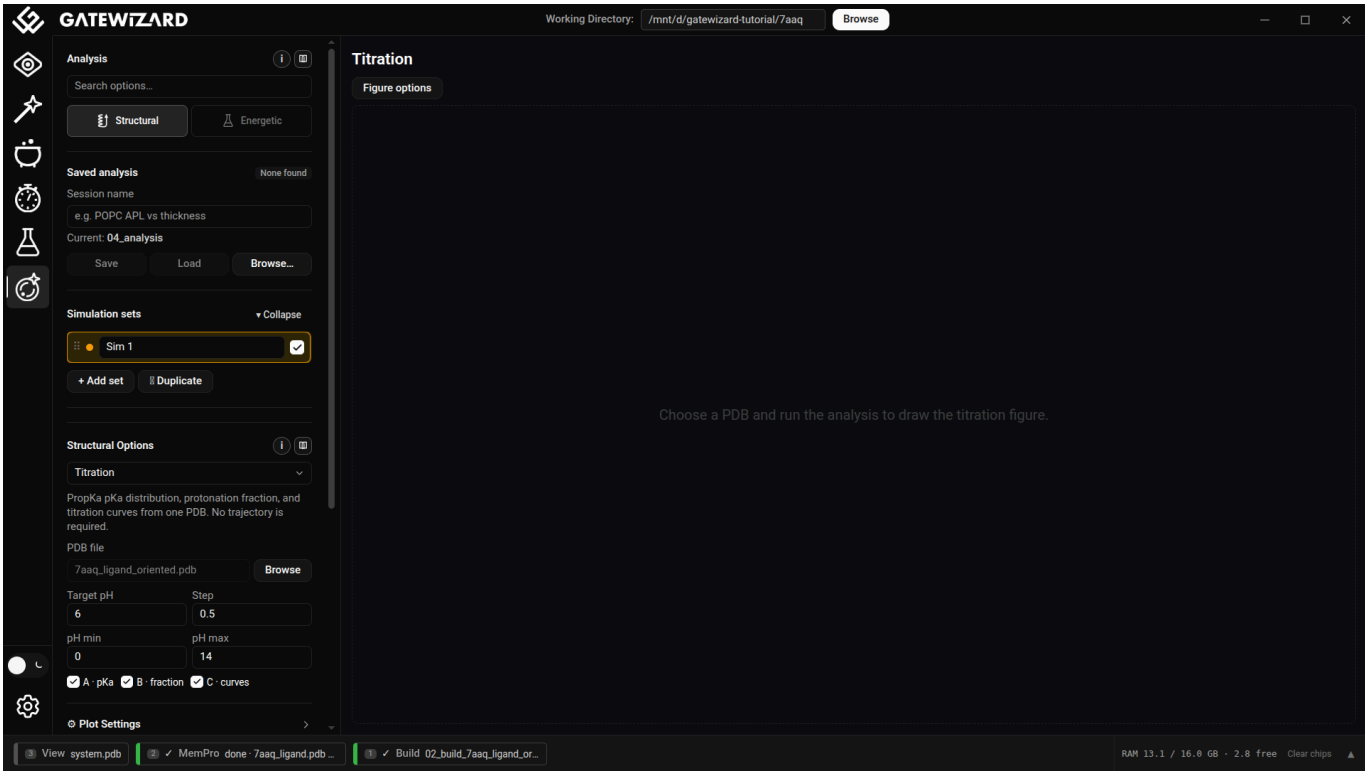
Choose RMSD. Use the selection `protein and name CA` and turn structural alignment on. Use a reference PDB for the comparison. Frame 0 is whichever frame was written first, so two replicas can start from different coordinates. The same reference PDB keeps one structure for every run. Click **Run Analysis**.

The x-axis stops short of 250 ns because the stride keeps every Nth frame, so the last point is the last frame that was kept. The run itself is 250 ns. In the plot settings, set the x-axis limit to 250 ns.

**Energetic**, with the engine set to OpenMM, reads the log files. Temperature, pressure, volume, and total energy are the usual traces. Titles, colors, units, and axis ranges are edited on the plot, and the figure or the data can be exported.




Stay on **Structural** and choose **Titration**. This reads one PDB. Browse `7aaq_ligand_oriented.pdb` and leave the target pH at 6, the same pH used in Preparation.




Click **Run Analysis**. Panel A is the pKa comparison, panel B is the protonated fraction, and panel C is the titration curves. Cysteines that form the disulfide stay out of A and B.

# STP10 API

## STP10 with the API

This walkthrough prepares *Arabidopsis thaliana* STP10 (PDB 7AAQ) with the Python API: MemPro orientation, protonation, a pre-oriented POPC membrane, and OpenMM equilibration inputs for a 100 ns production run. The same choices in the desktop app are STP10 in the desktop app.

The scripts and the cleaned protein are in `tests/study_cases/7aaq/`. Copy that folder to the directory where the outputs should be written. Each script uses the folder it lives in, not the shell's current directory.

| Stage | Script | Main input | Main output |
|---|---|---|---|
| Orientation | `orient_7aaq.py` | `7aaq_protein.pdb` | `7aaq_protein_oriented.pdb` |
| Preparation | `prepare_7aaq.py` | `7aaq_protein_oriented.pdb` | `7aaq_protein_oriented_protonated.pdb` |
| Builder | `build_7aaq.py` | `7aaq_protein_oriented_protonated.pdb` | `system.prmtop`, `system.inpcrd`, `system.pdb` |
| Equilibration | `equilibrate_7aaq.py` | the built system | `step*.inp`, `run_equilibration.sh` |

### Environment

Python, GateWizard, AmberTools, and OpenMM need to be installed first. The installation page covers the `gatewizard` conda environment. OpenMM is the engine that runs the equilibration.

```
conda activate gatewizard
cd /path/to/7aaq
python orient_7aaq.py
python prepare_7aaq.py
```

`7aaq_protein.pdb` is chain A of 7AAQ with the bound sugar BGC kept and the crystallization additives removed. The desktop walkthrough produces the same kind of file and names it `7aaq_ligand.pdb`.

Run `equilibrate_7aaq.py` with the same `python` that can import `openmm`. Compute target **Auto** leaves the platform empty, and OpenMM chooses it when the job starts.

### Orientation

`orient_7aaq.py` runs MemPro on `7aaq_protein.pdb`, then applies rank 1 to the whole file. BGC receives the same rigid move as the protein. The desktop steps are the MemPro figures in the GUI study case. The structure checker and the sequence bar on that page are desktop-only.

| Parameter | Value | Meaning |
|---|---|---|
| Iterations | `150` | MemPro minimisation steps |
| Grid size | `36` | Starting orientations |
| Device | `cpu` | Matches the desktop dialog |
| `SEED` | `7` | Same seed as the prepare-systems study case, so the pose repeats |

The desktop dialog can leave Seed empty. This script sets the seed so a second run keeps the same pose.

```
python orient_7aaq.py
```

The oriented structure is `7aaq_protein_oriented.pdb`. Ranked MemPro files stay in `mempro_7aaq_protein/`.

### Preparation

`prepare_7aaq.py` uses `PreparationManager` to predict pKa values and assign protonation at pH 6.

| Parameter | Value | Meaning |
|---|---|---|
| `INPUT_PDB` | `7aaq_protein_oriented.pdb` | Oriented structure |
| `PH` | `6.0` | Working pH |
| `PRESERVE_RESIDUE_NUMBERS` | `False` | Sequential Amber residue numbers |
| `OUTPUT_DIR` | `01_preparation_7aaq_protein_oriented` | Output directory |

The desktop app leaves **Keep original residue numbers and chains** on. This script turns that off on purpose. Set `PRESERVE_RESIDUE_NUMBERS = True` when the API run should match a desktop folder that kept the original numbers.

```
python prepare_7aaq.py
```

Inside `01_preparation_7aaq_protein_oriented/` the script prints `OK` or `MISSING` for each expected name:

- `7aaq_protein_oriented.pka` — pKa predictions
- `7aaq_protein_oriented_summary_of_prediction.txt` — prediction summary
- `7aaq_protein_oriented_protonated.pdb` — structure for the Builder

Also expect the non-protein PDB, the renumbering map, and the disulfide-link file. Continue only when `7aaq_protein_oriented_protonated.pdb` is present and readable.

PropKa assigns HIP when the pH is below the histidine pKa and HIE otherwise. The desktop walkthrough then sets those HIP rows to HID by hand. This script keeps the automatic assignment.

**Builder**

`build_7aaq.py` packs a POPC bilayer, TIP3P water, and 0.15 M KCl around the prepared protein, then parametrizes with tleap.

| Choice | Value |
|---|---|
| `water_model` | `tip3p` |
| `protein_ff` | `ff14SB` |
| `lipid_ff` | `lipid21` |
| `upper_lipids` / `lower_lipids` | `POPC` / `POPC` |
| `lipid_ratios` | `1//1` |
| `salt_concentration` | `0.15` |
| `cation` / `anion` | `K+` / `Cl-` |
| `dist` | `12` Å between the solute and the boundary |
| `dist_wat` | `26` Å water layer |
| `parametrize` | `True` |
| `notprotonate` | `True` so PropKa is not run again |
| `preoriented` | `True` because MemPro already oriented the protein |

```
python build_7aaq.py
```

The output folder is `02_build_7aaq_protein_oriented_protonated`. A folder that already contains a build is refused. Delete it, or set `OVERWRITE = True` in the script to reuse it. The script prints `OK` or `MISSING` for the files it expects. These four are the ones to check before equilibration:

- `system.prmtop` — Amber parameters and topology
- `system.inpcrd` — Amber coordinates
- `system.pdb` — the packed system
- `bilayer_7aaq_protein_oriented_protonated_lipid.pdb` — bilayer PDB used for the box

`packmol-memgen.log` and `leap.log` are the logs to read when a build fails.

### Equilibration

`equilibrate_7aaq.py` writes OpenMM inputs. It does not run OpenMM. The stage list starts with the same Minimization card as the desktop protocol (10,000 steps). `OpenMMEquilibrationManager.get_default_stage_params` then adds Equilibration 1 and 2 under NVT, Equilibration 3 through 6 under NPgT, and production under `SCHEME`. OpenMM writes the minimization steps into `step1_equilibration.inp`.

| Choice | Value |
|---|---|
| Engine | OpenMM |
| `SCHEME` | `NPT` for production |
| `TEMPERATURE` | `303.15` K |
| `PRODUCTION_NS` | `100.0` |
| Minimization | `10000` steps, written into step 1 |
| `cpu_cores` | `1` |
| `gpu_id` | `0` |
| `num_gpus` | `1` |
| Compute target | `auto` |
| `add_com_restraint` | `False` |
| `add_rotation_restraint` | `False` |

Those resource numbers are the desktop defaults after the engine is set to OpenMM.

```
python equilibrate_7aaq.py
```

The output folder is `03_equilibration_02_build_7aaq_protein_oriented_protonated`. The script checks `run_equilibration.sh`, `run_equilibration_cluster.sh`, `openmm_run.py`, `step1_equilibration.inp` through `step6_equilibration.inp`, `step7_production.inp`, and the copied system files.

### Run OpenMM

From the equilibration folder:

```
cd 03_equilibration_02_build_7aaq_protein_oriented_protonated
bash run_equilibration.sh
```

The script uses the `python` on `PATH` unless `PYTHON` is set. With compute target Auto, OpenMM picks the platform at startup. One CPU and GPU 0 are the counts written into the script.

After the run, read the log and confirm the coordinate, trajectory, and restart files exist. Temperature, energy, pressure, and RMSD are the usual checks that the system stayed stable. Those analyses are the last two sections of the desktop walkthrough.

## Prepare systems

This walkthrough times a membrane build for several proteins. One script prepares each system in POPC and writes how long each stage took. The membrane build itself is repeated three times. The entries below are the systems currently listed in `SYSTEMS`. Edit that list to use other PDB ids, chains, or replica counts.

Download or open the script from GitHub: `tests/study_cases/prepare_systems.py`. Run it from the directory where `work/` and `results/` should be written. AmberTools, MemPrO, and a GateWizard environment that can import `gatewizard` need to be available there.

### What the script builds

Every system uses the same membrane settings:

| Choice | Value |
|---|---|
| Leaflets | POPC / POPC |
| Protein | ff14SB |
| Lipids | Lipid21 |
| Water | TIP3P |
| Salt | 0.15 M |
| Ions added by the builder | K+ and Cl− |
| Protonation pH | 7.4 |
| Builder replicas | 3 |

Packmol-memgen on GateWizard was set to use a solute padding `dist` 12 Å, and water layer `dist_wat` 26 Å. Those are the builder defaults. Change them in the `prepare_system` call inside `build_once` if a study needs a different box.

The builder is the stage that is repeated. Download, chain selection, MemPrO, PropKa, pdb4amber, and GAFF run once per PDB, and every builder replica uses that same prepared PDB. Prepare one orientation, then pack the replicas from it. A second MemPrO run can shift the protein slightly and change the number of atoms. The same coordinates give packmol-memgen a reproducible system, so the replicas stay the same size and are easier to compare. The mean and the standard deviation are the packing plus tleap time.

```
BUILDER_REPLICAS = 3
PH = 7.4
```

A stopped run keeps finished stages. `python prepare_systems.py --fresh` times them again. `python prepare_systems.py 2MVH` runs one PDB and leaves the other rows in `results/timings.csv` and `results/summary.md`. Running that PDB again replaces only its rows.

### Decisions that are already written into the script

Each PDB is one entry in `SYSTEMS`. `chains` is the author chain list that is kept. `keep_hetero` is the residue names that stay with the protein. A name left off that list is left out of that example. Waters, bound lipids, and detergents belong on the list when they are part of the question. The builder still adds bulk TIP3P and POPC around what you keep.

Experimental water stays when its residue name is listed in `keep_hetero`. In most PDB files that name is `HOH`. Water is not a ligand, so it does not go in `parametrize`. The water must also sit on a chain in `chains`. A blank chain column is dropped. Those waters then move with the protein during orientation, and the builder adds the rest of the solvent around them. Whether to keep them depends on the system: a water in a channel or a binding site is often part of the biology, and a loosely ordered crystal water may not be.

```
{
    "pdb_id": "6RV2",
    "model": 1,
    "chains": "AB",
    "keep_hetero": ["K", "HOH"],
    "parametrize": {},
    "peripheral": False,
},
```

The systems on this page other than 3T4D do not list `HOH`, so those timing runs use TIP3P as the only solvent. 3T4D lists `HOH` because the crystal waters are part of that example. Add `"HOH"` when the crystallographic waters should stay. `WAT` is the other name this script recognizes.

| PDB | What it is | What this example keeps | Recommendation for a similar system |
|---|---|---|---|
| 2MVH | SpoVM, NMR, 26 residues, 10 models, chain A | Model 1, chain A, no hetero groups. MemPrO runs in peripheral mode. | For an amphipathic peptide, keep one NMR model and turn peripheral mode on. Transmembrane mode looks for a spanning axis that this kind of peptide does not have. |
| 1AFO | Glycophorin A transmembrane dimer, NMR, 20 models, chains A and B | Model 1 and both chains. | For an NMR dimer, keep one model and both chains of the biological assembly. One chain is a monomer, which is a different system. Leave peripheral mode off when the helices cross the bilayer. |
| 1M0L | Bacteriorhodopsin, 1.47 Å, chain A, retinal | Chain A and RET (GAFF2 with ABCG2, charge +1). Crystal lipids LI1 and SQU are left out of this example. | A deposited monomer can be built on its own when the biological assembly is a symmetry trimer. A retinal linked to a lysine as a protonated Schiff base needs charge +1. Keep a crystal lipid when it is biologically relevant, and give it a charge. Leave it out when POPC is standing in for the bilayer. |
| 7AAQ | STP10, 1.81 Å, chain A | Chain A and the bound sugar BGC (GAFF2 with ABCG2, charge 0). Acetate, the PEG detergent XPE, and monoolein OLC are left out of this example. | Keep a bound ligand and give antechamber its net charge. Acetate, the PEG, and the monoolein are crystallization additives in this deposit. Keep a detergent or a lipid when it is biologically relevant. |
| 3T4D | NaK2K channel Y55F, 1.70 Å, one monomer in the asymmetric unit | Biological assembly 2, the tetramer of the more complete chain. Crystal waters and the pore potassium stay. | Build the biological assembly when the deposit is only the asymmetric unit. Keep crystal water that belongs in the pore. The assembly step drops symmetry copies that land on the same point. |
| 6RV2 | TASK-1, 3.0 Å, four copies of the subunit | Chains A and B, plus the potassium ions named K. Crystal PC, decyl maltoside, and cholesterol hemisuccinate are left out of this example. | When the crystal holds two copies of a dimer, orient one copy. Chains A and B are one dimer; C and D are the other. Keep ions that belong in the protein, such as selectivity-filter potassium. Leave a detergent or a crystal lipid out only when it is not biologically relevant. |
| 3WU2 | Photosystem II, 1.9 Å, 38 protein chains | Both halves of the dimer, chains `AaBbCcDdEeFfHhIiJjKkLlMmOoTtUuVvXxYyZz`, plus CA, CL, MG, and FE2. Chlorophyll, heme, the metal cluster, and the thylakoid lipids are left out of this example. | Those cofactors and lipids are part of the complex. This example leaves them out so the protein-and-ions build is shorter. Include them when the question needs them and the parameter files exist. Match deposited ion atom names to the Amber templates before tleap. |

2MVH is the only peripheral protein:

```
results = MemPrO().run(
    str(protein_pdb),
    output_dir=str(mempro_dir),
    peripheral=bool(spec.get("peripheral")),
    seed=MEMPRO_SEED,
)
```

`MEMPRO_SEED` is `7`. MemPrO 0.3.1 writes the same orientation again for the same protein, seed, and CPU count.

MemPrO is run on protein atoms. The same rigid transform is then applied to the kept hetero groups, so a ligand moves with the protein.

### Atom counts and times

Every system is a symmetric POPC bilayer. The rows below, and the `SYSTEMS` list, follow the packed system size from smallest to largest. The protein atom count is `prepared.pdb` after protonation. The total atom count is every atom in `system.pdb` after packmol-memgen and tleap: protein, lipids, water, and ions. Those counts were measured before ACE and NME caps. A new run adds the caps unless the PDB already has them, so both counts will be larger. Builder times are packing plus parametrization. MemPrO, protonation, and GAFF run once and are not inside the builder mean.

| PDB | Prepared protein atoms | Total atoms | Builder mean | Note |
|---|---|---|---|---|
| 2MVH | 447 | 38,242 in each replica | 20.0 min (std 8 s) | Short peptide. A practical first system. |
| 1AFO | 1,291 | 45,542 in each replica | 20.0 min (std 5 s) | NMR dimer. One model, both chains. |
| 1M0L | 3,568 | 58,954 in each replica | 23.6 min (std 13 s) | Linked retinal uses charge +1. Long termini stay unbuilt. |
| 7AAQ | 7,580 | 84,695, 84,683, 84,689 | 31.6 min (std 9 s) | Bound sugar stays. Crystallization additives do not. |
| 3T4D | 6,143 | 88,914 in each replica | 30.9 min (std 9 s) | Assembly 2. Four potassium ions stay. This run's crystal waters were removed by pdb4amber `--dry` before packing. |
| 6RV2 | 8,266 | 129,701, 129,698, 129,707 | 43.6 min (std 2.0 min) | One dimer, chains A and B. |
| 3WU2 | 80,898 | 904,059, 904,068, 904,086 | 5.04 h (std 1.6 min) | Protein and monoatomic ions. All three replicas are in the mean. |

Prepare one orientation and pack every replica from it. A second MemPrO run can move the protein enough to change the atom count. The same prepared protein gives packmol-memgen matching systems that are easier to compare.

Orientation was the slow step before packing only for Photosystem II (18.5 min). The other proteins oriented in 83–116 s. GAFF ran for two ligands: retinal took 25 s and the bound sugar took 1.7 s. 3T4D has no organic ligand. Its assembly step took 0.09 s, and MemPrO took 116 s.

#### 2MVH — SpoVM

SpoVM is an amphipathic peptide, not a transmembrane helix bundle. The deposited NMR entry has 10 models and one chain. This example keeps model 1 and chain A, the RCSB representative, and drops every hetero group.

MemPrO needs peripheral mode for an amphipathic peptide. Transmembrane mode looks for a membrane-spanning axis that this peptide does not have.

```
{
    "pdb_id": "2MVH",
    "model": 1,
    "chains": "A",
    "keep_hetero": [],
    "parametrize": {},
    "peripheral": True,
}
```

The structure checker reports nothing before or after preparation: no missing heavy atoms, no gaps, no alternate conformations. Protonation adds one atom (446 coordinates become 447). Each built system has 38,242 atoms. The OpenMM protocol dict in the script is commented out. Uncomment it to run minimization, Equilibration 1–6, and 1 ns of production on a box of this size.

**1AFO — glycophorin A**

This is an NMR transmembrane dimer, 20 models, chains A and B. Sending every model to MemPrO stacks them into one solute. This example keeps model 1 and both chains.

```
{
    "pdb_id": "1AFO",
    "model": 1,
    "chains": "AB",
    "keep_hetero": [],
    "parametrize": {},
    "peripheral": False,
}
```

Leave `peripheral` off for a helix pair that spans the bilayer. One chain builds a monomer, which is a different system.

Before preparation the checker is empty. After preparation, chains A and B each have 40 residues and match at 100% identity. Hydrogens are present on 76 of 80 residues, which is the Amber protonation, not a missing-atom error. The prepared protein has 1,291 atoms (the subset had 1,322, because the NMR file already contained hydrogens that preparation replaces). Each built system has 45,542 atoms.

**1M0L — bacteriorhodopsin**

The deposited asymmetric unit is one chain at 1.47 Å, with retinal and two crystal lipids. The biological assembly is a symmetry trimer. This example builds the monomer. `LI1` and `SQU` are left out here because POPC stands in for the bilayer. Keep either lipid when it is biologically relevant, add its name to `keep_hetero`, and set its net charge in `parametrize`.

Retinal stays. The deposited formula looks like neutral retinal, but the oxygen is absent and a `LINK` record joins C15 to Lys216. After hydrogens are added the residue is C20H27. Charge 0 then has an odd electron count and antechamber refuses a singlet. Charge +1 is the protonated Schiff base. The lysine stays `LYS`, and the `LINK` is not rebuilt as a covalent bond.

```
{
    "pdb_id": "1M0L",
    "model": 1,
    "chains": "A",
    "keep_hetero": ["RET"],
    "parametrize": {"RET": 1},
    "peripheral": False,
}
```

The subset has 1,756 protein atoms and 20 retinal atoms. Preparation writes 3,568 protein atoms and 47 retinal atoms. GAFF for that retinal takes 25 s and is reused by all three replicas. Each built system has 58,954 atoms.

Before preparation, SEQRES lists 17 residues at one end and 18 at the other that have no coordinates. Both stretches are longer than 6 residues, so the short-gap fill leaves them open. The report after preparation does not repeat SEQRES, which is why that count becomes zero. Retinal remains listed as a non-standard residue. That is expected.

### 7AAQ — STP10

STP10 is a sugar/proton symporter at 1.81 Å, one protein chain. The bound sugar `BGC` stays at charge 0. Acetate (`ACT`), the PEG detergent `XPE`, and monoolein (`OLC`) are left out of this example. In this deposit they come from crystallization, and POPC is the membrane that gets packed. Keep a detergent or a lipid that is biologically relevant, and parametrize it the same way as the sugar.

```
{
    "pdb_id": "7AAQ",
    "model": 1,
    "chains": "A",
    "keep_hetero": ["BGC"],
    "parametrize": {"BGC": 0},
    "peripheral": False,
}
```

The subset has 3,771 protein atoms and 12 sugar atoms. Preparation writes 7,580 protein atoms and 24 sugar atoms. GAFF for the sugar takes 1.7 s. The three built systems have 84,695, 84,683, and 84,689 atoms.

SEQRES has 20 names without coordinates at one end and 7 at the other. Seven is already past the short-gap limit, so neither stretch is built. After preparation the checker only notes that `BGC` is non-standard and that hydrogens are present on 481 of 487 residues.

### 3T4D — NaK2K channel

The deposit is the asymmetric unit of the Y55F mutant, one copy of the subunit in chain A and one in chain B. The channel is a tetramer. Assembly 1 expands chain A. Assembly 2 expands chain B. Chain B has 96 of the 97 SEQRES residues (the N-terminal methionine has no coordinates). Chain A has 93. This example builds assembly 2, the more complete tetramer.

The build is the same one as Visualize → Build assembly, with crystal waters included and extra copies that land on the same point removed. The cutoff is 0.5 Å. On this structure that removes 12 potassium atoms. Each operator copies the pore ions onto one site. Four potassium ions remain, one set for the tetramer. The 19 crystal waters on chain B are copied onto the four subunits and do not land on each other, so all 76 water atoms stay.

The expanded chains are named B1, B2, B3, and B4. A PDB chain column holds one character, so the script renames them A, B, C, and D before MemPrO. SEQRES from deposited chain B is written onto each of those chains. `chains` is `ABCD` because the biological unit is the whole tetramer.

```
{
    "pdb_id": "3T4D",
    "model": 1,
    "assembly": {"id": "2", "waters": True, "drop_overlaps": True},
    "assembly_from": "B",
    "chains": "ABCD",
    "keep_hetero": ["HOH", "K"],
    "parametrize": {},
    "peripheral": False,
}
```

`assembly_from` is the deposited chain the operators apply to. Short-gap filling runs on that chain before the assembly is built. This entry has no internal gap of 1–6 residues. `waters` keeps `HOH`. `drop_overlaps` is the checkbox that removes the piled-up copies. Potassium is an ion, so it stays in `keep_hetero` without a GAFF charge.

The assembly file has 3,020 protein atoms, 76 crystal water atoms, and 4 potassium atoms. MemPrO takes 116 s. Preparation writes 6,143 protein atoms. Before preparation the checker lists one missing N-terminal methionine on each of the four chains. After preparation the four chains still match at 100% identity, and hydrogens are present on 380 of 384 residues.

Each packed system has 88,914 atoms. The builder mean is 30.9 min (sample standard deviation 9 s). Packmol is 30.6, 30.4, and 30.4 min of that. pdb4amber is about 20 s, and tleap is 2.3 s.

That timed run called pdb4amber with `--dry`, so the 76 crystal water atoms were removed before packing. The four potassium ions stayed. The script now leaves a water whose name is in `keep_hetero`. A new run of this PDB keeps those waters and will be larger than 88,914 atoms.

Run this PDB on its own when the other systems should be left as they are:

```
python tests/study_cases/prepare_systems.py 3T4D
```

#### 6RV2 — TASK-1

TASK-1 (K2P3.1) at 3.0 Å deposits two dimers in the asymmetric unit: A+B and C+D. The asymmetric unit is the crystal packing, not one channel. This example keeps one dimer, chains A and B. Set `chains` to `"CD"` for the other dimer.

```
{
    "pdb_id": "6RV2",
    "model": 1,
    "chains": "AB",
    "keep_hetero": ["K"],
    "parametrize": {},
    "peripheral": False,
}
```

Selectivity-filter potassium stays as residue `K` and atom `K`. pdb4amber in the builder renames those to `K+`, which is the same step the GUI uses. Crystal phosphatidylcholine (`PC1`), decyl maltoside (`DMU`), and cholesterol hemisuccinate (`Y01`) are left out of this example. Leave a detergent or a crystal lipid out only when it is not biologically relevant. A lipid that belongs in the channel goes in `keep_hetero` with a net charge in `parametrize`.

The subset has 4,066 protein atoms and 4 potassium atoms. Preparation fills 82 missing side-chain atoms and writes 8,266 protein atoms. The three built systems have 129,701, 129,698, and 129,707 atoms. This is the first case where the builder replicas spread by more than a few seconds (mean 43.6 min, sample standard deviation 2.0 min).

Chain B is 4 residues longer than chain A (261 versus 257) and the sequences match at 98% identity. That length difference is in the deposit. It is still there after preparation. SEQRES also lists 7 residues on chain A and 3 on chain B with no coordinates. The after report has no internal numbering gap, and it does not repeat those SEQRES-only residues.

#### 3WU2 — photosystem II

Photosystem II at 1.9 Å is a dimer. The author chains use both cases, and both halves are kept: `AaBbCcDdEeFfHhIiJjKkLlMmOoTtUuVvXxYyZz`. Monoatomic ions stay: `CA`, `CL`, `MG`, and `FE2`. Chlorophyll, pheophytin, carotenes, plastoquinone, heme, the Mn4Ca cluster, detergents, glycerol, and the thylakoid lipids are part of the biology of this complex. This example leaves them out so the protein-and-ions system packs faster. Include them when the question needs them. A POPC bilayer with ff14SB, Lipid21, and TIP3P can still be built from the protein and those ions while you decide.

```
{
    "pdb_id": "3WU2",
    "model": 1,
    "chains": "AaBbCcDdEeFfHhIiJjKkLlMmOoTtUuVvXxYyZz",
    "keep_hetero": ["CA", "CL", "MG", "FE2"],
    "parametrize": {},
    "manual_params": {},
    "peripheral": False,
}
```

The subset has 41,092 protein atoms and 15 ion atoms. Preparation writes 80,898 protein atoms and the same 15 ions. MemPrO takes 18.5 min. Protonation takes 86 s. Short-gap filling takes 15 s. The three systems have 904,059, 904,068, and 904,086 atoms. Their builder times are 5.01 h, 5.06 h, and 5.05 h (mean 5.04 h, sample standard deviation 1.6 min). Packmol is 4.95 h, 5.00 h, and 4.99 h of that. pdb4amber is 169 s, 172 s, and 168 s. tleap is 25 s, 24 s, and 24 s. Expect the large-system PDB rewrite: serials pass 99,999, and tleap needs the realigned file.

Before preparation the checker reports 314 missing heavy atoms, 63 alternate conformations, and 107 residues that are absent from the coordinates. Preparation fills the missing protein atoms, keeps the higher-occupancy alternate conformation, and leaves two residues open: homoserine lactone `HSK` at residue 336 on chains D and d. `HSK` is not a standard amino acid, so the short-gap builder does not invent it. The chain-difference list is long because the checker compares every pair of chains in the dimer. Different subunits are supposed to differ. `FE2` stays listed as a non-standard residue.

The deposited iron is residue `FE2` and atom `FE`. Before the build, that atom is renamed `FE2`, which is the name in Amber's ion template. A deposited `FE3` residue would be written as Amber residue `FE` (atom `FE`), the Fe3+ unit in the same library. `CU1`, `GD3`, `IOD`, `NH4`, and `U4+` get the library atom name when the deposited atom is only the element. `CA`, `CL`, `MG`, and `K` already match and stay as deposited.

### Photosystem II cofactors

Chlorophyll, pheophytin, carotenes, plastoquinone, heme, the oxygen-evolving cluster, and the thylakoid lipids are part of photosystem II. This example leaves them out for a shorter protein-and-ions pack. Add a name when that run should include it. They are listed in `PSII_ORGANIC` (`CLA`, `PHO`, `BCR`, `RRX`, `PL9`, `BCT`, the thylakoid lipids, `HEM`, `OEX`).

To carry an organic residue into the build, add its name to `keep_hetero` and its net charge to `parametrize`. The script then parametrizes it once and reuses the library for all three builder replicas. Expect a larger and slower pack than the protein-and-ions solute.

`HEM` and `OEX` need `manual_params` with an existing `frcmod` and `lib`. The oxygen-evolving cluster is not a GAFF molecule. Add those names when the libraries exist.

### The builder call

Shared settings live in one function. `notprotonate=True` keeps the PropKa residue names written earlier and stops packmol-memgen from running reduce, which renames `HA` to `HCA` and breaks tleap. `remove_protein_h=True` is the Builder option “Remove protein hydrogens”: it drops protein hydrogens left with those names, and tleap fills `HA` from the residue template. Ligand hydrogens stay. `preoriented=True` tells packmol-memgen that MemPrO already placed the protein.

```
builder.prepare_system(
    pdb_file=str(pdb_path),
    working_dir=str(job_root),
    upper_lipids=["POPC"],
    lower_lipids=["POPC"],
    lipid_ratios="1//1",
    protein_ff="ff14SB",
    lipid_ff="lipid21",
    water_model="tip3p",
    salt_concentration=0.15,
    cation="K+",
    anion="Cl-",
    preoriented=True,
    parametrize=True,
    notprotonate=True,
    remove_protein_h=True,
    ligand_params=ligand_params,
    wait=True,
    wait_verbose=True,
)
```

`lipid_ratios="1//1"` is required with `POPC//POPC`. This packmol-memgen treats the double slash as two leaflet compositions and stops unless a ratio is given. `ligand_params` is empty when the PDB has no organic hetero group. BGC (charge 0) and RET (charge +1) fill it for 7AAQ and 1M0L. The integer in `parametrize` is the net charge passed to antechamber.

The default ligand model is GAFF2 with ABCG2, which is one of the two pairings Amber recommends. The other is GAFF with AM1-BCC. Set `atom_type` to `gaff2` or `gaff`, and `charge_method` to `abcg2` or `bcc`, on the system entry. One residue can override both by using a dict instead of an integer. GAFF2 with AM1-BCC, and GAFF with ABCG2, still run, and the script prints Amber's warning for those pairs. Delete that PDB's `ligand_parameters.json` after changing the model, or the saved library is reused.

```
"atom_type": "gaff",
"charge_method": "bcc",
"parametrize": {"RET": 1},
```

```
"parametrize": {
    "RET": {"charge": 1, "atom_type": "gaff2", "charge_method": "abcg2"},
    "BGC": {"charge": 0, "atom_type": "gaff", "charge_method": "bcc"},
},
```

After packmol-memgen, the generated `run_preparation.sh` checks the packed PDB. Residue numbers past 9998, or a serial written against the atom name, are rewritten before `pdb4amber`. A readable membrane is left as packmol wrote it. tleap then receives disulfide bonds for CYX SG pairs and `set system box` from the CRYST1 record. `loadPDB` does not copy that box. The `system.pdb` written by `savePDB` is rewritten when its columns are shifted, so the equilibration restraints can read it. The original `bilayer_*_lipid.pdb` is kept for the box as well.

### Running minimization and equilibration

`PROTOCOL` at the top of the script is off. A system runs the membrane protocol only when its own `protocol` dict sets `enabled` to `True`. That dict can choose the engine (`amber`, `gromacs`, `openmm`, or `namd`), whether minimization and equilibration run, and `production_ns`. `stage_ns` replaces individual stage lengths. `None` keeps the GateWizard schedule: 10000 minimization steps, Equilibration 1–6 (Equilibration 6 is 47.625 ns), then production. COM and rotation restraints stay off, as on the Equilibration page. The run is `protocol/run_equilibration.sh` inside the builder replica, and only the first replica runs it unless `replicas` is raised.

2MVH is the small system. Its protocol dict is commented out, so a run only prepares that system. Uncomment the dict to run OpenMM minimization, every equilibration stage, and 1 ns of production.

```
"protocol": {
    "enabled": True,
    "engine": "openmm",
    "minimization": True,
    "equilibration": True,
    "production_ns": 1.0,
    "openmm_platform": "CUDA",
},
```

### What is timed

Stages, in order:

1. `download` — RCSB PDB file
2. `short_gaps` — numbering gaps of 1 to 6 standard residues are built from SEQRES before orientation. Only those new atoms are inserted into the downloaded PDB, and later atom serials move up. Ligands and the other records stay as deposited. Longer gaps stay open. A sequence that is not a standard amino acid, such as HSK, stays open. When nothing is built, the downloaded PDB is passed on unchanged. A system with `assembly_from` fills gaps on that deposited chain.

3. `assembly` — only when the system sets `assembly`. Biological assembly operators expand that chain, crystal waters stay when `waters` is set, and atoms that land on the same point are dropped when `drop_overlaps` is set. Copied chains are renamed to one letter so the later PDB stages can read them.
4. `subset` — chosen NMR model, chosen chains, and hetero groups that are not in `keep_hetero` removed
5. `structure_check_before` — the same report as Visualize → Structure checker, on the selected chains. SEQRES for those chains is included, so residues with no coordinates are listed. The report is `work/<PDB>/structure_check_before/report.txt`. Findings do not stop the run.
6. `mempro` — orientation, then the transform applied to the kept atoms
7. `protonation` — ACE and NME are added first, unless the PDB already has a terminal cap, then PropKa at pH 7.4 and Amber residue names, peptide names remapped, disulfide cysteines renamed to CYX, missing protein heavy atoms filled from the Amber templates, protein hydrogens removed, then `pdb4amber --reduce` adds Amber hydrogens. Ligands and ions keep their hydrogens. Capping matches Preparation with Cap protein termini on. A finished `protonation` stage is reused, so delete that PDB's `protonation.json` when this step must run again.
8. `structure_check_after` — the same checker on `prepared.pdb`. SEQRES is left out here because protonation renames residues. Numbering gaps and any heavy atoms the templates did not fill are still listed, in `work/<PDB>/structure_check_after/report.txt`.
9. `ligand_parameters` — GAFF2 with ABCG2 for each name in `parametrize`, unless that system sets `atom_type` and `charge_method`
10. `builder` — one row per replica, the whole packmol-memgen plus tleap run
11. `packmol`, `pdb4amber`, and `tleap` — the same replica split by the files that build writes. `packmol.log` ends packing, `system_for_tleap.pdb` ends pdb4amber, and `system.prmtop` ends tleap. A replica that is still packing stays as the single builder row.
12. `protocol` — only when that system's `protocol` is enabled, and only for the first replica unless `replicas` is higher. This is minimization, equilibration, and production.

`results/timings.csv` and `results/summary.md` are rewritten after every finished stage and every builder replica. Times under one second stay in the summary (they are not rounded to 0.0). Stopping the run keeps the rows that already finished, including protein atom counts from `prepared.pdb` and system atom counts from each finished `system.pdb`. A failed replica stays in the table and is left out of the mean. `python prepare_systems.py --report` rewrites that table from the saved stages and the build directories. It does not start a stage.

Successful stage times are stored under `work/<PDB>/*.json`. That is what lets a long run continue after an interruption. Delete that PDB's folder, or pass `--fresh`, when the table must come from a single uninterrupted pass.

## Recommendations

- Orient the protein once, then build the replicas from that prepared PDB. Repeating MemPrO can shift the protein and change the atom count. The same coordinates give packmol-memgen systems of equal size, which are easier to compare.
- Start with a short peptide such as SpoVM when learning the script. It is the peripheral case, the structure checker is empty, and a bilayer is ready in about 20 minutes.
- For an NMR entry, set `model` to one conformer. Glycophorin A has 20 models and SpoVM has 10. The whole file is not one protein.
- For a crystal that contains two copies of an oligomer, orient one copy. TASK-1 chains A and B are one dimer. Chains C and D are the other.
- When the deposit is the asymmetric unit, set `assembly` to the biological assembly that should be built. NaK2K uses assembly 2, keeps `HOH`, and drops the potassium copies that land on one point. The script renames the expanded chains to one letter.
- Put an organic residue in both `keep_hetero` and `parametrize`. The integer is the net charge antechamber receives. Retinal is `{"RET": 1}`. The sugar is `{"BGC": 0}`. Ions such as `K` and `FE2`, and waters such as `HOH`, stay in `keep_hetero` only. The default model is GAFF2 with ABCG2. Set `atom_type` to `gaff` and `charge_method` to `bcc` for GAFF with AM1-BCC.

- Keep experimental water, a bound lipid, or a detergent when it is biologically relevant. Add the residue name to `keep_hetero`, and add a net charge to `parametrize` when the residue is organic. Leave a detergent or a crystal lipid out only when it is not biologically relevant.
- Photosystem II chlorophyll, heme, the metal cluster, and the thylakoid lipids are part of the complex. The example leaves them out so the protein-and-ions pack is shorter. Add the names from `PSII_ORGANIC` when the run should include them. `HEM` and `OEX` need `manual_params`.
- Treat a SEQRES stretch longer than 6 residues as unmodeled. The script will not build it. Homoserine lactone on photosystem II stays open even though it is a single missing residue, because it is not a standard amino acid.
- Read `structure_check_after/report.txt` for gaps that are still open. Hydrogens on most residues, a named ligand, and chain pairs that are different subunits are listed there and are not failed builds.
- Plan on the builder, not MemPrO, for the wall time. Only photosystem II spends more than a couple of minutes in orientation.

# Roadmap

Future features!

## Tools

- **Concatenate / trim** a trajectory.
- **Extract frames** from a trajectory.
- **Align trajectory** on a chosen selection.
- **Convert format** between trajectory and structure files.
- **Enhanced sampling setup.**
- **Constant pH MD simulations.**
- **Computational electrophysiology.**

## Visualize

- **Strip extra molecules.** Delete chosen HET groups or chains before preparation. Offer ready-made selections and list the non-protein residues. Clicking an item focuses it in the view.

## Analysis

- **Check EVAPL.** The freud area-per-lipid path stays experimental until the same trajectories are compared with FATSLiM.

# Troubleshooting

Common issues and solutions for GateWizard.

## Installation Issues

### Issue: ImportError with numpy.compat

**Error Message**:

```
ImportError: cannot import name 'OLD_NUMERIC_TYPES' from 'numpy.compat'
```

**Cause**: Version conflict between NumPy and Parmed.

**Solution**:

```
conda activate gatewizard
conda install -c conda-forge parmed=4.3.0 --force-reinstall
```

### Issue: pdb4amber command not found

**Error Message**:

```
bash: pdb4amber: command not found
```

**Cause**: AmberTools not installed or environment not activated.

**Solution**:

```
# Check if environment is activated
conda activate gatewizard

# Verify AmberTools installation
which pdb4amber

# If not found, reinstall
conda install -c conda-forge ambertools=26
```

### Issue: CustomTkinter not found

**Error Message**:

```
ModuleNotFoundError: No module named 'customtkinter'
```

**Solution**:

```
conda activate gatewizard
pip install customtkinter --force-reinstall
```

### Issue: Display/GUI not appearing

**On Linux**:

```
# Install tkinter support
sudo apt-get install python3-tk

# Check DISPLAY variable
echo $DISPLAY

# If empty, set it
export DISPLAY=:0
```

**On macOS**:

```
# For the desktop app, use gatewizard-gui from Applications
# See https://github.com/franciscoadasme/gatewizard-gui#macos
```

**On Windows WSL**:

```
# Install X server on Windows (VcXsrv, X410)
# In WSL, set DISPLAY
export DISPLAY=$(cat /etc/resolv.conf | grep nameserver | awk '{print $2}'):0
```

## Runtime Issues

### Issue: GateWizard crashes on startup

**Check 1**: Verify Python version

```
python --version  # Should be 3.8+
```

**Check 2**: Check dependencies

```
python -c "import customtkinter"
python -c "import MDAnalysis"
python -c "import parmed"
```

**Check 3**: For the desktop app, check install logs (see gatewizard-gui README)

### Issue: GUI appears but is unresponsive

**Possible causes**: - Large file being processed - System resources exhausted - Display server issues

**Solutions**: 1. Wait for operation to complete (check terminal for progress) 2. Close other applications to free memory 3. Restart display server 4. Check system resources: `top` or `htop`

### Issue: Cannot load PDB file

**Error**: "Invalid PDB file" or "Cannot parse file"

**Solutions**: 1. **Verify file format**: Ensure it's a valid PDB file

```
head -n 5 your_file.pdb  # Should show ATOM/HETATM lines
```

1. **Clean the PDB**: Use Propka with "Clean PDB" option enabled
2. **Check file permissions**:

```
ls -l your_file.pdb
chmod 644 your_file.pdb  # If needed
```

3. **Try different source**: If from PDB database, download again

## Analysis Issues

### Issue: RMSD calculation fails

**Error**: "Cannot compute RMSD"

**Possible causes**: - Topology/trajectory mismatch - Atom selection returns no atoms - Missing frames

**Solutions**: 1. **Verify topology matches trajectory**:

```
# Check atom counts
grep "^ATOM" topology.pdb | wc -l
# Should match trajectory frame size
```

1. **Test atom selection**:
2. Try simpler selection: just "protein"

3. Verify selection syntax

4. **Check trajectory integrity**:

```
import MDAnalysis as mda
u = mda.Universe("topology.psf", "trajectory.dcd")
print(f"Frames: {len(u.trajectory)}")
```

### Issue: Trajectory loading is very slow

**Causes**: - Very large trajectory (>10GB) - Network storage (slow I/O) - Many atoms selected

**Solutions**: 1. **Use more specific selection**: "protein and name CA" instead of "protein" 2. **Copy to local disk**: Faster than network storage 3. **Split trajectory**: Analyze in smaller chunks 4. **Use frame stride**: Skip frames if possible

### Issue: Energy plot shows weird values

**Symptoms**: - Extremely large/small values - Discontinuities - Unexpected units

**Solutions**: 1. **Check unit selection**: - Verify kcal/mol vs kJ/mol - Verify ps vs ns vs μs

1. **Verify log files are sequential**:
2. Check file order

3. Verify time assignments

4. **Check for parsing errors**:

5. Look at status messages

6. Verify log files are from NAMD

7. **Plot individual files**:

8. Remove all files
9. Add one at a time
10. Find problematic file

### Issue: Time distribution is wrong

**Symptoms**: - Files overlap on time axis - Gaps in timeline - Wrong total duration

**Solutions**: 1. **Check time assignments**:

```
File1: 10 ns (not 10,000 ps)
File2: 50 ns
Result: 0–10 ns, 10–60 ns ✓
```

1. **Verify file order**:
2. Use drag-and-drop to reorder

3. Equilibration before production

4. **Check for typos**: 10 vs 100 ns makes big difference

5. **Clear and reassign**: Remove all files, re-add in order

### Issue: Plot is empty or not updating

**Solutions**: 1. **Verify data exists**: Check status messages 2. **Check plot ranges**: Auto-scale or adjust manually 3. **Verify columns selected**: For NAMD analysis 4. **Re-run analysis**: Click "Run Analysis" again

## Memory Issues

### Issue: Out of memory error

**Error**: "MemoryError" or system freezes

**Causes**: - Trajectory too large for available RAM - Multiple large analyses in memory - Memory leak (rare)

**Solutions**: 1. **Reduce atom selection**: Use "name CA" instead of "protein" 2. **Analyze in chunks**: Split trajectory temporally 3. **Close other applications**: Free up RAM 4. **Increase swap space** (Linux):

```
sudo fallocate -l 8G /swapfile
sudo chmod 600 /swapfile
sudo mkswap /swapfile
sudo swapon /swapfile
```

5. **Use a machine with more RAM**: For very large systems

## Performance Issues

### Issue: Analysis is very slow

**Symptoms**: - Progress bar barely moving - Hours for simple calculation - CPU not fully utilized

**Solutions**: 1. **Use simpler selections**: Fewer atoms = faster 2. **Check system load**: Close background processes 3. **Use SSD storage**: Faster I/O for trajectory reading 4. **Verify not swapping**: Check memory usage with `htop` 5. **Update libraries**: Newer versions may be faster

### Issue: GUI is laggy

**Solutions**: 1. **Reduce plot updates**: Don't update every frame 2. **Close unused tabs**: Free resources 3. **Lower display resolution**: If using remote X 4. **Restart application**: Clear accumulated state

## File/Path Issues

### Issue: Cannot find output directory

**Error**: "Output directory does not exist"

**Solutions**: 1. **Create directory**:

```
mkdir -p /path/to/output
```

2. **Use absolute path**: /home/user/output not ~/output 3. **Check permissions**: Must have write access 4. **Verify path exists**: No typos in directory name

### Issue: Cannot save results

**Error**: "Permission denied" or "Cannot write file"

**Solutions**: 1. **Check write permissions**:

```
ls -ld /path/to/directory
# Should show 'w' for write permission
```

2. **Change permissions**:

```
chmod u+w /path/to/directory
```

3. **Save to different location**: Your home directory 4. **Run with proper user**: Not as root (usually)

## Propka Issues

### Issue: Propka calculation fails

**Error**: "Propka failed" or "Cannot calculate pKa"

**Solutions**: 1. **Check PDB format**: Must be valid PDB 2. **Clean PDB first**: Remove heteroatoms 3. **Check residue names**: Must be standard amino acids 4. **Update Propka**:

```
pip install --upgrade propka
```

### Issue: Protonation states seem wrong

**Verify**: 1. **pH value**: Is it set correctly? 2. **pKa values**: Check against literature 3. **Environment effects**: Buried residues have shifted pKa

**Solutions**: - Review pKa table in results - Check pKa shifts - Compare with similar structures - Manually adjust if needed

## Platform-Specific Issues

### Linux

**Issue**: libGL error

```
libGL error: No matching fbConfigs or visuals found
```

**Solution**:

```
sudo apt-get install libgl1-mesa-glx
```

**Issue**: Permission denied for conda **Solution**:

```
# Fix conda ownership
sudo chown -R $USER:$USER ~/miniconda3
```

### macOS

**Issue**: App not trusted **Solution**:

```
# Allow running from unidentified developer
xattr -d com.apple.quarantine /path/to/gatewizard
```

**Issue**: Retina display scaling **Solution**: Adjust display settings in System Preferences

### Windows (WSL)

**Issue**: X server not connecting **Solution**:

```
# In WSL, set DISPLAY correctly
export DISPLAY=$(cat /etc/resolv.conf | grep nameserver | awk '{print $2}'):0

# In X server (VcXsrv), disable access control
```

**Issue**: File path issues (Windows vs WSL paths) **Solution**: Use WSL paths (`/mnt/c/...`) inside WSL

**Issue: VmmemWSL keeps using a lot of RAM after GateWizard closes**

WSL2 is a virtual machine. Linux keeps file cache (MD logs, Python, Electron) and does **not** give that RAM back to Windows right away. Other Apps and extra Ubuntu terminals also count toward `VmmemWSL`. On a 16 GB PC this can push Windows to 80–90% and cause other apps to crash.

This is normal WSL behavior, not a leftover GateWizard process (the backend log should already show a clean shutdown).

**Solution**: Cap the VM from Windows. Create `%UserProfile%\.wslconfig` (for example `C:\Users\YourName\.wslconfig`) with:

```
[wsl2]
memory=4GB
swap=2GB
```

On a 16 GB machine, `4GB` leaves headroom for Windows. Use `6GB` if WSL feels tight while GateWizard is open.

Then in **PowerShell** (Windows, not Ubuntu):

```
wsl --shutdown
```

Reopen Ubuntu / GateWizard. Task Manager should keep `VmmemWSL` near that cap instead of growing for minutes after you quit.

A local equilibration started from the GUI is detached on purpose and can keep using RAM until that job finishes. Remote cluster jobs do not run inside WSL.

**Issue**: App window not visible (icon appears in taskbar but window is off-screen or invisible)

This can happen after a crash, a display configuration change, or when the WSLg graphics server enters a bad state.

**Solution (step by step)**:

1. Kill any running GateWizard GUI process:

```
pkill -f gatewizard-gui
```

2. Reset the saved window position:

```
python3 -c "
import json
p = '/home/$USER/.config/gatewizard/config.json'
with open(p) as f:
    cfg = json.load(f)
cfg['gui']['last_window_x'] = None
cfg['gui']['last_window_y'] = None
with open(p, 'w') as f:
    json.dump(cfg, f, indent=2)
print('Window position reset')
"
```

3. If the window is still not visible, restart the WSL graphics server:

```
# Run in PowerShell (Windows side)
wsl --shutdown
```

   Then reopen your WSL terminal and launch **gatewizard-gui-linux** again. WSLg restarts automatically.

   **Note**: If you use VcXsrv or X410 instead of WSLg, close and reopen the X server application instead.

## Getting More Help

If you're still experiencing issues:

1. **Check logs**: GUI runtime log at `~/.config/gatewizard-gui/runtime-install.log` (Linux/WSL) — see gatewizard-gui install docs
2. **Search existing threads**: Discussions and GitHub issues
3. **Ask the community**: Post in Discussions (use labels `api` or `gui` as appropriate).
4. **Provide details**: When asking for help, include:
5. Operating system
6. Python version (`python --version`)
7. GateWizard API version (`python -c "import gatewizard; print(gatewizard.__version__)"`)
8. Error messages (full traceback)
9. Steps to reproduce
10. **Contact the project**: gatewizard.project@gmail.com

## FAQ

**Q: How much RAM do I need?** A: Minimum 4GB, 8GB+ recommended. Large trajectories may need 16GB+.

**Q: Can I run GateWizard remotely?** A: For the desktop app, use X forwarding or WSLg on a remote WSL host and launch **gatewizard-gui-linux**. For scripting, use the Python API (`import gatewizard`).

**Q: Does GateWizard work with GPU acceleration?** A: **OpenMM** can use CUDA when `openmm` + `cudatoolkit` are installed and an NVIDIA driver is present. **GROMACS** GPU builds are separate — prefer conda CPU `gromacs` (or a system CUDA `gmx`); see Installation. Trajectory analysis is CPU-based.

**Q: Can I automate GateWizard?** A: Python API is available for scripting (see API documentation).

**Q: conda/micromamba GROMACS CUDA install hangs — do I need to accept a license?** A: No. EULA / Terms-of-Service lines in the log are not interactive prompts (`-y` is already used). The hang is almost always the dependency solver. Cancel and install CPU `gromacs`, or for the GUI leave the default / see `GATEWIZARD_CONDA_GROMACS_CUDA` in Installation.

**Q: What trajectory formats are supported?** A: DCD, XTC, TRR, NetCDF, and others via MDAnalysis.

**Q: How do I cite GateWizard?** A: Citation information to be added.

# API

## Overview

### API Reference Overview

Welcome to the GateWizard API documentation. This reference provides detailed information about all modules, classes, methods, and functions available in GateWizard.

New to the library? Start with the Quick Reference. It downloads PDB 6RV2, keeps the chains that have short gaps, fills those gaps, and then shows the usual preparation, membrane, equilibration, and analysis calls.

**Available Modules**

**Method dictionary**

A–Z index of public classes and functions. Click a name to jump to its heading.

**Structure Manager**

Load, inspect, edit, and save PDB structures (selections, chain/residue edits, MemPrO orientation).

**Key Features:**

- Load from disk or PDB ID
- MDAnalysis selections and criteria helpers
- Rename / renumber / delete atoms
- Secondary structure assignment
- Apply a MemPrO orientation to the full system

**Main Classes:** `StructureManager`

---

**Structure checker**

Report missing atoms, gaps, alternate conformations, and chain differences in a PDB or mmCIF file.

**Key Features:**

- Missing heavy atoms, numbering gaps, SEQRES gaps, and backbone breaks
- Alternate conformations, with the highest occupancy marked as the most likely
- Protein chains that match, differ by a substitution, or differ in length
- Write a PDB that keeps one conformation and leaves the original file unchanged

---

**Biological assembly**

Expand the oligomer or capsid from operators already stored in a PDB or mmCIF file.

**Key Features:**

- List biological assemblies and their chain and atom counts
- Build an assembly as mmCIF with chain names such as `A1` and `A2`
- Leave out crystal waters unless requested
- Leave a file that is already the biological unit unchanged

---

#### Preparation Module

Module for predicting pKa values and managing protonation states in protein structures.

**Key Features:**

- pKa prediction and analysis
- Protonation state assignment based on pH
- Disulfide bond detection and application
- Protein capping with ACE/NME groups
- pH-dependent protein structure preparation
- Apply Amber protonation names, then complete missing protein atoms (`complete_missing_heavy_atoms`, tleap templates including ASH/GLH protons); `pdb4amber --reduce` can rebuild hydrogens when `reduce` is installed
- Restore original residue numbers after `pdb4amber` (`preserve_residue_numbers`; ACE/NME get N-terminus − 1 / C-terminus + 1)

**Main Classes:** `PreparationManager`, `ProteinCapper`

---

#### Builder Module

Module for building and preparing molecular dynamics simulation systems.

**Key Features:**

- System configuration and setup
- Integration with CHARMM-GUI
- Membrane protein system preparation

**Main Classes:** `Builder`

---

#### MemPrO Module

Module for orienting membrane proteins using MemPrO.

**Key Features:**

- Membrane protein orientation
- Ranked orientation results with scoring
- Oriented PDB file access by rank
- Command-line builder

**Main Classes:** `MemPrO`, `OrientationResult`

---

#### Hydration Module

Module for cavity hydration with standalone PACKMOL (AmberTools TIP3P waters).

**Key Features:**

- PACKMOL availability check
- Hydrogen status detection (heavy-atom-safe mode)
- Cavity volume estimation inside a 3D box
- PACKMOL input generation and execution

- Custom PACKMOL input support

**Main Functions:** `check_packmol_available`, `estimate_cavity_volume`, `hydrate_cavity`

---

#### Equilibration Module

Module for managing equilibration protocols and workflows for NAMD, GROMACS, OpenMM, and Amber.

**Key Features:**

- NAMD equilibration protocol generation (CHARMM-GUI template integration)
- GROMACS equilibration protocol generation
- OpenMM equilibration protocol generation
- Amber equilibration protocol generation (`pmemd` / `sander` mdin + GROUP restraints)
- Flexible restraint system (protein backbone/sidechain, lipid head/tail, ligand, ions)
- MDAnalysis-based atom selection for restraint files
- Progressive force-constant schedule across stages
- Custom stage parameters via `EquilibrationStage` dataclass

**Main Classes:** `NAMDEquilibrationManager`, `GROMACSEquilibrationManager`, `OpenMMEquilibrationManager`, `AmberEquilibrationManager`, `EquilibrationStage`

---

#### Analysis Module

Module for analyzing simulation results and monitoring equilibration progress.

**Key Features:**

- NAMD log file parsing and energy analysis
- OpenMM StateDataReporter log parsing
- MDAnalysis-based trajectory analysis (RMSD, RMSF, distances, radius of gyration)
- Lipid bilayer analysis: **FATSLiM** CLI default (`apl_method='fatslim'`); EVAPL experimental; official lipyphilic `AreaPerLipid` when `apl_method='lipyphilic'`; lipyphilic for leaflets and membrane thickness
- Dark-theme matplotlib plots
- Multi-stage log concatenation with per-file time override
- Comprehensive 2×2 energy summary plots

**Main Classes:** `EnergyAnalyzer` (`namd_analysis`), `TrajectoryAnalyzer` (`trajectory_analysis`), `BilayerTrajectoryAnalyzer` (`lipid_bilayer_analysis`), `OpenMMLogAnalyzer`

---

### Quick Links

- **Quick Reference** - Common patterns and snippets
- **User Guide** - Step-by-step tutorials
- **Method dictionary** - Clickable A–Z of public methods
- **Example scripts** - `tests/<module>_examples/` (included in these pages and run by pytest)

### Getting Help

If you need additional help:

1. Check the Troubleshooting Guide
2. Review the User Guide for practical examples
3. Ask in GitHub Discussions or report bugs via issues

4. See complete workflow examples in the `tests/` example directories

**Module Organization**

Each module documentation page includes:

- **Import statements** - How to import the module
- **Class/Function signatures** - Detailed method signatures
- **Parameters** - Complete parameter descriptions
- **Returns** - Return types and values
- **Examples** - Practical code examples
- **Workflows** - Complete usage patterns

## Quick Reference

### GateWizard Quick Reference

This page is the Python library. The desktop app is the User Guide.

The usual order is prepare the structure, build the membrane, write the equilibration, then analyze. Paste the blocks into the same Python session, in order. Each block reads a file the block before it wrote for PDB 6RV2. The linked module page has every argument.

**Open a session**

Create the `gatewizard` environment once, from the Installation page. Every new terminal then needs that environment turned on. The prompt shows `(gatewizard)` when it is on.

```
mkdir -p ~/gatewizard-work
cd ~/gatewizard-work
conda activate gatewizard
python -c "import gatewizard; print(gatewizard.__version__)"
```

Work in that folder. The first script saves `6RV2.pdb` in the current directory. Start Python and paste the next block:

```
python
```

Or save the block as `start_6rv2.py` and run `python start_6rv2.py`. Leave Python with `exit()`.

---

**Start from a PDB**

Download 6RV2 (TASK-1), keep chains A and B, and fill their short gaps. The crystal deposits two dimers, A+B and C+D. A and B are the copy with the short gaps, so C and D are left out. Later blocks can use `6RV2_AB.pdb`. The study case runs the full membrane build. This script does not protonate or pack. Chain selection and the download are in the Structure Manager. Listing and filling gaps are in Preparation. Filling needs `pdbfixer`, which `pip install "gatewizard[full]"` provides.

```
from gatewizard.core.structure_manager import StructureManager
from gatewizard.core.short_gaps import classify_short_gaps, fill_short_gaps

viewer = StructureManager()
info = viewer.load_from_pdb_id("6RV2", output_dir=".")
print(info["n_atoms"], "atoms,", info["n_chains"], "chains:", info["chains"])
print(info["title"])

report = classify_short_gaps("6RV2.pdb", chains=["A", "B"])
for gap in report["filled"]:
    print(f"fill {gap['count']} residues on chain {gap['chain']} after {gap['after_resid']}")
for gap in report["skipped"]:
    print(f"leave open ({gap['reason']}): chain {gap['chain']}")

filled = fill_short_gaps("6RV2.pdb", "6RV2_AB.pdb", chains=["A", "B"])
print("wrote", filled["output_file"])
print("built", len(filled["built"]), "gaps")
```

The two gaps that can be built are 3 residues each, at residue 149 on chain A and chain B. Longer stretches that exist only in SEQRES are reported and left open.

`6RV2_AB.pdb` still has the crystal phosphatidylcholine (`PC1`), decyl maltoside (`DMU`), and cholesterol hemisuccinate (`Y01`). Keep the protein and the potassium ions named `K` before PropKa. That is the same choice as the study case. Do this while the residue names are still the deposited ones. After PropKa, names such as ASH are no longer selected as protein.

```
from gatewizard.core.structure_manager import StructureManager

solute = StructureManager()
solute.load_structure("6RV2_AB.pdb")
drop = solute.select_atoms("not protein and not resname K")
solute.delete_atoms(drop)
solute.save_pdb("6RV2_solute.pdb")
```

---

### Propka Analysis

Predict pKa values for `6RV2_solute.pdb` and write protonation names. Every argument is in Preparation.

#### Basic Analysis

```
from gatewizard.core.preparation import PreparationManager

analyzer = PreparationManager(propka_version="3")
pka_file = analyzer.run_analysis("6RV2_solute.pdb")
summary_file = analyzer.extract_summary(pka_file)
residues = analyzer.parse_summary(summary_file)
```

#### Access Results

```
for res in residues:
    print(f"{res['residue']}{res['res_id']} Chain {res['chain']}: pKa={res['pka']:.2f}")
```

#### Filter by Type

```
acidic = [r for r in residues if r['residue'] in ['ASP', 'GLU']]
basic = [r for r in residues if r['residue'] in ['LYS', 'ARG', 'HIS']]
```

#### Write PDB with Protonation States

```
# Apply protonation states at target pH.
# This only changes residue names (ASP→ASH). It does not add hydrogens.
stats = analyzer.apply_protonation_states(
    input_pdb="6RV2_solute.pdb",
    output_pdb="6RV2_ph7.4.pdb",
    ph=7.4
)
print(f"Modified {stats['residue_changes']} residues")

# Optional: override one residue from the list printed above.
# The key is RESNAME + number, or RESNAME + number + _ + chain.
custom_states = {}
for res in residues:
    if res["residue"] == "ASP" and res["chain"] == "A":
        custom_states[f"ASP{res['res_id']}_A"] = "ASH"
        break
if custom_states:
    stats = analyzer.apply_protonation_states(
        input_pdb="6RV2_solute.pdb",
        output_pdb="6RV2_custom.pdb",
        ph=7.4,
        custom_states=custom_states
    )
```

The membrane step uses `6RV2_ph7.4.pdb`. `6RV2_custom.pdb` is only the override.

#### Generate Multiple pH States

```
for ph in [5.0, 6.0, 7.0, 8.0]:
    stats = analyzer.apply_protonation_states(
        input_pdb="6RV2_solute.pdb",
        output_pdb=f"6RV2_ph{ph:.1f}.pdb",
        ph=ph
    )
```

---

### System Preparation

Pack a membrane around `6RV2_ph7.4.pdb` and write the Amber topology. Details are in the Builder. Packing this protein takes about 45 minutes. `wait=True` blocks until that job finishes.

#### Basic Membrane System

```
from gatewizard.core.builder import Builder

builder = Builder()
builder.set_configuration(
    salt_concentration=0.15,
    dist=12,
    dist_wat=26
)

success, msg, job_dir = builder.prepare_system(
    pdb_file="6RV2_ph7.4.pdb",
    working_dir="6RV2_system",
    upper_lipids=["POPC", "POPE"],
    lower_lipids=["POPC", "POPE"],
    lipid_ratios="3:1//3:1",
    wait=True,
)
print(success, msg)
```

#### Check Output

```
if success:
    topology = job_dir / "system.prmtop"
    coords = job_dir / "system.inpcrd"
    pdb = job_dir / "system_solv.pdb"
```

---

### Equilibration Setup

Write an OpenMM equilibration for the packed 6RV2 system. OpenMM is part of `pip install "gatewizard[full]"`, which is the install on the Installation page, so there is no separate program to put on `PATH`. The files are the ones under `6RV2_system/`. Every stage field is on the Equilibration page.

The default membrane protocol is the real one: Equilibration 1–5 are 0.125, 0.5, 0.25, 0.5, and 1 ns, Equilibration 6 is 47.625 ns, and production is 50 ns. The block below is only for this tutorial. It sets Equilibration 6 and production to 1 ns each, like the longer of the short stages, so the run is a few nanoseconds. Delete those two assignments to keep 47.625 ns and 50 ns.

```
from pathlib import Path
from gatewizard.tools.equilibration import OpenMMEquilibrationManager

job_dir = Path("6RV2_system")
system_files = {
    "prmtop": str(job_dir / "system.prmtop"),
    "inpcrd": str(job_dir / "system.inpcrd"),
    "pdb": str(job_dir / "system_solv.pdb"),
}
stages = OpenMMEquilibrationManager.get_default_stage_params(
    "NPT", include_production=True
)
for stage in stages:
    if stage.name == "Equilibration 6":
        stage.time_ns = 1.0  # tutorial only; the real stage is 47.625 ns
    if stage.name == "Production":
        stage.time_ns = 1.0  # tutorial only; the real stage is 50 ns
manager = OpenMMEquilibrationManager(Path("."))
result = manager.setup_openmm_equilibration(
    system_files=system_files,
    stage_params_list=stages,
    output_name="6RV2_equilibration",
)
print(result["run_script"])
```

Run it from that folder:

```
cd 6RV2_equilibration
bash run_equilibration.sh
```

---

### Analysis

Read the OpenMM log after `bash run_equilibration.sh` has been run in `6RV2_equilibration`. The Analysis page covers trajectories and the other engines.

```
from pathlib import Path
from gatewizard.utils.openmm_analysis import get_equilibration_progress

progress = get_equilibration_progress(Path("6RV2_equilibration"))
stage = progress["equilibration_1"]
print(stage.status)
if stage.timing:
    print(stage.timing.steps_completed, "of", stage.timing.total_steps, "steps")
```

---

### Error Handling

Wrap a call so a missing file or a preparation error prints a message instead of stopping the session.

#### Standard Pattern

```
from gatewizard.core.preparation import PreparationError, PreparationManager

try:
    analyzer = PreparationManager()
    pka_file = analyzer.run_analysis("6RV2_solute.pdb")
except FileNotFoundError as e:
    print(f"File not found: {e}")
except PreparationError as e:
    print(f"Preparation error: {e}")
except Exception as e:
    print(f"Unexpected error: {e}")
```

---

### Common Dictionary Keys

These are the fields of each residue printed from the 6RV2 PropKa list, and of one equilibration stage.

#### Propka Results

```
residue = {
    "residue": "ASP",      # Residue name
    "res_id": 42,          # Residue number
    "chain": "A",          # Chain ID
    "pka": 3.85,           # Predicted pKa
    "model_pka": 3.80      # Model pKa
}
```

#### Stage Parameters

```
stage = {
    "time_ns": 0.125,              # Simulation time
    "timestep": 1.0,               # Timestep in fs
    "temperature": 310.15,         # Temperature in K
    "pressure": 1.0,               # Pressure in atm
    "constraints": {               # Restraints (kcal/mol/Å²)
        "protein_backbone": 10.0,
        "protein_sidechain": 5.0,
        "lipid_head": 2.0,
        "water": 0.0
    },
    "use_gpu": True,
    "cpu_cores": 4,
    "dcd_freq": 5000
}
```

---

### File Locations

These are the files the calls above leave on disk.

#### After Propka

```
6RV2_solute.pdb                    # Protein plus potassium, detergents removed
6RV2_solute.pka                    # Full PropKa output
6RV2_solute_summary_of_prediction.txt
6RV2_ph7.4.pdb                     # Names at pH 7.4; the membrane step uses this
```

#### After System Preparation

```
6RV2_system/
  ├── system.prmtop          # Topology
  ├── system.inpcrd          # Coordinates
  ├── system_solv.pdb        # Solvated structure
  └── step3_input.pdb        # Pre-solvation
```

#### After Equilibration Setup

```
6RV2_equilibration/
  ├── step1_equilibration.inp     # Config for stage 1
  ├── step1_equilibration.log     # Written when that stage is run
  ├── openmm_run.py               # OpenMM runner
  ├── run_equilibration.sh        # Run script
  ├── system.prmtop
  ├── system.inpcrd
  └── system.pdb
```

---

### Tips

Small choices that keep a first run from stalling or writing into the wrong folder.

#### Performance

- Use `use_gpu=True` for equilibration stages
- Increase `cpu_cores` for larger systems
- Use `timestep=1.0` for heavily restrained stages
- Use `timestep=2.0` for production

#### File Management

- Stay in `~/gatewizard-work` so the `6RV2_...` paths above keep resolving
- Use `pathlib.Path` for file operations
- Create output directories before writing

#### Debugging

- Check log files: `tail -f step*.log`
- Verify file existence before processing
- Use try-except for robust error handling

## Method dictionary

Scan the public Python API by name. Each entry jumps to the heading on the hand-written module page. Examples live in `tests/<module>_examples/` and are included on those pages.

Regenerate with `python scripts/gen_api_dictionary.py`.

### Structure Manager

- `assign_secondary_structure_map`
- `parse_pdb`
- `StructureManager.__init__`
- `StructureManager.align_to_axis`
- `StructureManager.apply_mempro_orientation`

- StructureManager.assign_secondary_structure
- StructureManager.auto_detect_molecules
- StructureManager.center_atoms
- StructureManager.delete_atoms
- StructureManager.get_chains
- StructureManager.get_residues
- StructureManager.get_secondary_structure_summary
- StructureManager.get_structure_info
- StructureManager.load_from_pdb_id
- StructureManager.load_structure
- StructureManager.rename_chain
- StructureManager.rename_chain_by_indices
- StructureManager.rename_residues
- StructureManager.rename_residues_by_indices
- StructureManager.renumber_residues
- StructureManager.renumber_residues_by_indices
- StructureManager.rotate_atoms
- StructureManager.save_pdb
- StructureManager.select_atoms
- StructureManager.select_by_criteria
- StructureManager.translate_atoms

### Structure checker

- apply_altloc_choices
- diagnose_structure
- keep_most_likely_altlocs

### Biological assembly

- build_assembly
- list_assemblies

### Preparation

- complete_missing_heavy_atoms
- count_protein_hydrogens
- extract_summary_section
- modify_pdb_based_on_summary
- parse_summary_section
- PreparationManager.__init__
- PreparationManager.apply_disulfide_bonds
- PreparationManager.apply_protonation_states
- PreparationManager.build_titration_figure
- PreparationManager.complete_missing_heavy_atoms
- PreparationManager.detect_disulfide_bonds
- PreparationManager.extract_summary
- PreparationManager.get_available_states
- PreparationManager.get_default_protonation_state
- PreparationManager.get_ph_titration_curve
- PreparationManager.get_residue_statistics
- PreparationManager.parse_summary
- PreparationManager.restore_original_residue_numbers
- PreparationManager.run_analysis

- PreparationManager.run_pdb4amber_with_cap_fix
- run_propka
- strip_protein_hydrogens

### Short gaps

- classify_short_gaps
- fill_short_gaps

### Minimize selection

- minimize_selection

### Residue mapping

- parse_residue_mapping_by_chain
- parse_residue_mapping_file

### Protein capping

- cap_protein
- detect_terminal_caps
- is_already_capped
- ProteinCapper.__init__
- ProteinCapper.remove_hydrogens_and_cap

### Builder

- Builder.__init__
- Builder.cancel_preparation
- Builder.generate_preparation_inputs
- Builder.modify_residue_names_for_propka
- Builder.prepare_system
- Builder.prepare_system_stage1_for_propka
- Builder.prepare_system_stage2_for_propka
- Builder.run_preparation
- Builder.set_configuration
- Builder.validate_system_inputs
- Builder.wait_for_completion

### Job monitor

- JobMonitor.__init__
- JobMonitor.cleanup_stale_jobs
- JobMonitor.export_job_summary
- JobMonitor.get_active_jobs
- JobMonitor.get_completed_jobs
- JobMonitor.get_job
- JobMonitor.get_job_statistics
- JobMonitor.refresh_job
- JobMonitor.remove_job
- JobMonitor.scan_for_jobs
- JobMonitor.set_working_directory

### Force fields

- `ForceFieldManager.__init__`
- `ForceFieldManager.get_available_anions`
- `ForceFieldManager.get_available_cations`
- `ForceFieldManager.get_available_lipids`
- `ForceFieldManager.get_force_field_info`
- `ForceFieldManager.get_lipid_catalog`
- `ForceFieldManager.get_lipid_force_fields`
- `ForceFieldManager.get_protein_force_fields`
- `ForceFieldManager.get_recommendations`
- `ForceFieldManager.get_water_models`
- `ForceFieldManager.validate_combination`
- `ForceFieldManager.validate_ion`
- `ForceFieldManager.validate_lipid`

### MemPrO

- `apply_orientation_transform`
- `compute_orientation_transform`
- `MemPrO.__init__`
- `MemPrO.build_command`
- `MemPrO.get_oriented_pdb`
- `MemPrO.is_available`
- `MemPrO.parse_results`
- `MemPrO.run`

### Hydration

- `build_hydrate_inp_text`
- `check_packmol_available`
- `detect_hydrogen_status`
- `estimate_cavity_volume`
- `hydrate_cavity`
- `prepare_hydration_job`
- `preview_hydrate_inp`
- `run_custom_packmol`

### Equilibration

- `adjust_protocol_for_missing_protein`
- `AmberEquilibrationManager.__init__`
- `AmberEquilibrationManager.build_group_restraint_block`
- `AmberEquilibrationManager.ensure_inpcrd_box`
- `AmberEquilibrationManager.ensure_prmtop_box`
- `AmberEquilibrationManager.find_system_files`
- `AmberEquilibrationManager.generate_input_file`
- `AmberEquilibrationManager.generate_mdin_file`
- `AmberEquilibrationManager.generate_run_script`
- `AmberEquilibrationManager.get_default_selections`
- `AmberEquilibrationManager.get_default_stage_params`
- `AmberEquilibrationManager.setup_amber_equilibration`
- `EquilibrationStage.replace`
- `EquilibrationStage.to_dict`

- GROMACSEquilibrationManager.__init__
- GROMACSEquilibrationManager.convert_from_amber
- GROMACSEquilibrationManager.find_system_files
- GROMACSEquilibrationManager.generate_com_colvars_config
- GROMACSEquilibrationManager.generate_index_ndx
- GROMACSEquilibrationManager.generate_mdp_file
- GROMACSEquilibrationManager.generate_posres_itp
- GROMACSEquilibrationManager.generate_run_script
- GROMACSEquilibrationManager.get_default_selections
- GROMACSEquilibrationManager.get_default_stage_params
- GROMACSEquilibrationManager.setup_gromacs_equilibration
- NAMDEquilibrationManager.__init__
- NAMDEquilibrationManager.count_all_selections
- NAMDEquilibrationManager.count_selection_atoms
- NAMDEquilibrationManager.find_system_files
- NAMDEquilibrationManager.generate_bilayer_thickness_colvar
- NAMDEquilibrationManager.generate_charmm_gui_config_file
- NAMDEquilibrationManager.generate_colvars_file
- NAMDEquilibrationManager.generate_com_colvars_config
- NAMDEquilibrationManager.generate_config_file
- NAMDEquilibrationManager.generate_restraints_file
- NAMDEquilibrationManager.generate_restraints_file_mda
- NAMDEquilibrationManager.generate_run_script
- NAMDEquilibrationManager.get_default_selections
- NAMDEquilibrationManager.get_default_stage_params
- NAMDEquilibrationManager.load_charmm_gui_template
- NAMDEquilibrationManager.run_equilibration
- NAMDEquilibrationManager.setup_namd_equilibration
- OpenMMEquilibrationManager.__init__
- OpenMMEquilibrationManager.find_system_files
- OpenMMEquilibrationManager.generate_openmm_config
- OpenMMEquilibrationManager.generate_openmm_restraint_files
- OpenMMEquilibrationManager.generate_run_script
- OpenMMEquilibrationManager.get_default_selections
- OpenMMEquilibrationManager.get_default_stage_params
- OpenMMEquilibrationManager.setup_openmm_equilibration
- pdb_has_protein

### Equilibration extras

- cluster_engine_executable
- resolve_cluster_launch_script
- write_cluster_run_script

### NAMD / energy analysis

- EnergyAnalyzer.__init__
- EnergyAnalyzer.get_available_properties
- EnergyAnalyzer.get_statistics
- EnergyAnalyzer.plot_energy
- EnergyAnalyzer.plot_properties
- parse_namd_log
- run_energetic_analysis

### Trajectory analysis

- prepare_structural_inputs
- run_structural_analysis
- TrajectoryAnalyzer.__init__
- TrajectoryAnalyzer.calculate_distances
- TrajectoryAnalyzer.calculate_radius_of_gyration
- TrajectoryAnalyzer.calculate_rmsd
- TrajectoryAnalyzer.calculate_rmsf
- TrajectoryAnalyzer.clear_analysis_cache
- TrajectoryAnalyzer.plot_distances
- TrajectoryAnalyzer.plot_radius_of_gyration
- TrajectoryAnalyzer.plot_rmsd
- TrajectoryAnalyzer.plot_rmsf
- TrajectoryAnalyzer.plot_summary
- TrajectoryAnalyzer.time_array_for_analysis

### Bilayer analysis

- BilayerTrajectoryAnalyzer.__init__
- BilayerTrajectoryAnalyzer.calculate_area_per_lipid
- BilayerTrajectoryAnalyzer.calculate_membrane_thickness
- BilayerTrajectoryAnalyzer.plot_area_per_lipid
- BilayerTrajectoryAnalyzer.plot_membrane_thickness
- BilayerTrajectoryAnalyzer.universe
- run_bilayer_analysis

### OpenMM analysis

- OpenMMLogAnalyzer.__init__
- OpenMMLogAnalyzer.get_statistics
- OpenMMLogAnalyzer.plot_energy
- OpenMMLogAnalyzer.plot_properties
- run_openmm_energetic_analysis

### GROMACS analysis

- GROMACSLogEnergyAnalyzer.__init__
- GROMACSLogEnergyAnalyzer.get_available_properties
- GROMACSLogEnergyAnalyzer.get_statistics

### Amber analysis

- AmberLogEnergyAnalyzer.__init__
- AmberLogEnergyAnalyzer.get_available_properties
- run_amber_energetic_analysis

### FATSLiM / GridMAT

- run_fatslim_apl
- run_gridmat_md_apl

### Cluster helpers

- connect_ssh

- `rsync_from_remote`
- `rsync_to_remote`
- `run_remote`

## Structure Manager

### Structure Manager Module

Module for loading, inspecting, editing, and saving molecular structures. Uses MDAnalysis for PDB parsing and atom selections.

- Load PDB files from disk or by PDB ID
- Query chains, residues, secondary structure
- Select atoms by criteria (protein, backbone, ligand, chain, range, etc.)
- Auto-detect molecules (protein, water, ligands)
- Edit: rename chains/residues, renumber residues, delete atoms
- Save modified structures as PDB

#### Import

```
from gatewizard.core.structure_manager import StructureManager, Selection
from gatewizard import StructureManager  # Also available at top level
```

#### Class: StructureManager

Main API class for programmatic structure viewing and editing.

##### Constructor

```
StructureManager()
```

Creates a new StructureManager instance. No structure is loaded initially.

##### Example 1: Create a StructureManager and inspect defaults

```
from gatewizard.core.structure_manager import StructureManager

viewer = StructureManager()
print(f"StructureManager created: {viewer}")
print(f"Structure loaded: {viewer.structure is not None}")
```

---

#### Loading Methods

##### load_structure

```
viewer.load_structure(filepath: str) -> dict
```

Load a PDB file from disk.

**Parameters:** - `filepath` (str): Path to the PDB file.

**Returns:** Dictionary with keys `n_atoms`, `n_residues`, `n_chains`, `n_bonds`, `title`.

**Raises:** `StructureError` if file not found or parse fails.

**Example 2: Load a PDB structure and print summary info**

```
import os
import tempfile
from gatewizard.core.structure_manager import StructureManager

viewer = StructureManager()

# Create a minimal PDB for testing
pdb_content = """\
HEADER    TEST PROTEIN
ATOM      1  N   ALA A   1       1.000   2.000   3.000  1.00  0.00           N
ATOM      2  CA  ALA A   1       2.000   2.000   3.000  1.00  0.00           C
ATOM      3  C   ALA A   1       3.000   2.000   3.000  1.00  0.00           C
ATOM      4  O   ALA A   1       3.500   3.000   3.000  1.00  0.00           O
ATOM      5  N   GLY A   2       4.000   1.000   3.000  1.00  0.00           N
ATOM      6  CA  GLY A   2       5.000   1.000   3.000  1.00  0.00           C
ATOM      7  C   GLY A   2       6.000   1.000   3.000  1.00  0.00           C
ATOM      8  O   GLY A   2       6.500   2.000   3.000  1.00  0.00           O
END
"""

with tempfile.NamedTemporaryFile(suffix=".pdb", mode="w", delete=False) as f:
    f.write(pdb_content)
    tmp_path = f.name

try:
    info = viewer.load_structure(tmp_path)
    print(f"Loaded: {info['n_atoms']} atoms, {info['n_residues']} residues")
    print(f"Chains: {info['n_chains']}, Bonds: {info['n_bonds']}")
    print(f"Title: {info.get('title', 'N/A')}")
finally:
    try:
        os.unlink(tmp_path)
    except OSError:
        pass
```

---

#### load_from_pdb_id

```
viewer.load_from_pdb_id(pdb_id: str, output_dir: str = ".") -> dict
```

Download a PDB from RCSB and load it.

**Parameters:** - `pdb_id` (str): 4-character PDB identifier (e.g. `"1CRN"`). - `output_dir` (str): Directory to save the downloaded file.

**Returns:** Same dict as `load_structure`.

---

### Query Methods

#### get_structure_info

```
viewer.get_structure_info() -> dict
```

Return summary of the loaded structure.

**Returns:** Dictionary with `n_atoms`, `n_residues`, `n_chains`, `n_bonds`, `title`.

---

#### get_chains

```
viewer.get_chains() -> dict
```

Return chain IDs and their residue counts.

**Returns:** `{"A": 150, "B": 120, ...}`

---

#### get_residues

```
viewer.get_residues(chain_id: str = None) -> list
```

List residues, optionally filtered by chain.

**Parameters:** - `chain_id` (str, optional): Filter by chain. If `None`, returns all residues.

**Returns:** List of dicts with keys `name`, `seq_id`, `chain_id`, `n_atoms`, `ss`.

---

#### get_secondary_structure_summary

```
viewer.get_secondary_structure_summary() -> dict
```

Count residues by secondary structure type.

**Returns:** `{"H": 45, "E": 30, "C": 75, ...}`

##### Example 3: Query chains, residues, secondary structure

```
import os
import tempfile
from gatewizard.core.structure_manager import StructureManager

viewer = StructureManager()

pdb_content = """\
ATOM      1  N   ALA A   1       1.000   2.000   3.000  1.00  0.00           N
ATOM      2  CA  ALA A   1       2.000   2.000   3.000  1.00  0.00           C
ATOM      3  C   ALA A   1       3.000   2.000   3.000  1.00  0.00           C
ATOM      4  O   ALA A   1       3.500   3.000   3.000  1.00  0.00           O
ATOM      5  N   GLY A   2       4.000   1.000   3.000  1.00  0.00           N
ATOM      6  CA  GLY A   2       5.000   1.000   3.000  1.00  0.00           C
ATOM      7  C   GLY A   2       6.000   1.000   3.000  1.00  0.00           C
ATOM      8  O   GLY A   2       6.500   2.000   3.000  1.00  0.00           O
ATOM      9  N   ALA B   1      11.000   2.000   3.000  1.00  0.00           N
ATOM     10  CA  ALA B   1      12.000   2.000   3.000  1.00  0.00           C
END
"""

with tempfile.NamedTemporaryFile(suffix=".pdb", mode="w", delete=False) as f:
    f.write(pdb_content)
    tmp_path = f.name

try:
    viewer.load_structure(tmp_path)
    chains = viewer.get_chains()
    print(f"Chains: {chains}")

    residues = viewer.get_residues(chain_id="A")
    print(f"Chain A residues: {len(residues)}")
    for r in residues:
        print(f"  {r['name']} {r['seq_id']} ({r['n_atoms']} atoms, SS: {r['ss']})")

    ss = viewer.get_secondary_structure_summary()
    print(f"SS summary: {ss}")
finally:
    try:
        os.unlink(tmp_path)
    except OSError:
        pass
```

---

### Selection Methods

#### select_atoms

```
viewer.select_atoms(selection_string: str) -> list[int]
```

Select atom indices using MDAnalysis selection language (`"protein"`, `"backbone"`, `"resname LIG"`, `"around 5 protein"`).

**Parameters:** - `selection_string` (str): MDAnalysis selection expression.

**Returns:** Atom indices into the current structure.

See Example 4 for the related convenience API.

#### select_by_criteria

```
viewer.select_by_criteria(criteria: str, extra: str = "") -> list
```

Select atoms using predefined criteria.

**Parameters:** - `criteria` (str): One of `"All"`, `"Protein"`, `"Backbone"`, `"Sidechain"`, `"Water"`, `"Ligand"`, `"Chain..."`, `"Residue range..."`. - `extra` (str): Required for `"Chain..."` (chain ID) and `"Residue range..."` (e.g. `"A:10-50"`).

**Returns:** List of atom indices.

**Example 4: Select atoms by criteria (protein, backbone, ligand, etc.)**

```
import os
import tempfile
from gatewizard.core.structure_manager import StructureManager

viewer = StructureManager()

pdb_content = """\
ATOM      1  N   ALA A   1       1.000   2.000   3.000  1.00  0.00           N
ATOM      2  CA  ALA A   1       2.000   2.000   3.000  1.00  0.00           C
ATOM      3  C   ALA A   1       3.000   2.000   3.000  1.00  0.00           C
ATOM      4  O   ALA A   1       3.500   3.000   3.000  1.00  0.00           O
ATOM      5  CB  ALA A   1       2.000   3.000   4.000  1.00  0.00           C
HETATM    6  O   HOH A 100      20.000  20.000  20.000  1.00  0.00           O
HETATM    7  C1  LIG A 200      30.000  30.000  30.000  1.00  0.00           C
END
"""

with tempfile.NamedTemporaryFile(suffix=".pdb", mode="w", delete=False) as f:
    f.write(pdb_content)
    tmp_path = f.name

try:
    viewer.load_structure(tmp_path)

    protein = viewer.select_by_criteria("Protein")
    print(f"Protein atoms: {len(protein)}")

    backbone = viewer.select_by_criteria("Backbone")
    print(f"Backbone atoms: {len(backbone)}")

    sidechain = viewer.select_by_criteria("Sidechain")
    print(f"Sidechain atoms: {len(sidechain)}")

    water = viewer.select_by_criteria("Water")
    print(f"Water atoms: {len(water)}")

    ligand = viewer.select_by_criteria("Ligand")
    print(f"Ligand atoms: {len(ligand)}")
finally:
    try:
        os.unlink(tmp_path)
    except OSError:
        pass
```

**Example 5: Select by chain and residue range**

```python
import os
import tempfile
from gatewizard.core.structure_manager import StructureManager

viewer = StructureManager()

pdb_content = """\
ATOM      1  N   ALA A   1       1.000   2.000   3.000  1.00  0.00           N
ATOM      2  CA  ALA A   1       2.000   2.000   3.000  1.00  0.00           C
ATOM      3  N   GLY A   2       4.000   1.000   3.000  1.00  0.00           N
ATOM      4  CA  GLY A   2       5.000   1.000   3.000  1.00  0.00           C
ATOM      5  N   ALA B   1      11.000   2.000   3.000  1.00  0.00           N
ATOM      6  CA  ALA B   1      12.000   2.000   3.000  1.00  0.00           C
END
"""

with tempfile.NamedTemporaryFile(suffix=".pdb", mode="w", delete=False) as f:
    f.write(pdb_content)
    tmp_path = f.name

try:
    viewer.load_structure(tmp_path)

    chain_a = viewer.select_by_criteria("Chain...", "A")
    print(f"Chain A atoms: {len(chain_a)}")

    rng = viewer.select_by_criteria("Residue range...", "A:1-2")
    print(f"A:1-2 atoms: {len(rng)}")

    all_atoms = viewer.select_by_criteria("All")
    print(f"All atoms: {len(all_atoms)}")
finally:
    try:
        os.unlink(tmp_path)
    except OSError:
        pass
```

---

#### auto_detect_molecules

```python
viewer.auto_detect_molecules() -> list[Selection]
```

Automatically group atoms into protein, water, and individual ligand selections.

**Returns:** List of `Selection` objects with sensible defaults (representation, color scheme).

##### Example 6: Auto-detect molecules (protein, water, ligands)

```
import os
import tempfile
from gatewizard.core.structure_manager import StructureManager

viewer = StructureManager()

pdb_content = """\
ATOM      1  N   ALA A   1       1.000   2.000   3.000  1.00  0.00           N
ATOM      2  CA  ALA A   1       2.000   2.000   3.000  1.00  0.00           C
ATOM      3  C   ALA A   1       3.000   2.000   3.000  1.00  0.00           C
HETATM    4  O   HOH A 100      20.000  20.000  20.000  1.00  0.00           O
HETATM    5  C1  LIG A 200      30.000  30.000  30.000  1.00  0.00           C
HETATM    6  C2  LIG A 200      31.000  30.000  30.000  1.00  0.00           C
END
"""

with tempfile.NamedTemporaryFile(suffix=".pdb", mode="w", delete=False) as f:
    f.write(pdb_content)
    tmp_path = f.name

try:
    viewer.load_structure(tmp_path)
    selections = viewer.auto_detect_molecules()
    for sel in selections:
        print(
            f"Selection '{sel.name}': {len(sel.atom_indices)} atoms, "
            f"rep={sel.representation}, cs={sel.color_scheme}"
        )
finally:
    try:
        os.unlink(tmp_path)
    except OSError:
        pass
```

---

### Edit Methods

#### rename_chain

```
viewer.rename_chain(old_chain: str, new_chain: str) -> int
```

Rename all atoms in a chain.

**Parameters:** - `old_chain` (str): Current chain ID. - `new_chain` (str): New chain ID (1 character).

**Returns:** Number of atoms renamed.

---

#### rename_residues

```
viewer.rename_residues(chain_id: str, start: int, end: int, new_name: str) -> int
```

Rename residues in a range.

**Parameters:** - `chain_id` (str): Chain to modify. - `start`, `end` (int): Residue number range (inclusive). - `new_name` (str): New residue name.

**Returns:** Number of atoms renamed.

##### Example 7: Rename chains and residues

```python
import os
import tempfile
from gatewizard.core.structure_manager import StructureManager

viewer = StructureManager()

pdb_content = """\
ATOM      1  N   ALA A   1       1.000   2.000   3.000  1.00  0.00           N
ATOM      2  CA  ALA A   1       2.000   2.000   3.000  1.00  0.00           C
ATOM      3  N   GLY A   2       4.000   1.000   3.000  1.00  0.00           N
ATOM      4  CA  GLY A   2       5.000   1.000   3.000  1.00  0.00           C
END
"""

with tempfile.NamedTemporaryFile(suffix=".pdb", mode="w", delete=False) as f:
    f.write(pdb_content)
    tmp_path = f.name

try:
    viewer.load_structure(tmp_path)

    # Rename chain A -> X
    count = viewer.rename_chain("A", "X")
    print(f"Renamed {count} atoms from chain A to X")
    chains = viewer.get_chains()
    print(f"Chains after rename: {chains}")

    # Rename residue
    count = viewer.rename_residues("X", 1, 1, "MET")
    print(f"Renamed {count} atoms to MET")
    residues = viewer.get_residues("X")
    for r in residues:
        print(f"  {r['name']} {r['seq_id']}")
finally:
    try:
        os.unlink(tmp_path)
    except OSError:
        pass
```

---

#### renumber_residues

```python
viewer.renumber_residues(chain_id: str, start: int, end: int, new_start: int) -> int
```

Renumber residues sequentially from `new_start`.

**Returns:** Number of atoms renumbered.

##### Example 8: Renumber residues

```python
import os
import tempfile
from gatewizard.core.structure_manager import StructureManager

viewer = StructureManager()

pdb_content = """\
ATOM      1  N   ALA A  10       1.000   2.000   3.000  1.00  0.00           N
ATOM      2  CA  ALA A  10       2.000   2.000   3.000  1.00  0.00           C
ATOM      3  N   GLY A  11       4.000   1.000   3.000  1.00  0.00           N
ATOM      4  CA  GLY A  11       5.000   1.000   3.000  1.00  0.00           C
ATOM      5  N   ALA A  12       7.000   1.000   3.000  1.00  0.00           N
ATOM      6  CA  ALA A  12       8.000   1.000   3.000  1.00  0.00           C
END
"""

with tempfile.NamedTemporaryFile(suffix=".pdb", mode="w", delete=False) as f:
    f.write(pdb_content)
    tmp_path = f.name

try:
    viewer.load_structure(tmp_path)

    # Renumber residues 10-12 to start at 1
    count = viewer.renumber_residues("A", 10, 12, new_start=1)
    print(f"Renumbered {count} atoms")
    residues = viewer.get_residues("A")
    for r in residues:
        print(f"  {r['name']} {r['seq_id']}")
finally:
    try:
        os.unlink(tmp_path)
    except OSError:
        pass
```

---

#### rename_chain_by_indices

```python
viewer.rename_chain_by_indices(indices: list[int], new_chain: str) -> int
```

Rename the chain of only the selected atoms. Sibling atoms that share a residue id but are not in `indices` are left unchanged.

#### rename_residues_by_indices

```python
viewer.rename_residues_by_indices(indices: list[int], new_name: str) -> int
```

Rename residues for the selected atoms only (does not rename a sibling residue with the same resid).

#### renumber_residues_by_indices

```python
viewer.renumber_residues_by_indices(indices: list[int], new_start: int) -> int
```

Renumber residues that contain the selected atoms, starting at `new_start`.

#### delete_atoms

```python
viewer.delete_atoms(indices: list) -> int
```

Remove atoms by index. Rebuilds residues, chains, and bonds.

**Returns:** Number of atoms removed.

#### save_pdb

```
viewer.save_pdb(filepath: str) -> str
```

Write the current structure to a PDB file.

**Returns:** Absolute path of the saved file.

**Raises:** `StructureError` if no structure is loaded.

##### Example 9: Delete atoms and save modified PDB

```
import os
import tempfile
from gatewizard.core.structure_manager import StructureManager

viewer = StructureManager()

pdb_content = """\
ATOM      1  N   ALA A   1       1.000   2.000   3.000  1.00  0.00           N
ATOM      2  CA  ALA A   1       2.000   2.000   3.000  1.00  0.00           C
ATOM      3  N   GLY A   2       4.000   1.000   3.000  1.00  0.00           N
ATOM      4  CA  GLY A   2       5.000   1.000   3.000  1.00  0.00           C
HETATM    5  O   HOH A 100      20.000  20.000  20.000  1.00  0.00           O
END
"""

with tempfile.NamedTemporaryFile(suffix=".pdb", mode="w", delete=False) as f:
    f.write(pdb_content)
    tmp_path = f.name

out_path = tmp_path + "_out.pdb"

try:
    viewer.load_structure(tmp_path)

    # Delete water atoms
    water_idx = viewer.select_by_criteria("Water")
    print(f"Deleting {len(water_idx)} water atoms")
    removed = viewer.delete_atoms(water_idx)
    print(f"Removed {removed} atoms")

    info = viewer.get_structure_info()
    print(f"After deletion: {info['n_atoms']} atoms")

    # Save modified structure
    saved = viewer.save_pdb(out_path)
    print(f"Saved to: {os.path.basename(saved)}")
finally:
    try:
        os.unlink(tmp_path)
    except OSError:
        pass
    if os.path.exists(out_path):
        try:
            os.unlink(out_path)
        except OSError:
            pass
```

### Coordinate Transformations

#### rotate_atoms

```
viewer.rotate_atoms(angle_degrees: float, axis: str,
                    indices: list = None, center: str = 'selection') -> int
```

Rotate atoms around a Cartesian axis.

**Parameters:**

| Parameter | Type | Default | Description |
|---|---|---|---|
| `angle_degrees` | float | — | Rotation angle in degrees |
| `axis` | str | — | `'x'`, `'y'`, or `'z'` |
| `indices` | list[int] | `None` | Atom indices to rotate. `None` = all atoms |
| `center` | str | `'selection'` | `'selection'` rotates around the centroid of the affected atoms; `'origin'` rotates around (0, 0, 0) |

**Returns:** Number of atoms rotated.

---

#### translate_atoms

```
viewer.translate_atoms(displacement: list, indices: list = None) -> int
```

Translate atoms by a displacement vector.

**Parameters:**

| Parameter | Type | Default | Description |
|---|---|---|---|
| `displacement` | list[float] | — | `[dx, dy, dz]` in angstroms |
| `indices` | list[int] | `None` | Atom indices to move. `None` = all atoms |

**Returns:** Number of atoms translated.

---

#### center_atoms

```
viewer.center_atoms(indices: list = None) -> numpy.ndarray
```

Move the entire structure so that the centroid of the selected atoms is at the origin. The shift is always applied to **all** atoms.

**Parameters:**

| Parameter | Type | Default | Description |
|---|---|---|---|
| `indices` | list[int] | `None` | Atoms whose centroid defines the shift. `None` = all atoms |

**Returns:** The displacement applied (old centroid position) as a NumPy array.

---

#### align_to_axis

```
viewer.align_to_axis(primary_indices: list, target_axis: str = 'z',
                     secondary_indices: list = None,
                     secondary_axis: str = None,
                     apply_to: list = None) -> int
```

Align a selection's principal direction to a reference axis using SVD. The first singular vector fitted through the primary atom positions is rotated onto the target axis. An optional secondary alignment adds a rotation around the primary axis so that the centroid of the secondary selection projects onto the secondary axis.

**Parameters:**

| Parameter | Type | Default | Description |
|---|---|---|---|
| `primary_indices` | list[int] | — | Atoms whose principal direction defines the alignment vector |
| `target_axis` | str | `'z'` | `'x'`, `'y'`, or `'z'` |
| `secondary_indices` | list[int] | `None` | Atoms for the secondary axis alignment |
| `secondary_axis` | str | `None` | `'x'`, `'y'`, or `'z'`; must differ from `target_axis` |
| `apply_to` | list[int] | `None` | Atom indices to actually transform. `None` = all atoms |

**Returns:** Number of atoms transformed.

> **Use case: ion channel alignment**
>
> For channels it is common to select the filter ions (`name K`) and align them to the Z-axis, then add the selectivity filter residues as secondary alignment to the X-axis. This orients the pore along Z with a known reference direction.

**Example 12: Rotate atoms around an axis**

```python
import os
import tempfile
import numpy as np
from gatewizard.core.structure_manager import StructureManager

viewer = StructureManager()

# A small chain along the X-axis so rotations are easy to verify
pdb_content = """\
ATOM      1  N   ALA A   1       0.000   0.000   0.000  1.00  0.00           N
ATOM      2  CA  ALA A   1       1.500   0.000   0.000  1.00  0.00           C
ATOM      3  C   ALA A   1       3.000   0.000   0.000  1.00  0.00           C
ATOM      4  O   ALA A   1       3.500   1.000   0.000  1.00  0.00           O
ATOM      5  N   ALA A   2       4.500   0.000   0.000  1.00  0.00           N
ATOM      6  CA  ALA A   2       6.000   0.000   0.000  1.00  0.00           C
ATOM      7  C   ALA A   2       7.500   0.000   0.000  1.00  0.00           C
ATOM      8  O   ALA A   2       8.000   1.000   0.000  1.00  0.00           O
END
"""

with tempfile.NamedTemporaryFile(suffix=".pdb", mode="w", delete=False) as f:
    f.write(pdb_content)
    tmp_path = f.name

try:
    viewer.load_structure(tmp_path)
    atoms = viewer.structure.atoms
    print(f"Loaded {len(atoms)} atoms")

    # --- Rotate all atoms 90° around Z (X → Y) ---
    before = np.array([a.coord.copy() for a in atoms])
    n = viewer.rotate_atoms(90, "z")
    after = np.array([a.coord for a in atoms])
    print(f"\nRotated {n} atoms 90° around Z:")
    print(
        f"  Atom 1 before: ({before[0][0]:.1f}, {before[0][1]:.1f}, {before[0][2]:.1f})"
    )
    print(f"  Atom 1 after:  ({after[0][0]:.1f}, {after[0][1]:.1f}, {after[0][2]:.1f})")

    # --- Rotate only first residue 45° around X ---
    viewer.load_structure(tmp_path)  # reload
    res1 = [i for i, a in enumerate(viewer.structure.atoms) if a.res_id == 1]
    n = viewer.rotate_atoms(45, "x", indices=res1)
    print(f"\nRotated {n} atoms (residue 1) 45° around X")

    # --- Rotate around origin instead of selection centroid ---
    viewer.load_structure(tmp_path)
    n = viewer.rotate_atoms(180, "y", center="origin")
    after180 = np.array([a.coord for a in viewer.structure.atoms])
    print(f"\nRotated {n} atoms 180° around Y (origin):")
    print(
        f"  Atom 1: ({after180[0][0]:.1f}, {after180[0][1]:.1f}, {after180[0][2]:.1f})"
    )
finally:
    try:
        os.unlink(tmp_path)
    except OSError:
        pass
```

**Expected output:**

```
Loaded 8 atoms

Rotated 8 atoms 90° around Z:
  Atom 1 before: (0.0, 0.0, 0.0)
  Atom 1 after:  (4.5, -4.0, 0.0)

Rotated 4 atoms (residue 1) 45° around X

Rotated 8 atoms 180° around Y (origin):
  Atom 1: (0.0, 0.0, 0.0)
```

**Example 13: Translate and center structure**

```python
import os
import tempfile
import numpy as np
from gatewizard.core.structure_manager import StructureManager

viewer = StructureManager()

# Structure offset from origin so centering is visible
pdb_content = """\
ATOM      1  N   ALA A   1      10.000  20.000  30.000  1.00  0.00           N
ATOM      2  CA  ALA A   1      11.500  20.000  30.000  1.00  0.00           C
ATOM      3  C   ALA A   1      13.000  20.000  30.000  1.00  0.00           C
ATOM      4  O   ALA A   1      13.500  21.000  30.000  1.00  0.00           O
ATOM      5  N   ALA A   2      14.500  20.000  30.000  1.00  0.00           N
ATOM      6  CA  ALA A   2      16.000  20.000  30.000  1.00  0.00           C
ATOM      7  C   ALA A   2      17.500  20.000  30.000  1.00  0.00           C
ATOM      8  O   ALA A   2      18.000  21.000  30.000  1.00  0.00           O
END
"""

with tempfile.NamedTemporaryFile(suffix=".pdb", mode="w", delete=False) as f:
    f.write(pdb_content)
    tmp_path = f.name

try:
    viewer.load_structure(tmp_path)
    atoms = viewer.structure.atoms

    # --- Translate all atoms by (5, -10, 0) Å ---
    before = np.array([a.coord.copy() for a in atoms])
    n = viewer.translate_atoms([5.0, -10.0, 0.0])
    after = np.array([a.coord for a in atoms])
    print(f"Translated {n} atoms by (5, -10, 0) Å:")
    print(
        f"  Atom 1 before: ({before[0][0]:.1f}, {before[0][1]:.1f}, {before[0][2]:.1f})"
    )
    print(f"  Atom 1 after:  ({after[0][0]:.1f}, {after[0][1]:.1f}, {after[0][2]:.1f})")

    # --- Translate only residue 2 ---
    viewer.load_structure(tmp_path)
    res2 = [i for i, a in enumerate(viewer.structure.atoms) if a.res_id == 2]
    n = viewer.translate_atoms([0.0, 0.0, 5.0], indices=res2)
    print(f"\nTranslated {n} atoms (residue 2) by (0, 0, 5) Å")

    # --- Center structure at origin ---
    viewer.load_structure(tmp_path)
    before_center = np.array([a.coord for a in viewer.structure.atoms]).mean(axis=0)
    shift = viewer.center_atoms()
    after_center = np.array([a.coord for a in viewer.structure.atoms]).mean(axis=0)
    print(f"\nCentered structure:")
    print(
        f"  Centroid before: ({before_center[0]:.1f}, {before_center[1]:.1f}, {before_center[2]:.1f})"
    )
    print(f"  Shift applied:   ({shift[0]:.1f}, {shift[1]:.1f}, {shift[2]:.1f})")
    print(
        f"  Centroid after:  ({after_center[0]:.4f}, {after_center[1]:.4f}, {after_center[2]:.4f})"
    )

    # --- Center using a subset as reference, shift applied to all ---
    viewer.load_structure(tmp_path)
    res1 = [i for i, a in enumerate(viewer.structure.atoms) if a.res_id == 1]
    shift = viewer.center_atoms(indices=res1)
    res1_center = np.array([viewer.structure.atoms[i].coord for i in res1]).mean(axis=0)
    print(f"\nCentered on residue 1:")
    print(
        f"  Residue 1 centroid: ({res1_center[0]:.4f}, {res1_center[1]:.4f}, {res1_center[2]:.4f})"
    )
finally:
    try:
        os.unlink(tmp_path)
    except OSError:
        pass
```

**Expected output:**

```
Translated 8 atoms by (5, -10, 0) Å:
  Atom 1 before: (10.0, 20.0, 30.0)
  Atom 1 after:  (15.0, 10.0, 30.0)

Translated 4 atoms (residue 2) by (0, 0, 5) Å

Centered structure:
  Centroid before: (14.2, 20.2, 30.0)
  Shift applied:   (14.2, 20.2, 30.0)
  Centroid after:  (0.0000, 0.0000, 0.0000)

Centered on residue 1:
  Residue 1 centroid: (0.0000, 0.0000, 0.0000)
```

**Example 14: Align structure to an axis**

```
import os
import tempfile
import numpy as np
from gatewizard.core.structure_manager import StructureManager

viewer = StructureManager()

# Structure extended along the X-axis (principal axis ≈ X)
# We will align it so the principal axis points along Z
pdb_content = """\
ATOM      1  N   ALA A   1       0.000   0.200   0.100  1.00  0.00           N
ATOM      2  CA  ALA A   1       1.500   0.100  -0.050  1.00  0.00           C
ATOM      3  C   ALA A   1       3.000  -0.100   0.200  1.00  0.00           C
ATOM      4  O   ALA A   1       3.500   0.900   0.100  1.00  0.00           O
ATOM      5  N   ALA A   2       4.500   0.050  -0.100  1.00  0.00           N
ATOM      6  CA  ALA A   2       6.000  -0.200   0.050  1.00  0.00           C
ATOM      7  C   ALA A   2       7.500   0.100   0.150  1.00  0.00           C
ATOM      8  O   ALA A   2       8.000   1.100  -0.050  1.00  0.00           O
ATOM      9  N   ALA A   3       9.000  -0.050   0.000  1.00  0.00           N
ATOM     10  CA  ALA A   3      10.500   0.150   0.100  1.00  0.00           C
ATOM     11  C   ALA A   3      12.000  -0.100  -0.050  1.00  0.00           C
ATOM     12  O   ALA A   3      12.500   0.800   0.200  1.00  0.00           O
END
"""

with tempfile.NamedTemporaryFile(suffix=".pdb", mode="w", delete=False) as f:
    f.write(pdb_content)
    tmp_path = f.name

try:
    viewer.load_structure(tmp_path)
    atoms = viewer.structure.atoms

    # Before alignment: measure span along each axis
    coords_before = np.array([a.coord for a in atoms])
    spans_before = coords_before.max(axis=0) - coords_before.min(axis=0)
    print("Before alignment (axis spans):")
    print(f"  X: {spans_before[0]:.2f} Å")
    print(f"  Y: {spans_before[1]:.2f} Å")
    print(f"  Z: {spans_before[2]:.2f} Å")
    print(f"  Principal axis: X (largest span)")

    # --- Align all atoms to Z-axis ---
    all_idx = list(range(len(atoms)))
    n = viewer.align_to_axis(all_idx, target_axis="z")
    coords_after = np.array([a.coord for a in atoms])
    spans_after = coords_after.max(axis=0) - coords_after.min(axis=0)
    print(f"\nAligned {n} atoms to Z-axis (axis spans):")
    print(f"  X: {spans_after[0]:.2f} Å")
    print(f"  Y: {spans_after[1]:.2f} Å")
    print(f"  Z: {spans_after[2]:.2f} Å")
    print(f"  Principal axis: Z (largest span)")

    # --- Align using only backbone CA atoms, transform all ---
    viewer.load_structure(tmp_path)
    ca_idx = [i for i, a in enumerate(viewer.structure.atoms) if a.name == "CA"]
    n = viewer.align_to_axis(ca_idx, target_axis="y")
    coords_ca = np.array([a.coord for a in viewer.structure.atoms])
    spans_ca = coords_ca.max(axis=0) - coords_ca.min(axis=0)
    print(f"\nAligned CA atoms to Y-axis, transformed all {n} atoms:")
    print(f"  X: {spans_ca[0]:.2f} Å")
    print(f"  Y: {spans_ca[1]:.2f} Å")
    print(f"  Z: {spans_ca[2]:.2f} Å")
finally:
    try:
        os.unlink(tmp_path)
    except OSError:
        pass
```

**Expected output:**

```
Before alignment (axis spans):
  X: 12.50 Å
  Y: 1.30 Å
  Z: 0.30 Å
  Principal axis: X (largest span)

Aligned 12 atoms to Z-axis (axis spans):
  X: 0.30 Å
  Y: 1.28 Å
  Z: 12.51 Å
  Principal axis: Z (largest span)

Aligned CA atoms to Y-axis, transformed all 12 atoms:
  X: 1.29 Å
  Y: 12.50 Å
  Z: 0.40 Å
```

**Example 15: Align with primary and secondary axes**

```
import os
import tempfile
import numpy as np
from gatewizard.core.structure_manager import StructureManager

viewer = StructureManager()

# A structure with two chains:
#   Chain A runs along the X-axis (the "channel pore")
#   Chain B has an atom offset in the Y direction (reference for secondary axis)
# This mimics aligning a channel pore to Z with a pore-lining residue on X.
pdb_content = """\
ATOM      1  N   ALA A   1       0.000   0.000   0.000  1.00  0.00           N
ATOM      2  CA  ALA A   1       2.000   0.200   0.100  1.00  0.00           C
ATOM      3  C   ALA A   1       4.000  -0.100   0.050  1.00  0.00           C
ATOM      4  O   ALA A   1       6.000   0.100  -0.100  1.00  0.00           O
ATOM      5  N   ALA A   2       8.000  -0.050   0.200  1.00  0.00           N
ATOM      6  CA  ALA A   2      10.000   0.150  -0.050  1.00  0.00           C
ATOM      7  C   ALA A   2      12.000   0.000   0.100  1.00  0.00           C
ATOM      8  O   ALA A   2      14.000  -0.200   0.000  1.00  0.00           O
ATOM      9  N   GLY B   1       5.000   4.000   0.500  1.00  0.00           N
ATOM     10  CA  GLY B   1       7.000   4.200   0.300  1.00  0.00           C
END
"""

with tempfile.NamedTemporaryFile(suffix=".pdb", mode="w", delete=False) as f:
    f.write(pdb_content)
    tmp_path = f.name

try:
    viewer.load_structure(tmp_path)
    atoms = viewer.structure.atoms

    # Primary: chain A backbone → align to Z-axis
    # Secondary: chain B atoms → align to X-axis
    chainA = [i for i, a in enumerate(atoms) if a.chain_id == "A"]
    chainB = [i for i, a in enumerate(atoms) if a.chain_id == "B"]

    print("Before alignment:")
    coordsA = np.array([atoms[i].coord for i in chainA])
    coordsB = np.array([atoms[i].coord for i in chainB])
    spansA = coordsA.max(axis=0) - coordsA.min(axis=0)
    print(f"  Chain A spans: X={spansA[0]:.1f}, Y={spansA[1]:.1f}, Z={spansA[2]:.1f}")
    print(
        f"  Chain B centroid: ({coordsB.mean(0)[0]:.1f}, "
        f"{coordsB.mean(0)[1]:.1f}, {coordsB.mean(0)[2]:.1f})"
    )

    # Align with primary + secondary axes
    n = viewer.align_to_axis(
        primary_indices=chainA,
        target_axis="z",
        secondary_indices=chainB,
        secondary_axis="x",
    )

    print(f"\nAligned {n} atoms (primary → Z, secondary → X):")
    coordsA2 = np.array([atoms[i].coord for i in chainA])
    coordsB2 = np.array([atoms[i].coord for i in chainB])
    spansA2 = coordsA2.max(axis=0) - coordsA2.min(axis=0)
    print(
        f"  Chain A spans: X={spansA2[0]:.2f}, Y={spansA2[1]:.2f}, Z={spansA2[2]:.2f}"
    )
    print(
        f"  Chain B centroid: ({coordsB2.mean(0)[0]:.2f}, "
        f"{coordsB2.mean(0)[1]:.2f}, {coordsB2.mean(0)[2]:.2f})"
    )
    print(f"  Chain A now mostly along Z (Z span >> X, Y).")
    print(f"  Chain B centroid now has largest offset along X.")

    # --- Align only chain A, keep chain B fixed ---
    viewer.load_structure(tmp_path)
    atoms = viewer.structure.atoms
    chainA = [i for i, a in enumerate(atoms) if a.chain_id == "A"]
    chainB_fixed = [i for i, a in enumerate(atoms) if a.chain_id == "B"]
```

```
    coordsB_before = np.array([atoms[i].coord.copy() for i in chainB_fixed])

    n = viewer.align_to_axis(
        primary_indices=chainA,
        target_axis="z",
        apply_to=chainA,  # only move chain A
    )
    coordsB_after = np.array([atoms[i].coord for i in chainB_fixed])
    print(f"\nAligned only chain A ({n} atoms), chain B unchanged:")
    print(f"  Chain B moved: {not np.allclose(coordsB_before, coordsB_after)}")
finally:
    try:
        os.unlink(tmp_path)
    except OSError:
        pass
```

**Expected output:**

```
Before alignment:
  Chain A spans: X=14.0, Y=0.4, Z=0.3
  Chain B centroid: (6.0, 4.1, 0.4)

Aligned 10 atoms (primary → Z, secondary → X):
  Chain A spans: X=0.30, Y=0.31, Z=14.00
  Chain B centroid: (10.07, 0.83, -0.73)
  Chain A now mostly along Z (Z span >> X, Y).
  Chain B centroid now has largest offset along X.

Aligned only chain A (8 atoms), chain B unchanged:
  Chain B moved: False
```

---

### Secondary Structure Assignment

#### apply_mempro_orientation

```
viewer.apply_mempro_orientation(oriented_pdb: str) -> int
```

Apply a MemPrO rigid-body orientation to the loaded structure. The oriented PDB supplies the target protein pose; every atom in the current structure (protein, lipids, ligands, water) receives the same transform.

**Parameters:** - `oriented_pdb` (str): Path to a MemPrO `oriented_rank_*.pdb` file.

**Returns:** Number of atoms transformed.

See also `compute_orientation_transform` and MemPrO Example 12.

#### assign_secondary_structure

```
viewer.assign_secondary_structure(method: str = 'auto') -> dict
```

Reassign secondary structure using a specific method.

**Parameters:** - `method` (str): Assignment method. One of: - `'auto'` – PDB HELIX/SHEET records → psique → heuristic (default, same as initial load). - `'psique'` – Use the psique tool (raises `StructureError` if psique is not available). - `'heuristic'` – CA-angle heuristic (always available). - `'pdb_records'` – Only read HELIX/SHEET from the PDB file (raises `StructureError` if none found).

**Returns:** Updated secondary structure summary `{"H": n, "E": n, ...}`.

**Raises:** `StructureError` if the requested method is not available or fails.

**Example 11: Reassign secondary structure (psique, heuristic, pdb_records)**

Uses backbone atoms from residues 1–20 of PDB 2MVJ (an alpha-helical region) so that `psique` can compute secondary structure from geometry. No HELIX/SHEET header records are included — this forces the `auto` method to fall through to `psique` instead of just reading PDB records.

```
import os
import tempfile
from gatewizard.core.structure_manager import StructureManager, StructureError

viewer = StructureManager()

# Backbone atoms (N, CA, C, O) from residues 1-20 of 2MVJ.
# Contains an alpha-helix (residues 5-19) that psique can detect.
pdb_content = """\
HEADER    BACKBONE OF 2MVJ RESIDUES 1-20
ATOM      1  N   MET A   1      12.463  -9.010  13.613  1.00  1.00           N
ATOM      2  CA  MET A   1      12.443  -7.552  13.317  1.00  1.00           C
ATOM      3  C   MET A   1      11.302  -6.897  14.099  1.00  1.00           C
ATOM      4  O   MET A   1      10.705  -7.511  14.982  1.00  1.00           O
ATOM      9  N   LYS A   2      11.011  -5.641  13.765  1.00  1.00           N
ATOM     10  CA  LYS A   2       9.947  -4.887  14.428  1.00  1.00           C
ATOM     11  C   LYS A   2       8.585  -5.480  14.086  1.00  1.00           C
ATOM     12  O   LYS A   2       7.578  -5.120  14.696  1.00  1.00           O
ATOM     18  N   PHE A   3       8.562  -6.358  13.080  1.00  1.00           N
ATOM     19  CA  PHE A   3       7.325  -6.995  12.609  1.00  1.00           C
ATOM     20  C   PHE A   3       6.359  -5.955  12.027  1.00  1.00           C
ATOM     21  O   PHE A   3       5.390  -6.304  11.351  1.00  1.00           O
ATOM     29  N   TYR A   4       6.650  -4.674  12.270  1.00  1.00           N
ATOM     30  CA  TYR A   4       5.852  -3.563  11.761  1.00  1.00           C
ATOM     31  C   TYR A   4       6.211  -3.358  10.285  1.00  1.00           C
ATOM     32  O   TYR A   4       5.510  -2.668   9.545  1.00  1.00           O
ATOM     41  N   THR A   5       7.302  -3.996   9.874  1.00  1.00           N
ATOM     42  CA  THR A   5       7.765  -3.930   8.497  1.00  1.00           C
ATOM     43  C   THR A   5       6.719  -4.546   7.560  1.00  1.00           C
ATOM     44  O   THR A   5       6.375  -3.963   6.532  1.00  1.00           O
ATOM     48  N   ILE A   6       6.190  -5.718   7.941  1.00  1.00           N
ATOM     49  CA  ILE A   6       5.155  -6.387   7.142  1.00  1.00           C
ATOM     50  C   ILE A   6       3.900  -5.510   7.126  1.00  1.00           C
ATOM     51  O   ILE A   6       3.257  -5.341   6.090  1.00  1.00           O
ATOM     56  N   LYS A   7       3.556  -4.976   8.296  1.00  1.00           N
ATOM     57  CA  LYS A   7       2.368  -4.139   8.440  1.00  1.00           C
ATOM     58  C   LYS A   7       2.417  -2.939   7.502  1.00  1.00           C
ATOM     59  O   LYS A   7       1.446  -2.649   6.802  1.00  1.00           O
ATOM     65  N   LEU A   8       3.537  -2.227   7.513  1.00  1.00           N
ATOM     66  CA  LEU A   8       3.683  -1.040   6.683  1.00  1.00           C
ATOM     67  C   LEU A   8       3.602  -1.416   5.212  1.00  1.00           C
ATOM     68  O   LEU A   8       2.948  -0.751   4.419  1.00  1.00           O
ATOM     73  N   ALA A   9       4.262  -2.505   4.854  1.00  1.00           N
ATOM     74  CA  ALA A   9       4.249  -2.968   3.470  1.00  1.00           C
ATOM     75  C   ALA A   9       2.803  -3.133   2.979  1.00  1.00           C
ATOM     76  O   ALA A   9       2.452  -2.708   1.880  1.00  1.00           O
ATOM     78  N   LYS A  10       1.968  -3.727   3.829  1.00  1.00           N
ATOM     79  CA  LYS A  10       0.551  -3.916   3.507  1.00  1.00           C
ATOM     80  C   LYS A  10      -0.150  -2.562   3.402  1.00  1.00           C
ATOM     81  O   LYS A  10      -0.936  -2.325   2.485  1.00  1.00           O
ATOM     87  N   PHE A  11       0.147  -1.681   4.350  1.00  1.00           N
ATOM     88  CA  PHE A  11      -0.448  -0.340   4.375  1.00  1.00           C
ATOM     89  C   PHE A  11      -0.163   0.373   3.058  1.00  1.00           C
ATOM     90  O   PHE A  11      -1.068   0.881   2.404  1.00  1.00           O
ATOM     98  N   LEU A  12       1.106   0.371   2.669  1.00  1.00           N
ATOM     99  CA  LEU A  12       1.514   0.988   1.405  1.00  1.00           C
ATOM    100  C   LEU A  12       0.887   0.229   0.237  1.00  1.00           C
ATOM    101  O   LEU A  12       0.425   0.834  -0.728  1.00  1.00           O
ATOM    106  N   GLY A  13       0.891  -1.100   0.324  1.00  1.00           N
ATOM    107  CA  GLY A  13       0.333  -1.932  -0.737  1.00  1.00           C
ATOM    108  C   GLY A  13      -1.169  -1.735  -0.857  1.00  1.00           C
ATOM    109  O   GLY A  13      -1.790  -2.148  -1.836  1.00  1.00           O
ATOM    110  N   GLY A  14      -1.757  -1.091   0.145  1.00  1.00           N
ATOM    111  CA  GLY A  14      -3.194  -0.806   0.137  1.00  1.00           C
ATOM    112  C   GLY A  14      -3.420   0.524  -0.569  1.00  1.00           C
ATOM    113  O   GLY A  14      -4.313   0.667  -1.404  1.00  1.00           O
ATOM    114  N   ILE A  15      -2.571   1.493  -0.221  1.00  1.00           N
ATOM    115  CA  ILE A  15      -2.618   2.835  -0.802  1.00  1.00           C
ATOM    116  C   ILE A  15      -2.152   2.813  -2.253  1.00  1.00           C
ATOM    117  O   ILE A  15      -2.687   3.540  -3.088  1.00  1.00           O
ATOM    122  N   VAL A  16      -1.114   2.020  -2.551  1.00  1.00           N
ATOM    123  CA  VAL A  16      -0.570   1.990  -3.912  1.00  1.00           C
ATOM    124  C   VAL A  16      -1.698   1.740  -4.925  1.00  1.00           C
ATOM    125  O   VAL A  16      -1.785   2.408  -5.956  1.00  1.00           O
ATOM    129  N   ARG A  17      -2.576   0.798  -4.587  1.00  1.00           N
```

```
ATOM    130  CA  ARG A  17      -3.730   0.477  -5.416  1.00  1.00           C
ATOM    131  C   ARG A  17      -4.695   1.668  -5.437  1.00  1.00           C
ATOM    132  O   ARG A  17      -5.274   1.987  -6.472  1.00  1.00           O
ATOM    140  N   ALA A  18      -4.880   2.304  -4.276  1.00  1.00           N
ATOM    141  CA  ALA A  18      -5.800   3.442  -4.169  1.00  1.00           C
ATOM    142  C   ALA A  18      -5.396   4.577  -5.114  1.00  1.00           C
ATOM    143  O   ALA A  18      -6.249   5.205  -5.738  1.00  1.00           O
ATOM    145  N   MET A  19      -4.092   4.828  -5.217  1.00  1.00           N
ATOM    146  CA  MET A  19      -3.578   5.878  -6.081  1.00  1.00           C
ATOM    147  C   MET A  19      -3.933   5.591  -7.539  1.00  1.00           C
ATOM    148  O   MET A  19      -4.546   6.409  -8.217  1.00  1.00           O
ATOM    153  N   LEU A  20      -3.542   4.417  -8.002  1.00  1.00           N
ATOM    154  CA  LEU A  20      -3.823   3.999  -9.374  1.00  1.00           C
ATOM    155  C   LEU A  20      -5.327   3.888  -9.559  1.00  1.00           C
ATOM    156  O   LEU A  20      -5.879   4.265 -10.593  1.00  1.00           O
END
"""

with tempfile.NamedTemporaryFile(suffix=".pdb", mode="w", delete=False) as f:
    f.write(pdb_content)
    tmp_path = f.name

try:
    viewer.load_structure(tmp_path)

    # Default SS (assigned automatically at load time)
    print("SS after load (auto):")
    print(f"  {viewer.get_secondary_structure_summary()}")

    # Reassign using the heuristic method
    ss = viewer.assign_secondary_structure("heuristic")
    print(f"SS after heuristic: {ss}")

    # Reassign from PDB HELIX/SHEET records (none in this file)
    try:
        ss = viewer.assign_secondary_structure("pdb_records")
        print(f"SS after pdb_records: {ss}")
    except StructureError as e:
        print(f"pdb_records: {e}")

    # Try psique (may not be installed)
    try:
        ss = viewer.assign_secondary_structure("psique")
        print(f"SS after psique: {ss}")
    except StructureError as e:
        print(f"psique not available: {e}")

    # Auto method (same priority as load)
    ss = viewer.assign_secondary_structure("auto")
    print(f"SS after auto: {ss}")
finally:
    try:
        os.unlink(tmp_path)
    except OSError:
        pass
```

**Expected output:**

```
SS after load (auto):
  {'C': 5, 'H': 15}
SS after heuristic: {'C': 4, 'H': 16}
pdb_records: No HELIX/SHEET records found in PDB file
SS after psique: {'C': 5, 'H': 15}
SS after auto: {'C': 5, 'H': 15}
```

---

### Full Workflow

#### Example 10: Full workflow: load → select → edit → save

```python
import os
import tempfile
from gatewizard.core.structure_manager import StructureManager

viewer = StructureManager()

pdb_content = """\
ATOM      1  N   ALA A  50       1.000   2.000   3.000  1.00  0.00           N
ATOM      2  CA  ALA A  50       2.000   2.000   3.000  1.00  0.00           C
ATOM      3  C   ALA A  50       3.000   2.000   3.000  1.00  0.00           C
ATOM      4  O   ALA A  50       3.500   3.000   3.000  1.00  0.00           O
ATOM      5  N   GLY A  51       4.000   1.000   3.000  1.00  0.00           N
ATOM      6  CA  GLY A  51       5.000   1.000   3.000  1.00  0.00           C
ATOM      7  C   GLY A  51       6.000   1.000   3.000  1.00  0.00           C
ATOM      8  O   GLY A  51       6.500   2.000   3.000  1.00  0.00           O
HETATM    9  O   HOH A 300      20.000  20.000  20.000  1.00  0.00           O
HETATM   10  C1  LIG B   1      30.000  30.000  30.000  1.00  0.00           C
HETATM   11  C2  LIG B   1      31.000  30.000  30.000  1.00  0.00           C
END
"""

with tempfile.NamedTemporaryFile(suffix=".pdb", mode="w", delete=False) as f:
    f.write(pdb_content)
    tmp_path = f.name
out_path = tmp_path + "_edited.pdb"

try:
    # 1. Load
    info = viewer.load_structure(tmp_path)
    print(f"Loaded: {info['n_atoms']} atoms, {info['n_chains']} chains")

    # 2. Inspect
    print(f"Chains: {viewer.get_chains()}")
    sels = viewer.auto_detect_molecules()
    for s in sels:
        print(f"  Detected: {s.name} ({len(s.atom_indices)} atoms)")

    # 3. Edit: rename chain A -> X, renumber residues
    viewer.rename_chain("A", "X")
    viewer.renumber_residues("X", 50, 51, new_start=1)

    # 4. Delete water
    water = viewer.select_by_criteria("Water")
    viewer.delete_atoms(water)

    # 5. Save
    viewer.save_pdb(out_path)
    print(f"Saved edited structure: {os.path.basename(out_path)}")

    # 6. Verify
    viewer2 = StructureManager()
    info2 = viewer2.load_structure(out_path)
    print(f"Verified: {info2['n_atoms']} atoms, chains: {viewer2.get_chains()}")
    residues = viewer2.get_residues("X")
    for r in residues:
        print(f"  {r['name']} {r['seq_id']}")
finally:
    try:
        os.unlink(tmp_path)
    except OSError:
        pass
    if os.path.exists(out_path):
        try:
            os.unlink(out_path)
        except OSError:
            pass
```

---

**parse_pdb**

```python
from gatewizard.core.structure_manager import parse_pdb

struct = parse_pdb(filepath: str) -> ProteinStructure
```

Parse a PDB file into a `ProteinStructure` without constructing a `StructureManager`.

#### assign_secondary_structure_map

```python
from gatewizard.core.structure_manager import assign_secondary_structure_map

ss = assign_secondary_structure_map(filepath: str, method: str = "auto") -> dict
```

Return a per-residue secondary-structure map. `method` is `auto`, `psique`, `pdb_records`, or `heuristic`. Used by callers that do not need a full `StructureManager`.

### Class: Selection

A named subset of atoms with display properties.

```python
Selection(name, atom_indices, *, representation='ball_stick',
          color_scheme='element', uniform_color=None, visible=True, ...)
```

#### Attributes

| Attribute | Type | Default | Description |
|---|---|---|---|
| `name` | str | — | Display name |
| `atom_indices` | list[int] | — | Indices into structure.atoms |
| `representation` | str | `'ball_stick'` | `'vdw'`, `'ball_stick'`, `'sticks'`, `'cartoon'`, `'tube_ss'`, `'backbone'`, `'surface'` |
| `color_scheme` | str | `'element'` | `'element'`, `'chain'`, `'ss'`, `'uniform'` |
| `uniform_color` | tuple | `None` | RGB tuple `(r, g, b)` when color_scheme is `'uniform'` |
| `visible` | bool | `True` | Show/hide |
| `quality` | int | `3` | 1–5, controls mesh resolution |
| `opacity` | float | `0.5` | Surface opacity |

---

### Representations

The viewer supports seven molecular representations:

| Key | Name | Description |
|---|---|---|
| `vdw` | VDW (Spacefill) | Atoms as spheres at van der Waals radii |
| `ball_stick` | Ball & Stick | Small spheres + bond sticks |
| `sticks` | Sticks | Bond sticks only |
| `cartoon` | Cartoon | Ribbon diagram with helix/sheet/coil |
| `tube_ss` | Tube SS | Colored tubes by secondary structure |
| `backbone` | Backbone | CA trace as tube |
| `surface` | Surface | Molecular surface |

## Structure checker

Look at a structure and report what is missing, broken, or ambiguous. The check does not build missing loops and does not score geometry.

Alternate conformations are the PDB altLoc letter. The likelihood is the occupancy, the column just before the B-factor. The most likely letter is the highest occupancy. A tie keeps the earlier letter (A before B).

### Import

```
from gatewizard.core.structure_checker import (
    diagnose_structure,
    apply_altloc_choices,
    keep_most_likely_altlocs,
)
```

#### diagnose_structure

```
diagnose_structure(path) -> dict
```

Read a PDB, mmCIF, or GRO file with MDAnalysis. Only the first model is inspected. GRO files have no altLoc, so that list is empty.

The result has `summary` counts and these lists:

- `missing_atoms` — heavy atoms absent from a standard amino acid, including Amber names such as HID, ASH, and CYX. Each entry has the chain, residue number, atom name, and a `selection`.
- `missing_residues` — numbering jumps inside a chain, and SEQRES names that have no coordinates.
- `gaps` — backbone breaks where the previous C and the next N of consecutively numbered residues are farther than about 2 Å.
- `altlocs` — residues that have two or more altLoc letters. A ligand, water, lipid, or other hetero group is its own residue: the identity is chain, residue number, insertion code, and residue name. `default` is the most likely letter for that residue. Each option includes `occupancy`, how many atoms use that letter, and `likely`. `selection` is the residue, so one conformation can be shown at a time. Atoms that overlap and have no altLoc letter are listed too, including two copies of one ligand in the same site. Those groups are marked `inferred`.
- `chain_pairs` — protein chains that are at least 80% identical or nearly the same length. A pair says whether the sequences match, which residues are substituted, and whether one chain is longer.
- `issues` — more than one model, duplicate atoms, unknown residue names, occupancy 0, and a note when the protein has no hydrogens. Each entry has a `selection` for the atoms it describes.

#### apply_altloc_choices

```
apply_altloc_choices(path, choices=None, output_path=None) -> dict
```

Write a PDB that keeps one altLoc for each disordered residue. `choices` is a list of `{chain, resid, icode, resname, altloc}`. `resname` keeps a ligand separate from a protein residue that uses the same number. Without `resname`, the letter applies only when one residue has that chain, number, and insertion code. Add `atom` only to override a single atom. A residue with no choice keeps the highest occupancy. Kept atoms have a blank altLoc and occupancy 1. `ATOM` and `HETATM` stay in their original records. SEQRES, LINK, and CONECT stay in the file. The original file is not changed.

When `output_path` is omitted, the file is written next to the source as `{name}_checked.pdb`.

The result has `output_file`, `atoms_kept`, and `atoms_removed`.

#### keep_most_likely_altlocs

```
keep_most_likely_altlocs(path, output_path) -> dict
```

Write a PDB that keeps the higher-occupancy letter of every alternate conformation, including ligands, waters, and other molecules. A tie keeps the earlier letter. A file with no altLoc and no overlapping copies is copied unchanged. This is the same choice as Preparation → **Keep the most likely alternate conformations**.

## Biological assembly

The coordinates in a PDB or mmCIF file are the asymmetric unit. The biological oligomer or capsid is described by operators already in that file: PDB `REMARK 350` / `BIOMT`, or mmCIF `_pdbx_struct_assembly`. Crystal neighbors from `CRYST1` are not built.

Chain copies are written as mmCIF with names such as `A1` and `A2`. That is the same idea as an official biological-assembly mmCIF (`A`, `A-2`, `A-3`). Classic PDB has a one-letter chain column, so it is not used for the assembly itself. Saving the result as PDB keeps only the first letter of each name.

### Import

```
from gatewizard.core.assembly import list_assemblies, build_assembly
```

#### list_assemblies

```
list_assemblies(path) -> dict
```

Read the original PDB or mmCIF. Each assembly reports its id, the author or software description, how many chains the expansion produces, and an atom estimate with and without crystal waters.

An assembly whose operators are all the identity is marked `already_biological_unit` and is not expanded. A file with no assembly records returns an empty list and a message that there is nothing to build. Strict NCS (`MTRIX`) is listed only when the file has no biological assembly that moves a chain.

#### build_assembly

```
build_assembly(path, assembly_id, include_waters=False, drop_overlaps=True, dest=None) -> dict
```

Write one assembly as mmCIF. `HOH`, `WAT`, and `DOD` are dropped unless `include_waters` is true. Ligands are kept. Identity operators do not duplicate atoms: the result has `built: false` and says there is nothing to build.

Ions and waters that sit on a symmetry axis are copied once per operator and land on the same coordinates. `drop_overlaps` keeps one hetatm at each of those points. Protein atoms that overlap are reported and left in place. The list entry counts those extras in `stacked_extra_atoms` and `stacked_extra_atoms_with_waters`.

Assemblies of about 80,000 atoms or more set `needs_confirmation` on the list entry so a caller can ask before writing.

## Preparation Module

Module for predicting pKa values and managing protonation states in protein structures. This module includes:

- pKa prediction and analysis
- Protonation state assignment based on pH
- Disulfide bond detection and application

- Protein capping with ACE/NME groups
- pH-dependent protein structure preparation

### Import

```
from gatewizard.core.preparation import PreparationManager
from gatewizard.utils.protein_capping import ProteinCapper
```

### Class: PreparationManager

Main class for running Propka analysis and managing protein protonation states.

#### Constructor

```
PreparationManager(propka_version: str = "3")
```

**Parameters:**

| Parameter | Type | Default | Description |
|---|---|---|---|
| `propka_version` | `str` | `"3"` | Version of Propka to use |

**Returns:** `PreparationManager` instance

**Example:**

```
from gatewizard.core.preparation import PreparationManager

analyzer = PreparationManager(propka_version="3")
print(f"Using PROPKA version: {analyzer.propka_version}")
```

---

### Core Methods

#### run_analysis

Run Propka analysis on a PDB file to predict pKa values.

```
run_analysis(pdb_file: str, output_dir: Optional[str] = None) -> str
```

**Parameters:**

| Parameter | Type | Required | Description |
|---|---|---|---|
| `pdb_file` | `str` | Yes | Path to input PDB file |
| `output_dir` | `str` | No | Directory for output files. If None, uses input file's directory |

**Returns:** `str` - Path to generated `.pka` file

**Raises:**

- `FileNotFoundError` - If input PDB file doesn't exist
- `PreparationError` - If Propka execution fails

**Output Files:**

- `{basename}.pka` - Full Propka output file

**Example:**

```
import shutil
import sys

if shutil.which("propka3") is None:
    sys.exit("Skip: PropKa 3 is not on PATH")

from gatewizard.core.preparation import PreparationManager

analyzer = PreparationManager()
pka_file = analyzer.run_analysis("protein.pdb")
```

---

#### extract_summary

Extract the summary section from a Propka output file.

```
extract_summary(
    propka_file: str,
    output_file: Optional[str] = None,
    output_dir: Optional[str] = None
) -> str
```

**Parameters:**

| Parameter | Type | Required | Description |
|---|---|---|---|
| `propka_file` | `str` | Yes | Path to `.pka` file from `run_analysis()` |
| `output_file` | `str` | No | Custom name for output file |
| `output_dir` | `str` | No | Directory for output file |

**Returns:** `str` - Path to summary file

**Raises:** - `PreparationError` - If summary section not found in `.pka` file

**Output Files:** - `{basename}_summary_of_prediction.txt` - Extracted summary section

**Example:**

```
import shutil
import sys

if shutil.which("propka3") is None:
    sys.exit("Skip: PropKa 3 is not on PATH")

from gatewizard.core.preparation import PreparationManager

analyzer = PreparationManager()

# First run analysis to generate the .pka file
analyzer.run_analysis("protein.pdb")

# Then extract summary
summary_file = analyzer.extract_summary("protein.pka")
# Returns: "protein_summary_of_prediction.txt"
```

---

#### parse_summary

Parse summary file and extract residue information as structured data.

```
parse_summary(summary_file: Optional[str] = None) -> List[Dict[str, Any]]
```

**Parameters:**

| Parameter | Type | Required | Description |
|---|---|---|---|
| `summary_file` | `str` | No | Path to summary file. If None, uses last extracted summary |

**Returns:** `List[Dict[str, Any]]` - List of residue dictionaries

**Dictionary Structure:**

| Key | Type | Description | Example |
|---|---|---|---|
| `"residue"` | `str` | Residue/ligand name | `"ASP"`, `"HIS"`, `"P5S"`, `"LPE"` |
| `"res_id"` | `int` | Residue number (0 for ligands) | `42`, `156`, `0` |
| `"chain"` | `str` | Chain identifier | `"A"`, `"B"` |
| `"pka"` | `float` | Predicted pKa value | `3.85`, `6.45`, `10.83` |
| `"atom"` | `str` | Atom name (for ligands only) | `""`, `"N"`, `"CAX"`, `"O15"` |
| `"atom_type"` | `str` | Atom type classification | `""`, `"N31"`, `"OCO"`, `"OP"` |
| `"model_pka"` | `float` or `None` | Model pKa value | `3.80`, `6.00`, `10.00` |

**Understanding Protein vs. Ligand Entries:**

PROPKA analyzes both **protein residues** and **ligand molecules** for ionizable groups:

**For Protein Residues:**

- `res_id`: Actual residue number (12, 52, 115, etc.)
- `atom`: Empty string (protein residues are treated at residue level)
- `atom_type`: Empty string
- Example: `ASP 52 A` means Aspartate at position 52 in chain A

**For Ligand Atoms:**

- `res_id`: Set to 0 (ligands don't have meaningful residue numbers)
- `atom`: Specific atom name in the ligand (`N`, `CAX`, `O15`, etc.)
- `atom_type`: Atom type classification (`N31`, `OCO`, `OP`, etc.)
- Example: `P5S N A` means atom N in ligand P5S in chain A

**PROPKA Summary Format:**

```
     Group      pKa  model-pKa   ligand atom-type
   ASP  52 A     4.33       3.80                     ← Protein residue
   GLU  65 A     4.87       4.50                     ← Protein residue
   HIS  77 A     6.07       6.50                     ← Protein residue
   N+    8 A     7.66       8.00                     ← N-terminus
   P5S   N A    10.83      10.00               N31   ← Ligand atom
   LPE   N A     9.87      10.00               N31   ← Ligand atom
   Y01 CAX A     4.62       4.50               OCO   ← Ligand atom
   P5S O15 A     5.46       6.00                OP   ← Ligand atom
```

**Common Atom Type Classifications:**

- **N31** - Tertiary amine nitrogen ($sp^3$, 3 bonds)
- **N33** - Quaternary nitrogen ($sp^3$, 4 bonds, charged)
- **OCO** - Carboxyl oxygen (in $-COO^-$)
- **OP** - Phosphate oxygen (in $-PO_4^{2-}$)
- **O3** - Hydroxyl oxygen ($sp^3$)
- **S3** - Thiol sulfur ($sp^3$)

**Raises:**

- `FileNotFoundError` - If summary file doesn't exist
- `PreparationError` - If no ionizable residues found

**Example:**

```
import shutil
import sys

if shutil.which("propka3") is None:
    sys.exit("Skip: PropKa 3 is not on PATH")

from gatewizard.core.preparation import PreparationManager

analyzer = PreparationManager()

# First run analysis and extract summary to generate the summary file
analyzer.run_analysis("protein.pdb")
analyzer.extract_summary("protein.pka")

# Now parse the summary file
residues = analyzer.parse_summary("protein_summary_of_prediction.txt")
print(residues)

# Separate protein residues from ligand atoms
protein_residues = [r for r in residues if r["res_id"] > 0]
ligand_atoms = [r for r in residues if r["res_id"] == 0]

print(f"Found {len(protein_residues)} ionizable protein residues")
print(f"Found {len(ligand_atoms)} ionizable ligand atoms")

# Analyze ligands
for lig in ligand_atoms:
    print(
        f"Ligand {lig['residue']} atom {lig['atom']} ({lig['atom_type']}): "
        f"pKa = {lig['pka']:.2f}"
    )

# Group by ligand type
from collections import defaultdict

ligands_by_type = defaultdict(list)
for lig in ligand_atoms:
    ligands_by_type[lig["residue"]].append(lig)

for ligand_name, atoms in ligands_by_type.items():
    print(f"\n{ligand_name}: {len(atoms)} ionizable atoms")
    for atom in atoms:
        print(f"  {atom['atom']:6s} pKa={atom['pka']:5.2f} ({atom['atom_type']})")
```

**Output:**

```
...
Found 272 ionizable protein residues
Found 22 ionizable ligand atoms
Ligand P5S atom N (N31): pKa = 10.83
Ligand LPE atom N (N31): pKa = 9.87
Ligand OJ0 atom N02 (N33): pKa = 8.82
Ligand P5S atom C (OCO): pKa = 4.16
Ligand Y01 atom CAX (OCO): pKa = 4.62
Ligand Y01 atom CAX (OCO): pKa = 4.56
Ligand Y01 atom CAX (OCO): pKa = 5.05
Ligand Y01 atom CAX (OCO): pKa = 4.52
Ligand Y01 atom CAX (OCO): pKa = 4.68
Ligand Y01 atom CAX (OCO): pKa = 4.65
Ligand P5S atom O15 (OP): pKa = 5.46
Ligand LPE atom O31 (OP): pKa = 5.15
Ligand PCW atom O1P (OP): pKa = 6.08
Ligand LPE atom O31 (OP): pKa = 6.46
Ligand PCW atom O1P (OP): pKa = 5.62
Ligand PCW atom O1P (OP): pKa = 7.02
Ligand LPE atom O32 (OP): pKa = 6.01
Ligand LPE atom O31 (OP): pKa = 6.54
Ligand LPE atom O32 (OP): pKa = 6.30
Ligand LPE atom O32 (OP): pKa = 6.81
Ligand PCW atom O1P (OP): pKa = 6.17
Ligand PCW atom O1P (OP): pKa = 4.64

P5S: 3 ionizable atoms
  N      pKa=10.83 (N31)
  C      pKa= 4.16 (OCO)
  O15    pKa= 5.46 (OP)

LPE: 7 ionizable atoms
  N      pKa= 9.87 (N31)
  O31    pKa= 5.15 (OP)
  O31    pKa= 6.46 (OP)
  O32    pKa= 6.01 (OP)
  O31    pKa= 6.54 (OP)
  O32    pKa= 6.30 (OP)
  O32    pKa= 6.81 (OP)

OJ0: 1 ionizable atoms
  N02    pKa= 8.82 (N33)

Y01: 6 ionizable atoms
  CAX    pKa= 4.62 (OCO)
  CAX    pKa= 4.56 (OCO)
  CAX    pKa= 5.05 (OCO)
  CAX    pKa= 4.52 (OCO)
  CAX    pKa= 4.68 (OCO)
  CAX    pKa= 4.65 (OCO)

PCW: 5 ionizable atoms
  O1P    pKa= 6.08 (OP)
  O1P    pKa= 5.62 (OP)
  O1P    pKa= 7.02 (OP)
  O1P    pKa= 6.17 (OP)
  O1P    pKa= 4.64 (OP)
```

---

### apply_protonation_states

Apply predicted protonation states to a PDB file based on pH and Propka results.

```
apply_protonation_states(
    input_pdb: str,
    output_pdb: str,
    ph: float,
    custom_states: Optional[Dict[str, str]] = None,
    residues: Optional[List[Dict[str, Any]]] = None
) -> Dict[str, int]
```

**Parameters:**

| Parameter | Type | Required | Description |
|---|---|---|---|
| `input_pdb` | `str` | Yes | Path to input PDB file |
| `output_pdb` | `str` | Yes | Path for output PDB file with modified residue names |
| `ph` | `float` | Yes | Target pH value for protonation state prediction |
| `custom_states` | `Dict[str, str]` | No | Custom state overrides (format: `{"ASP42": "ASH"}` or `{"ASP42_A": "ASH"}`) |
| `residues` | `List[Dict]` | No | Residue list. If None, uses last parsed results |

**Returns:** `Dict[str, int]` - Statistics dictionary

**Return Dictionary:**

| Key | Type | Description |
|---|---|---|
| `"residue_changes"` | `int` | Number of unique residues modified |
| `"record_changes"` | `int` | Total number of PDB records (atoms) modified |

**Raises:**

- `FileNotFoundError` - If input PDB file doesn't exist
- `PreparationError` - If no residue data available (need to run `parse_summary()` first)

**Protonation State Logic:**

The method automatically determines protonation states based on pKa vs pH:

| Residue | Condition | State | AMBER Code |
|---|---|---|---|
| ASP | pH < pKa | Protonated | ASH |
| ASP | pH ≥ pKa | Deprotonated | ASP |
| GLU | pH < pKa | Protonated | GLH |
| GLU | pH ≥ pKa | Deprotonated | GLU |
| HIS | pH < pKa | Protonated | HIP |
| HIS | pH ≥ pKa | Neutral (ε) | HIE |
| LYS | pH < pKa | Protonated | LYS |
| LYS | pH ≥ pKa | Neutral | LYN |
| CYS | pH < pKa | Free thiol | CYS |
| CYS | pH ≥ pKa | Deprotonated | CYM |

**Important Notes:**

- This method **only modifies residue names** in the PDB file (e.g., ASP → ASH, HIS → HIP)
- It does **NOT** add or remove hydrogen atoms
- For MD simulations, you may need additional preparation:
- Use `pdb4amber` or LEaP to add missing hydrogens
- Use system preparation tools to add solvent and parameterize

**Custom States Format:**

Custom states can be specified with or without chain identifiers:

- `{"ASP12": "ASH"}` - Applies to all chains
- `{"ASP12_A": "ASH"}` - Applies only to chain A

- `{"HIS15": "HID"}` - Use delta-protonated histidine instead of epsilon

**Example:**

```
import shutil
import sys

if shutil.which("propka3") is None:
    sys.exit("Skip: PropKa 3 is not on PATH")

from gatewizard.core.preparation import PreparationManager

analyzer = PreparationManager()
pka_file = analyzer.run_analysis("protein.pdb")
summary_file = analyzer.extract_summary(pka_file)
residues = analyzer.parse_summary(summary_file)

# Basic usage - automatic protonation at pH 7.4
stats = analyzer.apply_protonation_states(
    input_pdb="protein.pdb", output_pdb="protein_ph7.4.pdb", ph=7.4, residues=residues
)
print(f"Modified {stats['residue_changes']} residues ({stats['record_changes']} atoms)")

# With custom states - override specific residues
custom = {
    "ASP12": "ASH",  # Want to protonate original ASP12
    "GLU13": "GLH",  # Want to protonate original GLU13
    "HIS15": "HID",  # Want delta-protonated original HIS15
}

stats = analyzer.apply_protonation_states(
    input_pdb="protein.pdb",
    output_pdb="protein_custom.pdb",
    ph=7.4,
    custom_states=custom,
    residues=residues,
)
```

---

### get_default_protonation_state

Get the default protonation state for a residue at a given pH.

```
get_default_protonation_state(
    residue: Dict[str, Any],
    ph: float
) -> str
```

**Parameters:**

| Parameter | Type | Required | Description |
|---|---|---|---|
| `residue` | `Dict[str, Any]` | Yes | Residue dictionary from `parse_summary()` |
| `ph` | `float` | Yes | Target pH value |

**Returns:** `str` - Three-letter AMBER residue code

**Example:**

```python
import shutil
import sys

if shutil.which("propka3") is None:
    sys.exit("Skip: PropKa 3 is not on PATH")

from gatewizard.core.preparation import PreparationManager

analyzer = PreparationManager()
pka_file = analyzer.run_analysis("protein.pdb")
summary_file = analyzer.extract_summary(pka_file)
residues = analyzer.parse_summary(summary_file)

for res in residues:
    states = [
        analyzer.get_default_protonation_state(res, ph=p) for p in [2.0, 7.0, 11.0]
    ]
    print(
        f"{res['residue']}{res['res_id']} (pKa={res['pka']:.2f}): "
        f"pH2={states[0]}, pH7={states[1]}, pH11={states[2]}"
    )
```

---

## Protonation State Methods

### get_available_states

Get available protonation states for a residue type.

```python
get_available_states(residue_type: str) -> Dict[str, str]
```

**Parameters:**

| Parameter | Type | Required | Description |
|---|---|---|---|
| `residue_type` | `str` | Yes | Three-letter residue code (e.g., "ASP", "HIS") |

**Returns:** `Dict[str, str]` - Dictionary of available states

**Example:**

```python
from gatewizard.core.preparation import PreparationManager

analyzer = PreparationManager()
his_states = analyzer.get_available_states("HIS")
print(his_states)
```

---

### AMBER-Compatible Residue Names

| Standard | Protonated | Deprotonated | Notes |
|---|---|---|---|
| **ASP** | ASH | ASP | Aspartic acid |
| **GLU** | GLH | GLU | Glutamic acid |
| **HIS** | HIP | HIE/HID | HIP=both N protonated, HIE=ε, HID=δ |
| **LYS** | LYS | LYN | Lysine |
| **ARG** | ARG | - | Arginine (always protonated in AMBER) |
| **CYS** | CYS | CYM | Cysteine (free thiol) |
| **CYS** | - | CYX | Cysteine in disulfide bond |
| **TYR** | TYR | TYM | Tyrosine |

---

## Disulfide Bond Methods

### detect_disulfide_bonds

Automatically detect potential disulfide bonds based on sulfur-sulfur distance.

```
detect_disulfide_bonds(
    pdb_file: str,
    distance_threshold: float = 2.5
) -> List[Tuple[Tuple[str, int], Tuple[str, int]]]
```

**Parameters:**

| Parameter | Type | Default | Description |
|---|---|---|---|
| `pdb_file` | `str` | Required | Path to PDB file |
| `distance_threshold` | `float` | `2.5` | Maximum S-S distance in Ångströms |

**Returns:** List of bond pairs: `[((res_name, res_id), (res_name, res_id)), ...]`

**Algorithm:** 1. Extracts all cysteine SG (sulfur gamma) atoms from PDB 2. Calculates pairwise distances between all SG atoms 3. Identifies pairs within the distance threshold 4. Returns list of bonded residue pairs

**Example:**

```
from gatewizard.core.preparation import PreparationManager

analyzer = PreparationManager()
# Detect with default 2.5 Å threshold
bonds = analyzer.detect_disulfide_bonds("protein.pdb")
print(f"Found {len(bonds)} disulfide bonds:")

for bond in bonds:
    (res1_name, res1_id), (res2_name, res2_id) = bond
    print(f"  {res1_name}{res1_id} ↔ {res2_name}{res2_id}")
# Output:
# Found 1 disulfide bonds:
#   CYS18 ↔ CYS34

# Use stricter threshold
bonds_strict = analyzer.detect_disulfide_bonds("protein.pdb", distance_threshold=2.0)
print(f"Strict (2.0 Å): {len(bonds_strict)} bonds")
```

---

### apply_disulfide_bonds

Apply disulfide bond assignments by renaming CYS → CYX for bonded cysteines.

```
apply_disulfide_bonds(
    input_pdb: str,
    output_pdb: str,
    disulfide_bonds: Optional[List[Tuple[Tuple[str, int], Tuple[str, int]]]] = None,
    auto_detect: bool = True
) -> int
```

**Parameters:**

| Parameter | Type | Default | Description |
|---|---|---|---|
| `input_pdb` | `str` | Required | Input PDB file path |
| `output_pdb` | `str` | Required | Output PDB file path |
| `disulfide_bonds` | `List[Tuple]` | `None` | Pre-detected bonds (auto-detect if None) |

| Parameter | Type | Default | Description |
|---|---|---|---|
| `auto_detect` | `bool` | `True` | Auto-detect if bonds not provided |

**Returns:** `int` - Number of disulfide bonds applied (NOT the number of atoms changed)

**What it does:** 1. Identifies cysteines involved in disulfide bonds 2. Changes residue name from CYS to CYX in all ATOM/HETATM records 3. Preserves all other PDB information (coordinates, chains, etc.) 4. Returns count of bonds (e.g., 1 bond = 2 cysteines = ~20 atom records changed)

**Example:**

```
from gatewizard.core.preparation import PreparationManager

analyzer = PreparationManager()
# Auto-detect and apply
num_bonds = analyzer.apply_disulfide_bonds(
    input_pdb="protein.pdb", output_pdb="protein_ss_auto.pdb"
)
print(f"✓ Applied {num_bonds} disulfide bonds")

# Use pre-detected bonds with custom threshold
bonds = analyzer.detect_disulfide_bonds("protein.pdb", distance_threshold=2.3)
num_bonds = analyzer.apply_disulfide_bonds(
    input_pdb="protein.pdb",
    output_pdb="protein_ss_custom.pdb",
    disulfide_bonds=bonds,
    auto_detect=False,
)
```

**Integration with Protonation Workflow:**

Disulfide bonds should be applied **after** protonation states:

**Example:**

```
import shutil
import sys

if shutil.which("propka3") is None:
    sys.exit("Skip: PropKa 3 is not on PATH")

from gatewizard.core.preparation import PreparationManager

analyzer = PreparationManager()
pka_file = analyzer.run_analysis("protein.pdb")
summary_file = analyzer.extract_summary(pka_file)
residues = analyzer.parse_summary(summary_file)
# Step 1: Apply protonation states
stats = analyzer.apply_protonation_states(
    input_pdb="protein.pdb",
    output_pdb="protein_ph7.pdb",
    ph=7.4,
    residues=residues,
)

# Step 2: Apply disulfide bonds to protonated structure
num_bonds = analyzer.apply_disulfide_bonds(
    input_pdb="protein_ph7.pdb", output_pdb="protein_ph7_ss.pdb"
)
```

---

## Utility Methods

### get_residue_statistics

Get statistics about the protonable residues.

```
get_residue_statistics() -> Dict[str, int]
```

**Returns:** `Dict[str, int]` - Dictionary with residue type counts

**Example:**

```
import shutil
import sys

if shutil.which("propka3") is None:
    sys.exit("Skip: PropKa 3 is not on PATH")

from gatewizard.core.preparation import PreparationManager

analyzer = PreparationManager()
pka_file = analyzer.run_analysis("protein.pdb")
summary_file = analyzer.extract_summary(pka_file)
residues = analyzer.parse_summary(summary_file)

stats = analyzer.get_residue_statistics()
for res_type, count in stats.items():
    print(f"{res_type}: {count}")
```

---

### get_ph_titration_curve

Generate titration curves for all protonable residues.

```
get_ph_titration_curve(
    ph_range: Tuple[float, float] = (0, 14),
    ph_step: float = 0.5
) -> Dict[str, List[Tuple[float, str]]]
```

**Parameters:**

| Parameter | Type | Default | Description |
|---|---|---|---|
| `ph_range` | `Tuple[float, float]` | `(0, 14)` | pH range as (min, max) |
| `ph_step` | `float` | `0.5` | Step size for pH values |

**Returns:** `Dict[str, List[Tuple[float, str]]]` - Mapping of residue IDs to (pH, state) tuples

**Example:**

```
import shutil
import sys

if shutil.which("propka3") is None:
    sys.exit("Skip: PropKa 3 is not on PATH")

from gatewizard.core.preparation import PreparationManager

analyzer = PreparationManager()
pka_file = analyzer.run_analysis("protein.pdb")
summary_file = analyzer.extract_summary(pka_file)
residues = analyzer.parse_summary(summary_file)

curves = analyzer.get_ph_titration_curve(ph_range=(4, 10), ph_step=1.0)

for residue_id, curve in curves.items():
    print(f"\n{residue_id}:")
    for ph, state in curve:
        print(f"  pH {ph:.1f}: {state}")
```

**Plotting Example:**

You can visualize titration curves using matplotlib. For readability, it's best to plot only specific residues of interest:

```
import shutil
import sys

if shutil.which("propka3") is None:
    sys.exit("Skip: PropKa 3 is not on PATH")

from gatewizard.core.preparation import PreparationManager
import matplotlib.pyplot as plt

analyzer = PreparationManager()
pka_file = analyzer.run_analysis("protein.pdb")
summary_file = analyzer.extract_summary(pka_file)
residues = analyzer.parse_summary(summary_file)

# Get titration curves
curves = analyzer.get_ph_titration_curve(ph_range=(0, 14), ph_step=0.5)

# State encoding for plotting (1 = protonated, 0 = deprotonated)
state_map = {
    "ASH": 1,
    "ASP": 0,
    "GLH": 1,
    "GLU": 0,
    "HIP": 1,
    "HIE": 0,
    "HID": 0,
    "LYS": 1,
    "LYN": 0,
    "CYS": 0,
    "CYM": 0,
    "TYR": 0,
    "TYM": 0,
    "ARG": 1,
}

# Option 1: Select specific residues by name
selected_residues = ["ASP12", "GLU13", "LYS16"]
filtered_curves = {k: v for k, v in curves.items() if k in selected_residues}

# Option 2: Select by residue type (e.g., only histidines)
# filtered_curves = {k: v for k, v in curves.items() if k.startswith('HIS')}

# Option 3: Select residues with pKa near physiological pH (6-8)
# pka_dict = {f"{r['residue']}{r['res_id']}_{r['chain']}": r['pka']
#             for r in residues if r['res_id'] > 0}
# filtered_curves = {k: v for k, v in curves.items()
#                    if k in pka_dict and 6 <= pka_dict[k] <= 8}

# Plot selected curves
fig, ax = plt.subplots(figsize=(10, 6))

for residue_id, curve in filtered_curves.items():
    ph_values = [ph for ph, _ in curve]
    protonation = [state_map.get(state, 0.5) for _, state in curve]
    ax.plot(
        ph_values,
        protonation,
        marker="o",
        label=residue_id,
        linewidth=2,
        markersize=6,
        alpha=0.8,
    )

ax.axvline(
    x=7.4,
    color="gray",
    linestyle="--",
    linewidth=1.5,
    alpha=0.5,
    label="pH 7.4 (physiological)",
)
ax.set_xlabel("pH", fontsize=12)
ax.set_ylabel("Protonation State (1=protonated, 0=deprotonated)", fontsize=12)
ax.set_title("Protein Titration Curves", fontsize=14)
```

```
ax.set_ylim(-0.1, 1.1)
ax.grid(True, alpha=0.3)
ax.legend(fontsize=10)
plt.tight_layout()
plt.savefig("propka_titration_curves.png", dpi=600, bbox_inches="tight")
# plt.show()

print(f"Plotted {len(filtered_curves)} out of {len(curves)} total residues")
```

**Example Output:**

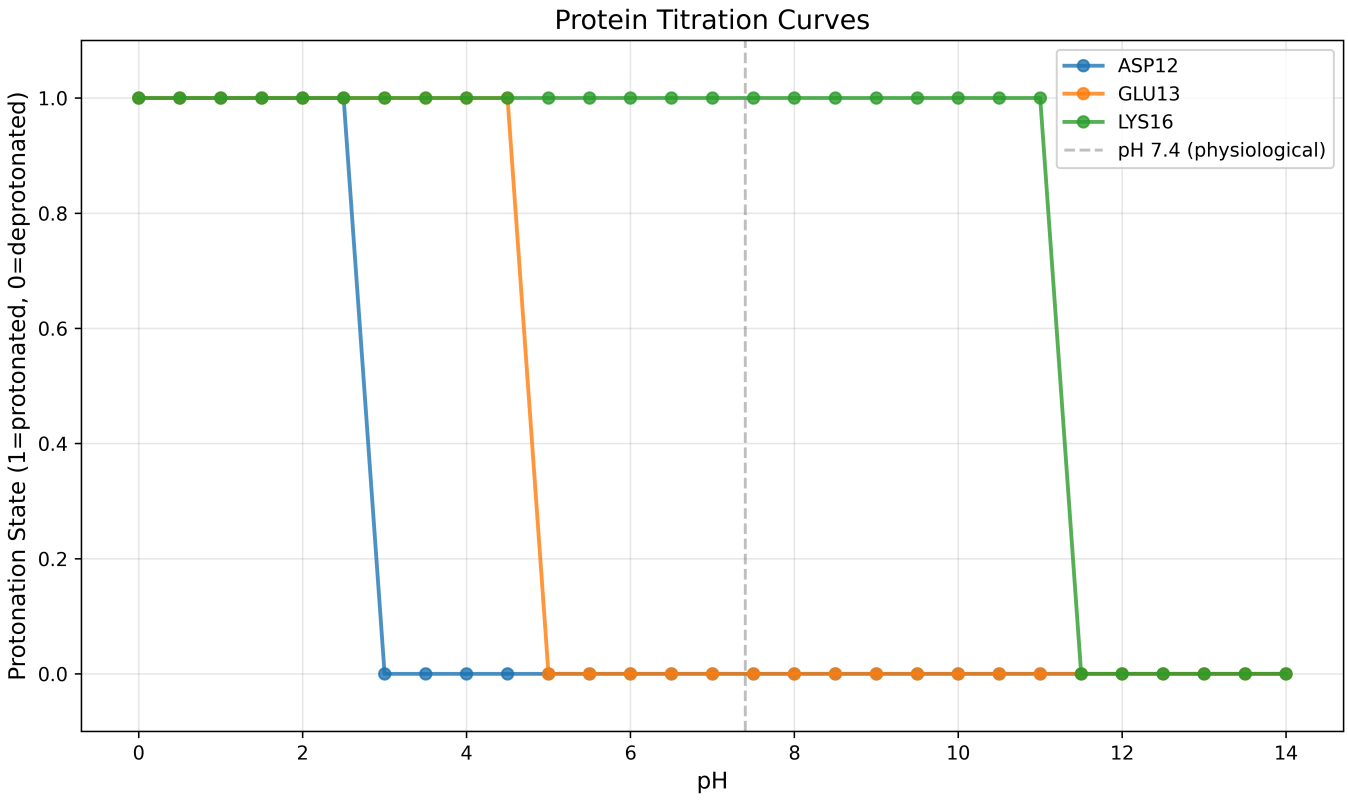



*Figure: Titration curves for selected residues showing protonation state transitions across pH range. The vertical dashed line indicates physiological pH (7.4).*

---

### build_titration_figure

Run PropKa and return the residue list, every curve, and the figure panels the GUI draws.

```
build_titration_figure(
    pdb_file: str,
    *,
    target_ph: float = 7.0,
    ph_min: float = 0.0,
    ph_max: float = 14.0,
    ph_step: float = 0.5,
    exclude_ids: Optional[List[str]] = None,
    curve_ids: Optional[List[str]] = None,
    type_filter: Optional[List[str]] = None,
) -> Dict[str, Any]
```

**Parameters:**

| Parameter | Type | Default | Description |
|---|---|---|---|
| `pdb_file` | `str` | — | PDB path |
| `target_ph` | `float` | `7.0` | pH used to mark a residue protonated (`pH < pKa`) |

| Parameter | Type | Default | Description |
|---|---|---|---|
| `ph_min` | `float` | `0.0` | First pH on the curves |
| `ph_max` | `float` | `14.0` | Last pH on the curves |
| `ph_step` | `float` | `0.5` | pH spacing |
| `exclude_ids` | `List[str]` | `None` | Residues left out of panels A and B. Each entry may be `CYS77`, `CYS77:A`, or the figure id |
| `curve_ids` | `List[str]` | `None` | Residues drawn in panel C. Omit to keep every curve |
| `type_filter` | `List[str]` | `None` | Residue names kept in A, B, and C. Empty means all types |

**Returns:** `Dict[str, Any]` — `residues` and `curves` are the full set. `figure` holds `types` (panels A and B), `curves` (panel C), and `excluded_ids`. Cysteines in a disulfide (SG–SG within 2.5 Å, or an SSBOND record) are omitted from panels A and B. `disulfide_warning` explains that PropKa writes pKa 99.99 for those residues as a cysteine-bridge placeholder, not a titration constant. `disulfide_hidden` lists the omitted residues.

`gatewizard.render_titration_png(spec)` draws that figure. Pass `layout.pixels` (each panel's on-screen width and height) so font sizes match the GUI. `c_ypad` is extra space above and below the panel C Y limits; ticks stay on the limits.

---

**Alternative: Plot pKa Distribution with Protonation States:**

```
import shutil
import sys

if shutil.which("propka3") is None:
    sys.exit("Skip: PropKa 3 is not on PATH")

from gatewizard.core.preparation import PreparationManager
import matplotlib.pyplot as plt
import numpy as np

analyzer = PreparationManager()
pka_file = analyzer.run_analysis("protein.pdb")
summary_file = analyzer.extract_summary(pka_file)
residues = analyzer.parse_summary(summary_file)

# Filter protein residues only
protein_residues = [r for r in residues if r["res_id"] > 0]

# Set target pH for protonation state analysis
target_ph = 5.0  # Change this to analyze different pH

# Group by residue type and calculate protonation states
residue_data = {}
for res in protein_residues:
    res_type = res["residue"]
    if res_type not in residue_data:
        residue_data[res_type] = {"pka": [], "protonated": [], "state": []}

    residue_data[res_type]["pka"].append(res["pka"])

    # Determine protonation state at target pH
    state = analyzer.get_default_protonation_state(res, ph=target_ph)
    residue_data[res_type]["state"].append(state)

    # Check if protonated (pH < pKa => protonated)
    is_protonated = target_ph < res["pka"]
    residue_data[res_type]["protonated"].append(is_protonated)

# Prepare data for plotting
res_names = []
pka_values_list = []
protonation_list = []
colors_list = []

color_map = {
    "ASP": "#e74c3c",
    "GLU": "#f92f94",
    "HIS": "#72e183",
    "LYS": "#60cae7",
    "CYS": "#9b59b6",
    "TYR": "#5b76f8",
    "ARG": "#f39c12",
    "N+": "#95a5a6",
    "C-": "#34495e",
}

for res_type in sorted(residue_data.keys()):
    res_names.append(res_type)
    pka_values_list.append(residue_data[res_type]["pka"])
    protonation_list.append(residue_data[res_type]["protonated"])
    colors_list.append(color_map.get(res_type, "#7f8c8d"))

# Create figure with two subplots
fig, (ax1, ax2) = plt.subplots(
    2, 1, figsize=(12, 10), gridspec_kw={"height_ratios": [2, 1]}
)

# --- Top plot: pKa distribution with protonation states ---
boxplot_kwargs = {
    "patch_artist": True,
    "showmeans": True,
    "meanline": False,
    "medianprops": {"color": "black", "linewidth": 2},
    "meanprops": {
        "marker": "D",
```

```
        "markerfacecolor": "red",
        "markeredgecolor": "darkred",
        "markersize": 6,
    },
}
try:
    bp = ax1.boxplot(pka_values_list, tick_labels=res_names, **boxplot_kwargs)
except TypeError:
    bp = ax1.boxplot(pka_values_list, labels=res_names, **boxplot_kwargs)

# Color each box
for patch, color in zip(bp["boxes"], colors_list):
    patch.set_facecolor(color)
    patch.set_alpha(0.6)

# Add individual points colored by protonation state
for i, (res_type, pka_vals, prot_states) in enumerate(
    zip(res_names, pka_values_list, protonation_list)
):
    x = np.random.normal(i + 1, 0.04, size=len(pka_vals))
    # Protonated = filled circles, Deprotonated = open circles
    for xi, pka, is_prot in zip(x, pka_vals, prot_states):
        if is_prot:
            ax1.scatter(
                xi,
                pka,
                s=50,
                color=colors_list[i],
                alpha=0.7,
                edgecolors="black",
                linewidth=1,
                marker="o",
                zorder=3,
            )
        else:
            ax1.scatter(
                xi,
                pka,
                s=50,
                facecolors="none",
                edgecolors=colors_list[i],
                alpha=0.7,
                linewidth=2,
                marker="o",
                zorder=3,
            )

# Add pH reference line
ax1.axhline(
    y=target_ph,
    color="red",
    linestyle="--",
    linewidth=2.5,
    alpha=0.8,
    label=f"pH {target_ph} (target)",
    zorder=2,
)

ax1.set_ylabel("pKa", fontsize=13, fontweight="bold")
ax1.set_title(
    f"pKa Distribution and Protonation States at pH {target_ph}",
    fontsize=15,
    fontweight="bold",
)
ax1.grid(True, alpha=0.3, axis="y", linestyle=":")
ax1.legend(loc="upper right", fontsize=11)
ax1.set_ylim(0, 20)

# Add legend for protonation states
from matplotlib.patches import Patch
from matplotlib.lines import Line2D

legend_elements = [
    Line2D(
        [0],
```

```
        [0],
        marker="o",
        color="w",
        markerfacecolor="gray",
        markersize=10,
        markeredgecolor="black",
        linewidth=1,
        label="Protonated (pH < pKa)",
    ),
    Line2D(
        [0],
        [0],
        marker="o",
        color="w",
        markerfacecolor="none",
        markersize=10,
        markeredgecolor="gray",
        linewidth=2,
        label="Deprotonated (pH ≥ pKa)",
    ),
]
ax1.legend(handles=legend_elements, loc="upper left", fontsize=10)

# --- Bottom plot: Protonation fraction bar chart ---
protonation_fractions = []
for prot_states in protonation_list:
    if len(prot_states) > 0:
        fraction = sum(prot_states) / len(prot_states)
    else:
        fraction = 0
    protonation_fractions.append(fraction)

bars = ax2.bar(
    range(1, len(res_names) + 1),
    protonation_fractions,
    color=colors_list,
    alpha=0.7,
    edgecolor="black",
    linewidth=1.5,
)

# Add percentage labels on bars
for i, (bar, frac) in enumerate(zip(bars, protonation_fractions)):
    height = bar.get_height()
    ax2.text(
        bar.get_x() + bar.get_width() / 2.0,
        height + 0.02,
        f"{frac*100:.0f}%",
        ha="center",
        va="bottom",
        fontsize=10,
        fontweight="bold",
    )

ax2.set_xlabel("Residue Type", fontsize=13, fontweight="bold")
ax2.set_ylabel("Protonated Fraction", fontsize=13, fontweight="bold")
ax2.set_title(
    f"Protonation State Distribution at pH {target_ph}", fontsize=14, fontweight="bold"
)
ax2.set_xticks(range(1, len(res_names) + 1))
ax2.set_xticklabels(res_names)
ax2.set_ylim(0, 1.1)
ax2.axhline(y=0.5, color="gray", linestyle=":", linewidth=1, alpha=0.5)
ax2.grid(True, alpha=0.3, axis="y", linestyle=":")

plt.tight_layout()
plt.savefig("propka_pka_distribution.png", dpi=600, bbox_inches="tight")
# plt.show()

# Print statistics
print(f"\npKa Statistics and Protonation States at pH {target_ph}:")
print("=" * 80)
for res_type, data in sorted(residue_data.items()):
    pka_vals = data["pka"]
    n_prot = sum(data["protonated"])
```

```
    n_total = len(pka_vals)
    prot_frac = n_prot / n_total if n_total > 0 else 0

    print(
        f"\n{res_type:4s}: n={n_total:2d}  "
        f"pKa range=[{min(pka_vals):5.2f}, {max(pka_vals):5.2f}]  "
        f"mean={np.mean(pka_vals):5.2f}"
    )
    print(
        f"      Protonated: {n_prot}/{n_total} ({prot_frac*100:.0f}%)  "
        f"Deprotonated: {n_total-n_prot}/{n_total} ({(1-prot_frac)*100:.0f}%)"
    )
    print(f"      States: {', '.join(set(data['state']))}")
```

**Example Output:**

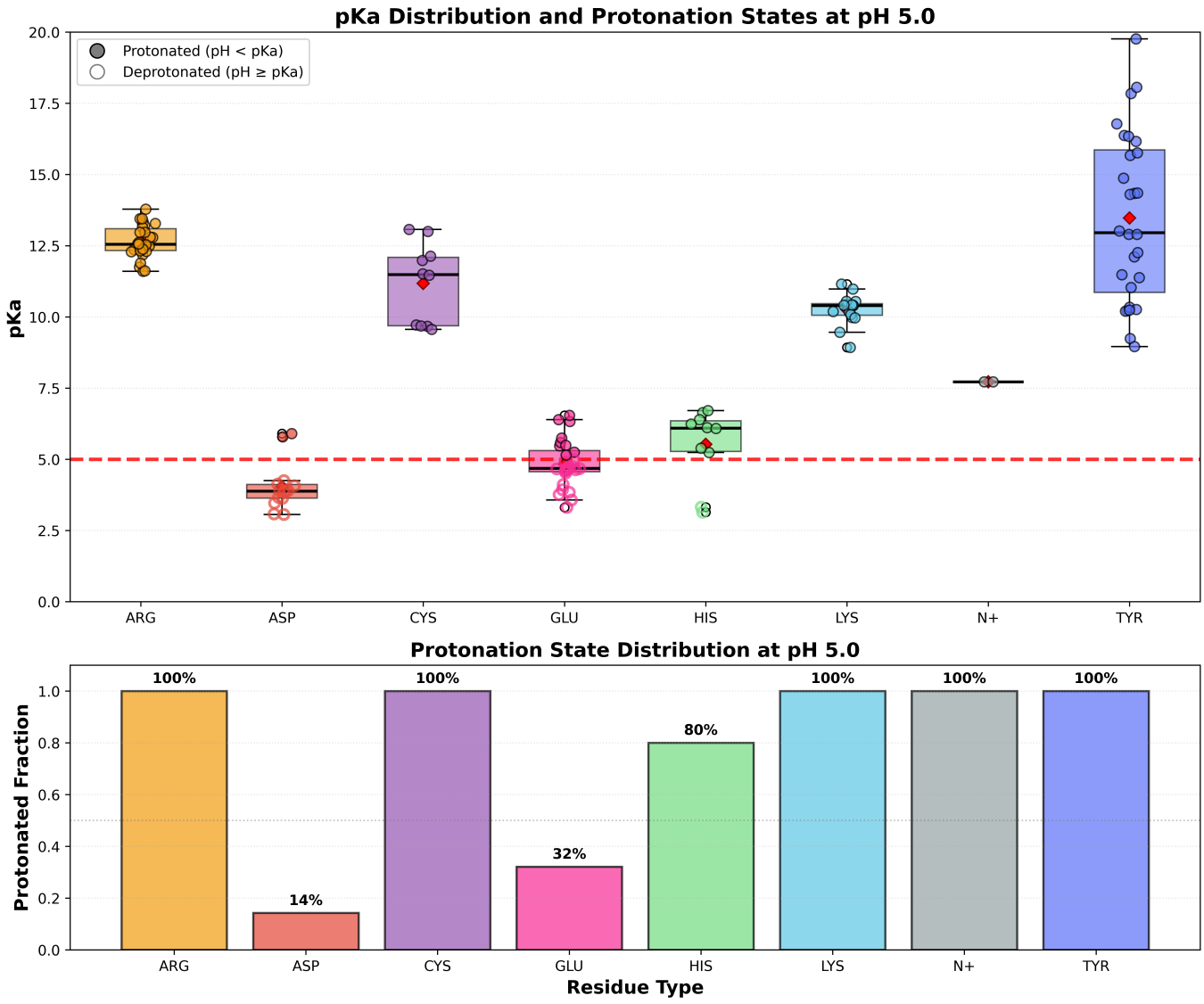



*Figure: (Top) pKa distribution by residue type with protonation states at pH 5.0. Filled circles indicate protonated residues (pH < pKa), open circles indicate deprotonated residues (pH ≥ pKa). The red dashed line shows the target pH. (Bottom) Fraction of each residue type that is protonated at pH 5.0.*

---

## Protein Capping

**Module:** `gatewizard.utils.protein_capping`

Protein capping adds **ACE** (N-acetyl) and **NME** (N-methylamide) groups to protein termini to:

- Neutralize charges at termini
- Improve Propka analysis accuracy
- Better represent membrane protein environments
- Mimic peptide bond continuation in protein fragments

**Class: ProteinCapper**

```
from gatewizard.utils.protein_capping import ProteinCapper, cap_protein

capper = ProteinCapper()
```

**remove_hydrogens_and_cap**

Main method to add ACE/NME caps and track residue renumbering.

```
remove_hydrogens_and_cap(
    input_file: Union[str, Path],
    output_file: Optional[Union[str, Path]] = None,
    target_dir: Optional[Union[str, Path]] = None
) -> Tuple[str, Dict]
```

**Parameters:**

| Parameter | Type | Default | Description |
|---|---|---|---|
| `input_file` | `str/Path` | Required | Input PDB file |
| `output_file` | `str/Path` | `None` | Output file (auto-generated if None) |
| `target_dir` | `str/Path` | `None` | Directory for mapping file |

**Returns:** `Tuple[str, Dict]`

- `str` - Path to capped PDB file
- `Dict` - Residue mapping dictionary

**Residue Mapping Dictionary:**

```
{
    (original_resname, chain, original_resid): (new_resname, chain, new_resid),
    ...
}
```

**What Happens During Capping:**

```
Before: MET1 - ALA2 - ... - GLY50
After:  ACE1 - MET2 - ALA3 - ... - GLY51 - NME52
```

All residue numbers shift by +1, and ACE/NME caps are added.

**Example:**

```python
from gatewizard.utils.protein_capping import ProteinCapper

capper = ProteinCapper()

# Add caps and get mapping
capped_file, residue_mapping = capper.remove_hydrogens_and_cap(
    input_file="protein.pdb", output_file="protein_capped.pdb", target_dir="output"
)

print(f"✓ Capped protein: {capped_file}")
print(f"✓ Residue mapping: {len(residue_mapping)} residues tracked")

# Show first few mappings
for (orig_resname, chain, orig_resid), (new_resname, new_chain, new_resid) in list(
    residue_mapping.items()
)[:5]:
    print(f"  {orig_resname}{orig_resid} (Chain {chain}) → {new_resname}{new_resid}")
# Output:
#  LEU1 (Chain A) → LEU2
#  PRO2 (Chain A) → PRO3
#  ALA3 (Chain A) → ALA4
#  GLY4 (Chain A) → GLY5
#  ILE5 (Chain A) → ILE6
```

#### cap_protein

Simplified wrapper for protein capping.

```python
from gatewizard.utils.protein_capping import cap_protein

capped_file, mapping = cap_protein(
    input_file="protein.pdb", output_file="protein_capped_convenient.pdb"
)
```

---

## pdb4amber Integration

### Important Distinction:

| Method | Runs pdb4amber? | Output |
|---|---|---|
| **GUI** "Apply States & Run pdb4amber" button | ✅ **YES** (automatic) | AMBER-ready PDB with hydrogens |
| **API** `apply_protonation_states()` | ❌ **NO** (manual) | Only residue name changes |
| **API** `apply_disulfide_bonds()` | ❌ **NO** (manual) | Only CYS→CYX changes |

The API methods only modify residue names in PDB files. For MD simulations, you must run pdb4amber separately:

### Option 1: Command Line

```
pdb4amber -i protein_protonated_ss.pdb -o protein_prepared.pdb
```

### Option 2: Python subprocess

```python
import subprocess

result = subprocess.run([
    "pdb4amber",
    "-i", "protein_protonated_ss.pdb",
    "-o", "protein_prepared.pdb"
], capture_output=True, text=True, check=True)
```

### ACE/NME Cap HETATM Fix

**Critical Note:** When using protein capping (ACE/NME), pdb4amber converts cap ATOM records to HETATM records, which can cause compatibility issues with downstream tools like packmol-memgen.

**The Problem:**

```
# Before pdb4amber (correct for caps):
ATOM      1  C   ACE A   1      12.640  -9.437  14.871  1.00  0.00      A
ATOM      2  CH3 ACE A   1      12.660 -10.927  15.173  1.00  0.00      A

# After pdb4amber (incorrect - becomes heteroatom):
HETATM    1  C   ACE A   1      12.640  -9.437  14.871  1.00  0.00           C
HETATM    2  CH3 ACE A   1      12.660 -10.927  15.173  1.00  0.00           C
```

### run_pdb4amber_with_cap_fix

Run pdb4amber to add hydrogens and prepare PDB for AMBER, with optional ACE/NME cap HETATM fix.

```
run_pdb4amber_with_cap_fix(
    input_pdb: str,
    output_pdb: str,
    fix_caps: bool = True,
    pdb4amber_options: Optional[Dict[str, Any]] = None,
    preserve_residue_numbers: bool = False,
    cap_mapping_path: Optional[str] = None,
    original_pdb: Optional[str] = None
) -> Dict[str, Any]
```

**Parameters:**

| Parameter | Type | Default | Description |
|---|---|---|---|
| `input_pdb` | `str` | Required | Path to input PDB file |
| `output_pdb` | `str` | Required | Path for output PDB file |
| `fix_caps` | `bool` | `True` | If True, automatically converts ACE/NME HETATM→ATOM after pdb4amber |
| `pdb4amber_options` | `Dict[str, Any]` | `None` | Optional dictionary of pdb4amber command-line options |
| `preserve_residue_numbers` | `bool` | `False` | After Amber, restore original residue numbers and chains. Default False keeps sequential Amber numbering for tleap / MD. |
| `cap_mapping_path` | `str` | `None` | Optional `*_gatewizard_residue_mapping.txt` so loop gaps survive a prior cap step |
| `original_pdb` | `str` | `None` | Optional pre-cap PDB used when composing originals |

**Returns:** `Dict[str, Any]` - Result dictionary with execution details

**Return Dictionary:**

| Key | Type | Description |
|---|---|---|
| `"success"` | `bool` | Whether pdb4amber execution succeeded |
| `"output_file"` | `str` | Path to the output PDB file |
| `"hetatm_fixed"` | `int` | Number of HETATM records converted to ATOM (0 if fix_caps=False) |
| `"stdout"` | `str` | Standard output from pdb4amber command |
| `"stderr"` | `str` | Standard error from pdb4amber command |
| `"residue_numbers_preserved"` | `bool` | True when originals were rewritten onto the output |
| `"remum_path"` | `str` | Path to pdb4amber `*_renum.txt` when found |

| Key | Type | Description |
|---|---|---|
| `"cap_assignments"` | `dict` | New ACE/NME residue numbers when preserve is on |

**Raises:**

- `FileNotFoundError` - If input PDB file doesn't exist
- `RuntimeError` - If pdb4amber command fails or is not found

**What This Method Does:**

**Runs pdb4amber**: Executes the AmberTools pdb4amber utility to:

- Prepare PDB structure for AMBER force field
- Remove problematic records
- Renumber atoms and residues if needed

**Optionally Fixes ACE/NME Caps** (if `fix_caps=True`):

- Detects ACE (N-acetyl) and NME (N-methylamide) cap residues
- Converts their HETATM records back to ATOM records
- This fixes compatibility issues with tools like packmol-memgen

**The ACE/NME Cap HETATM Problem:**

When protein capping is used, pdb4amber incorrectly converts cap ATOM records to HETATM:

```
# Before pdb4amber (correct):
ATOM      1  C   ACE A   1      12.640  -9.437  14.871  1.00  0.00           C
ATOM      2  CH3 ACE A   1      12.660 -10.927  15.173  1.00  0.00           C

# After pdb4amber (incorrect - becomes heteroatom):
HETATM    1  C   ACE A   1      12.640  -9.437  14.871  1.00  0.00           C
HETATM    2  CH3 ACE A   1      12.660 -10.927  15.173  1.00  0.00           C
```

This causes compatibility issues because:

- packmol-memgen expects caps as ATOM records
- Other MD preparation tools may skip HETATM records
- Caps should be treated as part of the protein chain

**pdb4amber_options Dictionary:**

You can pass custom pdb4amber command-line options:

```python
options = {
    "reduce": True,        # Use reduce for hydrogen addition
    "dry": False,          # Keep water molecules (default: False)
    "keep-altlocs": False, # Remove alternative locations (default: False)
    "strip-waters": True,  # Remove water molecules (default: True)
}
```

`reduce=True` requires the Amber `reduce` CLI. If that binary is not installed (common in slim conda envs that still have `pdb4amber`), GateWizard drops `--reduce`, strips stale CONECT/LINK records first, and keeps hydrogens from `complete_missing_heavy_atoms` / tleap. The result dict then has `reduce_skipped=True` and `conect_records_removed`.

CONECT/LINK warnings from ParmEd (missing serials after atoms were added, or K+ coordination LINKs) are not fatal. They are avoided by dropping those records before `pdb4amber`.

**strip_conect_and_link_records**

```
from gatewizard.core.preparation import strip_conect_and_link_records
```

Remove CONECT and LINK lines. Use this when serials no longer match (after tleap completion or any renumber).

**resolve_reduce_executable**

```
from gatewizard.core.preparation import resolve_reduce_executable
```

Return the Amber `reduce` path (`CONDA_PREFIX/bin` first), or `None` if it is not installed.

**Example - Basic Usage:**

```
from gatewizard.core.preparation import PreparationManager

analyzer = PreparationManager()

# Run pdb4amber with automatic cap fix (recommended for capped proteins)
result = analyzer.run_pdb4amber_with_cap_fix(
    input_pdb="protein_capped_ph7_ss.pdb",
    output_pdb="protein_prepared.pdb",
    fix_caps=True  # Default: automatically fix ACE/NME caps
)

if result['success']:
    print(f"✓ pdb4amber completed successfully")
    print(f"✓ Output: {result['output_file']}")
    if result['hetatm_fixed'] > 0:
        print(f"✓ Fixed {result['hetatm_fixed']} HETATM records for ACE/NME caps")
else:
    print(f"✗ pdb4amber failed")
    print(f"Error: {result['stderr']}")
```

**Example - Without Cap Fix:**

```
from gatewizard.core.preparation import PreparationManager

analyzer = PreparationManager()

# Run pdb4amber without cap fix (for uncapped proteins)
result = analyzer.run_pdb4amber_with_cap_fix(
    input_pdb="protein_uncapped_ph7_ss.pdb",
    output_pdb="protein_prepared.pdb",
    fix_caps=False  # No cap fix needed
)

print(f"Success: {result['success']}")
print(f"HETATM records fixed: {result['hetatm_fixed']}")  # Will be 0
```

**Example - With Custom pdb4amber Options:**

```python
from gatewizard.core.preparation import PreparationManager

analyzer = PreparationManager()

# Run with custom pdb4amber options
custom_options = {
    "dry": False,          # Keep crystallographic waters
    "keep-altlocs": True,  # Keep alternative locations
}

result = analyzer.run_pdb4amber_with_cap_fix(
    input_pdb="protein_capped_ph7_ss.pdb",
    output_pdb="protein_prepared.pdb",
    fix_caps=True,
    pdb4amber_options=custom_options
)

print(f"pdb4amber output:\n{result['stdout']}")
```

**Integration with Complete Workflow:**

```python
from gatewizard.core.preparation import PreparationManager
from gatewizard.utils.protein_capping import ProteinCapper

# Step 1: Add caps
capper = ProteinCapper()
capped_file, mapping = capper.remove_hydrogens_and_cap("protein.pdb")

# Step 2: Run Propka analysis
analyzer = PreparationManager()
pka_file = analyzer.run_analysis(capped_file)
summary_file = analyzer.extract_summary(pka_file)
residues = analyzer.parse_summary(summary_file)

# Step 3: Apply protonation states
analyzer.apply_protonation_states(
    input_pdb=capped_file,
    output_pdb="protein_capped_ph7.pdb",
    ph=7.4,
    residues=residues
)

# Step 4: Apply disulfide bonds
bonds = analyzer.detect_disulfide_bonds(capped_file)
analyzer.apply_disulfide_bonds(
    input_pdb="protein_capped_ph7.pdb",
    output_pdb="protein_capped_ph7_ss.pdb",
    disulfide_bonds=bonds
)

# Step 5: Run pdb4amber with automatic cap fix
result = analyzer.run_pdb4amber_with_cap_fix(
    input_pdb="protein_capped_ph7_ss.pdb",
    output_pdb="protein_prepared.pdb",
    fix_caps=True  # Critical for capped proteins!
)

print(f"✓ Final AMBER-ready structure: {result['output_file']}")
```

**GUI Behavior:**

When using the GUI:

- The "Apply States & Run pdb4amber" button automatically uses this method
- Automatically detects when capping was used (checks capping checkbox)
- Sets `fix_caps=True` if capping was applied
- Reports number of fixed HETATM records in success message
- Displays pdb4amber output in the GUI log

**Important Notes:**

⚠️ **When to use fix_caps=True:**

- When you used protein capping (ACE/NME groups added)
- When planning to use packmol-memgen or similar tools
- When you need caps to be part of the protein chain (ATOM records)

⚠️ **When to use fix_caps=False:**

- When no capping was applied
- When you want standard pdb4amber behavior
- When working with uncapped protein structures

⚠️ **Requirements:**

- Requires AmberTools installation (`pdb4amber` must be available). The Amber `reduce` binary is optional: when it is missing, `--reduce` is skipped and tleap hydrogens are kept.
- Input PDB should already have correct protonation states and disulfide bonds applied

---

## Complete Workflow Examples

### Advanced Workflow with Protonation and Disulfide Bonds

```
import shutil
import sys

if shutil.which("propka3") is None:
    sys.exit("Skip: PropKa 3 is not on PATH")

from gatewizard.core.preparation import PreparationManager

analyzer = PreparationManager()

# Step 1: Run Propka analysis
pka_file = analyzer.run_analysis("protein.pdb", output_dir="output")
summary_file = analyzer.extract_summary(pka_file, output_dir="output")
residues = analyzer.parse_summary(summary_file)
print(f"✓ Found {len(residues)} ionizable residues")

# Step 2: Detect disulfide bonds
bonds = analyzer.detect_disulfide_bonds("protein.pdb", distance_threshold=2.5)
print(f"✓ Detected {len(bonds)} disulfide bonds")

# Step 3: Apply protonation states
stats = analyzer.apply_protonation_states(
    input_pdb="protein.pdb",
    output_pdb="output/protein_ph7.pdb",
    ph=7.4,
    residues=residues,
)
print(
    f"✓ Protonation: {stats['residue_changes']} residues, "
    f"{stats['record_changes']} PDB records changed"
)

# Step 4: Apply disulfide bonds
num_bonds = analyzer.apply_disulfide_bonds(
    input_pdb="output/protein_ph7.pdb",
    output_pdb="output/protein_ph7_ss.pdb",
    disulfide_bonds=bonds,
    auto_detect=False,
)
print(f"✓ Applied {num_bonds} disulfide bonds (CYS → CYX)")
```

### Workflow with Protein Capping

```
import shutil
import sys

if shutil.which("propka3") is None:
    sys.exit("Skip: PropKa 3 is not on PATH")

from gatewizard.core.preparation import PreparationManager
from gatewizard.utils.protein_capping import ProteinCapper

# Step 1: Add ACE/NME caps
capper = ProteinCapper()
capped_file, residue_mapping = capper.remove_hydrogens_and_cap(
    input_file="protein.pdb", output_file="protein_capped.pdb", target_dir="output"
)
print(f"✓ Capped protein: {len(residue_mapping)} residues tracked")

# Step 2: Run Propka on capped structure
analyzer = PreparationManager()
pka_file = analyzer.run_analysis(capped_file, output_dir="output")
summary_file = analyzer.extract_summary(pka_file, output_dir="output")
residues = analyzer.parse_summary(summary_file)

# Step 3: Detect disulfide bonds
bonds = analyzer.detect_disulfide_bonds(capped_file)

# Step 4: Apply protonation
stats = analyzer.apply_protonation_states(
    input_pdb=capped_file,
    output_pdb="output/protein_capped_ph7.pdb",
    ph=7.4,
    residues=residues,
)

# Step 5: Apply disulfide bonds
num_bonds = analyzer.apply_disulfide_bonds(
    input_pdb="output/protein_capped_ph7.pdb",
    output_pdb="output/protein_capped_ph7_ss.pdb",
    disulfide_bonds=bonds,
    auto_detect=False,
)

# Step 6: Run pdb4amber with automatic ACE/NME HETATM fix
result = analyzer.run_pdb4amber_with_cap_fix(
    input_pdb="output/protein_capped_ph7_ss.pdb",
    output_pdb="output/protein_prepared.pdb",
    fix_caps=True,  # Automatically fix ACE/NME HETATM records
)

print(f"✓ pdb4amber completed: {result['output_file']}")
if result["hetatm_fixed"] > 0:
    print(f"✓ Fixed {result['hetatm_fixed']} HETATM records for ACE/NME caps")
print(f"✓ Final structure ready: protein_prepared.pdb")
```

### Multiple pH Variants

```
import shutil
import sys

if shutil.which("propka3") is None:
    sys.exit("Skip: PropKa 3 is not on PATH")

from gatewizard.core.preparation import PreparationManager

analyzer = PreparationManager()

# Run analysis once
pka_file = analyzer.run_analysis("protein.pdb", output_dir="output")
summary_file = analyzer.extract_summary(pka_file, output_dir="output")
residues = analyzer.parse_summary(summary_file)
bonds = analyzer.detect_disulfide_bonds("protein.pdb")

# Generate structures for different pH values
for ph in [5.0, 6.0, 7.0, 7.4, 8.0, 9.0]:
    # Apply protonation
    stats = analyzer.apply_protonation_states(
        input_pdb="protein.pdb",
        output_pdb=f"output/protein_ph{ph:.1f}.pdb",
        ph=ph,
        residues=residues,
    )

    # Apply disulfide bonds
    analyzer.apply_disulfide_bonds(
        input_pdb=f"output/protein_ph{ph:.1f}.pdb",
        output_pdb=f"output/protein_ph{ph:.1f}_ss.pdb",
        disulfide_bonds=bonds,
        auto_detect=False,
    )

    print(
        f"pH {ph:.1f}: {stats['residue_changes']} residues modified, "
        f"{len(bonds)} S-S bonds"
    )
```

---

## Advanced Topics

### Residue Mapping After Capping

When proteins are capped, residue numbers change (all shift by +1). Use the mapping dictionary to translate:

```python
import shutil
import sys

if shutil.which("propka3") is None:
    sys.exit("Skip: PropKa 3 is not on PATH")

import re
from gatewizard.core.preparation import PreparationManager
from gatewizard.utils.protein_capping import ProteinCapper

# Step 1: Add ACE/NME caps to get residue mapping
capper = ProteinCapper()
capped_file, residue_mapping = capper.remove_hydrogens_and_cap(
    input_file="protein.pdb", output_file="protein_capped.pdb", target_dir="output"
)
print(f"✓ Capped protein with {len(residue_mapping)} residues tracked")

# Step 2: Run Propka analysis on capped structure
analyzer = PreparationManager()
pka_file = analyzer.run_analysis(capped_file, output_dir="output")
summary_file = analyzer.extract_summary(pka_file, output_dir="output")
residues = analyzer.parse_summary(summary_file)
print(f"✓ Analyzed {len(residues)} ionizable residues")

# Step 3: Define custom states using ORIGINAL (uncapped) numbering
original_custom_states = {
    "ASP12": "ASH",  # Force protonate ASP at original position 12
    "GLU13": "GLH",  # Force protonate GLU at original position 13
}

# Step 4: Translate to NEW numbering after capping (all shifted by +1)
custom_states_new = {}
for orig_spec, state in original_custom_states.items():
    match = re.match(r"([A-Z]+)(\d+)(?:_([A-Z]))?", orig_spec)
    if match:
        resname, resid, chain = match.groups()
        resid = int(resid)
        chain = chain or "A"

        # Look up the new numbering from mapping
        original_key = (resname, chain, resid)
        if original_key in residue_mapping:
            new_resname, new_chain, new_resid = residue_mapping[original_key]
            new_spec = f"{new_resname}{new_resid}_{new_chain}"
            custom_states_new[new_spec] = state
            print(f"  Mapped {orig_spec} → {new_spec} ({state})")
        else:
            print(f"  Warning: {orig_spec} not found in mapping")

# Step 5: Apply protonation states using the TRANSLATED custom states
stats = analyzer.apply_protonation_states(
    input_pdb=capped_file,
    output_pdb="output/protein_capped_custom.pdb",
    ph=7.4,
    custom_states=custom_states_new,  # Use translated states
    residues=residues,
)
print(f"✓ Applied protonation: {stats['residue_changes']} residues modified")

# Step 6: Run pdb4amber with automatic ACE/NME HETATM fix
result = analyzer.run_pdb4amber_with_cap_fix(
    input_pdb="output/protein_capped_custom.pdb",
    output_pdb="output/protein_prepared.pdb",
    fix_caps=True,  # Automatically fix ACE/NME HETATM records
)

print(f"✓ pdb4amber completed: {result['output_file']}")
if result["hetatm_fixed"] > 0:
    print(f"✓ Fixed {result['hetatm_fixed']} HETATM records for ACE/NME caps")
print(f"✓ Final structure ready: protein_prepared.pdb")
```

**Example Output:**

```
✓ Capped protein with 50 residues tracked
✓ Analyzed 12 ionizable residues
  Mapped ASP12 → ASP13_A (ASH)
  Mapped GLU13 → GLU14_A (GLH)
✓ Applied protonation: 12 residues modified
```

### Custom Protonation States

Override automatic pH-based protonation with custom states for specific residues:

```python
import shutil
import sys

if shutil.which("propka3") is None:
    sys.exit("Skip: PropKa 3 is not on PATH")

from gatewizard.core.preparation import PreparationManager

# Step 1: Run standard Propka workflow
analyzer = PreparationManager()
pka_file = analyzer.run_analysis("protein.pdb", output_dir="output")
summary_file = analyzer.extract_summary(pka_file, output_dir="output")
residues = analyzer.parse_summary(summary_file)
print(f"✓ Analyzed {len(residues)} ionizable residues")

# Step 2: Specify custom states to override automatic pH-based assignment
custom_states = {
    "ASP12_A": "ASH",  # Force protonated aspartate in chain A
    "HIS15_A": "HID",  # Force delta-protonated histidine (instead of HIE or HIP)
    "GLU22": "GLH",  # Force protonated in all chains
}

# Step 3: Apply protonation with custom overrides
stats = analyzer.apply_protonation_states(
    input_pdb="protein.pdb",
    output_pdb="output/protein_custom.pdb",
    ph=7.4,
    custom_states=custom_states,
    residues=residues,
)
print(
    f"✓ Modified {stats['residue_changes']} residues ({stats['record_changes']} atoms)"
)
print(f"  Custom states applied: {len(custom_states)}")

# Step 4: Verify the changes
for spec, state in custom_states.items():
    print(f"  {spec} → {state}")
```

**Example Output:**

```
✓ Analyzed 13 ionizable residues
✓ Modified 3 residues (44 atoms)
  Custom states applied: 3
  ASP12_A → ASH
  HIS15_A → HID
  GLU22 → GLH
```

**Use Cases:**

- Force specific protonation for catalytic residues
- Match experimental conditions or known structures
- Test different protonation hypotheses
- Override PROPKA predictions based on domain knowledge

---

### Filtering and Analysis

Identify residues with unusual pKa shifts that may indicate important interactions:

```
import shutil
import sys

if shutil.which("propka3") is None:
    sys.exit("Skip: PropKa 3 is not on PATH")

from gatewizard.core.preparation import PreparationManager

# Step 1: Run Propka analysis
analyzer = PreparationManager()
pka_file = analyzer.run_analysis("protein.pdb", output_dir="output")
summary_file = analyzer.extract_summary(pka_file, output_dir="output")
residues = analyzer.parse_summary(summary_file)

# Step 2: Define expected model pKa values
expected_pka = {
    "ASP": 3.9,
    "GLU": 4.3,
    "HIS": 6.0,
    "LYS": 10.5,
    "ARG": 12.5,
    "CYS": 8.3,
    "TYR": 10.1,
}

# Step 3: Find residues with significant pKa shifts (>1.0 units)
print("Residues with unusual pKa shifts (>1.0 units):")
print("=" * 70)

shifted_residues = []
for res in residues:
    # Only analyze protein residues (skip ligands)
    if res["res_id"] == 0:
        continue

    res_name = res["residue"]
    if res_name in expected_pka:
        pka_diff = abs(res["pka"] - expected_pka[res_name])
        if pka_diff > 1.0:
            shifted_residues.append(
                {
                    "id": f"{res_name}{res['res_id']}_{res['chain']}",
                    "type": res_name,
                    "pka": res["pka"],
                    "expected": expected_pka[res_name],
                    "shift": res["pka"] - expected_pka[res_name],
                }
            )
            print(
                f"{res_name}{res['res_id']:3d} (Chain {res['chain']}): "
                f"pKa={res['pka']:5.2f} (expected ~{expected_pka[res_name]:.1f}, "
                f"shift={res['pka'] - expected_pka[res_name]:+.1f})"
            )

print(f"\nSummary: Found {len(shifted_residues)} residues with significant pKa shifts")

# Step 4: Categorize by shift direction
upshifted = [r for r in shifted_residues if r["shift"] > 1.0]
downshifted = [r for r in shifted_residues if r["shift"] < -1.0]

print(f"  Upshifted (more basic): {len(upshifted)}")
print(f"  Downshifted (more acidic): {len(downshifted)}")

# Step 5: Find extreme cases (>2.0 units shift)
extreme_shifts = [r for r in shifted_residues if abs(r["shift"]) > 2.0]
if extreme_shifts:
    print(f"\n⚠ Extreme shifts (>2.0 units): {len(extreme_shifts)}")
    for r in extreme_shifts:
        print(f"  {r['id']}: {r['shift']:+.2f} units")
```

**Example Output:**

```
Residues with unusual pKa shifts (>1.0 units):
======================================================================
ASP 42 (Chain A): pKa= 6.23 (expected ~3.9, shift=+2.3)
GLU 58 (Chain A): pKa= 2.15 (expected ~4.3, shift=-2.2)
HIS 77 (Chain A): pKa= 8.45 (expected ~6.0, shift=+2.5)
LYS115 (Chain A): pKa= 8.32 (expected ~10.5, shift=-2.2)
CYS128 (Chain A): pKa=10.87 (expected ~8.3, shift=+2.6)

Summary: Found 5 residues with significant pKa shifts
  Upshifted (more basic): 3
  Downshifted (more acidic): 2

⚠ Extreme shifts (>2.0 units): 5
  ASP42_A: +2.33 units
  GLU58_A: -2.15 units
  HIS77_A: +2.45 units
  LYS115_A: -2.18 units
  CYS128_A: +2.57 units
```

**Interpretation:**

- **Large upshifts** (more basic): May indicate buried residues, salt bridges, or hydrogen bonding networks
- **Large downshifts** (more acidic): May indicate proximity to positive charges or unusual electrostatic environments
- **Extreme shifts (>2 units)**: Often mark functionally important residues like active sites, binding pockets, or structural switches

---

## Module-level helpers

### run_propka

```python
from gatewizard.core.preparation import run_propka
```

Run the PropKa binary on a PDB (used internally by `PreparationManager.run_analysis`).

### extract_summary_section

```python
from gatewizard.core.preparation import extract_summary_section
```

Write the “summary of prediction” block from a `.pka` file.

### parse_summary_section

```python
from gatewizard.core.preparation import parse_summary_section
```

Parse that summary into residue dictionaries.

### modify_pdb_based_on_summary

```python
from gatewizard.core.preparation import modify_pdb_based_on_summary
```

Apply protonation names from a parsed summary to a PDB.

### count_protein_hydrogens

```
from gatewizard.core.preparation import count_protein_hydrogens
```

Count protein hydrogen atoms (ligands/hetero H ignored).

### complete_missing_heavy_atoms

```
from gatewizard.core.preparation import complete_missing_heavy_atoms
```

Fill **missing protein atoms** from Amber residue templates (`tleap` + `leaprc.protein.ff19SB`). Ligands, waters, and ions are copied unchanged. Missing loop residues are built only by `fill_short_gaps` (1–6 residues). CONECT/LINK records are dropped because atom serials change after `savePdb`.

Call this **after** PropKa / Amber name changes (ASP→ASH, GLU→GLH, HIS→HID/HIE/HIP). `tleap` then adds the extra protons from those templates — the same step packmol-memgen uses at final parametrization, but protein-only (no water or ions). If you complete on ASP and only rename to ASH afterward, HD2 is never placed.

`pdb4amber --reduce` can still rebuild Amber hydrogens when the Amber `reduce` binary is on `PATH` (or `CONDA_PREFIX/bin`). If `reduce` is missing, Prepare keeps the tleap template hydrogens and runs `pdb4amber` without `--reduce` instead of failing. The GUI Prepare button applies names first, then this tleap pass.

### classify_short_gaps

```
from gatewizard.core.short_gaps import classify_short_gaps
```

List numbering gaps. A gap of **1 to 6** residues whose sequence is in SEQRES is marked `fill`. A longer gap, a gap with no SEQRES sequence, or a sequence that is not a standard amino acid is marked `skip` and is not built.

### fill_short_gaps

```
from gatewizard.core.short_gaps import fill_short_gaps
```

Build every short gap with PDBFixer, then insert only those new residue atoms into a copy of the original PDB. Later atom serials increase by the number of atoms inserted before them, and CONECT serials follow. Ligands, waters, `LINK`, and the other original lines stay as they were. Pass `chains` to keep only those chains. Pass `sequences` to use those residue names instead of SEQRES for the matching gaps. Gaps longer than 6 residues stay open. Requires `pdbfixer` next to OpenMM.

### minimize_selection

```
from gatewizard.core.minimize_selection import minimize_selection
```

Minimize the selected protein or ion atoms with OpenMM (`amber14-all.xml` plus the Amber ion parameters, a short L-BFGS pass by default). Every other atom Amber can build stays fixed. Selected residues Amber cannot build are relaxed with a repulsive Lennard-Jones bump and bond springs, and the rest of the structure stays fixed. A mixed selection runs OpenMM first, then that bump, and writes one PDB. `tolerance` is the OpenMM force tolerance in kJ/mol/nm. `max_rmsd` caps how far the selected atoms of each pass move, in angstroms. The rest of the file is copied. This is a clash nudge, not a stable energy minimum. It is separate from filling a short gap: PDBFixer already minimizes only the atoms it just added.

**restore_original_residue_numbers**

Rewrite ATOM/HETATM residue numbers **and chain IDs** back to the pre-`pdb4amber` (and, when a cap map is given, pre-cap) ids. Missing loop residues are not built; a gap of 200 then 205 stays a gap. After tleap completion, protein is often one blank chain with sequential ids; restore pairs polymer residues in file order to the original PDB.

**ACE / NME when preserve is on** (ProteinCapper itself still writes ACE1 / protein+1 for the MD path):

| Original N-terminus | ACE | Protein | NME |
|---|---|---|---|
| 1 | **0** | unchanged | last + 1 |
| 21 | 20 | unchanged | last + 1 |
| 200 then 205 (gap) | 199 | 200 and 205 kept | 206 |

Residue 0 is legal PDB. AmberTools / tleap usually want sequential ids starting at 1 — uncheck preserve or call this method with `preserve_residue_numbers=False` for that path.

```
from pathlib import Path

from gatewizard.core.preparation import PreparationManager

here = Path(__file__).resolve().parent
out_dir = here / "output"
out_dir.mkdir(exist_ok=True)

sequential = out_dir / "preserve_numbers_sequential.pdb"
sequential.write_text(
    "ATOM      1  CA  ALA A   1       1.000   1.000   1.000  1.00  0.00           C\n"
    "ATOM      2  CA  GLY A   2       2.000   2.000   2.000  1.00  0.00           C\n"
    "HETATM    3  C1  LIG A   3       3.000   3.000   3.000  1.00  0.00           C\n"
    "ATOM      4  C   ACE B   4       4.000   4.000   4.000  1.00  0.00           C\n"
    "ATOM      5  CA  VAL B   5       5.000   5.000   5.000  1.00  0.00           C\n"
    "ATOM      6  N   NME B   6       6.000   6.000   6.000  1.00  0.00           N\n"
    "END\n",
    encoding="utf-8",
)
remum = out_dir / "preserve_numbers_sequential_renum.txt"
remum.write_text(
    "ALA A      1    ALA    10\n"
    "GLY A      2    GLY    20\n"
    "LIG A      3    LIG   900\n"
    "ACE B      4    ACE     1\n"
    "VAL B      5    VAL     2\n"
    "NME B      6    NME     3\n",
    encoding="utf-8",
)
cap_map = out_dir / "preserve_numbers_gatewizard_residue_mapping.txt"
cap_map.write_text(
    "# Format: ORIGINAL_RESNAME CHAIN ORIGINAL_ID FINAL_RESNAME FINAL_ID\n"
    "  - B      -    ACE     1\n"
    "VAL B     21    VAL     2\n"
    "  - B      -    NME     3\n",
    encoding="utf-8",
)

restored = out_dir / "preserve_numbers_restored.pdb"
manager = PreparationManager()
result = manager.restore_original_residue_numbers(
    str(sequential),
    remum_path=str(remum),
    output_pdb=str(restored),
    cap_mapping_path=str(cap_map),
)

print(f"✓ Restored residue numbers: {result['output_file']}")
print(f"  Cap assignments: {result['cap_assignments']}")
assert result["residue_numbers_preserved"]
assert restored.is_file()
text = restored.read_text(encoding="utf-8")
assert "ALA A  10" in text
assert "GLY A  20" in text
assert "LIG A 900" in text
assert "ACE B  20" in text
assert "VAL B  21" in text
assert "NME B  22" in text
```

### parse_residue_mapping_by_chain

```
from gatewizard.utils.residue_mapping import parse_residue_mapping_by_chain
```

Load `(chain, final_resid) → original_resid`. Cap lines with original `-` are `None`. Two chains that share residue 21 stay distinct.

### parse_residue_mapping_file

```
from gatewizard.utils.residue_mapping import parse_residue_mapping_file
```

Chain-blind `final_resid → original_resid` for Analysis overlays. Caps with original `-` are skipped. Prefer `parse_residue_mapping_by_chain` when restoring a PDB.

#### strip_protein_hydrogens

```
from gatewizard.core.preparation import strip_protein_hydrogens
```

Write a PDB with protein hydrogens removed; ligand hydrogens stay.

#### detect_terminal_caps

```
from gatewizard.utils.protein_capping import detect_terminal_caps
```

Return ACE/NME (or similar) cap residue names present in a PDB.

#### is_already_capped

```
from gatewizard.utils.protein_capping import is_already_capped
```

True when terminal caps are already present.

## Important Notes

### Output Directory Behavior

⚠️ **Important**: When using `output_dir`, the code automatically:

- Creates the directory if it doesn't exist
- Converts input file paths to absolute paths
- Places all output files in the specified directory

### PDB Records vs Residues

⚠️ **Important distinction**:

- **Residue changes**: Number of unique residues modified
- **PDB record changes**: Number of ATOM/HETATM lines modified

Example: Changing 1 histidine residue with 10 atoms:

- `residue_changes` = 1
- `record_changes` = 10

### Dictionary Key Names

⚠️ **Common Mistake**: The parsed residues use these exact key names:

- ✓ Correct: `res['residue']` (not `res['residue_name']`)
- ✓ Correct: `res['res_id']` (not `res['residue_number']`)
- ✓ Correct: `res['chain']` (not `res['chain_id']`)
- ✓ Correct: `res['atom']` (atom name for ligands, empty for protein residues)
- ✓ Correct: `res['atom_type']` (atom type classification like 'N31', 'OCO', 'OP')

**Note on Ligands:**

- Protein residues have `res_id > 0`, `atom == ''`, `atom_type == ''`

- Ligand atoms have `res_id == 0`, `atom` contains atom name, `atom_type` contains classification
- Use `res_id` to distinguish: `if res['res_id'] > 0:` → protein, else → ligand

#### Output Files

The analysis generates several files: - `*.pka` - Full Propka output - `*_summary_of_prediction.txt` - Extracted summary section - `*_protonated.pdb` - After applying protonation states - `*_ss.pdb` - After applying disulfide bonds - `*_gatewizard_residue_mapping.txt` - Residue mapping (if capping used)

## Builder Module

Module for preparing membrane protein systems with automated lipid bilayer construction, solvation, and parametrization. This module includes:

- Membrane system building with packmol-memgen
- Custom lipid composition (asymmetric membranes supported)
- Force field selection and validation
- Salt addition and ionic strength control
- Two-stage Propka workflow support
- Automated parametrization with pdb4amber and tleap

### Import

```
from gatewizard.core.builder import Builder
from gatewizard.tools.force_fields import ForceFieldManager
```

### Class: Builder

Main class for building membrane protein systems with complete control over lipid composition, force fields, and system parameters.

#### Constructor

```
Builder()
```

**Parameters:** None

**Returns:** `Builder` instance

**Default Configuration:**

| Parameter | Default | Description |
|---|---|---|
| `water_model` | `"tip3p"` | Water model for solvation |
| `protein_ff` | `"ff19SB"` | Protein force field |
| `lipid_ff` | `"lipid21"` | Lipid force field |
| `preoriented` | `True` | Protein already oriented for membrane insertion |
| `parametrize` | `True` | Run parametrization with tleap |
| `salt_concentration` | `0.15` | Salt concentration in M (molar) |
| `cation` | `"K+"` | Cation type for salt |
| `anion` | `"Cl-"` | Anion type for salt |
| `dist` | `12` | Minimum solute-to-box-boundary distance in Å |
| `dist_wat` | `26` | Water layer thickness in Å |

| Parameter | Default | Description |
|---|---|---|
| `notprotonate` | `True` | Skip protonation (preserve PropKa residue names) |

### Example 1: Basic Configuration

```
from gatewizard.core.builder import Builder

builder = Builder()
print(f"Default water model: {builder.config['water_model']}")
print(f"Default protein force field: {builder.config['protein_ff']}")
print(f"Default lipid parameters: {builder.config['lipid_ff']}")
```

---

## Configuration Methods

### set_configuration

Update configuration parameters for system building.

```
set_configuration(**kwargs)
```

**Parameters:**

| Parameter | Type | Default | Description |
|---|---|---|---|
| `water_model` | `str` | `"tip3p"` | Water model (tip3p, tip4p, spce, opc) |
| `protein_ff` | `str` | `"ff19SB"` | Protein force field (ff14SB, ff19SB) |
| `lipid_ff` | `str` | `"lipid21"` | Lipid force field (lipid17, lipid21) |
| `preoriented` | `bool` | `True` | Whether protein is pre-oriented |
| `parametrize` | `bool` | `True` | Run parametrization after packing |
| `salt_concentration` | `float` | `0.15` | Salt concentration in M |
| `cation` | `str` | `"K+"` | Cation type (Na+, K+, etc.) |
| `anion` | `str` | `"Cl-"` | Anion type (Cl-, Br-, etc.) |
| `dist` | `float` | `12` | Minimum solute-to-box-boundary distance in Å |
| `dist_wat` | `float` | `26` | Water layer thickness in Å |
| `notprotonate` | `bool` | `True` | Skip protonation during parametrization |
| `add_salt` | `bool` | `True` | Whether to add salt to system |

**Returns:** None

### Example 2: Custom Configuration

```
from gatewizard.core.builder import Builder

builder = Builder()

# Configure for specific system
builder.set_configuration(
    water_model="tip3p",
    protein_ff="ff14SB",
    lipid_ff="lipid21",
    salt_concentration=0.15,
    cation="Na+",
    anion="Cl-",
    dist_wat=20.0,  # Larger water layer
    preoriented=True,
)
```

### Example 3: Available Water Models

```
from gatewizard.tools.force_fields import ForceFieldManager

ff_manager = ForceFieldManager()

# Get available water models
water_models = ff_manager.get_water_models()

print("\nAvailable Water Models:")
for water in water_models:
    print(f"  - {water}")

print(f"\nTotal: {len(water_models)} water models available")
```

Similarly can be done for lipids:

### Example 4: Available Lipid Models

```
from gatewizard.tools.force_fields import ForceFieldManager

ff_manager = ForceFieldManager()

# Residue names from packmol-memgen --available_lipids_all
lipids = ff_manager.get_available_lipids()
catalog = {row["name"]: row for row in ff_manager.get_lipid_catalog()}

print(f"Total available lipids: {len(lipids)}\n")
for name in ("POPC", "POPE", "CHL1", "1LAC"):
    row = catalog[name]
    print(f"  {name}  charge {row['charge']}  {row['full_name']}")

print(f"\nTotal: {len(lipids)} lipid models available")
```

### Example 5: Available Protein Force Fields

```
from gatewizard.tools.force_fields import ForceFieldManager

ff_manager = ForceFieldManager()

# Get available protein force fields
protein_ffs = ff_manager.get_protein_force_fields()

print("\nAvailable Protein Force Fields:")
for protein_ff in protein_ffs:
    print(f"  - {protein_ff}")

print(f"\nTotal: {len(protein_ffs)} protein force fields available")
```

### Example 6: Available Lipid Force Fields

```
from gatewizard.tools.force_fields import ForceFieldManager

ff_manager = ForceFieldManager()

# Get available lipid force fields
lipid_ffs = ff_manager.get_lipid_force_fields()

print("\nAvailable Lipid Force Fields:")
for lipid_ff in lipid_ffs:
    print(f"  - {lipid_ff}")

print(f"\nTotal: {len(lipid_ffs)} lipid force fields available")
```

---

## Validation Methods

Before preparing systems, you can validate your inputs and check force field compatibility.

### validate_system_inputs

Validate all inputs before system preparation.

```
validate_system_inputs(
    pdb_file: str,
    upper_lipids: List[str],
    lower_lipids: List[str],
    lipid_ratios: str = "",
    **kwargs
) -> Tuple[bool, str]
```

**Parameters:**

| Parameter | Type | Required | Description |
|---|---|---|---|
| `pdb_file` | `str` | No | Path to input PDB file, or empty/None for a bilayer-only build |
| `upper_lipids` | `List[str]` | Yes | List of lipids for upper leaflet |
| `lower_lipids` | `List[str]` | Yes | List of lipids for lower leaflet |
| `lipid_ratios` | `str` | No | Lipid ratios string |
| `**kwargs` | `Any` | No | Additional parameters to validate |

**Returns:** `Tuple[bool, str]`

- `bool`: Validation status
- `str`: Error message (empty if valid)

**Validation Checks:**

1. PDB file exists and is readable
2. All lipid types are valid/available
3. Lipid ratios match number of lipids
4. Ratios are positive numbers
5. Force field combinations are compatible
6. Ion types are valid

### Example 7: Input Validation

```
from gatewizard.core.builder import Builder

builder = Builder()

# Validate inputs before preparation
valid, msg = builder.validate_system_inputs(
    pdb_file="protein_protonated_prepared.pdb",
    upper_lipids=["POPC", "POPE"],
    lower_lipids=["POPC", "POPE"],
    lipid_ratios="7:3//7:3",
    water_model="tip3p",
    protein_ff="ff14SB",
    lipid_ff="lipid21",
)

if valid:
    if "WARNING" in msg:
        print(f"[!] Inputs valid with warning: {msg}")
        print("You may proceed at your own risk")
    else:
        print("[OK] All inputs are valid, proceed with preparation")
    # Now call prepare_system()
else:
    print(f"[ERROR] Validation failed: {msg}")
    # Fix issues before proceeding
```

### Example 8: Force Field Validation

```
from gatewizard.tools.force_fields import ForceFieldManager

ff_manager = ForceFieldManager()
valid, message, is_warning = ff_manager.validate_combination(
    "tip3p", "ff14SB", "lipid21"
)

if valid:
    if is_warning:
        print(f"[!] Warning: {message}")
    else:
        print("[OK] Force field combination is compatible")
else:
    print(f"[ERROR] Incompatible: {message}")
```

---

## Core Preparation Methods

### prepare_system

Prepare a complete membrane protein system with lipid bilayer, water, and ions.

```
prepare_system(
    pdb_file: str,
    working_dir: str,
    upper_lipids: List[str],
    lower_lipids: List[str],
    lipid_ratios: str = "",
    **kwargs
) -> Tuple[bool, str, Optional[Path]]
```

**Parameters:**

| Parameter | Type | Required | Description |
|---|---|---|---|
| `pdb_file` | `str` | No | Path to input PDB file (oriented protein). Omit or pass `None` for a bilayer-only build |
| `working_dir` | `str` | Yes | Working directory for output files |

| Parameter | Type | Required | Description |
|---|---|---|---|
| `upper_lipids` | `List[str]` | Yes | List of lipid types for upper leaflet |
| `lower_lipids` | `List[str]` | Yes | List of lipid types for lower leaflet |
| `lipid_ratios` | `str` | No | Lipid molar ratios (format: "ratio1:ratio2//ratio3:ratio4") |
| `output_folder_name` | `str` | No | Custom output folder name (kwarg) |
| `distxy_fix` | `float` | No | Membrane XY size in Å (`--distxy_fix`). **Required** when `pdb_file` is omitted |
| `solutes` | `list` | No | Free molecules to pack: `[{"pdb": "TEA.pdb", "concentration": "4", "in_membrane": False}]` |
| `solute_inmem` | `bool` | No | Place free molecules in the membrane instead of water (`--solute_inmem`) |
| `solute_prot_dist` | `float` | No | Keep free molecules this far from the protein, in Å (`--solute_prot_dist`) |
| `wait` | `bool` | No | Block until the job completes or errors (default `False`) |
| `wait_timeout` | `float` | No | Maximum seconds to wait when `wait=True` (`None` = unlimited) |
| `wait_poll_interval` | `float` | No | Seconds between status checks (default `5.0`) |
| `wait_verbose` | `bool` | No | Print elapsed-time progress while waiting (default `True`) |
| `**kwargs` | `Any` | No | Additional configuration parameters (override defaults) |

**Returns:** `Tuple[bool, str, Optional[Path]]`

- `bool`: Success status
- `str`: Status message
- `Optional[Path]`: Job directory path (None if failed)

**Lipid Ratios Format:**

The `lipid_ratios` string specifies molar ratios for each leaflet:

- Format: `"upper_ratio1:upper_ratio2//lower_ratio1:lower_ratio2"`
- Ratios must match the order of lipids in the lipid lists
- Ratios are normalized automatically (don't need to sum to 1.0)
- Use `//` to separate upper and lower leaflet ratios

**Available lipids:**

GateWizard supports **4028** atomistic lipids from `packmol-memgen --available_lipids_all` (version 2025.1.29). Each row has the residue name, the charge, and the full chemical name. `ForceFieldManager.get_available_lipids()` returns the residue names, and `get_lipid_catalog()` returns the charge, the chemical name, and the note.

The catalog lists 4055 names: 3770 are marked `Parameters from PACKMOL-Memgen Lipid_ext`, 251 have no extra note, 7 are marked Lipid21 only, and 27 are marked SIRAH only. The supported total is 4028 (4055 minus the 27 SIRAH lipids).

SIRAH and coarse-grained lipids are not supported. The 27 SIRAH names start with `si`. The 258 Amber lipids (no Lipid_ext note, including POPC) are parametrized with lipid21 or lipid17 alone. The 3770 Lipid_ext lipids are parametrized by sourcing `leaprc.lipid_ext` from the installed PACKMOL-Memgen package after that Amber lipid leaprc. Those parameters are not part of Amber's `lipid21.lib`. A mix of POPC and one Lipid_ext lipid still loads the extension. tleap has to exit with `Errors = 0` and write a non-empty `system.prmtop` and `system.inpcrd`; a failed tleap fails the build.

A familiar subset:

| Category | Lipid Types |
|---|---|
| **Phosphatidylcholine (PC)** | DHPC, DLPC, DMPC, DPPC, DSPC, DOPC, POPC, PLPC, SOPC |
| **Phosphatidylethanolamine (PE)** | DHPE, DLPE, DMPE, DPPE, DSPE, DOPE, POPE, PLPE, SOPE |

| Category | Lipid Types |
|---|---|
| **Phosphatidylserine (PS)** | DHPS, DLPS, DMPS, DPPS, DSPS, DOPS, POPS, PLPS, SOPS |
| **Phosphatidylglycerol (PG)** | DHPG, DLPG, DMPG, DPPG, DSPG, DOPG, POPG, PLPG, SOPG |
| **Phosphatidic Acid (PA)** | DHPA, DLPA, DMPA, DPPA, DSPA, DOPA, POPA, PLPA, SOPA |
| **Sphingomyelins** | PSM, ASM, LSM, MSM, HSM, OSM |
| **Cholesterol** | CHL1 |

**Output Files:**

In `{output_folder_name}/` directory:

- `bilayer_*.pdb` - Packed system (protein in membrane)
- `bilayer_*_lipid.pdb` - Lipid bilayer with CRYST1 info (for equilibration)
- `system.prmtop` - AMBER topology file (if parametrize=True)
- `system.inpcrd` - AMBER coordinate file (if parametrize=True)
- `system_solv.pdb` - Solvated system in PDB format
- `system_4amber.pdb` - PDB prepared by pdb4amber
- `tleap.in` - tleap input script
- `logs/preparation.log` - Main execution log
- `logs/parametrization.log` - Parametrization log
- `status.json` - Job status tracking
- `run_preparation.sh` - Execution script

A packed PDB with residue numbers past 9998 or atom serials past 99999 is rewritten before `pdb4amber`. The leap input sets the CRYST1 box, because `loadPDB` does not copy it, and adds disulfide bonds for CYX SG atoms. `system.pdb` is rewritten after `savePDB` when tleap shifts the columns, so equilibration restraints can read it.

**Raises:**

- `FileNotFoundError` - If input PDB file doesn't exist
- `BuilderError` - If validation fails or system preparation fails

**Note:** By default, system preparation runs in the background and the method returns immediately. Pass `wait=True` to block until the job finishes — useful for scripting sequential preparations. For asynchronous monitoring, use the log files or the `JobMonitor` class (see examples below).

**Bilayer-only and free molecules**

packmol-memgen can pack a membrane **without a protein**. Omit `pdb_file` and set `distxy_fix` (Å) for the XY size. Free copies of a molecule go in `solutes` (`--solute` / `--solute_con`); parametrize them first and pass `ligand_params` plus GAFF2. The membrane-protein path (`pdb_file` required, `--pdb` + `--preoriented`) is unchanged.

```python
from gatewizard.core.builder import Builder

builder = Builder()

# Membrane only (no protein): XY size is required
success, msg, job_dir = builder.generate_preparation_inputs(
    pdb_file=None,
    working_dir="./systems",
    upper_lipids=["DOPE", "DOPG"],
    lower_lipids=["DOPE", "DOPG"],
    lipid_ratios="3:1//3:1",
    distxy_fix=100,
)

# Free ligand in water around a protein
success, msg, job_dir = builder.generate_preparation_inputs(
    pdb_file="1BL8.pdb",
    working_dir="./systems",
    upper_lipids=["DOPE", "DOPG"],
    lower_lipids=["DOPE", "DOPG"],
    lipid_ratios="3:1//3:1",
    solutes=[{"pdb": "TEA.pdb", "concentration": "4"}],
    solute_prot_dist=10,
    ligand_params={"TEA": {"frcmod": "TEA.frcmod", "lib": "TEA.lib"}},
)

# Ligand + bilayer, no protein (place copies in the membrane)
success, msg, job_dir = builder.generate_preparation_inputs(
    pdb_file=None,
    working_dir="./systems",
    upper_lipids=["POPC"],
    lower_lipids=["POPC"],
    lipid_ratios="1//1",
    distxy_fix=75,
    solutes=[{"pdb": "TEA.pdb", "concentration": "0.1M", "in_membrane": True}],
    ligand_params={"TEA": {"frcmod": "TEA.frcmod", "lib": "TEA.lib"}},
)
```

`concentration` is a molecule count (`"4"`) or a packmol-memgen concentration (`"0.1M"`, `"2%"`). `--solute_inmem` is set when any solute has `in_membrane=True` or when `solute_inmem=True`. `--solute_prot_dist` is only passed when a protein PDB is present.

### Example 9: Simple Symmetric Membrane

```
from gatewizard.core.builder import Builder

builder = Builder()

# Configure system
builder.set_configuration(
    water_model="tip3p",
    protein_ff="ff14SB",
    lipid_ff="lipid21",
    salt_concentration=0.15,
    cation="K+",
    anion="Cl-",
)

# Prepare system with 100% POPC (symmetric)
success, message, job_dir = builder.prepare_system(
    pdb_file="protein_protonated_prepared.pdb",
    working_dir="./systems",
    upper_lipids=["POPC"],
    lower_lipids=["POPC"],
    lipid_ratios="1//1",  # 100% POPC both leaflets
    output_folder_name="popc_membrane",
)

if success:
    print(f"✓ System preparation started in background")
    print(f"  {message}")
    print(f"  Job directory: {job_dir}")
    print(f"  Monitor progress: {job_dir / 'preparation.log'}")
    print(f"  When complete, files will be at:")
    print(f"    - Topology: {job_dir / 'system.prmtop'}")
    print(f"    - Coordinates: {job_dir / 'system.inpcrd'}")
else:
    print(f"✗ Preparation failed: {message}")
```

### Example 10: Monitoring Job Progress

```python
from gatewizard.core.job_monitor import JobMonitor
from pathlib import Path

# Create monitor for your working directory
monitor = JobMonitor(working_directory=Path("./systems"))

# Scan for jobs
monitor.scan_for_jobs(force=True)

# Get active jobs
active_jobs = monitor.get_active_jobs()

if active_jobs:
    print(f"✓ Found {len(active_jobs)} active job(s)")

    for job_id, job_info in active_jobs.items():
        print(f"Job: {job_info.job_dir.name}")
        print(f"  Status: {job_info.status.value}")
        print(f"  Progress: {job_info.progress:.1f}%")
        print(f"  Current step: {job_info.current_step}")
        print(f"  Elapsed: {job_info.elapsed_time:.1f}s")

        # Show completed steps
        if job_info.steps_completed:
            print(f"  Completed steps:")
            for step in job_info.steps_completed:
                print(f"    ✓ {step}")
else:
    print("No active jobs found")

# Check for completed jobs
completed_jobs = monitor.get_completed_jobs()
if completed_jobs:
    print(f"✓ Found {len(completed_jobs)} completed job(s)")
    for job_id, job_info in completed_jobs.items():
        print(f"  - {job_info.job_dir.name}: {job_info.status.value}")
```

### Example 11: Asymmetric Membrane with Multiple Lipids

```
from gatewizard.core.builder import Builder

builder = Builder()

# Configure for asymmetric membrane
builder.set_configuration(
    water_model="tip3p",
    protein_ff="ff14SB",
    lipid_ff="lipid21",
    # Note: When using anionic lipids (POPS, POPG, etc.), the system may require
    # a higher salt concentration for neutralization. If you get an error like:
    # "The concentration of ions required to neutralize the system is higher than
    # the concentration specified", increase salt_concentration or use add_salt=False.
    # POPS is anionic (-1 charge), so 50% POPS in lower leaflet adds significant
    # negative charge requiring more cations for neutralization.
    salt_concentration=0.5,  # Increased from default 0.15 M due to anionic lipids
    dist_wat=20.0,  # Thicker water layer
)

# Upper leaflet: 70% POPC + 30% cholesterol
# Lower leaflet: 50% POPE + 50% POPS (anionic)
success, message, job_dir = builder.prepare_system(
    pdb_file="protein_protonated_prepared.pdb",
    working_dir="./systems",
    upper_lipids=["POPC", "CHL1"],
    lower_lipids=["POPE", "POPS"],
    lipid_ratios="7:3//5:5",  # Ratios normalized automatically
    output_folder_name="asymmetric_membrane",
)

if success:
    print(f"✓ System preparation started in background")
    print(f"  {message}")
    print(f"  Upper leaflet: 70% POPC, 30% CHL1")
    print(f"  Lower leaflet: 50% POPE, 50% POPS")
    print(f"  Job directory: {job_dir}")
    print(f"  Monitor: {job_dir / 'logs/preparation.log'}")
else:
    print(f"✗ Preparation failed: {message}")
```

### Example 12: Complex Composition (Plasma Membrane Mimic)

```
from gatewizard.core.builder import Builder

builder = Builder()

# Plasma membrane-like composition
# Upper: PC-rich with cholesterol
# Lower: PE/PS-rich (cytoplasmic side)
success, message, job_dir = builder.prepare_system(
    pdb_file="protein_protonated_prepared.pdb",
    working_dir="./systems",
    upper_lipids=["POPC", "POPE", "CHL1"],
    lower_lipids=["POPE", "POPS", "CHL1"],
    lipid_ratios="5:2:3//4:4:2",  # Upper: 50% POPC, 20% POPE, 30% CHL1
    # Lower: 40% POPE, 40% POPS, 20% CHL1
    output_folder_name="plasma_membrane",
    salt_concentration=0.5,
    cation="Na+",  # Use sodium instead of potassium
    dist_wat=25.0,  # Extra water for large protein
)

if success:
    print(f"✓ System preparation started in background")
    print(f"  {message}")
    print(f"  Job directory: {job_dir}")
    print(f"  Monitor: {job_dir / 'logs/preparation.log'}")
else:
    print(f"✗ Preparation failed: {message}")
```

### Example 13: Packing Only (No Parametrization)

```
from gatewizard.core.builder import Builder

builder = Builder()

# Only pack the system, don't parametrize
# Useful for visual inspection before parametrization
success, message, job_dir = builder.prepare_system(
    pdb_file="protein_protonated_prepared.pdb",
    working_dir="./systems",
    upper_lipids=["POPC"],
    lower_lipids=["POPC"],
    lipid_ratios="1//1",
    output_folder_name="packed_only",
    parametrize=False,  # Skip parametrization
)

if success:
    print(f"✓ Packing started in background (no parametrization)")
    print(f"  {message}")
    print(f"  Job directory: {job_dir}")
    print(f"  Monitor: {job_dir / 'logs/preparation.log'}")
    print(f"  When complete, inspect: {job_dir / 'bilayer_*.pdb'}")
else:
    print(f"✗ Preparation failed: {message}")
```

### Example 14: Custom Salt Concentration

```
from gatewizard.core.builder import Builder

builder = Builder()

# High salt concentration for ionic strength studies
success, message, job_dir = builder.prepare_system(
    pdb_file="protein_protonated_prepared.pdb",
    working_dir="./systems",
    upper_lipids=["POPC", "POPS"],
    lower_lipids=["POPC", "POPS"],
    lipid_ratios="8:2//8:2",  # 80% POPC, 20% POPS
    output_folder_name="high_salt",
    salt_concentration=2.0,  # 2000 mM (high salt)
    cation="Na+",
    anion="Cl-",
)

if success:
    print(f"✓ System preparation started in background")
    print(f"  {message}")
    print(f"  High salt: 2.0 M NaCl")
    print(f"  Job directory: {job_dir}")
    print(f"  Monitor: {job_dir / 'logs/preparation.log'}")
else:
    print(f"✗ Preparation failed: {message}")
```

**Example 15: No Salt (Charge Neutralization Only)**

```
from gatewizard.core.builder import Builder

builder = Builder()

# Neutralize system charges only (no extra salt)
success, message, job_dir = builder.prepare_system(
    pdb_file="protein_protonated_prepared.pdb",
    working_dir="./systems",
    upper_lipids=["POPC"],
    lower_lipids=["POPC"],
    lipid_ratios="1//1",
    output_folder_name="no_salt",
    salt_concentration=0.0,  # Only neutralize, no extra salt
    add_salt=True,  # Still add ions for neutralization
)

if success:
    print(f"✓ System preparation started in background")
    print(f"  {message}")
    print(f"  Neutralization only, no extra salt")
    print(f"  Job directory: {job_dir}")
    print(f"  Monitor: {job_dir / 'logs/preparation.log'}")
else:
    print(f"✗ Preparation failed: {message}")
```

---

**run_preparation**

```
success, message = builder.run_preparation(job_dir)
```

Launch a previously generated preparation job (`run_preparation.sh` in `job_dir`). Returns `(False, ...)` if the script is missing or a job is already running.

**Parameters:** - `job_dir` (str or Path): Job directory created by `prepare_system` / `generate_preparation_inputs`.

**cancel_preparation**

```
result = builder.cancel_preparation(job_dir) -> dict
```

Cancel a running preparation job. Kills the detached `run_preparation.sh` process group (PID in `process.pid`) and marks `status.json` as `cancelled`. If the job already finished, returns `stopped=False` and the existing status.

**Returns:** `{success, job_dir, stopped, status, message}`.

**prepare_system_stage1_for_propka**

```
success, message, job_dir = builder.prepare_system_stage1_for_propka(
    pdb_file, working_dir, upper_lipids, lower_lipids, lipid_ratios="", **kwargs
)
```

Stage 1 of the two-stage PropKa workflow: pack only (`parametrize=False`). After packing, apply PropKa residue names, then call stage 2.

**prepare_system_stage2_for_propka**

```
success, message, job_dir = builder.prepare_system_stage2_for_propka(
    packed_pdb_file, working_dir, **kwargs
)
```

Stage 2: parametrize a packed PDB that already has PropKa residue names (`notprotonate=True`).

**modify_residue_names_for_propka**

```
ok, message = builder.modify_residue_names_for_propka(
    packed_pdb_file, propka_results, output_file=None
)
```

Rewrite residue names in a packed PDB from a `{resid: new_name}` (or similar) PropKa map before stage 2.

See Example 25 for `wait=True` on `prepare_system`.

**wait_for_completion**

Block until a preparation job completes or fails by polling `status.json`.

```
wait_for_completion(
    job_dir,
    poll_interval: float = 5.0,
    timeout: Optional[float] = None,
    verbose: bool = True,
) -> Tuple[bool, str]
```

**Parameters:**

| Parameter | Type | Default | Description |
|---|---|---|---|
| `job_dir` | `Path \| str` | — | Path to the job directory |
| `poll_interval` | `float` | `5.0` | Seconds between status checks |
| `timeout` | `float \| None` | `None` | Maximum seconds to wait (`None` = no limit) |
| `verbose` | `bool` | `True` | Print elapsed-time progress to stdout |

**Returns:** `Tuple[bool, str]` — (success, message)

**Example 25: Blocking (synchronous) Preparation with wait=True**

```
"""
Builder Example 25: Blocking (synchronous) preparation with wait=True

Demonstrates how to run prepare_system() in blocking mode so that the
Python script does not continue until the job finishes (or errors).
This is especially useful for scripting multiple sequential preparations.

Two approaches are shown:
  A) Inline ``wait=True`` inside prepare_system()
  B) Launch in background + call wait_for_completion() later

NOTE: This example requires AmberTools and packmol-memgen.
      It uses the in-repo two-ligand PDB tests/2MVJ_2ligs.pdb.
"""

import shutil
import sys
from pathlib import Path

from gatewizard.core.builder import Builder
from gatewizard.tools.ligand_parametrization import (
    detect_ligands,
    parametrize_all_ligands,
)

if shutil.which("antechamber") is None:
    sys.exit("Skip: AmberTools antechamber is not on PATH")

pdb_file = str(Path(__file__).resolve().parent.parent / "2MVJ_2ligs.pdb")
working_dir = "./systems"

# ── Step 1: Detect and parametrize ligands (same as example 23) ──────
print("Step 1: Detecting ligands...")
ligands = detect_ligands(pdb_file)
for lig in ligands:
    print(f"  Found: {lig.name} ({lig.num_atoms} atoms, {lig.formula})")

print("\nStep 2: Parametrizing ligands...")
ligand_results = parametrize_all_ligands(
    pdb_file=pdb_file,
    output_dir=f"{working_dir}/ligand_params",
    charges={"AAA": 0, "BBB": 0},
    charge_method="bcc",
    atom_type="gaff2",  # GAFF2 atom types
)
print(f"  Parametrized: {list(ligand_results.keys())}")

# ── Step 3: Configure builder ─────────────────────────────────────────
builder = Builder()
builder.set_configuration(
    water_model="tip3p",
    protein_ff="ff14SB",
    lipid_ff="lipid21",
    preoriented=True,
    parametrize=True,
    salt_concentration=0.15,
    dist=12,
    dist_wat=26,
    notprotonate=True,
    ligand_params=ligand_results,
)

# ── Approach A: inline wait ──────────────────────────────────────────
# With wait=True the call blocks until the job finishes.
# prepare_system returns (success, message, job_dir) where
#   success reflects the *final* outcome, not just "launched OK".
#
# NOTE: The actual preparation is long-running.  Set a short timeout
#       so the example returns quickly during tests; remove or increase
#       the timeout for real production runs.
print("\n── Approach A: prepare_system with wait=True ──")
success_a, message_a, job_dir_a = builder.prepare_system(
```

```
    pdb_file=pdb_file,
    working_dir=working_dir,
    upper_lipids=["POPC"],
    lower_lipids=["POPC"],
    lipid_ratios="1//1",
    output_folder_name="membrane_2MVJ_wait",
    wait=True,  # ← block until done or error
    wait_timeout=10,  # short timeout for demo (use 3600+ for real runs)
    wait_poll_interval=2,  # check every 2 s
    wait_verbose=True,  # print elapsed time
)
print(f"Result : {success_a}")
print(f"Message: {message_a}")

# ── Approach B: launch then wait later ──────────────────────────────
print("\n── Approach B: launch + wait_for_completion() ──")
success_b, message_b, job_dir_b = builder.prepare_system(
    pdb_file=pdb_file,
    working_dir=working_dir,
    upper_lipids=["POPC"],
    lower_lipids=["POPC"],
    lipid_ratios="1//1",
    output_folder_name="membrane_2MVJ_bg",
)

if success_b and job_dir_b is not None:
    # Do other work here …
    print("Doing other work while system builds …")

    # Then block until that specific job is done
    completed, wait_msg = builder.wait_for_completion(
        job_dir_b,
        poll_interval=2,
        timeout=10,  # short timeout for demo
        verbose=True,
    )
    print(f"Completed: {completed}")
    print(f"Message  : {wait_msg}")
else:
    print(f"Not launched: {message_b}")

print("\nExample 25 finished.")
```

---

## Class: ForceFieldManager

```
from gatewizard.tools.force_fields import ForceFieldManager

ff = ForceFieldManager()
```

### get_water_models

```
ff.get_water_models() -> list[str]
```

See Example 3.

### get_protein_force_fields

See Example 5.

### get_lipid_force_fields

See Example 6.

#### get_available_lipids

Residue names from `packmol-memgen --available_lipids_all`. The stored catalog has 4055 names. GateWizard supports 4028 of them; the 27 SIRAH names are not supported.

#### get_lipid_catalog

The same lipids as records: `name`, `charge`, `full_name`, and `note`.

#### get_available_cations

#### get_available_anions

#### validate_combination

```
valid, message, is_warning = ff.validate_combination(water, protein_ff, lipid_ff)
```

See Example 8.

#### get_recommendations

```
ff.get_recommendations(system_type="membrane") -> dict
```

#### get_force_field_info

Return metadata for one force-field name.

#### validate_lipid

```
ff.validate_lipid(lipid_name) -> bool
```

#### validate_ion

```
ok, charge = ff.validate_ion(ion_name)
```

### Job Monitoring

Since system preparation runs in the background, GateWizard provides the `JobMonitor` class to track progress, check status, and manage running jobs.

#### Class: JobMonitor

Monitor and track system preparation jobs.

#### Example 16: JobMonitor class

```
from gatewizard.core.job_monitor import JobMonitor
from pathlib import Path

monitor = JobMonitor(working_directory=Path("./systems"))
```

#### Constructor Parameters:

| Parameter | Type | Default | Description |
|---|---|---|---|
| `working_directory` | `Path` or `str` | `Path.cwd()` | Directory to monitor for jobs |

**Returns:** `JobMonitor` instance

---

#### scan_for_jobs

Scan the working directory for preparation jobs.

```
monitor.scan_for_jobs(force=False)
```

**Parameters:**

| Parameter | Type | Default | Description |
|---|---|---|---|
| `force` | `bool` | `False` | Force scan even if within scan interval (3 seconds) |

**Returns:** None (updates internal job list)

**Description:** Scans the working directory for subdirectories containing `status.json` files, which indicate active or completed preparation jobs.

---

#### get_active_jobs

Get all currently running jobs.

```
active_jobs = monitor.get_active_jobs()
```

**Returns:** `Dict[str, JobInfo]`

- Dictionary mapping job IDs to `JobInfo` objects
- Only includes jobs with status `RUNNING`

---

#### get_completed_jobs

Get all completed or errored jobs.

```
completed_jobs = monitor.get_completed_jobs()
```

**Returns:** `Dict[str, JobInfo]`

- Dictionary mapping job IDs to `JobInfo` objects
- Includes jobs with status `COMPLETED` or `ERROR`

---

#### get_job

Get specific job by ID.

```
job = monitor.get_job(job_id)
```

**Parameters:**

| Parameter | Type | Required | Description |
|---|---|---|---|
| `job_id` | `str` | Yes | Job identifier (usually the full job directory path) |

**Returns:** `Optional[JobInfo]`

- `JobInfo` object if job exists
- `None` if job not found

---

#### remove_job

```
removed = monitor.remove_job(job_id) -> bool
```

Drop a job from the in-memory monitor. Does **not** delete files on disk.

#### cleanup_stale_jobs

```
monitor.cleanup_stale_jobs(max_age_seconds: float = 3600)
```

Remove jobs whose last update is older than `max_age_seconds`.

#### get_job_statistics

```
stats = monitor.get_job_statistics() -> dict
```

Counts of `total`, `running`, `completed`, `error`, and `unknown` jobs.

See Example 16 and Example 18.

#### refresh_job

Force refresh of a specific job's status.

```
success = monitor.refresh_job(job_id)
```

**Parameters:**

| Parameter | Type | Required | Description |
|---|---|---|---|
| `job_id` | `str` | Yes | Job identifier to refresh |

**Returns:** `bool`

- `True` if job was refreshed successfully
- `False` if job not found or refresh failed

---

#### JobInfo Object

Each job is represented by a `JobInfo` object with the following attributes:

| Attribute | Type | Description |
|---|---|---|
| `job_id` | `str` | Unique job identifier |
| `job_dir` | `Path` | Job directory path |
| `status` | `JobStatus` | Current status (RUNNING, COMPLETED, ERROR, UNKNOWN) |

| Attribute | Type | Description |
|---|---|---|
| `progress` | `float` | Progress percentage (0.0 - 100.0) |
| `current_step` | `str` | Name of current processing step |
| `steps_completed` | `List[str]` | List of completed step names |
| `elapsed_time` | `float` | Elapsed time in seconds |
| `start_time` | `datetime` | Job start timestamp |
| `end_time` | `Optional[datetime]` | Job end timestamp (None if running) |

**JobStatus Values:**

- `RUNNING` - Job is currently executing
- `COMPLETED` - Job finished successfully
- `ERROR` - Job encountered an error
- `UNKNOWN` - Status cannot be determined

---

### Example 17: Real-time Progress Tracking

```python
from gatewizard.core.job_monitor import JobMonitor
from pathlib import Path
import time

# Start a system preparation (runs in background)
from gatewizard.core.builder import Builder

builder = Builder()
success, message, job_dir = builder.prepare_system(
    pdb_file="protein_protonated_prepared.pdb",
    working_dir="./systems",
    upper_lipids=["POPC"],
    lower_lipids=["POPC"],
    lipid_ratios="1//1",
)

if not success:
    print(f"✗ Preparation not started: {message}")
else:
    print(f"✓ Job started: {job_dir}")

    # Monitor progress; cap wait so a missing packmol-memgen cannot hang pytest.
    monitor = JobMonitor(working_directory=Path("./systems"))
    deadline = time.time() + 30

    while time.time() < deadline:
        monitor.scan_for_jobs(force=True)
        active_jobs = monitor.get_active_jobs()

        if not active_jobs:
            completed = monitor.get_completed_jobs()
            if completed:
                for job_id, job_info in completed.items():
                    if str(job_dir) in job_id:
                        print(f"\n✓ Job completed: {job_info.status.value}")
                        print(f"  Total time: {job_info.elapsed_time:.1f}s")
                        break
            break

        for job_id, job_info in active_jobs.items():
            if str(job_dir) in job_id:
                print(
                    f"\rProgress: {job_info.progress:.1f}% - {job_info.current_step}",
                    end="",
                    flush=True,
                )

        time.sleep(2)
    else:
        print("\nStopped waiting after 30s (job still running)")
```

---

### Example 18: Monitoring Batch Job Management

```python
from gatewizard.core.job_monitor import JobMonitor
from pathlib import Path

# Monitor all jobs in a directory
monitor = JobMonitor(working_directory=Path("./systems"))
monitor.scan_for_jobs(force=True)

# Check active jobs
active_jobs = monitor.get_active_jobs()
print(f"\n{'='*60}")
print(f"ACTIVE JOBS: {len(active_jobs)}")
print(f"{'='*60}")

for job_id, job_info in active_jobs.items():
    print(f"\n📁 {job_info.job_dir.name}")
    print(f"   Status: {job_info.status.value}")
    print(f"   Progress: {job_info.progress:.1f}%")
    print(f"   Current: {job_info.current_step}")
    print(f"   Runtime: {job_info.elapsed_time:.0f}s")

    if job_info.steps_completed:
        print(f"   Completed steps:")
        for step in job_info.steps_completed:
            print(f"     ✓ {step}")

# Check completed jobs
completed_jobs = monitor.get_completed_jobs()
print(f"\n{'='*60}")
print(f"COMPLETED JOBS: {len(completed_jobs)}")
print(f"{'='*60}")

for job_id, job_info in completed_jobs.items():
    status_icon = "✓" if job_info.status.value == "completed" else "✗"
    print(
        f"{status_icon} {job_info.job_dir.name}: {job_info.status.value} ({job_info.elapsed_time:.0f}s)"
    )
```

---

### Monitoring Tips

**Real-time monitoring:**

- Use `scan_for_jobs(force=True)` to bypass the 3-second scan interval
- Check status every 1-2 seconds for responsive progress updates
- Use `flush=True` in print statements for immediate display

**Long-running jobs:**

- Check status periodically (every 30-60 seconds)
- Log progress to file for later review
- Use `elapsed_time` to estimate remaining time

**Multiple jobs:**

- Monitor entire working directory for batch processing
- Filter jobs by status for targeted monitoring
- Use job_dir.name for user-friendly display

**Debugging:**

- Check `job_dir / 'logs/preparation.log'` for detailed output
- Check `job_dir / 'logs/parametrization.log'` if parametrization fails
- Examine `status.json` for detailed job state

**Manual log inspection:**

```
# Follow preparation log in real-time
tail -f systems/popc_membrane/logs/preparation.log

# Check for errors
grep -i error systems/popc_membrane/logs/*.log

# View job status
cat systems/popc_membrane/status.json
```

---

## Tips and Best Practices

### Lipid Selection

**For simple systems:**

- Use 100% POPC for both leaflets (symmetric membrane)
- Well-characterized, stable, and widely used
- Good starting point for method development

**For realistic membranes:**

- Include cholesterol (10-30%) for membrane stability
- Use PE lipids (POPE) for inner leaflet (more physiological)
- Add PS lipids (POPS) for negative charge (cytoplasmic side)

**For specialized studies:**

- Use asymmetric compositions (different upper/lower)
- Match experimental lipid compositions when available
- Consider protein's native membrane environment

### Force Field Selection

**Default recommendation (most stable):**

- Water: tip3p
- Protein: ff14SB
- Lipid: lipid21

**For latest parameters:**

- Water: opc
- Protein: ff19SB
- Lipid: lipid21

### Salt Concentration

**Physiological (default):**

- 0.15 M (150 mM) NaCl or KCl
- Mimics intracellular or extracellular conditions
- Packmol-memgen passes this number to `--saltcon`. That flag has no maximum; 0.15 M is only the default. A negative value is rejected.

**High salt studies:**

- 0.5-1.0 M for ionic strength effects
- May require longer equilibration

**Charge neutralization only:**

- Set `salt_concentration=0.0`
- System will still be neutralized with minimum ions

**Water Layer Thickness**

**Default (26 Å):**

- Sufficient for most membrane proteins
- ~3-4 water layers above/below membrane

**Boundary distance (12 Å default):**

- Set `dist` to control the minimum distance from the solute extents to box boundaries
- packmol-memgen may choose a larger real distance because it uses the worst-case scenario

**Large proteins or complexes:**

- Increase to 20-25 Å
- Ensures adequate solvation

**Minimize system size:**

- Decrease to 15 Å (minimum recommended)
- Faster simulations but ensure full solvation

**Protein Orientation**

**Pre-oriented proteins:**

- Use OPM database
- Protein already positioned in membrane plane
- Set `preoriented=True` (default)

**Non-oriented proteins:**

- Let packmol-memgen use MEMEMBED for orientation
- Set `preoriented=False`
- Adds extra time to preparation

**Troubleshooting**

**"PDB file not found":**

- Check absolute/relative paths
- Verify file permissions
- Ensure PDB file is valid format

**"Packing failed":**

- Protein may be too large for default settings
- Try increasing `dist_wat` parameter
- Check protein orientation (OPM database)
- Verify protein is pre-oriented if using `preoriented=True`

**"Parametrization failed":**

- Check logs/parametrization.log for details
- May have non-standard residues
- Verify protonation states (use Propka workflow)
- Check for missing atoms or unusual residue names

**"Invalid lipid ratios":**

- Ensure ratios match number of lipids
- Format: "ratio1:ratio2//ratio3:ratio4"
- Ratios don't need to sum to 1.0 (auto-normalized)
- Example: "7:3//5:5" for 2 lipids per leaflet

**"Force field compatibility error":**

- Use ForceFieldManager to check combinations
- Not all force fields are compatible
- Use recommended combinations for stability

**Performance Tips**

**Faster preparation:**

- Use symmetric membranes (same upper/lower)
- Fewer lipid types (1-2 per leaflet)
- Pre-oriented proteins
- Smaller water layers (15-20 Å)

**More realistic systems:**

- Asymmetric membranes (different leaflets)
- Multiple lipid types (3-4 per leaflet)
- Include cholesterol
- Larger water layers (20-25 Å)
- Use Propka workflow for correct protonation

---

## Ligand Parametrization

Module for detecting, extracting, and parametrizing non-standard (ligand) residues found in PDB files. Uses the AMBER/GAFF workflow (antechamber → parmchk2 → tleap) to generate force field parameters that integrate with the Builder's packmol-memgen and final tleap steps. Supports both GAFF and GAFF2 atom type sets, with recommended pairings per the AMBER manual: **gaff/bcc** and **gaff2/abcg2**.

### Import

```
from gatewizard.tools.ligand_parametrization import (
    detect_ligands,
    extract_ligand_pdb,
    parametrize_ligand,
    parametrize_ligand_from_system_pdb,
    parametrize_all_ligands,
    get_ligand_2d_image,
    get_ligand_2d_image_from_pdb_lines,
    build_ligand_param_args,
    build_tleap_ligand_lines,
    LigandInfo,
    LigandParametrizationError,
    STANDARD_RESIDUES,
    CHARGE_METHODS,
    ATOM_TYPES,
    DEFAULT_CHARGE_METHOD,
    DEFAULT_ATOM_TYPE,
    RECOMMENDED_COMBOS,
    NON_RECOMMENDED_COMBOS,
)
```

### Class: LigandInfo

Information container for a detected ligand residue.

| Attribute | Type | Description |
|---|---|---|
| `name` | `str` | 3-letter residue name (e.g., `"AAA"`) |
| `chain` | `str` | Chain identifier |
| `res_id` | `int` | Residue sequence number |
| `num_atoms` | `int` | Number of atoms in the ligand |
| `elements` | `Dict[str, int]` | Element counts (e.g., `{'C': 9, 'H': 10, 'O': 1}`) |
| `pdb_lines` | `List[str]` | Raw HETATM lines from the PDB file |
| `formula` | `str` (property) | Molecular formula string (e.g., `"C9H10O"`) |

### Methods:

| Method | Returns | Description |
|---|---|---|
| `to_dict()` | `Dict[str, Any]` | Convert to serializable dictionary |

### Constants

| Constant | Type | Description |
|---|---|---|
| `STANDARD_RESIDUES` | `set` | Residue names excluded from detection (amino acids, water, ions, lipids, capping groups) |
| `CHARGE_METHODS` | `dict` | Antechamber charge methods: `'bcc'` (AM1-BCC), `'abcg2'` (ABCG2), `'gas'` (Gasteiger), `'mul'` (Mulliken), `'cm2'` (CM2), `'rc'` (Read-in), `'resp'` (RESP — requires Gaussian), `'esp'` (ESP — requires Gaussian) |
| `ATOM_TYPES` | `dict` | Antechamber atom type sets: `'gaff2'` → `'GAFF2'`, `'gaff'` → `'GAFF'` |
| `DEFAULT_CHARGE_METHOD` | `str` | `'abcg2'` |
| `DEFAULT_ATOM_TYPE` | `str` | `'gaff2'` |
| `RECOMMENDED_COMBOS` | `set` | Recommended (atom_type, charge_method) pairings per AMBER manual: `{('gaff', 'bcc'), ('gaff2', 'abcg2')}` |
| `NON_RECOMMENDED_COMBOS` | `set` | Non-recommended pairings (warning shown in GUI): `{('gaff2', 'bcc'), ('gaff', 'abcg2')}` |

**Recommended Atom Type / Charge Method Pairings**

Per the AMBER manual, the efficient charge models `bcc` (AM1-BCC) and `abcg2` (ABCG2) should be paired with specific atom type sets:

| Atom Type | Charge Method | Status |
|---|---|---|
| `gaff` | `bcc` | **Recommended** ✓ |
| `gaff2` | `abcg2` | **Recommended** ✓ |
| `gaff2` | `bcc` | Not recommended ✗ |
| `gaff` | `abcg2` | Not recommended ✗ |

The GUI shows an amber warning label when a non-recommended combination is selected, and displays a confirmation dialog before proceeding with parametrization.

**External QM Software Requirements**

The `resp` and `esp` charge methods require **Gaussian** (external quantum-mechanics software, not included in AmberTools). All other methods (`bcc`, `abcg2`, `gas`, `mul`, `cm2`) use **sqm**, which is bundled with AmberTools.

**detect_ligands**

Detect non-standard (ligand) residues in a PDB file by scanning HETATM records.

```
detect_ligands(pdb_file: str) -> List[LigandInfo]
```

**Parameters:**

| Parameter | Type | Description |
|---|---|---|
| `pdb_file` | `str` | Path to the PDB file to analyze |

**Returns:** `List[LigandInfo]` — one entry per unique ligand residue name

**Raises:** `LigandParametrizationError` if the file cannot be read

### Example 19: Detect Ligands

```
"""
Builder Example 19: Detect ligands in a PDB file

Demonstrates how to detect non-standard residues (ligands)
in a PDB file using the ligand parametrization tools.
"""

from pathlib import Path

from gatewizard.tools.ligand_parametrization import detect_ligands

# Detect ligands in a PDB file with two ligands (AAA and BBB)
pdb_file = str(Path(__file__).resolve().parent.parent / "2MVJ_2ligs.pdb")
ligands = detect_ligands(pdb_file)

print(f"Detected {len(ligands)} ligand(s) in {pdb_file}:")
print(f"{'='*60}")

for lig in ligands:
    print(f"\nLigand: {lig.name}")
    print(f"  Chain: {lig.chain}")
    print(f"  Residue ID: {lig.res_id}")
    print(f"  Number of atoms: {lig.num_atoms}")
    print(f"  Molecular formula: {lig.formula}")
    print(f"  Elements: {lig.elements}")
    print(f"  As dict: {lig.to_dict()}")
```

---

### extract_ligand_pdb

Extract a single ligand from a PDB file into its own file.

```
extract_ligand_pdb(pdb_file: str, ligand_name: str, output_dir: str) -> str
```

**Parameters:**

| Parameter | Type | Description |
|---|---|---|
| `pdb_file` | `str` | Path to the source PDB file |
| `ligand_name` | `str` | 3-letter residue name of the ligand |
| `output_dir` | `str` | Directory to write the extracted PDB file |

**Returns:** `str` — path to the extracted PDB file (`output_dir/LIGAND.pdb`)

**Raises:** `LigandParametrizationError` if the ligand is not found or extraction fails

### Example 20: Extract a Ligand

```
"""
Builder Example 20: Extract a ligand from a PDB file

Demonstrates how to extract a specific ligand into its own PDB file
for individual parametrization.
"""

from pathlib import Path
from gatewizard.tools.ligand_parametrization import detect_ligands, extract_ligand_pdb

pdb_file = str(Path(__file__).resolve().parent.parent / "2MVJ_2ligs.pdb")
output_dir = "./systems/ligand_extraction"

# First detect ligands
ligands = detect_ligands(pdb_file)
print(f"Detected ligands: {[l.name for l in ligands]}")

# Extract each ligand to its own subdirectory
for lig in ligands:
    lig_dir = str(Path(output_dir) / lig.name)
    extracted_pdb = extract_ligand_pdb(pdb_file, lig.name, lig_dir)

    # Verify extraction
    with open(extracted_pdb) as f:
        lines = [l for l in f if l.startswith("HETATM")]

    print(f"\nExtracted {lig.name}:")
    print(f"  Output: {extracted_pdb}")
    print(f"  Atoms: {len(lines)}")
    print(f"  First line: {lines[0].strip()[:60]}...")

print(f"\nAll extracted ligands saved in: {output_dir}")
```

---

### parametrize_ligand

Parametrize a single ligand using the AMBER/GAFF workflow (antechamber → parmchk2 → tleap).

```
parametrize_ligand(
    ligand_pdb: str,
    ligand_name: str,
    output_dir: str,
    charge: int = 0,
    charge_method: str = 'abcg2',
    multiplicity: int = 1,
    atom_type: str = 'gaff2',
    sqm_keywords: str = '',
    timeout: Optional[int] = None,
) -> Dict[str, str]
```

**Parameters:**

| Parameter | Type | Default | Description |
|---|---|---|---|
| `ligand_pdb` | `str` | — | Path to the ligand PDB file |
| `ligand_name` | `str` | — | 3-letter residue name (must match PDB) |
| `output_dir` | `str` | — | Directory for output files |
| `charge` | `int` | `0` | Net charge of the ligand |
| `charge_method` | `str` | `'abcg2'` | Charge method (see `CHARGE_METHODS`). The default pairing is GAFF2 with ABCG2. |
| `multiplicity` | `int` | `1` | Spin multiplicity |
| `atom_type` | `str` | `'gaff2'` | Atom type set — `'gaff2'` or `'gaff'` (see `ATOM_TYPES`) |
| `sqm_keywords` | `str` | `''` | Extra SQM keywords for antechamber `-ek`. Empty leaves the Amber default. |

| Parameter | Type | Default | Description |
|---|---|---|---|
| `timeout` | `int` or `None` | `None` | Antechamber wall-clock limit in seconds. `None` waits until it exits. parmchk2 and tleap each stop after 120 seconds. |

**Returns:** Dictionary with paths to generated files:

| Key | Description |
|---|---|
| `'mol2'` | Typed MOL2 file (GAFF/GAFF2 atom types + charges) |
| `'frcmod'` | Force field modification file (missing parameters) |
| `'lib'` | Residue library file for tleap |
| `'prmtop'` | AMBER topology file |
| `'inpcrd'` | AMBER coordinate file |

**Raises:** `LigandParametrizationError` if any step fails

> **Critical: Variable Name Must Match Residue Name**
>
> The tleap variable name **must** match the residue name in the PDB. For example, `AAA = loadmol2 AAA.mol2` — using a generic name like `mol = loadmol2 AAA.mol2` will cause `saveoff` to store the unit under the wrong name and tleap will fail to recognize the residue when loading the full system PDB.

---

#### parametrize_ligand_from_system_pdb

Extract one ligand from a system PDB and parametrize it. The GUI uses this for a ligand detected in the protein file. A standalone ligand PDB uses `parametrize_ligand` instead.

Files are written to `{output_dir}/ligand_params/{ligand_name}/`.

```
parametrize_ligand_from_system_pdb(
    pdb_file: str,
    ligand_name: str,
    output_dir: str,
    charge: int = 0,
    charge_method: str = 'abcg2',
    multiplicity: int = 1,
    atom_type: str = 'gaff2',
    sqm_keywords: str = '',
    timeout: Optional[int] = None,
) -> Dict[str, str]
```

**Parameters:**

| Parameter | Type | Default | Description |
|---|---|---|---|
| `pdb_file` | `str` | — | System PDB that contains the ligand |
| `ligand_name` | `str` | — | Residue name to extract and parametrize |
| `output_dir` | `str` | — | Directory where `ligand_params/` is created |
| `charge` | `int` | `0` | Net charge of the ligand |
| `charge_method` | `str` | `'abcg2'` | Charge method (see `CHARGE_METHODS`) |
| `multiplicity` | `int` | `1` | Spin multiplicity |
| `atom_type` | `str` | `'gaff2'` | Atom type set — `'gaff2'` or `'gaff'` |
| `sqm_keywords` | `str` | `''` | Extra SQM keywords for antechamber `-ek` |

| Parameter | Type | Default | Description |
|---|---|---|---|
| `timeout` | `int` or `None` | `None` | Antechamber wall-clock limit in seconds. `None` waits until it exits. parmchk2 and tleap each stop after 120 seconds. |

**Returns:** The same file-path dictionary as `parametrize_ligand()`.

**Raises:** `LigandParametrizationError` if any step fails

---

### parametrize_all_ligands

Detect and parametrize all ligands in a PDB file in one call.

```
parametrize_all_ligands(
    pdb_file: str,
    output_dir: str,
    charges: Optional[Dict[str, int]] = None,
    charge_method: str = 'abcg2',
    atom_type: str = 'gaff2',
    sqm_keywords: str = '',
    timeout: Optional[int] = None,
) -> Dict[str, Dict[str, str]]
```

**Parameters:**

| Parameter | Type | Default | Description |
|---|---|---|---|
| `pdb_file` | `str` | — | PDB file containing ligands |
| `output_dir` | `str` | — | Base directory (each ligand gets a subdirectory) |
| `charges` | `Dict[str, int]` or `None` | `None` | Map of ligand name → net charge (default: 0 for all) |
| `charge_method` | `str` | `'abcg2'` | Charge method for antechamber. The default pairing is GAFF2 with ABCG2. |
| `atom_type` | `str` | `'gaff2'` | Atom type set — `'gaff2'` or `'gaff'` (see `ATOM_TYPES`) |
| `sqm_keywords` | `str` | `''` | Extra SQM keywords for antechamber `-ek`. Empty leaves the Amber default. |
| `timeout` | `int` or `None` | `None` | Antechamber wall-clock limit in seconds for every ligand. `None` waits until each one exits. parmchk2 and tleap each stop after 120 seconds. |

**Returns:** `Dict[str, Dict[str, str]]` — maps ligand names to their file paths (same structure as `parametrize_ligand()`)

**Example 22: Parametrize All Ligands**

```
"""
Builder Example 22: Parametrize all ligands in a PDB file

Demonstrates the full ligand parametrization workflow:
1. Detect ligands from PDB
2. Extract each ligand
3. Run antechamber + parmchk2 + tleap for each
4. Collect .frcmod and .lib files

NOTE: This example requires AmberTools (antechamber, parmchk2, tleap)
to be installed and accessible in the PATH.
"""

import shutil
import sys
from pathlib import Path
from gatewizard.tools.ligand_parametrization import (
    parametrize_all_ligands,
    build_ligand_param_args,
    build_tleap_ligand_lines,
)

if shutil.which("antechamber") is None:
    sys.exit("Skip: AmberTools antechamber is not on PATH")

pdb_file = str(Path(__file__).resolve().parent.parent / "2MVJ_2ligs.pdb")
output_dir = "./systems/ligand_params"

# Set charges for each ligand (default is 0 if not specified)
charges = {
    "AAA": 0,
    "BBB": 0,
}

print(f"Output directory: {output_dir}")

# Parametrize all ligands
results = parametrize_all_ligands(
    pdb_file=pdb_file,
    output_dir=output_dir,
    charges=charges,
    charge_method="bcc",  # AM1-BCC charges
    atom_type="gaff2",  # GAFF2 atom types (recommended with abcg2; bcc also works)
)

print(f"\nParametrized {len(results)} ligand(s):")
for name, files in results.items():
    print(f"\n  {name}:")
    for file_type, file_path in files.items():
        exists = Path(file_path).exists()
        print(f"    {file_type}: {file_path} ({'exists' if exists else 'MISSING'})")

# Now these can be passed to Builder.prepare_system()
print("\n\npackmol-memgen arguments:")
pmm_args = build_ligand_param_args(results)
print(f"  {' '.join(pmm_args)}")

print("\ntleap ligand lines:")
tleap_lines = build_tleap_ligand_lines(results)
print(tleap_lines)

print(f"\nAll parameter files saved in: {output_dir}")
```

---

**parametrize_peptide_cap**

Parametrize one polymer cap (FVA, FOR, ETA, and the other Amber-unsupported termini). The residue stays in the chain. tleap stores the head and tail connect atoms in the OFF library.

`parametrize_peptide_cap_from_system_pdb` extracts that residue from a system PDB into `peptide_cap_params/{name}/` and calls this function. `parametrize_all_peptide_caps_from_system_pdb` does that for every cap that needs GAFF.

```
parametrize_peptide_cap(
    residue_pdb: str,
    residue_name: str,
    output_dir: str,
    charge: int = 0,
    charge_method: str = 'abcg2',
    multiplicity: int = 1,
    atom_type: str = 'gaff2',
    sqm_keywords: str = '',
    timeout: Optional[int] = None,
) -> Dict[str, Any]
```

| Parameter | Type | Default | Description |
|---|---|---|---|
| `timeout` | `int` or `None` | `None` | Antechamber wall-clock limit in seconds. `None` waits until it exits. parmchk2 and tleap each stop after 120 seconds. |

The other parameters match `parametrize_ligand`. The system-PDB helpers take `pdb_file` and `output_dir` instead of `residue_pdb`, and the all-caps helper takes `charges` the same way as `parametrize_all_ligands`. Each helper also accepts `timeout`.

---

### get_ligand_2d_image

Generate a publication-quality 2D molecular structure image using RDKit. A GAFF mol2 bond stored as type 3 is drawn as a triple bond only when the atoms are that close. A ring of shorter bonds is drawn aromatic, and a longer bond is drawn single.

```
get_ligand_2d_image(
    ligand_pdb_or_mol2: str,
    output_image: str,
    width: int = 400,
    height: int = 300,
    *,
    remove_nonpolar_h: bool = True,
    remove_all_h: bool = False,
    dpi: int = 150,
    bond_line_width: float = 2.5,
    atom_label_font_size: int = 0,
    background_color: tuple = (0.11, 0.11, 0.11, 1.0),
    padding: float = 0.15,
    kekulize: bool = True,
    wedge_bonds: bool = True,
    atom_palette: dict | None = None,
    highlight_atoms: list[int] | None = None,
    highlight_color: tuple = (1.0, 0.8, 0.0, 0.3),
    transparent_background: bool = False,
) -> Optional[str]
```

**Parameters:**

| Parameter | Type | Default | Description |
|---|---|---|---|
| `ligand_pdb_or_mol2` | `str` | — | Path to PDB or MOL2 file |
| `output_image` | `str` | — | Path for the output PNG |
| `width` | `int` | `400` | Image width in pixels (before DPI scaling) |
| `height` | `int` | `300` | Image height in pixels (before DPI scaling) |
| `remove_nonpolar_h` | `bool` | `True` | Remove non-polar hydrogens (C-H) for cleaner figures while keeping polar H (O-H, N-H, etc.) |
| `remove_all_h` | `bool` | `False` | Remove **all** explicit hydrogens. Overrides `remove_nonpolar_h` |

| Parameter | Type | Default | Description |
|---|---|---|---|
| `dpi` | `int` | `150` | DPI multiplier. Use `300` for print-quality images |
| `bond_line_width` | `float` | `2.5` | Thickness of bond lines |
| `atom_label_font_size` | `int` | `0` | Font size for atom labels (0 = auto) |
| `background_color` | `tuple` | `(0.11, 0.11, 0.11, 1.0)` | RGBA background colour (0–1 range) |
| `padding` | `float` | `0.15` | Fractional padding around the molecule (0–1) |
| `kekulize` | `bool` | `True` | Draw aromatic bonds as alternating single/double |
| `wedge_bonds` | `bool` | `True` | Draw stereo wedge/dash bonds |
| `atom_palette` | `dict` | `None` | Custom `{atomic_number: (r, g, b)}` colour map. Uses built-in dark palette when `None` |
| `highlight_atoms` | `list[int]` | `None` | 0-based atom indices to highlight |
| `highlight_color` | `tuple` | `(1.0, 0.8, 0.0, 0.3)` | RGBA colour for highlights |
| `transparent_background` | `bool` | `False` | Produce a transparent PNG |

**Returns:** Path to the generated PNG image, or `None` if generation failed

**Built-in palettes:**

| Palette | Import | Best for |
|---|---|---|
| `_DEFAULT_DARK_PALETTE` | (used automatically) | Dark backgrounds |
| `LIGHT_PALETTE` | `from gatewizard.tools.ligand_parametrization import LIGHT_PALETTE` | White / light backgrounds |

### get_ligand_2d_image_from_pdb_lines

Generate a 2D image directly from PDB HETATM lines (no file required). All keyword arguments are forwarded to `get_ligand_2d_image`.

```
get_ligand_2d_image_from_pdb_lines(
    pdb_lines: List[str],
    output_image: str,
    width: int = 400,
    height: int = 300,
    **kwargs,
) -> Optional[str]
```

**Parameters:**

| Parameter | Type | Default | Description |
|---|---|---|---|
| `pdb_lines` | `List[str]` | — | HETATM lines for the ligand |
| `output_image` | `str` | — | Path for the output PNG |
| `width` | `int` | `400` | Image width |
| `height` | `int` | `300` | Image height |
| `**kwargs` | | | All keyword options from `get_ligand_2d_image` |

**Returns:** Path to the generated PNG image, or `None` if failed

**Example 24: Generate 2D Structure Images**

```
"""
Builder Example 24: Generate 2D structure images of ligands

Demonstrates generating 2D molecular structure images from PDB files
using RDKit, including options for hydrogen removal, DPI control,
custom colour palettes, and transparent backgrounds.
"""

from pathlib import Path
from gatewizard.tools.ligand_parametrization import (
    detect_ligands,
    get_ligand_2d_image_from_pdb_lines,
    get_ligand_2d_image,
    extract_ligand_pdb,
    LIGHT_PALETTE,
)

pdb_file = str(Path(__file__).resolve().parent.parent / "2MVJ_2ligs.pdb")
output_dir = "./systems/ligand_images"
Path(output_dir).mkdir(parents=True, exist_ok=True)

# Detect ligands
ligands = detect_ligands(pdb_file)
print(f"Detected {len(ligands)} ligands\n")

for lig in ligands:
    # --- Style 1: Default (dark background, non-polar H removed) --------
    img = str(Path(output_dir) / f"{lig.name}_default.png")
    result = get_ligand_2d_image_from_pdb_lines(
        lig.pdb_lines,
        img,
        width=400,
        height=300,
        remove_nonpolar_h=True,  # cleaner look (default)
    )
    if result:
        print(f"{lig.name} default : {Path(result).stat().st_size:>6} bytes")

    # --- Style 2: All hydrogens removed ----------------------------------
    img = str(Path(output_dir) / f"{lig.name}_no_h.png")
    result = get_ligand_2d_image_from_pdb_lines(
        lig.pdb_lines,
        img,
        width=400,
        height=300,
        remove_all_h=True,  # skeleton only
    )
    if result:
        print(f"{lig.name} no-H    : {Path(result).stat().st_size:>6} bytes")

    # --- Style 3: High-DPI for publication (white background) ------------
    img = str(Path(output_dir) / f"{lig.name}_hires.png")
    lig_dir = str(Path(output_dir) / lig.name)
    extracted_pdb = extract_ligand_pdb(pdb_file, lig.name, lig_dir)
    result = get_ligand_2d_image(
        extracted_pdb,
        img,
        width=800,
        height=600,
        dpi=300,  # high DPI
        remove_all_h=True,
        background_color=(1, 1, 1, 1),  # white
        atom_palette=LIGHT_PALETTE,  # colours for light background
        bond_line_width=1.5,
    )
    if result:
        print(f"{lig.name} hi-res  : {Path(result).stat().st_size:>6} bytes")

    # --- Style 4: Transparent background ---------------------------------
    img = str(Path(output_dir) / f"{lig.name}_transparent.png")
    result = get_ligand_2d_image_from_pdb_lines(
        lig.pdb_lines,
```

```
        img,
        width=400,
        height=300,
        remove_nonpolar_h=True,
        transparent_background=True,
    )
    if result:
        print(f"{lig.name} transp. : {Path(result).stat().st_size:>6} bytes")

    # --- Style 5: All hydrogens visible, thicker bonds -------------------
    img = str(Path(output_dir) / f"{lig.name}_all_h.png")
    result = get_ligand_2d_image_from_pdb_lines(
        lig.pdb_lines,
        img,
        width=500,
        height=400,
        remove_nonpolar_h=False,
        remove_all_h=False,
        bond_line_width=3.5,
        padding=0.2,
    )
    if result:
        print(f"{lig.name} all-H   : {Path(result).stat().st_size:>6} bytes")

    print()

print(f"All images saved in: {output_dir}")
```

---

### build_ligand_param_args

Build `--ligand_param` command-line arguments for packmol-memgen.

```
build_ligand_param_args(
    ligand_files: Dict[str, Dict[str, str]]
) -> List[str]
```

**Parameters:**

| Parameter | Type | Description |
|---|---|---|
| `ligand_files` | `Dict[str, Dict[str, str]]` | Ligand name → file paths (as returned by `parametrize_ligand`) |

**Returns:** List of CLI arguments — each ligand produces `['--ligand_param', 'path.frcmod:path.lib']`

> **One Flag Per Ligand**
>
> packmol-memgen requires a **separate** `--ligand_param` flag per ligand. Combining multiple ligands into a single flag will not work.

### build_tleap_ligand_lines

Build tleap input lines to load GAFF/GAFF2 and ligand parameters. Insert these **before** `loadPDB`.

```
build_tleap_ligand_lines(
    ligand_files: Dict[str, Dict[str, str]],
    atom_type: str = 'gaff2',
) -> str
```

**Parameters:**

| Parameter | Type | Description |
|---|---|---|
| `ligand_files` | `Dict[str, Dict[str, str]]` | Ligand name → file paths |
| `atom_type` | `str` | Atom type set — `'gaff2'` or `'gaff'` (default: `'gaff2'`). Determines the `leaprc` source line |

**Returns:** Multi-line string with tleap commands (`source leaprc.gaff2` or `source leaprc.gaff`, `loadamberparams`, `loadoff`)

**Example 21: Build packmol-memgen and tleap Arguments**

```
"""
Builder Example 21: Build packmol-memgen and tleap commands with ligand parameters

Demonstrates how to construct the packmol-memgen --ligand_param arguments
and tleap input lines for systems with ligands.
"""

from gatewizard.tools.ligand_parametrization import (
    build_ligand_param_args,
    build_tleap_ligand_lines,
)

# Simulated parametrization results (paths to .frcmod and .lib files)
ligand_files = {
    "AAA": {
        "frcmod": "ligand_params/AAA/AAA.frcmod",
        "lib": "ligand_params/AAA/AAA.lib",
        "mol2": "ligand_params/AAA/AAA.mol2",
    },
    "BBB": {
        "frcmod": "ligand_params/BBB/BBB.frcmod",
        "lib": "ligand_params/BBB/BBB.lib",
        "mol2": "ligand_params/BBB/BBB.mol2",
    },
}

# Build packmol-memgen arguments
# Each ligand gets its own --ligand_param flag (CANNOT combine in one flag)
pmm_args = build_ligand_param_args(ligand_files)
print("packmol-memgen arguments:")
for i in range(0, len(pmm_args), 2):
    print(f"  {pmm_args[i]} {pmm_args[i+1]}")

print()

# Build tleap input lines
# These go BEFORE loadPDB in the tleap input
tleap_lines = build_tleap_ligand_lines(ligand_files)
print("tleap input lines:")
print(tleap_lines)

# With GAFF atom types instead of GAFF2
tleap_lines_gaff = build_tleap_ligand_lines(ligand_files, atom_type="gaff")
print("\ntleap input lines (GAFF):")
print(tleap_lines_gaff)
```

---

**Builder Integration: `ligand_params` Config Key**

When ligands are parametrized (either through the GUI or programmatically), the Builder uses the `ligand_params` configuration key to integrate them into the build process.

**Config Structure:**

```
config['ligand_params'] = {
    'AAA': {
        'frcmod': '/path/to/AAA/AAA.frcmod',
        'lib': '/path/to/AAA/AAA.lib',
    },
    'BBB': {
        'frcmod': '/path/to/BBB/BBB.frcmod',
        'lib': '/path/to/BBB/BBB.lib',
    },
}
```

**What happens during build:**

1. **packmol-memgen** receives `--ligand_param frcmod:lib` for each ligand, plus `--gaff2` (or `--gaff` depending on the atom type used)
2. **tleap parametrization** loads `source leaprc.gaff2` (or `source leaprc.gaff`), then `loadamberparams` and `loadoff` for each ligand before `loadPDB`
3. The atom type is automatically extracted from the parametrized ligand results, so matching `leaprc` is always used

### Example 23: Full Membrane System with Ligand Parametrization

```
"""
Builder Example 23: Full membrane system with ligand parametrization

Demonstrates setting up a complete membrane system that includes
non-standard ligands. The ligand .frcmod/.lib files are passed
to both packmol-memgen and the final tleap parametrization.

NOTE: This example requires AmberTools and packmol-memgen.
"""

import shutil
import sys
from pathlib import Path

from gatewizard.core.builder import Builder
from gatewizard.tools.ligand_parametrization import (
    detect_ligands,
    parametrize_all_ligands,
)

if shutil.which("antechamber") is None:
    sys.exit("Skip: AmberTools antechamber is not on PATH")

# Create builder
builder = Builder()

pdb_file = str(Path(__file__).resolve().parent.parent / "2MVJ_2ligs.pdb")
working_dir = "./systems"

# Step 1: Detect and parametrize ligands
print("Step 1: Detecting ligands...")
ligands = detect_ligands(pdb_file)
for lig in ligands:
    print(f"  Found: {lig.name} ({lig.num_atoms} atoms, {lig.formula})")

print("\nStep 2: Parametrizing ligands...")
ligand_results = parametrize_all_ligands(
    pdb_file=pdb_file,
    output_dir=f"{working_dir}/ligand_params",
    charges={"AAA": 0, "BBB": 0},
    charge_method="bcc",
    atom_type="gaff2",  # GAFF2 atom types
)

print(f"  Parametrized: {list(ligand_results.keys())}")

# Step 3: Configure builder with ligand parameters
builder.set_configuration(
    water_model="tip3p",
    protein_ff="ff14SB",
    lipid_ff="lipid21",
    preoriented=True,
    parametrize=True,
    salt_concentration=0.15,
    dist=12,
    dist_wat=26,
    notprotonate=True,
    ligand_params=ligand_results,  # Pass parametrized ligand files
)

print("\nStep 3: Builder configured with ligand parameters")
print(f"  Ligands in config: {list(builder.config['ligand_params'].keys())}")

# Step 4: Prepare system
success, message, job_dir = builder.prepare_system(
    pdb_file=pdb_file,
    working_dir=working_dir,
    upper_lipids=["POPC"],
    lower_lipids=["POPC"],
    lipid_ratios="1.0//1.0",
)
print(f"\nResult: {message}")
```

```
# The builder will:
# 1. Add --ligand_param AAA/AAA.frcmod:AAA/AAA.lib to packmol-memgen
# 2. Add --ligand_param BBB/BBB.frcmod:BBB/BBB.lib to packmol-memgen
# 3. Add --gaff2 to packmol-memgen
# 4. Load ligand .frcmod and .lib in the tleap parametrization step
print("\nWorkflow complete. The builder will pass ligand params to:")
print("  - packmol-memgen (--ligand_param flags)")
print("  - tleap (loadamberparams/loadoff commands)")
```

### Example 26: Atom Type Selection and Recommended Pairings

```
"""
Builder Example 26: Atom type selection and recommended pairings

Demonstrates how to choose between GAFF and GAFF2 atom types,
check recommended pairings per the AMBER manual, and use
the ABCG2 charge method with GAFF2.

NOTE: This example requires AmberTools (antechamber, parmchk2, tleap)
to be installed and accessible in the PATH.
"""

import shutil
import sys
from pathlib import Path

from gatewizard.tools.ligand_parametrization import (
    parametrize_all_ligands,
    build_tleap_ligand_lines,
    ATOM_TYPES,
    CHARGE_METHODS,
    DEFAULT_ATOM_TYPE,
    DEFAULT_CHARGE_METHOD,
    RECOMMENDED_COMBOS,
    NON_RECOMMENDED_COMBOS,
)

if shutil.which("antechamber") is None:
    sys.exit("Skip: AmberTools antechamber is not on PATH")

# ── Available options ──────────────────────────────────────────
print("Available atom types:")
for key, label in ATOM_TYPES.items():
    default = " (default)" if key == DEFAULT_ATOM_TYPE else ""
    print(f"  {key}: {label}{default}")

print("\nAvailable charge methods:")
for key, label in CHARGE_METHODS.items():
    default = " (default)" if key == DEFAULT_CHARGE_METHOD else ""
    print(f"  {key}: {label}{default}")

# ── Recommended pairings ──────────────────────────────────────
print("\nRecommended pairings (AMBER manual):")
for at, cm in sorted(RECOMMENDED_COMBOS):
    print(f"  {at} + {cm}  ✓")

print("\nNon-recommended pairings (will show warning):")
for at, cm in sorted(NON_RECOMMENDED_COMBOS):
    print(f"  {at} + {cm}  ✗")

# ── Check a pairing before parametrizing ──────────────────────
atom_type = "gaff2"
charge_method = "abcg2"

if (atom_type, charge_method) in RECOMMENDED_COMBOS:
    print(f"\n{atom_type}/{charge_method} is a recommended pairing.")
elif (atom_type, charge_method) in NON_RECOMMENDED_COMBOS:
    print(f"\nWARNING: {atom_type}/{charge_method} is NOT recommended.")
else:
    print(f"\n{atom_type}/{charge_method} has no specific recommendation.")

# ── Parametrize with gaff2/abcg2 (recommended) ────────────────
pdb_file = str(Path(__file__).resolve().parent.parent / "2MVJ_2ligs.pdb")
output_dir = "./systems/ligand_params_gaff2_abcg2"

results = parametrize_all_ligands(
    pdb_file=pdb_file,
    output_dir=output_dir,
    charges={"AAA": 0, "BBB": 0},
    charge_method="abcg2",  # ABCG2 charges
    atom_type="gaff2",  # GAFF2 atom types (recommended with abcg2)
)
```

```
print(f"\nParametrized {len(results)} ligand(s) with gaff2/abcg2:")
for name, files in results.items():
    print(f"  {name}: {files.get('frcmod', 'N/A')}")

# ── tleap lines reflect the chosen atom type ──────────────────────
tleap_gaff2 = build_tleap_ligand_lines(results, atom_type="gaff2")
print(f"\ntleap lines (GAFF2):\n{tleap_gaff2}")

tleap_gaff = build_tleap_ligand_lines(results, atom_type="gaff")
print(f"\ntleap lines (GAFF):\n{tleap_gaff}")
```

#### Ligand Parametrization Troubleshooting

**"Antechamber failed":**

- Check that AMBER/AmberTools is installed and in `$PATH`
- Verify ligand PDB has proper HETATM records
- Try a different charge method (e.g., `gas` is faster and may work when `bcc` fails)
- Check `logs/antechamber.log` for detailed error messages

**"No ligands detected":**

- Ligands must use HETATM records, not ATOM
- Residue names must NOT be in `STANDARD_RESIDUES` set
- Water (HOH, WAT), ions (NA, CL), and lipids (POPC, etc.) are excluded by design

**"tleap: unknown residue name":**

- The tleap variable name must match the 3-letter residue name exactly
- Ensure `.lib` file was generated with the correct residue name in `saveoff`
- Check that `source leaprc.gaff2` (or `source leaprc.gaff`) is loaded before `loadamberparams`

**"Non-recommended combination" warning (gaff2/bcc or gaff/abcg2):**

- The AMBER manual recommends `gaff/bcc` and `gaff2/abcg2` pairings
- Using `gaff2/bcc` or `gaff/abcg2` may produce suboptimal parameters
- The GUI shows an amber warning label and a confirmation dialog
- To silence the warning, switch to a recommended pairing
- Check `RECOMMENDED_COMBOS` and `NON_RECOMMENDED_COMBOS` constants for the full list

**"resp/esp: Gaussian not found":**

- The `resp` and `esp` charge methods require Gaussian (external QM software)
- Gaussian is **not** included in AmberTools
- Use `bcc` or `abcg2` instead (both use the built-in `sqm` engine)

**"RDKit not available":**

- Install with `conda install -c conda-forge rdkit`
- RDKit is a required dependency for ligand 2D visualization

---

## See Also

- Preparation Module - Protonation state analysis
- Equilibration Module - MD equilibration protocols
- User Guide - Builder - GUI workflow
- Builder examples - Numbered scripts included on this page

## MemPrO Module

Module for orienting membrane proteins using MemPrO. MemPrO positions proteins correctly in the membrane prior to system building with packmol-memgen.

- Run MemPrO orientation from Python
- Parse ranked orientation results
- Access oriented PDB files by rank
- Build command lines without execution
- Check tool availability

### Import

```
from gatewizard.core.mempro import MemPrO, OrientationResult, MemProError
```

### Class: MemPrO

Main class for orienting membrane proteins. Wraps the `mempro` CLI tool.

#### Constructor

```
MemPrO()
```

**Parameters:** None

**Returns:** `MemPrO` instance

#### Example 1: Create a MemPrO instance and check availability

```
from gatewizard.core.mempro import MemPrO

mp = MemPrO()
print(f"MemPrO available: {MemPrO.is_available()}")
```

---

### Static Methods

#### is_available

Check whether the `mempro` executable is on PATH.

```
MemPrO.is_available() -> bool
```

**Returns:** `True` if `mempro` is found, `False` otherwise.

#### Example 2: Check if MemPrO is installed

```
from gatewizard.core.mempro import MemPrO

if MemPrO.is_available():
    print("MemPrO is installed and ready to use")
else:
    print('Install MemPrO: pip install "gatewizard[full]"')
```

---

## Core Methods

### run

Run MemPrO orientation on a PDB file.

```
run(
    pdb_file: str,
    output_dir: Optional[str] = None,
    n_cpus: Optional[int] = None,
    n_iters: int = 150,
    grid_size: int = 36,
    dual_membrane: bool = False,
    peripheral: bool = False,
    use_weights: bool = False,
    flip: bool = False,
    membrane_thickness: Optional[float] = None,
    seed: Optional[int] = None,
    device: Optional[str] = None,
    single_precision: bool = False,
    rank: Optional[str] = None,
    predict_pg: bool = False,
    pg_guess: Optional[float] = None,
    charge: Optional[float] = None,
    charge_outer: Optional[float] = None,
    outer_membrane_thickness: Optional[float] = None,
    optimize_thickness: bool = False,
    transmembrane_residues: Optional[str] = None,
    write_bfactors: bool = False,
    curvature: bool = False,
    itp_file: Optional[str] = None,
    insane_path: Optional[str] = None,
    build_system: Optional[int] = None,
    build_arguments: Optional[str] = None,
    additional_residues: Optional[str] = None,
    additional_residues_itp: Optional[str] = None,
    residue_cg_file: Optional[str] = None,
    extra_args: Optional[List[str]] = None,
) -> List[OrientationResult]
```

**Parameters:**

A value that matches the MemPrO default is left off the command.

| Parameter | Type | Default | Description |
|---|---|---|---|
| `pdb_file` | `str` | — | Path to the input PDB file |
| `output_dir` | `str` | `None` | Output directory name (default: `Orient`) |
| `n_cpus` | `int` | `None` | CPU cores (`-nc`). Omit to use MemPrO's default |
| `n_iters` | `int` | `150` | Minimisation iterations (`-ni`) |
| `grid_size` | `int` | `36` | Starting configurations (`-ng`) |
| `dual_membrane` | `bool` | `False` | Dual membrane (`-dm`) |
| `peripheral` | `bool` | `False` | Peripheral protein (`-pr`). Not with dual membrane |
| `use_weights` | `bool` | `False` | Weight by B-factors (`-w`). Not with transmembrane residues |
| `flip` | `bool` | `False` | Flip in Z after orientation (`-flip`) |
| `membrane_thickness` | `float` | `None` | Inner membrane thickness in Å (`-mt`, default 28) |
| `seed` | `int` | `None` | Random seed (`-seed`). Omit for an unseeded run. The same seed, protein, and CPU count repeat |
| `device` | `str` | `None` | `auto`, `cpu`, or `gpu` (`-dev`). `auto` is omitted |
| `single_precision` | `bool` | `False` | 32-bit floats (`-sp`) |
| `rank` | `str` | `None` | `auto`, `h` (hits), or `p` (potential). `auto` is omitted |

| Parameter | Type | Default | Description |
|---|---|---|---|
| `predict_pg` | `bool` | `False` | Peptidoglycan layer (`-pg`). Needs dual membrane |
| `pg_guess` | `float` | `None` | Peptidoglycan position guess in Å (`-pg_guess`) |
| `charge` | `float` | `None` | Inner membrane charge (`-ch`, default 0) |
| `charge_outer` | `float` | `None` | Outer membrane charge (`-ch_o`). Needs dual membrane |
| `outer_membrane_thickness` | `float` | `None` | Outer thickness in Å (`-mt_o`, default 24). Needs dual membrane |
| `optimize_thickness` | `bool` | `False` | Optimise thickness (`-mt_opt`). Not with curvature |
| `transmembrane_residues` | `str` | `None` | Ranges such as `10-40,50-60` (`-tm`). Not with B-factor weights |
| `write_bfactors` | `bool` | `False` | Write the potential into B-factors (`-wb`). Not with curvature |
| `curvature` | `bool` | `False` | Curvature minimisation (`-c`). Not with dual membrane |
| `itp_file` | `str` | `None` | Martini topology (`-itp`) |
| `insane_path` | `str` | `None` | Path to insane (`-insane`) |
| `build_system` | `int` | `None` | Build a coarse-grained system for ranks below this (`-bd`). `0` is omitted |
| `build_arguments` | `str` | `None` | Extra insane arguments (`-bd_args`) |
| `additional_residues` | `str` | `None` | Comma-separated residue names (`-res`) |
| `additional_residues_itp` | `str` | `None` | Topology for those residues (`-res_itp`) |
| `residue_cg_file` | `str` | `None` | Folder of coarse-grained residue files (`-res_cg`) |
| `extra_args` | `list` | `None` | Additional CLI arguments passed verbatim |

**Returns:** List of `OrientationResult` sorted by rank.

**Raises:**

- `MemProError` — If `mempro` is not installed or execution fails.
- `FileNotFoundError` — If the input PDB file does not exist.

**Output Structure:**

```
Orient/
├── orientation.txt        # Summary of all ranked orientations
├── Rank_1/
│   └── oriented_rank_1.pdb
├── Rank_2/
│   └── oriented_rank_2.pdb
└── ...
```

**Example 3: Run MemPrO with default settings**

```python
from pathlib import Path
from gatewizard.core.mempro import MemPrO

pdb_file = str(Path(__file__).parent.parent / "6RV3_AB.pdb")
mp = MemPrO()
if MemPrO.is_available():
    results = mp.run(pdb_file)
    for r in results:
        print(
            f"Rank {r.rank}: potential={r.relative_potential:.2f}, hits={r.hits_pct:.1f}%"
        )
        print(f"  PDB: {r.pdb_path}")
```

### Example 4: Run with custom parameters

```
from pathlib import Path
from gatewizard.core.mempro import MemPrO

pdb_file = str(Path(__file__).parent.parent / "6RV3_AB.pdb")
mp = MemPrO()
if MemPrO.is_available():
    results = mp.run(
        pdb_file,
        output_dir="my_orient",
        n_cpus=4,
        n_iters=200,
        grid_size=72,
    )
    print(f"Found {len(results)} orientations")
```

### Example 5: Run with dual membrane mode

```
from pathlib import Path
from gatewizard.core.mempro import MemPrO

pdb_file = str(Path(__file__).parent.parent / "6RV3_AB.pdb")
mp = MemPrO()
if MemPrO.is_available():
    results = mp.run(pdb_file, dual_membrane=True)
    print(f"Dual membrane: {len(results)} orientations")
```

### Example 6: Run with peripheral mode

```
from pathlib import Path
from gatewizard.core.mempro import MemPrO

pdb_file = str(Path(__file__).parent.parent / "6RV3_AB.pdb")
mp = MemPrO()
if MemPrO.is_available():
    results = mp.run(pdb_file, peripheral=True)
    print(f"Peripheral: {len(results)} orientations")
```

---

### parse_results

Parse MemPrO results from an existing Orient directory (static method).

```
MemPrO.parse_results(orient_dir: str) -> List[OrientationResult]
```

**Parameters:**

| Parameter | Type | Required | Description |
|---|---|---|---|
| `orient_dir` | `str` | Yes | Path to the Orient output directory |

**Returns:** List of `OrientationResult` sorted by rank.

**Raises:** `MemProError` — If `orientation.txt` is not found or cannot be parsed.

**Example 7: Parse results from an existing Orient directory**

```
import os
from gatewizard.core.mempro import MemPrO, MemProError

orient_dir = os.environ.get("MEMPRO_ORIENT_DIR", "Orient")
try:
    results = MemPrO.parse_results(orient_dir)
    for r in results:
        print(
            f"Rank {r.rank}: potential={r.relative_potential:.2f}, "
            f"hits={r.hits_pct:.1f}%, pdb={r.pdb_path}"
        )
except (MemProError, FileNotFoundError):
    print(f"Orient directory not found at '{orient_dir}', skipping.")
```

---

### get_oriented_pdb

Get the path to an oriented PDB file for a specific rank (static method).

```
MemPrO.get_oriented_pdb(orient_dir: str, rank: int = 1) -> str
```

**Parameters:**

| Parameter | Type | Default | Description |
|---|---|---|---|
| `orient_dir` | `str` | — | Path to the Orient output directory |
| `rank` | `int` | `1` | Desired rank number |

**Returns:** Absolute path to the oriented PDB file.

**Raises:** `FileNotFoundError` — If the PDB for the rank does not exist.

**Example 8: Get the best oriented PDB**

```
from gatewizard.core.mempro import MemPrO

try:
    pdb = MemPrO.get_oriented_pdb("Orient", rank=1)
    print(f"Best orientation: {pdb}")
except FileNotFoundError as e:
    print(f"Not found: {e}")
```

---

### build_command

Build the MemPrO command line without executing it.

```
build_command(
    pdb_file: str,
    output_dir: Optional[str] = None,
    ...
) -> List[str]
```

Accepts the same parameters as `run()`.

**Returns:** List of command-line tokens.

#### Example 9: Build and inspect command

```python
from gatewizard.core.mempro import MemPrO

mp = MemPrO()
if MemPrO.is_available():
    cmd = mp.build_command("protein.pdb", n_cpus=4, dual_membrane=True)
    print("Command:", " ".join(cmd))
```

---

### Class: OrientationResult

Data class representing a single ranked orientation from a MemPrO run.

#### Attributes

| Attribute | Type | Description |
|---|---|---|
| `rank` | `int` | Rank number (1 = best) |
| `relative_potential` | `float` | Relative potential score |
| `hits_pct` | `float` | Percentage of hits |
| `rerank_potential` | `float` | Re-ranked potential |
| `rerank_depth` | `float` | Re-rank minima depth |
| `rerank_value` | `float` | Re-rank value |
| `pdb_path` | `str` | Path to the oriented PDB file |

#### Example 10: Inspect OrientationResult attributes

```python
import os
from gatewizard.core.mempro import MemPrO, MemProError

orient_dir = os.environ.get("MEMPRO_ORIENT_DIR", "Orient")
try:
    results = MemPrO.parse_results(orient_dir)
    if results:
        best = results[0]
        print(f"Rank: {best.rank}")
        print(f"Relative potential: {best.relative_potential}")
        print(f"Hits: {best.hits_pct}%")
        print(f"Re-rank potential: {best.rerank_potential}")
        print(f"Re-rank depth: {best.rerank_depth}")
        print(f"Re-rank value: {best.rerank_value}")
        print(f"PDB path: {best.pdb_path}")
except (MemProError, FileNotFoundError):
    print(f"Orient directory not found at '{orient_dir}', skipping.")
```

---

#### compute_orientation_transform

```python
from gatewizard.core.mempro import compute_orientation_transform

R, t = compute_orientation_transform(source_pdb, oriented_pdb)
```

Compute the rigid transform that maps protein atoms in `source_pdb` onto a MemPrO `oriented_rank_*.pdb`. Matching uses standard amino-acid atoms keyed by `(chain, resid, atom_name)`.

**Returns:** Rotation matrix `R` and translation `t` such that `oriented ≈ R @ source + t`.

**Raises:** `MemProError` if fewer than three atom pairs match.

#### apply_orientation_transform

```
from gatewizard.core.mempro import apply_orientation_transform

out = apply_orientation_transform(source_pdb, oriented_pdb, output_pdb)
```

Apply the MemPrO transform to every atom in `source_pdb` (protein, ligands, water) and write `output_pdb`. Dummy MemPrO atoms are not copied.

See Example 12 and `StructureManager.apply_mempro_orientation`.

### Class: MemProError

Custom exception for MemPrO-related errors.

```
from gatewizard.core.mempro import MemPrO, MemProError

mp = MemPrO()
try:
    results = mp.run("nonexistent.pdb")
except FileNotFoundError:
    print("PDB file not found")
except MemProError as e:
    print(f"MemPrO error: {e}")
```

---

## Complete Workflow

#### Example 11: Full orientation workflow

```
from pathlib import Path
from gatewizard.core.mempro import MemPrO, MemProError

pdb_file = str(Path(__file__).parent.parent / "6RV3_AB.pdb")
mp = MemPrO()

# Check availability
if not MemPrO.is_available():
    print('Install: pip install "gatewizard[full]"')
else:
    # Run orientation
    try:
        results = mp.run(pdb_file, n_cpus=4)
        print(f"Found {len(results)} orientations\n")

        # Display results table
        print(f"{'Rank':>4}  {'Potential':>10}  {'Hits%':>6}  {'Re-rank':>10}")
        print("-" * 36)
        for r in results:
            print(
                f"{r.rank:>4d}  {r.relative_potential:>10.3f}  "
                f"{r.hits_pct:>5.1f}%  {r.rerank_potential:>10.3f}"
            )

        # Load the best orientation
        if results and results[0].pdb_path:
            print(f"\nBest PDB: {results[0].pdb_path}")
    except MemProError as e:
        print(f"Error: {e}")
```

#### Example 12: Parse existing results and load into StructureManager

```
import os
from gatewizard.core.mempro import MemPrO, MemProError

orient_dir = os.environ.get("MEMPRO_ORIENT_DIR", "Orient")
try:
    # Parse pre-computed results
    results = MemPrO.parse_results(orient_dir)

    # Load the best orientation into the viewer
    if results and results[0].pdb_path:
        from gatewizard.core.structure_manager import StructureManager

        viewer = StructureManager()
        info = viewer.load_structure(results[0].pdb_path)
        print(f"Loaded rank 1: {info['n_atoms']} atoms, {info['n_chains']} chains")
except (MemProError, FileNotFoundError):
    print(f"Orient directory not found at '{orient_dir}', skipping.")
```

## Hydration Module

### Packmol Hydration Module

Fill internal protein cavities and pores with TIP3P waters using standalone **PACKMOL** (AmberTools). Designed for heavy-atom structures from Visualize using **heavy-atom-safe inflated exclusion** radii.

- Check PACKMOL availability
- Detect hydrogen status on the protein
- Estimate free volume inside a 3D box
- Build and run PACKMOL input for cavity hydration
- Optional custom PACKMOL input (advanced)

#### Import

```
from gatewizard.tools.packmol_hydration import (
    check_packmol_available,
    detect_hydrogen_status,
    estimate_cavity_volume,
    build_hydrate_inp_text,
    prepare_hydration_job,
    hydrate_cavity,
    preview_hydrate_inp,
    run_custom_packmol,
    PackmolHydrationError,
)
```

---

##### check_packmol_available

```
check_packmol_available() -> dict
```

**Returns:** `{ "available": bool, "version": str|None, "resolved_path": str|None }`

###### Example 1: Check PACKMOL availability

```python
from gatewizard.tools.packmol_hydration import check_packmol_available

info = check_packmol_available()
print(f"PACKMOL available: {info['available']}")
if info["resolved_path"]:
    print(f"Path: {info['resolved_path']}")
if info["version"]:
    print(f"Version: {info['version']}")
```

---

#### detect_hydrogen_status

```python
detect_hydrogen_status(
    pdb_file: str | None = None,
    atoms: Sequence[Atom] | None = None,
) -> str
```

**Returns:** `"full"`, `"partial"`, or `"none"` (protein heavy-atom vs H ratio).

##### Example 2: Detect hydrogen status on 6RV3_AB

```python
from pathlib import Path

from gatewizard.tools.packmol_hydration import detect_hydrogen_status

PDB_FILE = str(Path(__file__).parent.parent / "6RV3_AB.pdb")
status = detect_hydrogen_status(pdb_file=PDB_FILE)
print(f"Hydrogen status for 6RV3_AB: {status}")
```

---

#### estimate_cavity_volume

```python
estimate_cavity_volume(
    pdb_file: str,
    box_min: Sequence[float],
    box_max: Sequence[float],
    solute_radius: float | None = None,
    exclusion_mode: str | None = None,
    grid_spacing: float | None = None,
    atom_indices: Sequence[int] | None = None,
) -> VolumeEstimate
```

**Parameters:**

| Parameter | Type | Default | Description |
|---|---|---|---|
| `pdb_file` | `str` | — | Input PDB path |
| `box_min` | `(x, y, z)` | — | Box minimum corner (Å) |
| `box_max` | `(x, y, z)` | — | Box maximum corner (Å) |
| `solute_radius` | `float` | auto | PACKMOL solute `radius`; default 2.5 Å (heavy-atom) or 1.5 Å (explicit) |
| `exclusion_mode` | `str` | auto | `"heavy_atom_safe"` or `"explicit"` |
| `grid_spacing` | `float` | `0.5` | Volume grid spacing (Å) |
| `atom_indices` | `list[int]` | `None` | Optional subset of atoms for exclusion |

**Returns:** `VolumeEstimate` dataclass (also `.as_dict()`).

##### Example 3: Estimate cavity volume with a fixed box

```python
from pathlib import Path

from gatewizard.tools.packmol_hydration import estimate_cavity_volume

PDB_FILE = str(Path(__file__).parent.parent / "6RV3_AB.pdb")
BOX_MIN = (-13.03, -49.98, 18.67)
BOX_MAX = (6.97, -29.98, 38.67)

result = estimate_cavity_volume(
    pdb_file=PDB_FILE,
    box_min=BOX_MIN,
    box_max=BOX_MAX,
)
print(f"Box volume: {result.box_volume_A3:.1f} Å³")
print(f"Free volume: {result.free_volume_A3:.1f} Å³")
print(f"Suggested waters: {result.suggested_waters}")
print(f"Exclusion mode: {result.exclusion_mode}")
print(f"Hydrogen status: {result.hydrogen_status}")
```

---

#### build_hydrate_inp_text

```python
build_hydrate_inp_text(
    protein_path: str,
    tip3p_path: str,
    output_pdb: str,
    box_min: Sequence[float],
    box_max: Sequence[float],
    n_waters: int,
    solute_radius: float = 2.5,
    tolerance: float = 2.0,
    nloop: int = 20,
) -> str
```

**Returns:** PACKMOL input file contents as a string.

##### Example 4: Build PACKMOL input text (preview)

```python
from pathlib import Path

from gatewizard.tools.packmol_hydration import build_hydrate_inp_text, estimate_cavity_volume

PDB_FILE = str(Path(__file__).parent.parent / "6RV3_AB.pdb")
BOX_MIN = (-13.03, -49.98, 18.67)
BOX_MAX = (6.97, -29.98, 38.67)

vol = estimate_cavity_volume(PDB_FILE, BOX_MIN, BOX_MAX)
n_waters = max(1, min(vol.suggested_waters, 50))

inp = build_hydrate_inp_text(
    protein_path=PDB_FILE,
    tip3p_path="TIP3P.pdb",
    output_pdb="6RV3_AB_hydrated.pdb",
    box_min=BOX_MIN,
    box_max=BOX_MAX,
    n_waters=n_waters,
    solute_radius=2.5,
)
print(inp)
```

---

#### prepare_hydration_job

```
prepare_hydration_job(
    pdb_file: str,
    job_dir: str,
    box_min: Sequence[float],
    box_max: Sequence[float],
    n_waters: int,
    ...
) -> dict
```

Creates `job_dir`, copies the protein and `TIP3P.pdb`, writes `packmol.inp`.

**Example 5: Prepare hydration job files**

```
import shutil
import tempfile
from pathlib import Path

from gatewizard.tools.packmol_hydration import estimate_cavity_volume, prepare_hydration_job

PDB_FILE = str(Path(__file__).parent.parent / "6RV3_AB.pdb")
BOX_MIN = (-13.03, -49.98, 18.67)
BOX_MAX = (6.97, -29.98, 38.67)

vol = estimate_cavity_volume(PDB_FILE, BOX_MIN, BOX_MAX)
n_waters = max(1, min(vol.suggested_waters, 50))

tmp = Path(tempfile.mkdtemp())
try:
    job = prepare_hydration_job(
        pdb_file=PDB_FILE,
        job_dir=str(tmp),
        box_min=BOX_MIN,
        box_max=BOX_MAX,
        n_waters=n_waters,
    )
    print(f"Job dir: {job['job_dir']}")
    print(f"Input file: {job['packmol_inp_path']}")
    print(f"Output PDB name: {job['output_pdb_name']}")
    for name in ("packmol.inp", "TIP3P.pdb", Path(PDB_FILE).name):
        path = tmp / name
        print(f"  {name}: {'OK' if path.is_file() else 'MISSING'}")
finally:
    shutil.rmtree(tmp, ignore_errors=True)
```

---

**hydrate_cavity**

```
hydrate_cavity(
    pdb_file: str,
    working_dir: str,
    output_folder_name: str,
    box_min: Sequence[float],
    box_max: Sequence[float],
    n_waters: int | None = None,
    ...
) -> HydrationJobResult
```

Prepares the job under `working_dir/output_folder_name`, runs PACKMOL, returns paths and log.

**Example 6: Full cavity hydration workflow**

```python
import tempfile
from pathlib import Path

from gatewizard.tools.packmol_hydration import (
    check_packmol_available,
    estimate_cavity_volume,
    hydrate_cavity,
)

PDB_FILE = str(Path(__file__).parent.parent / "6RV3_AB.pdb")
BOX_MIN = (-13.03, -49.98, 18.67)
BOX_MAX = (6.97, -29.98, 38.67)

if check_packmol_available()["available"]:
    vol = estimate_cavity_volume(PDB_FILE, BOX_MIN, BOX_MAX)
    n_waters = max(1, min(vol.suggested_waters, 20))
    with tempfile.TemporaryDirectory() as tmp:
        result = hydrate_cavity(
            pdb_file=PDB_FILE,
            working_dir=tmp,
            output_folder_name="hydration_6RV3_AB",
            box_min=BOX_MIN,
            box_max=BOX_MAX,
            n_waters=n_waters,
        )
        print(f"Success: {result.success}")
        print(f"Output: {result.output_pdb}")
        print(f"Message: {result.message}")
else:
    print("PACKMOL not installed; skipping hydration run")
```

---

**Complete workflow**

Typical GateWizard usage:

1. Load a heavy-atom PDB in **Visualize**
2. Define a hydration box around an internal cavity
3. Run **Example 3** to estimate volume (or use the GUI)
4. Run **Example 6** (or the GUI) to fill with TIP3P waters into `hydration_{pdb}/`
5. Load the hydrated PDB and continue with **Preparation** → **Builder**

When no hydrogens are present, the module uses **heavy-atom-safe** inflated radii and a default solute `radius` of **2.5 Å** so waters do not clash with polar H added later in Builder/tleap.

---

#### preview_hydrate_inp

```python
from gatewizard.tools.packmol_hydration import preview_hydrate_inp

text = preview_hydrate_inp(...)
```

Return PACKMOL input text without writing a job directory. Same arguments as `build_hydrate_inp_text`.

#### run_custom_packmol

Run user-supplied PACKMOL input text (see GUI custom tab). Returns `{ job_dir, packmol_log, output_pdb, success, message }`.

# Equilibration

## Equilibration Module

Module for setting up NAMD, OpenMM, GROMACS, and Amber equilibration protocols for membrane protein systems. Generates configuration files, restraint files, and run scripts for multi-stage equilibration simulations using AMBER force fields.

The three engine managers share a similar user-facing API, with engine-specific file formats and run scripts.

Systems without a protein (for example a packed bilayer) omit `protein_backbone` / `protein_sidechain` restraints and protein COM/rotation. Detection uses the system PDB or GRO (`pdb_has_protein`). Lipid restraints are unchanged.

### Import

```
from gatewizard.tools.equilibration import (
    NAMDEquilibrationManager,
    OpenMMEquilibrationManager,
    GROMACSEquilibrationManager,
    AmberEquilibrationManager,
)
```

### Remote cluster helpers (Slurm-first)

Programmatic helpers for probing modules, rendering batch scripts, and recording remote `execution` metadata live in `gatewizard.utils.cluster` (used by the GUI). Typical flow: generate inputs locally (produces `run_equilibration.sh` for local runs and `run_equilibration_cluster.sh` for Slurm), then wrap the cluster runner with `render_batch_script(...)` (`run_command` defaults to `bash run_equilibration_cluster.sh`; workdir strategy e.g. `scratch_job_id`).

```
from gatewizard.utils.cluster import BatchScriptRequest, render_batch_script

script = render_batch_script(
    BatchScriptRequest(
        job_name="eq_popc",
        cpus=8,
        gpus=1,
        modules=["md/namd/3.0b6+cuda"],
        workdir_strategy="scratch_job_id",
        scratch_root="$SCRATCH_DIR",
    )
)
```

See the user guide section **Running on HPC Clusters** for the GUI workflow (Settings → Clusters, Equilibration Run target).

### Class: NAMDEquilibrationManager

Manager for NAMD equilibration simulations with support for multi-stage protocols, flexible restraints, and ensemble control.

#### Constructor

```
NAMDEquilibrationManager(working_dir: Path, namd_executable: str = "namd3")
```

**Parameters:**

| Parameter | Type | Default | Description |
|---|---|---|---|
| `working_dir` | `Path` | Required | Working directory containing system files |

| Parameter | Type | Default | Description |
|---|---|---|---|
| `namd_executable` | `str` | `"namd3"` | NAMD executable name or path |

**Returns:** `NAMDEquilibrationManager` instance

---

### Quick Start

#### Example 1: Automatic File Detection (Simplest)

```python
from pathlib import Path
from gatewizard.tools.equilibration import NAMDEquilibrationManager

# Point to folder with system files
work_dir = Path(__file__).parent / "popc_membrane"

# Define equilibration stages
stages = [
    {
        "name": "Equilibration 1",
        "time_ns": 0.125,
        "steps": 125000,
        "ensemble": "NVT",
        "temperature": 310.15,
        "timestep": 1.0,
        "minimize_steps": 10000,
        "constraints": {
            "protein_backbone": 10.0,
            "protein_sidechain": 5.0,
            "lipid_head": 2.5,
            "lipid_tail": 2.5,
            "water": 0.0,
            "ions": 10.0,
            "other": 0.0,
        },
    }
]

# Setup with automatic file detection (no system_files needed!)
# scheme_type is auto-detected from the 'ensemble' field in stages
manager = NAMDEquilibrationManager(work_dir)
result = manager.setup_namd_equilibration(
    stage_params_list=stages, output_name="equilibration_example_01"
)

# Note that output folder is created inside the system_folder

print(f"Setup complete: {result['namd_dir']}")
# Run with: cd {result['namd_dir']} && ./run_equilibration.sh
```

#### Example 2: Explicit File Paths (Alternative)

```python
from pathlib import Path
from gatewizard.tools.equilibration import NAMDEquilibrationManager

# Point to folder with system files
work_dir = Path(__file__).parent / "popc_membrane"

# Explicitly define system files (if auto-detection doesn't work)
system_files = {
    "prmtop": str(work_dir / "system.prmtop"),
    "inpcrd": str(work_dir / "system.inpcrd"),
    "pdb": str(work_dir / "system.pdb"),
    "bilayer_pdb": str(work_dir / "bilayer_protein_protonated_prepared_lipid.pdb"),
}

# Define equilibration stages
stages = [
    {
        "name": "Equilibration 1",
        "time_ns": 0.125,
        "steps": 125000,
        "ensemble": "NVT",
        "temperature": 310.15,
        "timestep": 1.0,
        "minimize_steps": 10000,
        "constraints": {
            "protein_backbone": 10.0,
            "protein_sidechain": 5.0,
            "lipid_head": 2.5,
            "lipid_tail": 2.5,
            "water": 0.0,
            "ions": 10.0,
            "other": 0.0,
        },
    }
]

# Setup with explicit file paths
# scheme_type is auto-detected from the 'ensemble' field in stages
manager = NAMDEquilibrationManager(work_dir)
result = manager.setup_namd_equilibration(
    system_files=system_files,
    stage_params_list=stages,
    output_name="equilibration_example_02",
)

print(f"Setup complete: {result['namd_dir']}")
# Run with: cd {result['namd_dir']} && ./run_equilibration.sh
```

---

### Helper Methods

#### find_system_files

Automatically detect system files in the working directory. Useful for verifying which files will be used before running setup.

```python
find_system_files() -> Optional[Dict[str, str]]
```

**Parameters:** None

**Returns:** `Optional[Dict[str, str]]`

- Dictionary with detected file paths, or `None` if required files not found
- Keys: `'prmtop'`, `'inpcrd'`, `'pdb'`, `'bilayer_pdb'`

**Detection Priority:**

1. **Topology:** `*.prmtop` files

2. **Coordinates:** `*.inpcrd` → `*.crd` → `*.rst`
3. **System PDB:** `system.pdb` → any non-bilayer `*.pdb`
4. **Bilayer PDB:** `bilayer*_lipid.pdb` (with CRYST1) → `bilayer_*.pdb` → `*_bilayer.pdb`

**Example 3:**

```
from pathlib import Path
from gatewizard.tools.equilibration import NAMDEquilibrationManager

manager = NAMDEquilibrationManager(Path("popc_membrane"))

# Check which files will be used
system_files = manager.find_system_files()
if system_files:
    print("Detected files:")
    for key, path in system_files.items():
        print(f"  {key}: {Path(path).name}")

    # Now run setup with auto-detection
    # result = manager.setup_namd_equilibration(
    #     stage_params_list=stages
    # )
else:
    print("Required files not found - please check working directory")
```

---

### Core Method

#### setup_namd_equilibration

Complete equilibration setup with automatic file generation. This method:

1. Auto-detects system files (if not provided)
2. Creates output directory structure
3. Copies system files (prmtop, inpcrd, pdb) to NAMD directory
4. Generates NAMD configuration files for each stage
5. Creates restraint files based on constraints
6. Generates executable run script
7. Creates protocol summary JSON

```
setup_namd_equilibration(
    system_files: Optional[Dict[str, str]] = None,
    stage_params_list: List[Dict[str, Any]] = None,
    output_name: str = "equilibration",
    scheme_type: str = "NPT",
    namd_executable: str = "namd3"
) -> Dict[str, Any]
```

**Parameters:**

| Parameter | Type | Default | Description |
|---|---|---|---|
| `system_files` | `Optional[Dict[str, str]]` | `None` | System file paths (see below). If `None`, auto-detects files in working_dir |
| `stage_params_list` | `List[Dict[str, Any]]` | Required | List of stage parameter dictionaries (see below). Each stage must include `ensemble` field |
| `output_name` | `str` | `"equilibration"` | Base name for output folder |
| `scheme_type` | `Optional[str]` | `None` | (Optional) Equilibration scheme (NVT, NPT, NPAT, NPgT). If `None`, auto-detected from first stage's `ensemble` field |
| `namd_executable` | `str` | `"namd3"` | NAMD executable name/path |

**Returns:** `Dict[str, Any]` with keys:

- `namd_dir` (`Path`): NAMD output directory path
- `config_files` (`List[Path]`): Generated .conf files
- `run_script` (`Path`): Executable run script path
- `summary_file` (`Path`): Protocol summary JSON path

##### System Files Dictionary

**Auto-Detection (Recommended):**

If `system_files=None` (default), the method will automatically search the working directory for:

- `*.prmtop` - AMBER topology file
- `*.inpcrd`, `*.crd`, or `*.rst` - AMBER coordinate file
- `system.pdb` - System PDB file
- `bilayer*_lipid.pdb` - Bilayer PDB with CRYST1 record (highest priority)

**Manual Specification (Alternative):**

If auto-detection fails or you need specific files, provide a dictionary with these keys:

| Key | Type | Description |
|---|---|---|
| `prmtop` | `str` | Path to AMBER topology file (system.prmtop) |
| `inpcrd` | `str` | Path to AMBER coordinate file (system.inpcrd) |
| `pdb` | `str` | Path to system PDB file (system.pdb) |
| `bilayer_pdb` | `str` | **REQUIRED** Path to bilayer PDB with CRYST1 record |

**Important: CRYST1 Box Dimensions**

The `bilayer_pdb` file **must contain** a CRYST1 record that defines the periodic box dimensions. This file is typically:

- Named `bilayer_*_lipid.pdb` (from packmol-memgen --parametrize)
- Generated during system preparation
- Contains the header: `CRYST1 70.335 70.833 85.067 90.00 90.00 90.00 P 1`

  **Warning:** Without proper CRYST1 information, NAMD will use incorrect box dimensions, leading to simulation failures.

##### Stage Parameters Dictionary

Each stage in `stage_params_list` must be a dictionary with the following structure:

**Required Parameters:**

| Parameter | Type | Description |
|---|---|---|
| `name` | `str` | Stage name (e.g., "Equilibration 1"). **Note:** Names are optional and used primarily for logging and user feedback. The actual configuration file names are generated sequentially (step1, step2, etc.) regardless of the name you provide |
| `time_ns` | `float` | Simulation time in nanoseconds |
| `steps` | `int` | Number of MD steps |
| `ensemble` | `str` | Ensemble type (NVT, NPT, NPAT, NPgT) |
| `temperature` | `float` | Temperature in Kelvin |
| `timestep` | `float` | Timestep in femtoseconds (1.0 or 2.0) |

| Parameter | Type | Description |
|---|---|---|
| `constraints` | `Dict[str, float]` | Restraint forces (see below) |

**Optional Parameters:**

| Parameter | Type | Default | Description |
|---|---|---|---|
| `minimize_steps` | `int` | `0` | Minimization steps before MD (first stage only) |
| `pressure` | `float` | `1.0` | Pressure in bar (NPT, NPAT, NPgT only) |
| `surface_tension` | `float` | `0.0` | Surface tension in dyn/cm (NPgT, and OpenMM NPT). NPAT holds the area fixed and does not use this value. |
| `dcd_freq` | `int` | `5000` | DCD output frequency (steps) |
| `use_gpu` | `bool` | `True` | Enable GPU acceleration |
| `cpu_cores` | `int` | `1` | Number of CPU cores |
| `gpu_id` | `int` | `0` | GPU device ID |
| `num_gpus` | `int` | `1` | Number of GPUs to use |
| `custom_template` | `str` | `None` | Explicitly specify CHARMM-GUI template file (e.g., `'step6.3_equilibration.inp'`) |

**Constraints Dictionary:**

Restraint forces in kcal/mol/Å². Applied to atomic groups:

| Key | Description | Typical Values |
|---|---|---|
| `protein_backbone` | Protein backbone atoms (CA, C, N, O) | 10.0 → 5.0 → 1.0 → 0.0 |
| `protein_sidechain` | Protein sidechain heavy atoms | 5.0 → 2.5 → 0.5 → 0.0 |
| `lipid_head` | Lipid head group atoms (P, O) | 5.0 → 2.5 → 1.0 → 0.0 |
| `lipid_tail` | Lipid tail carbon atoms | 5.0 → 2.5 → 0.5 → 0.0 |
| `water` | Water molecules (usually unrestrained) | 0.0 |
| `ions` | Ion atoms (K+, Cl-, etc.) | 10.0 → 0.0 |
| `other` | Other molecules/ligands | 0.0 |

---

### Ensemble Types

| Ensemble | Full Name | Control | Use Case |
|---|---|---|---|
| **NVT** | Canonical | Temperature | Initial heating, fixed box |
| **NPT** | Isothermal-isobaric | Temp + Pressure (isotropic) | General equilibration |
| **NPAT** | Constant surface area | Temp + pressure along Z; the membrane area stays fixed | Membrane area held constant |
| **NPgT** | Constant surface tension | Temp + pressure + surface tension | Membrane with a surface-tension target |

---

### Advanced Features

#### Per-Stage Ensemble Selection

Each stage can use a **different ensemble** by specifying the `ensemble` field. This allows flexible equilibration protocols:

```
stages = [
    {'name': 'Equilibration 1', 'ensemble': 'NVT', ...},   # Heat with NVT
    {'name': 'Equilibration 2', 'ensemble': 'NPT', ...},   # Equilibrate with NPT
    {'name': 'Equilibration 3', 'ensemble': 'NPAT', ...},  # Relax membrane with NPAT
]
```

**How it works:**

- The `scheme_type` parameter is auto-detected from the first stage's `ensemble` field
- Each stage uses its own `ensemble` to select the appropriate CHARMM-GUI template
- When a stage's ensemble differs from the protocol default, a **warning is logged**
- Templates are automatically loaded from the correct ensemble folder (01_NVT, 02_NPT, 03_NPAT, 04_NPgT)

**Example: Mixed Ensemble Protocol**

```
stages = [
    {
        'name': 'Initial Heating',
        'ensemble': 'NVT',      # Uses NVT template
        'time_ns': 0.25,
        'temperature': 310.15,
        'timestep': 1.0,
        'minimize_steps': 10000,
        'constraints': {'protein_backbone': 10.0, ...}
    },
    {
        'name': 'Pressure Equilibration',
        'ensemble': 'NPT',      # Uses NPT template (warning logged)
        'time_ns': 0.5,
        'temperature': 310.15,
        'pressure': 1.0,
        'timestep': 1.0,
        'constraints': {'protein_backbone': 5.0, ...}
    },
    {
        'name': 'Membrane Relaxation',
        'ensemble': 'NPAT',     # Uses NPAT template (warning logged)
        'time_ns': 1.0,
        'temperature': 310.15,
        'pressure': 1.0,
        'surface_tension': 0.0,
        'timestep': 2.0,
        'constraints': {'protein_backbone': 1.0, ...}
    }
]

# Setup automatically detects scheme_type='NVT' from first stage
# Stages 2-3 use different ensembles with appropriate templates
result = manager.setup_namd_equilibration(stage_params_list=stages)
```

**Output:**

```
INFO - Auto-detected scheme_type from stages: NVT
INFO - Generated: step1_equilibration.conf
WARNING - Stage 2 (Pressure Equilibration) uses ensemble 'NPT' but protocol default is 'NVT'. Using 'NPT'
template.
INFO - Generated: step2_equilibration.conf
WARNING - Stage 3 (Membrane Relaxation) uses ensemble 'NPAT' but protocol default is 'NVT'. Using 'NPAT'
template.
INFO - Generated: step3_equilibration.conf
```

#### Custom Template Selection

For advanced control, explicitly specify which CHARMM-GUI template to use with the `custom_template` parameter:

```
stages = [
    {
        'name': 'Strong Restraints',
        'ensemble': 'NPT',
        'custom_template': 'step6.1_equilibration.inp',  # Use template from stage 1
        'time_ns': 0.25,
        'constraints': {'protein_backbone': 10.0, ...}
    },
    {
        'name': 'Medium Restraints',
        'ensemble': 'NPT',
        'custom_template': 'step6.3_equilibration.inp',  # Use template from stage 3
        'time_ns': 0.5,
        'constraints': {'protein_backbone': 5.0, ...}
    },
    {
        'name': 'Light Restraints',
        'ensemble': 'NPAT',
        'custom_template': 'step6.5_equilibration.inp',  # Use template from stage 5
        'time_ns': 1.0,
        'constraints': {'protein_backbone': 1.0, ...}
    }
]
```

**Use cases for custom templates:**

- Skip intermediate equilibration stages (use step6.5 directly)
- Repeat a specific stage with different parameters
- Mix templates from different equilibration phases
- Test different template configurations

**Available templates per ensemble:**

- `step6.1_equilibration.inp` through `step6.6_equilibration.inp` (6 equilibration stages)
- `step7_production.inp` (production stage)

#### MDAnalysis Atom Counts and Selection Editing (GUI)

When using the Equilibration GUI frame, the constraint section for each stage now displays:

- **Atom count labels** next to each constraint entry, showing how many atoms match the selection
- **Gear button** (⚙) on each constraint row to edit the MDAnalysis selection string
- **Add Selection button** (+) at the bottom of each stage to add custom named selections
- **Auto-detect ligands** when an input folder is selected — non-standard residues are automatically added as `ligand_<RESNAME>` entries

**Gear Button (Selection Editor):**

Clicking the gear icon opens a modal dialog where you can:

- View the current MDAnalysis selection string
- Edit the selection expression
- Click **Test** to count matching atoms in the loaded PDB
- Click **Apply** to save the modified selection

**Add Selection Button:**

Clicking the **+** button opens a dialog to:

- Enter a custom name (e.g., `drug_A`)
- Provide an MDAnalysis selection string (e.g., `resname LIG and around 5 protein`)
- Set the default restraint force (kcal/mol/Å$^2$)

- Test the selection before adding

New selections are added to **all stages** in the protocol.

---

#### Custom Stage Names

Stage names can be anything - they don't need to follow the "Equilibration N" convention:

```
stages = [
    {'name': 'Initial Heating', ...},           # Maps to step1
    {'name': 'Pressure Equilibration', ...},    # Maps to step2
    {'name': 'Membrane Relaxation', ...},       # Maps to step3
    {'name': 'Production Preparation', ...},    # Maps to step4
]
```

**How it works:**

- Custom names are automatically mapped to sequential step numbers (step1, step2, step3, ...)
- Config files use sequential naming: `step1_equilibration.conf`, `step2_equilibration.conf`, etc.
- Restart files are properly chained: `step2` loads from `step1`, `step3` loads from `step2`, etc.
- `inputname` variables are correctly set for all stages

---

### Working Examples

#### Example 4: Three-Stage Protocol

```python
from pathlib import Path
from gatewizard.tools.equilibration import NAMDEquilibrationManager

# Point to system folder
work_dir = Path(__file__).parent / "popc_membrane"

stages = [
    {
        "name": "Equilibration 1",
        "time_ns": 0.125,
        "steps": 125000,
        "ensemble": "NVT",
        "temperature": 303.15,
        "timestep": 1.0,
        "minimize_steps": 10000,
        "constraints": {
            "protein_backbone": 10.0,
            "protein_sidechain": 5.0,
            "lipid_head": 2.5,
            "lipid_tail": 2.5,
            "water": 0.0,
            "ions": 10.0,
            "other": 0.0,
        },
    },
    {
        "name": "Equilibration 2",
        "time_ns": 0.125,
        "steps": 125000,
        "ensemble": "NVT",
        "temperature": 303.15,
        "timestep": 1.0,
        "constraints": {
            "protein_backbone": 5.0,
            "protein_sidechain": 2.5,
            "lipid_head": 2.5,
            "lipid_tail": 2.5,
            "water": 0.0,
            "ions": 0.0,
            "other": 0.0,
        },
    },
    {
        "name": "Equilibration 3",
        "time_ns": 0.125,
        "steps": 125000,
        "ensemble": "NPT",
        "temperature": 303.15,
        "pressure": 1.0,
        "timestep": 1.0,
        "constraints": {
            "protein_backbone": 2.5,
            "protein_sidechain": 1.0,
            "lipid_head": 1.0,
            "lipid_tail": 1.0,
            "water": 0.0,
            "ions": 0.0,
            "other": 0.0,
        },
    },
]

# Auto-detect files and setup
# scheme_type auto-detected from stages (NVT from first stage)
# Stage 3 uses NPT ensemble - warning will be logged
manager = NAMDEquilibrationManager(work_dir)
result = manager.setup_namd_equilibration(
    stage_params_list=stages,
    output_name="equilibration_example_04",
    namd_executable="namd3",
)

print(f"\n✓ Setup complete!")
```

```
print(f"  Config files: {len(result['config_files'])}")
print(f"  Run script: {result['run_script'].name}")
```

**Example 5: Custom Four-Stage Protocol**

```python
from pathlib import Path
from gatewizard.tools.equilibration import NAMDEquilibrationManager

# Point to system folder
work_dir = Path(__file__).parent / "popc_membrane"
system_files = {
    "prmtop": str(work_dir / "system.prmtop"),
    "inpcrd": str(work_dir / "system.inpcrd"),
    "pdb": str(work_dir / "system.pdb"),
    "bilayer_pdb": str(work_dir / "bilayer_protein_protonated_prepared_lipid.pdb"),
}

custom_protocol = [
    {
        "name": "Initial Equilibration",
        "time_ns": 0.25,
        "steps": 250000,
        "ensemble": "NVT",
        "temperature": 310.15,
        "timestep": 1.0,
        "minimize_steps": 10000,
        "constraints": {
            "protein_backbone": 10.0,
            "protein_sidechain": 5.0,
            "lipid_head": 5.0,
            "lipid_tail": 5.0,
            "water": 0.0,
            "ions": 10.0,
            "other": 0.0,
        },
    },
    {
        "name": "Pressure Equilibration",
        "time_ns": 0.5,
        "steps": 500000,
        "ensemble": "NPT",
        "temperature": 310.15,
        "pressure": 1.0,
        "timestep": 1.0,
        "constraints": {
            "protein_backbone": 5.0,
            "protein_sidechain": 2.5,
            "lipid_head": 2.5,
            "lipid_tail": 2.5,
            "water": 0.0,
            "ions": 0.0,
            "other": 0.0,
        },
    },
    {
        "name": "Membrane Relaxation",
        "time_ns": 1.0,
        "steps": 500000,
        "ensemble": "NPAT",
        "temperature": 310.15,
        "pressure": 1.0,
        "surface_tension": 0.0,
        "timestep": 2.0,
        "constraints": {
            "protein_backbone": 2.0,
            "protein_sidechain": 1.0,
            "lipid_head": 1.0,
            "lipid_tail": 0.5,
            "water": 0.0,
            "ions": 0.0,
            "other": 0.0,
        },
    },
    {
        "name": "Production Preparation",
        "time_ns": 2.0,
        "steps": 1000000,
        "ensemble": "NPAT",
        "temperature": 310.15,
        "pressure": 1.0,
```

```
        "surface_tension": 0.0,
        "timestep": 2.0,
        "constraints": {
            "protein_backbone": 0.5,
            "protein_sidechain": 0.0,
            "lipid_head": 0.0,
            "lipid_tail": 0.0,
            "water": 0.0,
            "ions": 0.0,
            "other": 0.0,
        },
    },
]

# Auto-detect and setup
# scheme_type auto-detected from stages (NVT from first stage)
# Stages 2-4 use different ensembles - warnings will be logged
manager = NAMDEquilibrationManager(work_dir)
result = manager.setup_namd_equilibration(
    stage_params_list=custom_protocol,
    output_name="equilibration_example_05",
    namd_executable="namd3",
)

print(f"\n✓ Setup complete!")
print(f"  Total stages: {len(custom_protocol)}")
print(f"  Total time: {sum(s['time_ns'] for s in custom_protocol):.1f} ns")
print(f"\nTo run:")
print(f"  cd {result['namd_dir']}")
print(f"  ./run_equilibration.sh")
```

**Example 6: Complete CHARMM-GUI 7-Stage Protocol in NPT ensemble**

```
from pathlib import Path
from gatewizard.tools.equilibration import NAMDEquilibrationManager

# System folder
work_dir = Path(__file__).parent / "popc_membrane"

stages = [
    {
        "name": "Equilibration 1",
        "time_ns": 0.125,
        "steps": 125000,
        "ensemble": "NPT",
        "temperature": 303.15,
        "minimize_steps": 10000,
        "timestep": 1.0,
        "constraints": {
            "protein_backbone": 10.0,
            "protein_sidechain": 5.0,
            "lipid_head": 2.5,
            "lipid_tail": 2.5,
            "water": 0.0,
            "ions": 10.0,
            "other": 0.0,
        },
    },
    {
        "name": "Equilibration 2",
        "time_ns": 0.125,
        "steps": 125000,
        "ensemble": "NPT",
        "temperature": 303.15,
        "timestep": 1.0,
        "constraints": {
            "protein_backbone": 5.0,
            "protein_sidechain": 2.5,
            "lipid_head": 2.5,
            "lipid_tail": 2.5,
            "water": 0.0,
            "ions": 0.0,
            "other": 0.0,
        },
    },
    {
        "name": "Equilibration 3",
        "time_ns": 0.125,
        "steps": 125000,
        "ensemble": "NPT",
        "temperature": 303.15,
        "pressure": 1.0,
        "surface_tension": 0.0,
        "timestep": 1.0,
        "constraints": {
            "protein_backbone": 2.5,
            "protein_sidechain": 1.0,
            "lipid_head": 1.0,
            "lipid_tail": 1.0,
            "water": 0.0,
            "ions": 0.0,
            "other": 0.0,
        },
    },
    {
        "name": "Equilibration 4",
        "time_ns": 0.5,
        "steps": 500000,
        "ensemble": "NPT",
        "temperature": 303.15,
        "pressure": 1.0,
        "surface_tension": 0.0,
        "timestep": 1.0,
        "constraints": {
            "protein_backbone": 1.0,
            "protein_sidechain": 0.5,
            "lipid_head": 0.5,
            "lipid_tail": 0.5,
```

```
            "water": 0.0,
            "ions": 0.0,
            "other": 0.0,
        },
    },
    {
        "name": "Equilibration 5",
        "time_ns": 0.5,
        "steps": 250000,
        "ensemble": "NPT",
        "temperature": 303.15,
        "pressure": 1.0,
        "surface_tension": 0.0,
        "timestep": 2.0,
        "constraints": {
            "protein_backbone": 0.5,
            "protein_sidechain": 0.1,
            "lipid_head": 0.1,
            "lipid_tail": 0.1,
            "water": 0.0,
            "ions": 0.0,
            "other": 0.0,
        },
    },
    {
        "name": "Equilibration 6",
        "time_ns": 0.5,
        "steps": 250000,
        "ensemble": "NPT",
        "temperature": 303.15,
        "pressure": 1.0,
        "surface_tension": 0.0,
        "timestep": 2.0,
        "constraints": {
            "protein_backbone": 0.1,
            "protein_sidechain": 0.0,
            "lipid_head": 0.0,
            "lipid_tail": 0.0,
            "water": 0.0,
            "ions": 0.0,
            "other": 0.0,
        },
    },
    {
        "name": "Production",
        "time_ns": 10.0,
        "steps": 5000000,
        "ensemble": "NPT",
        "temperature": 303.15,
        "pressure": 1.0,
        "surface_tension": 0.0,
        "timestep": 2.0,
        "constraints": {
            "protein_backbone": 0.0,
            "protein_sidechain": 0.0,
            "lipid_head": 0.0,
            "lipid_tail": 0.0,
            "water": 0.0,
            "ions": 0.0,
            "other": 0.0,
        },
    },
]

# Auto-detect and setup
# scheme_type auto-detected from first stage's ensemble
manager = NAMDEquilibrationManager(work_dir)
result = manager.setup_namd_equilibration(
    stage_params_list=stages,
    output_name="equilibration_example_06",
    namd_executable="namd3",
)

print(f"\n✓ Complete! Generated {len(result['config_files'])} configuration files")
```

```
print(f"  Total equilibration: {sum(s['time_ns'] for s in stages[:-1]):.3f} ns")
print(f"  Production: {stages[-1]['time_ns']:.1f} ns")
```

#### Example 7: Custom Template Selection

This example demonstrates explicit template control for advanced workflows:

```python
from pathlib import Path
from gatewizard.tools.equilibration import NAMDEquilibrationManager

# System folder
work_dir = Path(__file__).parent / "popc_membrane"

system_files = {
    "prmtop": str(work_dir / "system.prmtop"),
    "inpcrd": str(work_dir / "system.inpcrd"),
    "pdb": str(work_dir / "system.pdb"),
    "bilayer_pdb": str(work_dir / "bilayer_protein_protonated_prepared_lipid.pdb"),
}

# Use explicit templates to skip intermediate stages
stages = [
    {
        "name": "Strong Restraints Phase",
        "ensemble": "NPT",
        "custom_template": "step6.1_equilibration.inp",  # Use stage 1 template
        "time_ns": 0.25,
        "timestep": 1.0,
        "temperature": 310.15,
        "pressure": 1.0,
        "minimize_steps": 10000,
        "constraints": {
            "protein_backbone": 10.0,
            "protein_sidechain": 5.0,
            "lipid_head": 5.0,
            "lipid_tail": 5.0,
            "water": 0.0,
            "ions": 10.0,
            "other": 0.0,
        },
    },
    {
        "name": "Medium Restraints Phase",
        "ensemble": "NPT",
        "custom_template": "step6.3_equilibration.inp",  # Skip to stage 3 template
        "time_ns": 0.5,
        "timestep": 1.0,
        "temperature": 310.15,
        "pressure": 1.0,
        "constraints": {
            "protein_backbone": 5.0,
            "protein_sidechain": 2.5,
            "lipid_head": 2.5,
            "lipid_tail": 2.5,
            "water": 0.0,
            "ions": 0.0,
            "other": 0.0,
        },
    },
    {
        "name": "Light Restraints Phase",
        "ensemble": "NPAT",
        "custom_template": "step6.5_equilibration.inp",  # Use stage 5 template
        "time_ns": 1.0,
        "timestep": 2.0,
        "temperature": 310.15,
        "pressure": 1.0,
        "surface_tension": 0.0,
        "constraints": {
            "protein_backbone": 1.0,
            "protein_sidechain": 0.5,
            "lipid_head": 0.5,
            "lipid_tail": 0.0,
            "water": 0.0,
            "ions": 0.0,
            "other": 0.0,
        },
    },
]

# Setup with custom template selection
manager = NAMDEquilibrationManager(work_dir)
```

```
result = manager.setup_namd_equilibration(
    system_files=system_files,
    stage_params_list=stages,
    output_name="equilibration_example_07",
    namd_executable="namd3",
)

print(f"\n✓ Setup complete with custom templates!")
print(f"  Stage 1: Using template step6.1")
print(f"  Stage 2: Using template step6.3 (skipped step6.2)")
print(f"  Stage 3: Using template step6.5 (skipped step6.4)")
```

**When to use custom templates:**

- **Skip intermediate stages**: Jump from step6.1 → step6.3 → step6.5
- **Repeat a stage**: Use step6.2 multiple times with different parameters
- **Mix templates**: Combine templates from different equilibration phases
- **Test protocols**: Experiment with different template combinations

---

#### Example 8: MDAnalysis Selections for Restraints

This example demonstrates the MDAnalysis-based selection system for precise atom counting and restraint generation, including auto-detection of non-standard residues (ligands, ions):

```python
#!/usr/bin/env python3
"""
Equilibration Example 08 - MDAnalysis Selections for Restraints

Demonstrates:
  1. Listing default MDAnalysis selections and their atom counts.
  2. Auto-detecting extra ligand residues in the PDB as named selections.
  3. Using custom MDAnalysis selection strings when generating restraint
     PDB files so that the B-factor column is filled via MDAnalysis
     rather than the built-in heuristic.

Requirements:
  - MDAnalysis >= 2.0
  - A system PDB file (the example uses the bundled popc_membrane data).
"""

from pathlib import Path
from gatewizard.tools.equilibration import NAMDEquilibrationManager

# ── 1. Point to the test system ──────────────────────────────────────────
work_dir = Path(__file__).parent / "popc_membrane"
system_pdb = work_dir / "bilayer_protein_protonated_prepared_lipid.pdb"

manager = NAMDEquilibrationManager(work_dir)

# ── 2. Inspect the default selections ────────────────────────────────────
print("=== Default MDAnalysis selections ===")
for name, sel in NAMDEquilibrationManager.DEFAULT_SELECTIONS.items():
    count = NAMDEquilibrationManager.count_selection_atoms(str(system_pdb), sel)
    print(f"  {name:25s}  →  {count:>7d} atoms")

# ── 3. Auto-detect ligands / non-standard residues ──────────────────────
print("\n=== Auto-detected selections (includes ligands) ===")
all_sels = NAMDEquilibrationManager.get_default_selections(str(system_pdb))
for name, sel in all_sels.items():
    if name.startswith("ligand_"):
        count = NAMDEquilibrationManager.count_selection_atoms(str(system_pdb), sel)
        print(f"  {name:25s}  →  {count:>7d} atoms  |  {sel}")

# ── 4. Count all selections at once ──────────────────────────────────────
print("\n=== Full atom count summary ===")
counts = NAMDEquilibrationManager.count_all_selections(str(system_pdb))
for name, count in counts.items():
    print(f"  {name:25s}  →  {count:>7d} atoms")

# ── 5. Generate a restraints PDB using MDAnalysis selections ─────────────
output_dir = work_dir / "equilibration_example_08" / "namd" / "restraints"
output_dir.mkdir(parents=True, exist_ok=True)

# Build {name: (mda_selection, force)} dict
selections_with_forces = {
    "protein_backbone": ("protein and backbone", 10.0),
    "protein_sidechain": ("protein and not backbone", 5.0),
    "lipid_head": (NAMDEquilibrationManager.DEFAULT_SELECTIONS["lipid_head"], 2.5),
    "lipid_tail": (NAMDEquilibrationManager.DEFAULT_SELECTIONS["lipid_tail"], 2.5),
    "water": (NAMDEquilibrationManager.DEFAULT_SELECTIONS["water"], 0.0),
    "ions": (NAMDEquilibrationManager.DEFAULT_SELECTIONS["ions"], 10.0),
}

# Add any auto-detected ligand with force 1.0
for name, sel in all_sels.items():
    if name.startswith("ligand_"):
        selections_with_forces[name] = (sel, 1.0)

output_file = output_dir / "step1_equilibration_restraints.pdb"
manager.generate_restraints_file_mda(
    system_pdb, selections_with_forces, output_file, stage_name="Equilibration 1"
)
print(f"\n✓ Restraints PDB written: {output_file}")

# ── 6. Use the high-level API (generate_restraints_file with selections) ─
constraints = {
    "protein_backbone": 10.0,
    "protein_sidechain": 5.0,
    "lipid_head": 2.5,
```

```
    "lipid_tail": 2.5,
    "water": 0.0,
    "ions": 10.0,
}
selections = {name: sel for name, (sel, _) in selections_with_forces.items()}

output_file2 = output_dir / "step1_via_high_level.pdb"
manager.generate_restraints_file(
    system_pdb,
    constraints,
    output_file2,
    stage_name="Eq1 (high-level)",
    selections=selections,
)
print(f"✓ High-level restraints PDB written: {output_file2}")

print("\nDone!")
```

**Output:**

```
  protein_backbone            →      204 atoms
  protein_sidechain           →      488 atoms
  lipid_head                  →     2904 atoms
  lipid_tail                  →    13310 atoms
  water                       →    14685 atoms
  ions                        →        0 atoms
  other                       →       21 atoms
  ligand_Cl-                  →       10 atoms  |  resname Cl-
  ligand_K+                   →       11 atoms  |  resname K+
```

---

#### Example 9: Custom Restraints — Three Levels of Customisation

This example demonstrates three progressive levels of customisation for NAMD restraints using `EquilibrationStage.replace()`:

```
"""
equilibration_example_09.py
===========================
NAMD custom restraints — three levels of customisation.

Prerequisites
-------------
- A prepared membrane-protein system in the ``popc_membrane/`` test folder
- MDAnalysis (``conda install -c conda-forge mdanalysis``) for levels 2 and 3

"""

from pathlib import Path
from gatewizard.tools.equilibration import NAMDEquilibrationManager, EquilibrationStage

work_dir = Path(__file__).parent / "popc_membrane"
system_files = {
    "prmtop": str(work_dir / "system.prmtop"),
    "inpcrd": str(work_dir / "system.inpcrd"),
    "pdb": str(work_dir / "system.pdb"),
    "bilayer_pdb": str(work_dir / "bilayer_protein_protonated_prepared_lipid.pdb"),
}

WORK_DIR = work_dir
manager = NAMDEquilibrationManager(working_dir=WORK_DIR)

# ---------------------------------------------------------------------------
# Level 1 — Override a single force constant key (no MDAnalysis needed)
# ---------------------------------------------------------------------------
# Turn off sidechain restraints entirely; keep default backbone + lipid forces.

print("=== Level 1: Override protein_sidechain to 0 ===")
stages_l1 = [
    s.replace(constraints={**s.constraints, "protein_sidechain": 0.0})
    for s in NAMDEquilibrationManager.get_default_stage_params()
]

result_l1 = manager.setup_namd_equilibration(
    system_files=system_files,
    stage_params_list=stages_l1,
    output_name="level1_no_sc",
)
print(f"Output: {result_l1['namd_dir']}")

# ---------------------------------------------------------------------------
# Level 2 — Override selections for standard categories (MDAnalysis)
# ---------------------------------------------------------------------------
# Useful when your PSF uses non-standard segment names or residue types.

print("\n=== Level 2: Custom selections for standard categories ===")
stages_l2 = NAMDEquilibrationManager.get_default_stage_params()

result_l2 = manager.setup_namd_equilibration(
    system_files=system_files,
    stage_params_list=stages_l2,
    output_name="level2_custom_sel",
    selections={
        "protein_backbone": "backbone",
        "protein_sidechain": "protein and not backbone",
        "lipid_head": "resname POPC and name P O11 O12 O13 O14",
        "lipid_tail": "resname POPC and not (name P O11 O12 O13 O14 N)",
    },
)
print(f"Output: {result_l2['namd_dir']}")

# ---------------------------------------------------------------------------
# Level 3 — Full MDAnalysis control with a custom atom category
# ---------------------------------------------------------------------------
# Restrain ions in the first 3 stages at 10 kcal/mol/Å², then release.
# Replace "ions" with "ligand_ABC" and "resname ABC" for a real ligand system.

print("\n=== Level 3: Custom ion restraints (demonstrates ligand-style) ===")
stages_l3 = NAMDEquilibrationManager.get_default_stage_params()

# Apply 10 kcal/mol/Å² to ions in stages 1–3; zero thereafter
```

```
stages_l3_dicts = []
for i, s in enumerate(stages_l3):
    ion_force = 10.0 if i < 3 else 0.0
    new_constraints = {**s.constraints, "custom_ions": ion_force}
    stages_l3_dicts.append(s.replace(constraints=new_constraints).to_dict())

result_l3 = manager.setup_namd_equilibration(
    system_files=system_files,
    stage_params_list=stages_l3_dicts,
    output_name="level3_custom_ions",
    selections={
        "custom_ions": "resname SOD CLA POT",  # MDAnalysis selection
    },
)
print(f"Output:           {result_l3['namd_dir']}")
```

**When to use each level:**

- **Level 1** (`replace(constraints=...)`): Change only force constants. No MDAnalysis needed — fastest approach.
- **Level 2** (`selections=` parameter): Override which atoms are selected for standard categories (backbone, sidechain, lipid head/tail) using your own MDAnalysis selection strings.
- **Level 3** (custom keys): Add entirely new atom categories (ligands, ions, cofactors) not covered by the default keys. Set the selection string and per-stage force schedule.

---

## Best Practices

#### Restraint Progression

**Recommended restraint schedule** (kcal/mol/Å$^2$):

| Component | Stage 1 | Stage 2 | Stage 3 | Stage 4 | Production |
|---|---|---|---|---|---|
| Protein backbone | 10.0 | 5.0 | 2.5 | 1.0 | 0.0 |
| Protein sidechain | 5.0 | 2.5 | 1.0 | 0.5 | 0.0 |
| Lipid heads | 5.0 | 2.5 | 1.0 | 0.5 | 0.0 |
| Lipid tails | 5.0 | 2.5 | 1.0 | 0.0 | 0.0 |
| Ions | 10.0 | 0.0 | 0.0 | 0.0 | 0.0 |
| Water | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 |

**Guidelines:**

- Start with **strong restraints** (10.0) on protein backbone and ions
- **Gradually reduce** restraints over 3-6 stages
- Always keep **water unrestrained** (0.0)
- Release **ions early** (after stage 1)
- Release **lipid tails** before heads
- Final stage should have **minimal or zero** restraints

#### Timestep Progression

| Stage | Timestep | Restraints | Notes |
|---|---|---|---|
| 1-2 | 1.0 fs | Strong (10.0-5.0) | Initial equilibration, constrained |
| 3-4 | 1.0-2.0 fs | Medium (2.5-1.0) | Transition to larger timestep |
| 5+ | 2.0 fs | Light (< 1.0) | Standard production timestep |

**Rules:**

- Use **1.0 fs** with strong restraints (> 5.0 kcal/mol/Å²)
- Switch to **2.0 fs** when restraints < 2.5 kcal/mol/Å²
- NAMD supports **2.0 fs** with SHAKE/SETTLE for bonds to hydrogen
- Never use > 2.0 fs for equilibration

#### Stage Lengths

**Minimum recommended times:**

| Ensemble | Stage Purpose | Min Time | Typical Time |
|---|---|---|---|
| NVT | Initial heating | 0.1 ns | 0.125-0.25 ns |
| NPT | Pressure equilibration | 0.1 ns | 0.125-0.5 ns |
| NPAT | Membrane equilibration | 0.5 ns | 0.5-2.0 ns |
| Production | Data collection | 10 ns | 50-500 ns |

**Total equilibration time:**

- **Minimum:** 1.0 ns (3-4 stages)
- **Standard:** 2.0 ns (6 stages, CHARMM-GUI protocol)
- **Thorough:** 3-5 ns (custom research protocols)

#### Ensemble Selection

**For membrane protein systems:**

**Start with NVT** (0.1-0.25 ns)

- Heat system from 0 K to target temperature
- Strong restraints on protein and ions
- Fixed box dimensions

**Switch to NPT** (0.1-0.5 ns)

- Equilibrate pressure (isotropic)
- Medium restraints
- Box can change size uniformly

**Finish with NPAT** (0.5-2.0 ns)

- Equilibrate membrane (anisotropic pressure)
- Light restraints
- Lateral box dimensions independent from Z-axis
- **Recommended for membrane systems**

#### Minimization

- Use **minimize_steps** only in **first stage**
- Typical value: **5000-10000 steps**
- Removes bad contacts/clashes
- Always minimize before MD

#### GPU Acceleration

```python
# Enable GPU acceleration (default)
stage = {
    'use_gpu': True,
    'gpu_id': 0,      # First GPU
    'num_gpus': 1,    # Single GPU
    'cpu_cores': 1    # Minimal CPU usage with GPU. Recommended > 4 depending on the system´s size.
}
```

---

### Output Structure

```
{output_name}/
└── namd/
    ├── system.prmtop              # Copied AMBER topology
    ├── system.inpcrd              # Copied AMBER coordinates
    ├── system.pdb                 # Copied system PDB (for structure/restraints)
    ├── bilayer_*_lipid.pdb        # Copied bilayer PDB (for CRYST1 box info only)
    ├── step1.conf                 # Stage 1 config
    ├── step2.conf                 # Stage 2 config
    ├── ...
    ├── run_equilibration.sh       # Executable run script
    ├── protocol_summary.json      # Protocol metadata
    ├── restraints/
        ├── step1_equilibration_restraints.pdb   # Stage 1 restraints
        ├── step2_equilibration_restraints.pdb   # Stage 2 restraints
    └── logs/
        ├── step1.log              # Stage 1 output
        ├── step2.log              # Stage 2 output
        └── ...
```

**Generated Files:**

| File | Purpose |
|---|---|
| `system.pdb` | System structure for NAMD simulations and restraints |
| `bilayer_*_lipid.pdb` | Original bilayer file (kept for reference, CRYST1 info read from this) |
| `stepN.conf` | NAMD configuration for stage N |
| `stepN_restraints.txt` | Harmonic restraints for stage N |
| `run_equilibration.sh` | Bash script to run all stages sequentially |
| `protocol_summary.json` | Protocol metadata (stages, parameters, files) |
| `logs/stepN.log` | NAMD output log for stage N |

**Important Notes:**

- `system.pdb` is the actual system file used for simulations and restraint generation
- `bilayer_*_lipid.pdb` is kept with its original name for reference and CRYST1 box dimensions
- Both files are copied to the output directory but serve different purposes

**Run Script Features:**

- Sequential execution of all stages
- Automatic dependency management (restart files)
- Progress tracking
- Error handling
- Timing information

---

### Internal Methods (Advanced)

#### Class Attribute: DEFAULT_SELECTIONS

Default MDAnalysis selection strings for the seven standard restraint categories. These selections are used when MDAnalysis-based restraint generation is enabled.

```
NAMDEquilibrationManager.DEFAULT_SELECTIONS = {
    'protein_backbone':  'protein and backbone',
    'protein_sidechain': 'protein and not backbone',
    'lipid_head':        '(resname POPC POPE POPS DPPC ...) and (name P O11 O12 ...)',
    'lipid_tail':        '(resname POPC POPE POPS DPPC ...) and not (name P O11 ...)',
    'water':             'resname TIP3 HOH WAT SOL TIP4 SPC T3P T4P',
    'ions':              'resname NA CL K CA MG ZN FE CU SOD CLA POT CAL MAG ZIN IRN COP',
    'other':             'not (protein or lipids or water or ions)',
}
```

Each value is a valid MDAnalysis selection string. Users can override any selection through the GUI gear button or the API `selections` parameter.

---

#### Static Method: count_selection_atoms()

Count atoms matching an MDAnalysis selection expression.

```
NAMDEquilibrationManager.count_selection_atoms(pdb_path: str, selection: str) -> int
```

**Parameters:**

| Parameter | Type | Description |
|---|---|---|
| `pdb_path` | `str` | Path to a PDB file |
| `selection` | `str` | MDAnalysis selection string |

**Returns:** `int` — number of matching atoms (0 if selection is invalid or MDAnalysis unavailable)

```
count = NAMDEquilibrationManager.count_selection_atoms(
    "system.pdb", "protein and backbone"
)
print(f"Backbone atoms: {count}")
```

---

#### Static Method: get_default_selections()

Build the default selection dict, auto-detecting extra ligands / non-standard residues.

```
NAMDEquilibrationManager.get_default_selections(pdb_path: str) -> Dict[str, str]
```

**Parameters:**

| Parameter | Type | Description |
|---|---|---|
| `pdb_path` | `str` | Path to a PDB file |

**Returns:** `Dict[str, str]` — `{category_name: mda_selection_string, ...}` including any auto-detected `ligand_<RESNAME>` entries.

The seven standard categories are always present. Any residue that falls into the *other* category is additionally split into individual `ligand_<RESNAME>` entries so users can assign per-ligand restraint forces.

```
sels = NAMDEquilibrationManager.get_default_selections("system.pdb")
for name, sel in sels.items():
    if name.startswith("ligand_"):
        print(f"  Detected: {name} → {sel}")
# Output:
#   Detected: ligand_Cl- → resname Cl-
#   Detected: ligand_K+  → resname K+
```

---

#### Static Method: count_all_selections()

Count atoms for every selection in a dictionary in one call.

```
NAMDEquilibrationManager.count_all_selections(
    pdb_path: str,
    selections: Optional[Dict[str, str]] = None
) -> Dict[str, int]
```

**Parameters:**

| Parameter | Type | Default | Description |
|---|---|---|---|
| `pdb_path` | `str` | Required | Path to a PDB file |
| `selections` | `Optional[Dict[str, str]]` | `None` | `{name: mda_selection_string}`. If `None`, uses `get_default_selections()` (with auto-detected ligands) |

**Returns:** `Dict[str, int]` — `{name: atom_count, ...}`

```
counts = NAMDEquilibrationManager.count_all_selections("system.pdb")
for name, n in counts.items():
    print(f"  {name:25s}  {n:>7d} atoms")
```

---

#### generate_restraints_file_mda

Generate a restraints PDB using MDAnalysis selections instead of the built-in heuristic. Each entry maps a category name to a `(mda_selection_string, force)` tuple. For every ATOM/HETATM line the **first matching** selection determines the B-factor. Atoms matching no selection get B-factor 0.0.

```
generate_restraints_file_mda(
    system_pdb: Path,
    selections_with_forces: Dict[str, Tuple[str, float]],
    output_file: Path,
    stage_name: str = ""
) -> None
```

**Parameters:**

| Parameter | Type | Description |
|---|---|---|
| `system_pdb` | `Path` | Path to the system PDB file |
| `selections_with_forces` | `Dict[str, Tuple[str, float]]` | `{name: (selection_string, force), ...}` |
| `output_file` | `Path` | Destination path for the restraints PDB |
| `stage_name` | `str` | Label for log messages (optional) |

```
selections_with_forces = {
    "protein_backbone":  ("protein and backbone", 10.0),
    "protein_sidechain": ("protein and not backbone", 5.0),
    "lipid_head":        (DEFAULT_SELS["lipid_head"], 2.5),
    "lipid_tail":        (DEFAULT_SELS["lipid_tail"], 2.5),
    "water":             (DEFAULT_SELS["water"], 0.0),
    "ions":              (DEFAULT_SELS["ions"], 10.0),
    "ligand_Cl-":        ("resname Cl-", 1.0),
}

manager.generate_restraints_file_mda(
    system_pdb, selections_with_forces, output_file,
    stage_name="Equilibration 1",
)
```

---

#### generate_restraints_file

Generates restraint PDB files with B-factors encoding restraint forces for each atom type. Now supports optional MDAnalysis selections for precise atom classification.

```
generate_restraints_file(
    system_pdb: Path,
    constraints: Dict[str, float],
    output_file: Path,
    stage_name: str = "",
    selections: Optional[Dict[str, str]] = None
) -> None
```

**Parameters:**

| Parameter | Type | Default | Description |
|---|---|---|---|
| `system_pdb` | `Path` | Required | Path to the **system.pdb** file (full system: protein + lipids + water + ions) |
| `constraints` | `Dict[str, float]` | Required | Dictionary of restraint forces (kcal/mol/Å²) for each atom type |
| `output_file` | `Path` | Required | Path for output restraints PDB file |
| `stage_name` | `str` | `""` | Stage name for logging (optional) |
| `selections` | `Optional[Dict[str, str]]` | `None` | `{constraint_name: mda_selection_string}`. When provided, MDAnalysis is used instead of the built-in heuristic |

**Behavior:**

When `selections` is `None` (legacy mode):

A. Classifies each atom by type using built-in residue/atom-name heuristics:

- `protein_backbone`: CA, C, N, O atoms in protein residues
- `protein_sidechain`: Heavy sidechain atoms in protein residues
- `lipid_head`: P, O atoms in lipid head groups
- `lipid_tail`: C atoms in lipid tails
- `water`: H2O molecules (TIP3, HOH, WAT, SOL)
- `ions`: Na+, Cl-, K+, Ca2+, Mg2+, etc.
- `other`: Ligands and other molecules

B. Assigns B-factor values based on constraints dictionary

C. Writes restraints PDB file with modified B-factors

When `selections` is provided (MDAnalysis mode):

A. Pairs each constraint name with its MDAnalysis selection string

B. Delegates to `generate_restraints_file_mda()` for precise MDAnalysis-based classification

C. Useful for custom selections, auto-detected ligands, or non-standard residues

```
# Legacy mode (heuristic-based)
manager.generate_restraints_file(system_pdb, constraints, output_file)

# MDAnalysis mode (selection-based)
selections = {
    "protein_backbone": "protein and backbone",
    "protein_sidechain": "protein and not backbone",
    "ligand_Cl-": "resname Cl-",
}
manager.generate_restraints_file(
    system_pdb, constraints, output_file,
    selections=selections,
)
```

**Example Output Log:**

```
INFO - Generated restraints file: step1_equilibration_restraints.pdb
INFO - Source PDB: system.pdb
INFO - Stage: Equilibration 1
INFO - Total atoms processed: 45678
INFO -   protein_backbone: 1234 atoms, force = 10.0 kcal/mol/Å²
INFO -   protein_sidechain: 2345 atoms, force = 5.0 kcal/mol/Å²
INFO -   lipid_head: 512 atoms, force = 5.0 kcal/mol/Å²
INFO -   lipid_tail: 3456 atoms, force = 5.0 kcal/mol/Å²
INFO -   ions: 89 atoms, force = 10.0 kcal/mol/Å²
```

---

#### generate_charmm_gui_config_file

Generates NAMD configuration files using CHARMM-GUI templates with GateWizard customizations.

```
generate_charmm_gui_config_file(
    stage_name: str,
    stage_params: Dict[str, Any],
    stage_index: int,
    system_files: Dict[str, str],
    scheme_type: str,
    previous_stage_name: Optional[str] = None,
    all_stage_settings: Optional[Dict[str, Dict[str, Any]]] = None,
    force_scheme_type: bool = False
) -> str
```

**Parameters:**

| Parameter | Type | Description |
|---|---|---|
| `stage_name` | `str` | Name of the equilibration stage |
| `stage_params` | `Dict[str, Any]` | Stage parameters dictionary |
| `stage_index` | `int` | Stage index (0-based) |
| `system_files` | `Dict[str, str]` | System file paths |
| `scheme_type` | `str` | CHARMM-GUI scheme type (NVT, NPT, NPAT, NPgT) |
| `previous_stage_name` | `Optional[str]` | Previous stage name for restart files |
| `all_stage_settings` | `Optional[Dict]` | All stages settings for context |
| `force_scheme_type` | `bool` | **[NEW]** If True, always use `scheme_type` for all stages (GUI mode) |

**Returns:** NAMD configuration file content as string

**Behavior:**

**API Mode (force_scheme_type=False, default):**

- Each stage can specify its own `ensemble` field
- Template folder is selected based on the stage's ensemble value
- Allows mixing NVT/NPT/NPAT templates across stages
- Warning logged when stage ensemble differs from protocol scheme_type

**GUI Mode (force_scheme_type=True):**

- ALL stages use the global `scheme_type` for template selection
- Individual stage `ensemble` values are ignored for template selection
- Ensures CHARMM-GUI protocol consistency when selected from GUI dropdown
- Example: If "NPT" scheme selected, all stages use NPT templates (02_NPT folder)

**Template Selection Logic:**

1. If `custom_template` specified → use that template file
2. If `force_scheme_type=True` → use `scheme_type` for folder selection
3. If `force_scheme_type=False` → use stage's `ensemble` for folder selection
4. Templates loaded from: `{scheme_folder}/{template_file}` under `equilibration/namd/`

**Scheme to Folder Mapping:**

- `NVT` → `01_NVT/`
- `NPT` → `02_NPT/`
- `NPAT` → `03_NPAT/`
- `NPgT` → `04_NPgT/`

**Example (API Mode):**

```
# Stage 1 uses NVT template, Stage 2 uses NPT template
config1 = manager.generate_charmm_gui_config_file(
    stage_name="Heating",
    stage_params={'ensemble': 'NVT', ...},
    stage_index=0,
    system_files=files,
    scheme_type='NVT',
    force_scheme_type=False  # API mode - use stage's ensemble
)

config2 = manager.generate_charmm_gui_config_file(
    stage_name="Pressure Eq",
    stage_params={'ensemble': 'NPT', ...},  # Different ensemble
    stage_index=1,
    system_files=files,
    scheme_type='NVT',  # Protocol default is NVT
    force_scheme_type=False  # Stage uses NPT template (warning logged)
)
```

**Example (GUI Mode):**

```
# User selected "NPT" scheme in GUI dropdown
# ALL stages use NPT templates regardless of individual ensemble values
config1 = manager.generate_charmm_gui_config_file(
    stage_name="Stage 1",
    stage_params={'ensemble': 'NVT', ...},  # Ignored for template selection
    stage_index=0,
    system_files=files,
    scheme_type='NPT',  # From GUI dropdown
    force_scheme_type=True  # GUI mode - enforce scheme_type
)
# Uses template from: 02_NPT/step6.1_equilibration.inp

config2 = manager.generate_charmm_gui_config_file(
    stage_name="Stage 2",
    stage_params={'ensemble': 'NVT', ...},  # Ignored for template selection
    stage_index=1,
    system_files=files,
    scheme_type='NPT',  # From GUI dropdown
    force_scheme_type=True  # GUI mode - enforce scheme_type
)
# Uses template from: 02_NPT/step6.2_equilibration.inp
```

**Rationale for force_scheme_type:**

The `force_scheme_type` parameter was added to support two different use cases:

1. **GUI Users:** Select a CHARMM-GUI scheme (NPT, NPAT, etc.) and expect all stages to use templates from that scheme folder for consistency with CHARMM-GUI protocols.
2. **API Users:** Have flexibility to mix different ensemble templates across stages for custom equilibration protocols (e.g., NVT → NPT → NPAT progression).

---

### Common Issues

#### Missing CRYST1 Record

**Error:** Box dimensions not found in PDB file

**Solution:** Ensure `bilayer_pdb` contains CRYST1 record:

```
head bilayer_protein_protonated_prepared_lipid.pdb
# Should show: CRYST1   70.335   70.833   85.067  90.00  90.00  90.00 P 1
```

**Correct file:**

- From packmol-memgen `--parametrize` step
- Named `bilayer_*_lipid.pdb`
- Contains proper PDB header with CRYST1
- Not an intermediate Packmol file

#### Restraint File Errors

**Error:** Cannot generate restraint file OR restraints only applied to protein

**Solution:** Ensure **system.pdb** is used (not protein.pdb):

- The restraints generation requires the **full system.pdb** file
- This file must contain: protein + lipids + water + ions
- Do NOT use protein.pdb (protein-only) for restraint generation
- Check that system.pdb exists in the working directory or input folder
- Verify atom names match AMBER topology

- Verify segnames/chains are present (PROA, MEMB, SOLV)

**Common mistake:** Using protein.pdb instead of system.pdb results in:

- No restraints applied to lipids (lipid_head, lipid_tail)
- No restraints applied to ions
- Membrane collapse during equilibration
- System instability

**Fix:** The GUI now automatically uses system.pdb from the output directory where files are copied. If you see restraints being generated with protein.pdb, check:

1. Is system.pdb present in the input folder?
2. Is system.pdb copied to the output directory?
3. Check the log for "Using PDB file for restraints generation: ..."

#### Timestep Too Large

**Error:** NAMD crashes with "Atoms moving too fast"

**Solution:** Reduce timestep or increase restraints:

- Use 1.0 fs with restraints > 5.0 kcal/mol/Å²
- Switch to 2.0 fs only when restraints < 2.5 kcal/mol/Å²
- Add minimization step if starting from poor geometry

#### Ensemble Switching

**Error:** System unstable when switching NPT → NPAT

**Solution:** Ensure pressure is equilibrated first:

- Run NPT for at least 0.25 ns before NPAT
- Check box dimensions are stable in NPT
- Use medium restraints during transition
- Don't switch ensemble and reduce restraints simultaneously

---

### Class: OpenMMEquilibrationManager

Manager for OpenMM equilibration simulations using CHARMM-GUI templates and the AMBER force field. Uses the same user-facing API as `NAMDEquilibrationManager`.

#### Constructor

```
OpenMMEquilibrationManager(working_dir: Path)
```

| Parameter | Type | Default | Description |
|---|---|---|---|
| `working_dir` | `Path` | Required | Working directory containing system files |

---

#### Example 1: Single NVT Stage

```python
from pathlib import Path
from gatewizard.tools.equilibration import OpenMMEquilibrationManager

# Point to folder with system files
work_dir = Path(__file__).parent / "popc_membrane"

# Define a single NVT equilibration stage
stages = [
    {
        "name": "Equilibration 1",
        "time_ns": 0.125,
        "ensemble": "NVT",
        "temperature": 310.15,
        "timestep": 1.0,
        "minimize_steps": 5000,
        "constraints": {
            "protein_backbone": 10.0,
            "protein_sidechain": 5.0,
            "lipid_head": 2.5,
            "lipid_tail": 0.0,
        },
    }
]

# Setup with automatic file detection (no system_files needed!)
# scheme_type is auto-detected from the 'ensemble' field in stages
manager = OpenMMEquilibrationManager(work_dir)
result = manager.setup_openmm_equilibration(
    stage_params_list=stages,
    output_name="openmm_example_01",
)

print(f"Setup complete: {result['openmm_dir']}")
# Run with: cd {result['openmm_dir']} && bash run_equilibration.sh
```

---

**Example 2: Full CHARMM-GUI Protocol (7 stages)**

```python
from pathlib import Path
from gatewizard.tools.equilibration import OpenMMEquilibrationManager

# Point to folder with system files
work_dir = Path(__file__).parent / "popc_membrane"

# Full 6-stage NPT membrane equilibration protocol (CHARMM-GUI style)
# Gradual relaxation of restraints following the standard protocol
stages = [
    {
        "name": "Equilibration 1 - NVT with strong restraints",
        "time_ns": 0.125,
        "ensemble": "NVT",
        "temperature": 303.15,
        "timestep": 1.0,
        "minimize_steps": 5000,
        "constraints": {
            "protein_backbone": 10.0,
            "protein_sidechain": 5.0,
            "lipid_head": 2.5,
            "lipid_tail": 0.0,
        },
    },
    {
        "name": "Equilibration 2 - NVT relaxing restraints",
        "time_ns": 0.125,
        "ensemble": "NVT",
        "temperature": 303.15,
        "timestep": 1.0,
        "constraints": {
            "protein_backbone": 5.0,
            "protein_sidechain": 2.5,
            "lipid_head": 1.0,
            "lipid_tail": 0.0,
        },
    },
    {
        "name": "Equilibration 3 - NPT with pressure coupling",
        "time_ns": 0.125,
        "ensemble": "NPT",
        "temperature": 303.15,
        "timestep": 1.0,
        "constraints": {
            "protein_backbone": 2.5,
            "protein_sidechain": 1.0,
            "lipid_head": 0.5,
            "lipid_tail": 0.0,
        },
    },
    {
        "name": "Equilibration 4 - NPT further relaxing",
        "time_ns": 0.25,
        "ensemble": "NPT",
        "temperature": 303.15,
        "timestep": 1.0,
        "constraints": {
            "protein_backbone": 1.0,
            "protein_sidechain": 0.5,
            "lipid_head": 0.0,
            "lipid_tail": 0.0,
        },
    },
    {
        "name": "Equilibration 5 - NPT backbone only",
        "time_ns": 0.25,
        "ensemble": "NPT",
        "temperature": 303.15,
        "timestep": 2.0,
        "constraints": {
            "protein_backbone": 0.5,
            "protein_sidechain": 0.0,
            "lipid_head": 0.0,
            "lipid_tail": 0.0,
        },
    },
```

```
    {
        "name": "Equilibration 6 - NPT light backbone restraints",
        "time_ns": 0.5,
        "ensemble": "NPT",
        "temperature": 303.15,
        "timestep": 2.0,
        "constraints": {
            "protein_backbone": 0.1,
            "protein_sidechain": 0.0,
            "lipid_head": 0.0,
            "lipid_tail": 0.0,
        },
    },
    {
        "name": "Production - NPT unrestrained",
        "time_ns": 50.0,
        "ensemble": "NPT",
        "temperature": 303.15,
        "timestep": 2.0,
        "constraints": {
            "protein_backbone": 0.0,
            "protein_sidechain": 0.0,
            "lipid_head": 0.0,
            "lipid_tail": 0.0,
        },
    },
]

# Setup with automatic file detection
# Mini + Eq1-6 always load equilibration/openmm/eq/; production uses
# equilibration/openmm/production/{NVT|NPT|NPAT|NPgT}/ from scheme_type.
# Mixed ensembles (NVT heat/scaffold, NPgT packing, NPT production) are
# handled automatically - pass scheme_type="NPT" for the production folder.
manager = OpenMMEquilibrationManager(work_dir)
result = manager.setup_openmm_equilibration(
    stage_params_list=stages,
    output_name="openmm_example_02",
    scheme_type="NPT",
)

print(f"Setup complete: {result['openmm_dir']}")
print(f"Config files: {len(result['config_files'])}")
print(f"Run script: {result['run_script'].name}")
# Run with: cd {result['openmm_dir']} && bash run_equilibration.sh
```

---

**Example 3: Custom Ligand Restraints**

```
"""
openmm_example_03.py
====================
OpenMM equilibration with custom positional restraints for a ligand (residue
name ABC) plus standard protein/lipid restraints.

Prerequisites
-------------
- AMBER-format system: system.prmtop + system.inpcrd + system.pdb
- MDAnalysis (``conda install -c conda-forge mdanalysis``)

The example demonstrates:
  1. Default 6-stage CHARMM-GUI-style protocol
  2. Adding a ligand_ABC restraint in the first 3 stages
  3. Inspecting the generated restraint files
  4. Per-stage force-constant taper for backbone
"""

from pathlib import Path
from gatewizard.tools.equilibration import (
    OpenMMEquilibrationManager,
    EquilibrationStage,
)

WORK_DIR = Path("openmm_ligand_restraints")
WORK_DIR.mkdir(exist_ok=True)

manager = OpenMMEquilibrationManager(working_dir=WORK_DIR)

system_files = {
    "prmtop": "system.prmtop",
    "inpcrd": "system.inpcrd",
    "pdb": "system.pdb",
}

# ---------------------------------------------------------------------------
# Example 1 — Standard protein + lipid restraints (auto-detected)
# ---------------------------------------------------------------------------
print("=== Example 1: Standard protein/lipid restraints ===")
stages = OpenMMEquilibrationManager.get_default_stage_params()

result = manager.setup_openmm_equilibration(
    system_files=system_files,
    stage_params_list=stages,
    output_name="standard_restraints",
)
print(f"OpenMM dir:      {result['openmm_dir']}")
print(f"Restraint files: {result['restraint_files']}")
# → restraint_files["prot_pos"]  = Path(".../restraints/prot_pos.txt")
# → restraint_files["lipid_pos"] = Path(".../restraints/lipid_pos.txt")  (if lipid forces > 0)
# → restraint_files["custom_pos"] = None

# ---------------------------------------------------------------------------
# Example 2 — Add ligand ABC restraints in stages 1-3
# ---------------------------------------------------------------------------
print("\n=== Example 2: Ligand ABC restraints in stages 1-3 ===")
raw_stages = OpenMMEquilibrationManager.get_default_stage_params()
stage_objs = [
    s if isinstance(s, EquilibrationStage) else EquilibrationStage(**s)
    for s in raw_stages
]

# Apply 5 kcal/mol/Å² to ligand ABC in the first 3 stages; zero thereafter
stage_dicts = []
for i, s in enumerate(stage_objs):
    ligand_force = 5.0 if i < 3 else 0.0
    new_constraints = {**s.constraints, "ligand_ABC": ligand_force}
    stage_dicts.append(s.replace(constraints=new_constraints).to_dict())

result2 = manager.setup_openmm_equilibration(
    system_files=system_files,
    stage_params_list=stage_dicts,
    output_name="ligand_ABC_restraints",
    selections={
        "ligand_ABC": "resname ABC",  # MDAnalysis selection string
```

```
    },
)
print(f"OpenMM dir:      {result2['openmm_dir']}")
print(f"Restraint files: {result2['restraint_files']}")
# → restraint_files["custom_pos"] = Path(".../restraints/custom_pos.txt")
#   custom_pos.txt force = 5.0 kcal/mol/Å² × 418.4 = 2092.0 kJ/mol/nm²

# Verify rest = yes in stage 1 config (ligand_ABC = 5.0 > 0)
stage1_cfg = result2["config_files"][0].read_text()
assert "rest        = yes" in stage1_cfg, "Stage 1 should have rest = yes"

# Verify rest = no in stage 4 config (ligand_ABC = 0, no other active forces assumed)
# Note: if standard protein forces are also 0 in stage 4, rest = no there.
print("Assertions passed.")

# ------------------------------------------------------------------------
# Example 3 — Custom backbone taper + ligand restraints
# ------------------------------------------------------------------------
print("\n=== Example 3: Custom backbone taper + ligand ABC ===")
raw_stages = OpenMMEquilibrationManager.get_default_stage_params()

# Apply a linear backbone taper and add the ligand
bb_schedule = [10.0, 5.0, 2.5, 1.0, 0.5, 0.0]
sc_schedule = [5.0, 2.5, 1.0, 0.5, 0.0, 0.0]
lig_schedule = [5.0, 5.0, 5.0, 0.0, 0.0, 0.0]

stage_dicts3 = []
for i, item in enumerate(raw_stages):
    stage = (
        item
        if isinstance(item, EquilibrationStage)
        else EquilibrationStage(**item)
    )
    constraints = {
        **stage.constraints,
        "protein_backbone": bb_schedule[i],
        "protein_sidechain": sc_schedule[i],
        "ligand_ABC": lig_schedule[i],
    }
    stage_dicts3.append(stage.replace(constraints=constraints).to_dict())

result3 = manager.setup_openmm_equilibration(
    system_files=system_files,
    stage_params_list=stage_dicts3,
    output_name="taper_plus_ligand",
    selections={"ligand_ABC": "resname ABC"},
)
print(f"OpenMM dir:      {result3['openmm_dir']}")
print(f"Config files:    {[p.name for p in result3['config_files']]}")
print(f"Restraint files: {result3['restraint_files']}")
```

---

#### Ensemble Types (OpenMM)

| Ensemble | Folder | Pressure Coupling | Use Case |
|---|---|---|---|
| **NVT** | `01_NVT` | None (`pcouple = no`) | Initial heating / fixed box |
| **NPT** | `02_NPT` | `MonteCarloMembraneBarostat` | General membrane equilibration |
| **NPAT** | `03_NPAT` | `MonteCarloAnisotropicBarostat` (Z-free) | Anisotropic pressure control |
| **NPgT** | `04_NPgT` | `MonteCarloMembraneBarostat` + surface tension | Membrane with explicit surface tension |

---

#### Important Notes

##### Force constant schedule

The CHARMM-GUI template files contain a built-in force constant schedule (`fc_bb`, `fc_sc`, `fc_lpos`) that decreases progressively across the six equilibration steps (4000 → 2000 → 1000 → 500 → 200 → 50 kJ/mol/nm²). These values are **hardcoded in the templates** and are not controlled by the user's `constraints` dict.

The `constraints` dict controls **which atom types are included in the restraint index files**:

- A non-zero value → that atom type is written to `prot_pos.txt` or `lipid_pos.txt`
- A zero value → that atom type is excluded from the restraint files

#### Units

| Parameter | NAMD | OpenMM template | User API |
|---|---|---|---|
| Force constants | kcal/mol/Å² (B-factor) | kJ/mol/nm² (hardcoded in template) | kcal/mol/Å² (controls inclusion only) |
| Timestep | fs | ps (converted internally) | fs |
| Temperature | K | K | K |

#### Lipid dihedral restraints

`fc_ldih = 0` is hardcoded in all templates. OpenMM dihedral restraints require equilibrium dihedral angles from the structure, which GateWizard does not compute. Only positional restraints (`prot_pos.txt`, `lipid_pos.txt`) are used.

---

#### setup_openmm_equilibration

```
setup_openmm_equilibration(
    system_files: Optional[Dict[str, str]] = None,
    stage_params_list: Optional[List[Dict[str, Any]]] = None,
    output_name: str = "equilibration",
    scheme_type: Optional[str] = None,
) -> Dict[str, Any]
```

**Parameters:**

| Parameter | Type | Default | Description |
|---|---|---|---|
| `system_files` | `Optional[Dict[str, str]]` | `None` | System file paths. If `None`, auto-detects in `working_dir` |
| `stage_params_list` | `List[Dict[str, Any]]` | Required | List of stage parameter dicts |
| `output_name` | `str` | `"equilibration"` | Name for output folder |
| `scheme_type` | `Optional[str]` | `None` | Equilibration scheme (NVT, NPT, NPAT, NPgT). Auto-detected from first stage if `None` |

**Returns:** `Dict[str, Any]` with keys:

- `openmm_dir` (`Path`): Output directory path
- `config_files` (`List[Path]`): Generated `.inp` files
- `run_script` (`Path`): Bash run script path

**What it does:**

1. Auto-detects system files if not provided
2. Creates `{working_dir}/{output_name}/` output directory
3. Copies system files (`prmtop`, `inpcrd`, `pdb`) into the output directory
4. Copies the 7 CHARMM-GUI Python helper scripts into the output directory
5. Generates `.inp` config files for each stage from the appropriate template
6. Generates restraint index files (`prot_pos.txt`, `lipid_pos.txt`, `dihe.txt`) in `restraints/`

7. Generates `run_equilibration.sh` bash script

---

#### Stage Parameters (OpenMM)

Each entry in `stage_params_list` accepts:

| Parameter | Type | Required | Default | Description |
|---|---|---|---|---|
| `name` | `str` | Yes | — | Stage label (for logging) |
| `time_ns` | `float` | Yes | — | Simulation time in nanoseconds |
| `ensemble` | `str` | Yes | — | NVT, NPT, NPAT, or NPgT |
| `temperature` | `float` | Yes | — | Temperature in Kelvin |
| `timestep` | `float` | No | `2.0` | Timestep in femtoseconds |
| `minimize_steps` | `int` | No | `0` | Minimization steps (first stage only) |
| `dcd_freq` | `int` | No | `5000` | DCD output frequency (steps) |
| `constraints` | `Dict[str, float]` | No | all zero | Restraint inclusion dict (kcal/mol/Å²) |

**Constraints keys:** `protein_backbone`, `protein_sidechain`, `lipid_head`, `lipid_tail`

---

#### Output Structure (OpenMM)

```
{output_name}/
├── system.prmtop              # Copied AMBER topology
├── system.inpcrd              # Copied AMBER coordinates
├── system.pdb                 # Copied system PDB
├── openmm_run.py              # CHARMM-GUI runner script
├── omm_readinputs.py          # Config parser
├── omm_readparams.py          # File I/O
├── omm_restraints.py          # Restraint forces
├── omm_barostat.py            # Pressure coupling
├── omm_vfswitch.py            # vdW force switching
├── omm_rewrap.py              # Coordinate rewrapping
├── step6.1_equilibration.inp  # Stage 1 config
├── step6.2_equilibration.inp  # Stage 2 config
├── ...
├── step7_production.inp       # Production config
├── run_equilibration.sh       # Bash run script
└── restraints/
    ├── prot_pos.txt           # Protein atom indices (BB / SC labels)
    ├── lipid_pos.txt          # Lipid head-group atom indices
    └── dihe.txt               # Dihedral restraints (always empty)
```

---

#### How to Run (OpenMM)

```
cd {output_name}/
bash run_equilibration.sh

# Override Python interpreter:
PYTHON=python3 bash run_equilibration.sh
```

The run script chains all stages sequentially. Each stage uses `-irst <prev>.rst` to restart from the previous stage's checkpoint, except the first stage which reads from the AMBER coordinate file.

---

### EquilibrationStage Dataclass

`EquilibrationStage` is a typed, immutable-friendly dataclass that wraps the stage-parameter dictionary used by both engines. It is the recommended way to build and manipulate individual stages.

#### Fields

| Field | Type | Default | Description |
|---|---|---|---|
| `name` | `str` | `"Stage"` | Human-readable label |
| `ensemble` | `str` | `"NPT"` | Thermodynamic ensemble (`NVT`, `NPT`, `NPAT`, `NPgT`) |
| `time_ns` | `float` | `0.125` | Simulation time per stage (ns) |
| `temperature` | `float` | `310.15` | Temperature (K) |
| `timestep` | `float` | `2.0` | Integration timestep (fs) |
| `pressure` | `float` | `1.0` | Target pressure (bar) |
| `minimize_steps` | `int` | `0` | Energy minimization steps (first stage only) |
| `dcd_freq` | `int` | `5000` | Trajectory save frequency (steps) |
| `constraints` | `Dict[str, float]` | `{}` | Force constants (kcal/mol/Å²) per category |

#### Methods

```
# Replace specific fields → new object (original unchanged)
new_stage = stage.replace(time_ns=0.5, temperature=298.15)

# Convert to plain dict for the manager API
d = stage.to_dict()
```

#### Example

```
from gatewizard.tools.equilibration import EquilibrationStage

s = EquilibrationStage(
    name="NVT Heating",
    ensemble="NVT",
    time_ns=0.25,
    temperature=310.15,
    timestep=2.0,
    minimize_steps=5000,
    constraints={"protein_backbone": 10.0, "protein_sidechain": 5.0},
)
# Create a NPT variant at different temperature
s_npt = s.replace(ensemble="NPT", name="NPT Stage 1", minimize_steps=0)
```

---

### Default Equilibration Protocol

Both engines expose `get_default_stage_params()` to generate a standard CHARMM-GUI-compatible six-stage schedule with gradually decreasing positional restraints.

#### NAMD — zero-config defaults

```python
from pathlib import Path
from gatewizard.tools.equilibration import NAMDEquilibrationManager

manager = NAMDEquilibrationManager(working_dir=Path("/tmp/work"))

result = manager.setup_namd_equilibration(
    system_files={
        "psf":  "system.psf",
        "pdb":  "system.pdb",
        "prm":  "par_all36m_prot.prm",
    },
    # stage_params_list omitted → uses 6-stage default
)
print(result["namd_dir"])   # Path to output folder
```

#### OpenMM — zero-config defaults

```python
from pathlib import Path
from gatewizard.tools.equilibration import OpenMMEquilibrationManager

manager = OpenMMEquilibrationManager(working_dir=Path("/tmp/work"))

result = manager.setup_openmm_equilibration(
    system_files={
        "prmtop": "system.prmtop",
        "inpcrd": "system.inpcrd",
        "pdb":    "system.pdb",
    },
    # stage_params_list omitted → uses 6-stage CHARMM-GUI default
)
print(result["openmm_dir"])
```

#### Retrieving the default stages

```python
# NAMD
stages = NAMDEquilibrationManager.get_default_stage_params()

# OpenMM  (NPT, NPAT, or NPgT scheme)
stages = OpenMMEquilibrationManager.get_default_stage_params(scheme_type="NPT")
```

---

### Modifying Stage Parameters

#### Direct attribute assignment

```python
stages = OpenMMEquilibrationManager.get_default_stage_params()

# Run all stages at 298 K instead of 310.15 K
for s in stages:
    s["temperature"] = 298.15

# Double the simulation time for stage 6
stages[5]["time_ns"] = 0.5
```

#### Using `EquilibrationStage.replace()` for immutable updates

```python
from gatewizard.tools.equilibration import EquilibrationStage

raw = OpenMMEquilibrationManager.get_default_stage_params()
stages = [EquilibrationStage(**s) for s in raw]

# Build a taper schedule: halve backbone restraint each stage
bb_forces = [10.0, 5.0, 2.5, 1.0, 0.5, 0.0]
stages = [
    s.replace(constraints={**s.constraints, "protein_backbone": bb_forces[i]})
    for i, s in enumerate(stages)
]

result = manager.setup_openmm_equilibration(
    system_files=system_files,
    stage_params_list=[s.to_dict() for s in stages],
)
```

---

### Custom Restraints — NAMD

Three levels of customisation are available, from simplest to most flexible.

#### Level 1 — Override a single force constant key

Use this when you only need to change one category (e.g. turn off sidechain restraints entirely):

```python
stages = NAMDEquilibrationManager.get_default_stage_params()
for s in stages:
    s["constraints"]["protein_sidechain"] = 0.0  # no SC restraints

result = manager.setup_namd_equilibration(
    system_files=system_files,
    stage_params_list=stages,
)
```

#### Level 2 — Auto-detected MDAnalysis selections

GateWizard can auto-detect standard selections from your PDB. Pass a custom `selections` dict to override specific categories:

```python
result = manager.setup_namd_equilibration(
    system_files=system_files,
    stage_params_list=stages,
    selections={
        "protein_backbone": "backbone and segid PROA PROB",
        "protein_sidechain": "protein and not backbone and segid PROA PROB",
        "lipid_head":        "resname POPC and name P O11 O12 O13 O14",
    },
)
```

Supported keys: `protein_backbone`, `protein_sidechain`, `lipid_head`, `lipid_tail`, `water`, `ions`, `other`.

#### Level 3 — Full MDAnalysis control with ligands

Add a custom `ligand_<RESNAME>` or `other` key to the `constraints` dict, and provide a matching MDAnalysis selection string:

```
stages = NAMDEquilibrationManager.get_default_stage_params()
for s in stages[:3]:                          # restrain ligand only in stages 1-3
    s["constraints"]["ligand_ABC"] = 5.0      # kcal/mol/Å²

result = manager.setup_namd_equilibration(
    system_files=system_files,
    stage_params_list=stages,
    selections={
        "ligand_ABC": "resname ABC",          # MDAnalysis selection string
    },
)
# Generates: restraints/ligand_ABC_restraints.txt (PDB-format NAMD restraint file)
```

> **Requires MDAnalysis.** Install with `conda install -c conda-forge mdanalysis`.

---

### Custom Restraints — OpenMM

OpenMM restraints use atom-index files consumed by `omm_restraints.py` at runtime.

#### Standard protein / lipid restraints

These are enabled automatically when the `constraints` dict contains non-zero forces for the standard keys:

```
stages = OpenMMEquilibrationManager.get_default_stage_params()
# stages already contain protein_backbone, protein_sidechain, lipid_head, etc.

result = manager.setup_openmm_equilibration(
    system_files=system_files,
    stage_params_list=stages,
)
# Generates:
#   restraints/prot_pos.txt   (BB / SC labels)
#   restraints/lipid_pos.txt  (lipid atom indices)
```

GateWizard writes `rest = yes` automatically for any stage where at least one force constant is > 0, and injects the correct `fc_bb`, `fc_sc`, and `fc_lpos` values (in kJ/mol/nm$^2$) into the CHARMM-GUI template.

#### Ligand / cofactor restraints (`custom_pos.txt`)

Add a `ligand_<RESNAME>` (or any non-standard) key to the `constraints` dict and supply a matching MDAnalysis selection:

```
stages = OpenMMEquilibrationManager.get_default_stage_params()

# Restrain ligand ABC at 5 kcal/mol/Å² in the first three stages
for s in stages[:3]:
    s["constraints"]["ligand_ABC"] = 5.0

result = manager.setup_openmm_equilibration(
    system_files=system_files,
    stage_params_list=stages,
    selections={
        "ligand_ABC": "resname ABC",   # MDAnalysis selection string
    },
)
# Generates:
#   restraints/custom_pos.txt  (per-atom force constants in kJ/mol/nm²)
```

The `custom_pos.txt` file is read at runtime by the GateWizard extension block inside `omm_restraints.py` via a `CustomExternalForce` with a periodic harmonic potential (`k*periodicdistance(x,y,z,x0,y0,z0)²`).

#### How force constants are determined

| Key | Restraint file | Force constant source |
|---|---|---|
| `protein_backbone` | `prot_pos.txt` (BB) | Per-stage `fc_bb` in the `.inp` config |
| `protein_sidechain` | `prot_pos.txt` (SC) | Per-stage `fc_sc` in the `.inp` config |
| `lipid_head` / `lipid_tail` | `lipid_pos.txt` | Per-stage `fc_lpos` in the `.inp` config |
| Any other key | `custom_pos.txt` | Fixed at **maximum across all stages**; baked into the file |

> **Note — `custom_pos.txt` limitation**: force constants in `custom_pos.txt` are fixed at the maximum value found across all stages. Per-stage scaling of custom forces is not supported in the current implementation. If you need different forces per stage, generate the files manually and place them in the `restraints/` directory before running.

#### Using pre-computed atom indices (PSF + PDB)

MDAnalysis can load topology files for more accurate selections. If you have already computed the 0-based atom indices externally, pass them as a literal index selection:

```
selections = {
    "ligand_ABC": "index 1234 1235 1236 1237",  # 0-based, space-separated
}
```

Or generate the `restraints/` files manually and simply omit `selections` from the call.

> **Requires MDAnalysis** for custom categories. Install with `conda install -c conda-forge mdanalysis`.

#### Return value

`setup_openmm_equilibration()` now returns a `"restraint_files"` key:

```
result = manager.setup_openmm_equilibration(...)
print(result["restraint_files"])
# {"prot_pos": PosixPath("...restraints/prot_pos.txt"),
#  "lipid_pos": None,
#  "custom_pos": PosixPath("...restraints/custom_pos.txt")}
```

---

### Class: GROMACSEquilibrationManager

Manager for GROMACS equilibration simulations using CHARMM-GUI-style MDP templates. Mirrors the NAMD and OpenMM manager APIs.

- Accepts GROMACS-native (`.gro` + `topol.top`) **or** AMBER (`.prmtop` + `.inpcrd`) files. AMBER files are automatically converted using **ParmEd**.
- Position restraints are handled via MDP `define` macros (`POSRES_FC_BB`, `POSRES_FC_SC`, `POSRES_FC_LIPID`) — identical to the CHARMM-GUI scheme.
- Force constants are specified in **kcal/mol/Å²** and converted to kJ/mol/nm² internally.
- COM restraints via GROMACS Colvars (`distanceX/Y/Z` + optional `orientation`).

#### Import

```
from gatewizard.tools.equilibration import GROMACSEquilibrationManager
```

#### Constructor

```
GROMACSEquilibrationManager(working_dir: Path, gmx_executable: str = "gmx")
```

**Parameters:**

| Parameter | Type | Default | Description |
|---|---|---|---|
| `working_dir` | `Path` | Required | Directory containing system files |
| `gmx_executable` | `str` | `"gmx"` | GROMACS executable name or full path |

---

#### get_default_stage_params

```
GROMACSEquilibrationManager.get_default_stage_params(
    scheme_type: str = "NPT",
    temperature: float = 310.15,
    include_production: bool = False,
) -> List[EquilibrationStage]
```

Returns 7 (or 8) CHARMM-GUI-style equilibration stages with gradually decreasing positional restraints.

| Parameter | Type | Default | Description |
|---|---|---|---|
| `scheme_type` | `str` | `"NPT"` | Ensemble: NVT, NPT, NPAT, or NPgT |
| `temperature` | `float` | `310.15` | Temperature in Kelvin |
| `include_production` | `bool` | `False` | Append a 50 ns unrestrained production stage |

**Returns:** `List[EquilibrationStage]`

**Example:**

```
stages = GROMACSEquilibrationManager.get_default_stage_params("NPT", temperature=300.0)
stages[-1].time_ns = 5.0   # extend last equilibration stage
```

---

#### setup_gromacs_equilibration

```
manager.setup_gromacs_equilibration(
    system_files: Optional[Dict[str, str]] = None,
    stage_params_list: Optional[List] = None,
    output_name: str = "equilibration",
    scheme_type: Optional[str] = None,
    selections: Optional[Dict[str, str]] = None,
    gmx_executable: str = "gmx",
    add_com_restraint: bool = False,
    com_restraint_k: float = 10.0,
    add_rotation_restraint: bool = False,
    rotation_restraint_k: float = 2000.0,
) -> Dict[str, Any]
```

**Parameters:**

| Parameter | Type | Default | Description |
|---|---|---|---|
| `system_files` | `dict` | Auto-detect | Keys: `gro` + `top` (GROMACS) **or** `prmtop` + `inpcrd` + `pdb` + `bilayer_pdb` (AMBER) |
| `stage_params_list` | `list` | 7-stage default | List of `EquilibrationStage` objects or dicts |
| `output_name` | `str` | `"equilibration"` | Subdirectory name under `working_dir` |
| `scheme_type` | `str` | From first stage | Ensemble: NVT, NPT, NPAT, NPgT |
| `gmx_executable` | `str` | `"gmx"` | GROMACS executable |
| `add_com_restraint` | `bool` | `False` | Generate Colvars COM restraint file |
| `com_restraint_k` | `float` | `10.0` | COM force constant in kcal/mol/Å² |

| Parameter | Type | Default | Description |
|---|---|---|---|
| `add_rotation_restraint` | `bool` | `False` | Add orientation restraint in Colvars |
| `rotation_restraint_k` | `float` | `2000.0` | Rotation force constant in kcal/mol/Å² |

**Returns:**

```
{
    "gromacs_dir":   Path,          # output directory
    "mdp_files":     List[Path],    # generated MDP files
    "run_script":    Path,          # run_equilibration.sh
    "system_files":  dict,          # detected input files
    "posres_files":  dict,          # backbone/sidechain/lipid .itp files
    "gro":           Path,
    "top":           Path,
    "ndx":           Path,
    "com_colvars":   Path | None,   # com_restraint.dat (if requested)
}
```

**Example — minimal (AMBER files, auto-detected):**

```
from pathlib import Path
from gatewizard.tools.equilibration import GROMACSEquilibrationManager

manager = GROMACSEquilibrationManager(Path("popc_membrane"))
stages = GROMACSEquilibrationManager.get_default_stage_params("NPT",
                                                              include_production=True)
stages[-1].time_ns = 100.0     # 100 ns production
result = manager.setup_gromacs_equilibration(stage_params_list=stages)
# cd result["gromacs_dir"] && bash run_equilibration.sh
```

**Example — with COM restraint:**

```
result = manager.setup_gromacs_equilibration(
    stage_params_list=stages,
    add_com_restraint=True,
    com_restraint_k=5.0,           # kcal/mol/Å²
    add_rotation_restraint=True,
    rotation_restraint_k=500.0,
)
# Activate Colvars in the desired MDP files:
#   colvars-active         = yes
#   colvars-configfile     = com_restraint.dat
```

---

#### generate_mdp_file

```
manager.generate_mdp_file(
    stage_name: str,
    stage_params: Dict[str, Any],
    stage_index: int,      # 0=minimization, 1-6=equilibration, 7=production
    scheme_type: str,
) -> str
```

Returns the contents of an MDP file for a single stage, generated by substituting runtime parameters (temperature, timestep, nsteps, force constants) into the appropriate template from `equilibration/gromacs/{ensemble}/`.

---

#### convert_from_amber

```
manager.convert_from_amber(
    prmtop: Path,
    inpcrd: Path,
    output_dir: Path,
    bilayer_pdb: Optional[Path] = None,
) -> Dict[str, Path]   # {"gro": ..., "top": ...}
```

Converts AMBER topology + coordinates to GROMACS format using **ParmEd**. When the prmtop has no box information (common in membrane systems), the box is read from the CRYST1 record of `bilayer_pdb`.

**Requires:** `conda install -c conda-forge parmed`

---

#### generate_com_colvars_config

```
manager.generate_com_colvars_config(
    pdb_path: Path,
    output_file: Path,
    com_restraint_k: float = 10.0,
    add_rotation_restraint: bool = False,
    rotation_restraint_k: float = 2000.0,
    selection: str = "name CA",
) -> Optional[Path]
```

Generates a Colvars configuration file that restrains the geometric centre of the selected atoms to its initial position. Uses `distanceX/Y/Z` CVs for translation and an `orientation` CV for rotation.

When COM restraints are enabled through `setup_namd_equilibration()` or `setup_gromacs_equilibration()`, GateWizard writes the corresponding colvars activation block directly into the generated input files:

- NAMD: `colvars on` + `colvarsConfig com_restraint.col`
- GROMACS: `colvars-active = yes` + `colvars-configfile = com_restraint.dat`

**Requires:** `conda install -c conda-forge mdanalysis`

---

### Class: AmberEquilibrationManager

Manager for Amber (`pmemd` / `sander`) equilibration using GateWizard mdin templates under `equilibration/amber/{01_NVT,02_NPT,03_NPAT,04_NPgT}/`.

Amber `01_NVT` is **true NVT** (constant volume) for all stages through production: no `barostat` / `ntp` / surface-tension controls. Early heating in the other ensembles remains NVT-like; from `step3` onward those packs enable their ensemble-specific pressure settings (`02_NPT` semi-isotropic, `03_NPAT` anisotropic Z-scaling, `04_NPgT` surface tension). Positional restraints use Amber `ntr=1` with a GROUP block generated from MDAnalysis selections (no dihedral / `nmropt` / `DISANG`). Stages include a separate minimization (`step0_minimization.mdin`) plus six equilibration stages and optional production, matching the GROMACS stage layout. Timestep ladder: **1.0 fs through equilibration 4**, then **2.0 fs**.

Generated inputs are stamped with the GateWizard API version, local generation time (with timezone), and the shared equilibration **templates version** so runs remain traceable across releases.

```python
from pathlib import Path
from gatewizard.tools.equilibration import AmberEquilibrationManager

manager = AmberEquilibrationManager(Path("popc_membrane"), amber_executable="pmemd.cuda")
stages = AmberEquilibrationManager.get_default_stage_params("NPT", include_production=True)
result = manager.setup_amber_equilibration(
    stage_params_list=stages,
    amber_executable="pmemd.cuda",
)
# cd result["amber_dir"] && bash run_equilibration.sh
# Resume unfinished stages: RESUME=1 bash run_equilibration.sh
```

Executable preference when several are found: `pmemd.cuda` → `pmemd` → `pmemd.MPI` / `pmemd.cuda.MPI` → `sander`. Minimization prefers a CPU binary when CUDA was selected (CHARMM-GUI caution). GPU device selection uses `CUDA_VISIBLE_DEVICES`.

#### Output Structure (Amber)

```
equilibration/
├── system.prmtop / system.inpcrd / system.pdb
├── step0_minimization.mdin … step6_equilibration.mdin
├── step7_production.mdin
└── run_equilibration.sh
```

Stage outputs: `*.mdout`, `*.rst7`, `*.nc` (dynamics), `*.mdinfo`.

---

### COM Restraints (all engines)

GateWizard supports **centre-of-mass / centre-of-geometry restraints** for NAMD, OpenMM, and GROMACS. Unlike per-atom positional restraints, COM restraints act on the *centroid* of the selected group — they prevent rigid-body translation (and optionally rotation) of the protein without introducing bias on individual atoms. Amber equilibration currently uses GROUP positional restraints only (no COM colvars).

#### Motivation

In membrane simulations, proteins can undergo slow lateral drift or rotation within the membrane plane over long equilibration runs. COM restraints counteract this while still allowing internal conformational flexibility.

#### API summary

| Engine | Parameter | Effect |
|---|---|---|
| **NAMD** | `add_com_restraint=True` in `setup_namd_equilibration()` | Writes `com_restraint.col` and injects `colvars on` / `colvarsConfig ...` into the generated `.conf` files |
| **OpenMM** | `add_com_restraint=True` in `setup_openmm_equilibration()` | Writes `com_restraint_params.json`; `omm_restraints.py` reads it automatically |
| **GROMACS** | `add_com_restraint=True` in `setup_gromacs_equilibration()` | Writes `com_restraint.dat` and injects `colvars-active = yes` / `colvars-configfile = ...` into the generated `.mdp` files |

All engines use kcal/mol/Å² for `com_restraint_k` and `rotation_restraint_k`.

#### NAMD example

```python
from pathlib import Path
from gatewizard.tools.equilibration import NAMDEquilibrationManager

manager = NAMDEquilibrationManager(Path("popc_membrane"))
result = manager.setup_namd_equilibration(
    add_com_restraint=True,
    com_restraint_k=5.0,
    add_rotation_restraint=True,
    rotation_restraint_k=200.0,
)
# The generated .conf files already include the colvars activation block.
```

#### OpenMM example

```python
from gatewizard.tools.equilibration import OpenMMEquilibrationManager

manager = OpenMMEquilibrationManager(Path("popc_membrane"))
result = manager.setup_openmm_equilibration(
    add_com_restraint=True,
    com_restraint_k=5.0,
)
# com_restraint_params.json is auto-loaded by omm_restraints.py
```

#### GROMACS example

```python
from gatewizard.tools.equilibration import GROMACSEquilibrationManager

manager = GROMACSEquilibrationManager(Path("popc_membrane"))
stages = GROMACSEquilibrationManager.get_default_stage_params("NPT")
result = manager.setup_gromacs_equilibration(
    stage_params_list=stages,
    add_com_restraint=True,
    com_restraint_k=5.0,
)
# The generated .mdp files already include the colvars activation block.
```

---

### Resume, metadata, and cluster scripts

These helpers sit beside the engine managers. They are used by the Equilibration page and cluster launch path.

#### pdb_has_protein

```python
from gatewizard.tools.equilibration import pdb_has_protein
```

Return whether a structure file contains protein residues (used to strip protein restraints from bilayer-only protocols).

#### adjust_protocol_for_missing_protein

```python
from gatewizard.tools.equilibration import adjust_protocol_for_missing_protein
```

Drop protein-targeted restraints and COM groups when the system has no protein.

#### cluster_engine_executable

```python
from gatewizard.utils.equilibration_cluster_script import cluster_engine_executable
```

Resolve the engine binary name written into a cluster run script.

#### write_cluster_run_script

```
from gatewizard.utils.equilibration_cluster_script import write_cluster_run_script
```

Write `run_equilibration.sh` (or engine equivalent) into an equilibration directory.

#### resolve_cluster_launch_script

```
from gatewizard.utils.equilibration_cluster_script import resolve_cluster_launch_script
```

Find the script the GUI/cluster launcher should submit.

Resume checkpoints live in `gatewizard.utils.equilibration_resume` (`resume_checkpoint_paths`). Job metadata JSON is written next to each engine directory when you generate inputs.

#### SSHSession

Dataclass/handle returned by `connect_ssh` (host, user, identity, control path).

#### connect_ssh

```
from gatewizard.utils.cluster.ssh import connect_ssh, run_remote, rsync_to_remote, rsync_from_remote
```

Low-level SSH/rsync helpers used by **Run on cluster**. See also `SSHSession`.

#### run_remote

Remote command helper. Requires an active `connect_ssh` session.

#### rsync_to_remote

Upload a local job directory.

#### rsync_from_remote

Pull results (and mid-run scratch when configured).

### See Also

- User Guide - Desktop Equilibration page
- Examples
- Troubleshooting

## Analysis

### Analysis Module API

Module for analyzing MD trajectories and NAMD energy data with easy-to-use wrapper classes supporting all GUI options.

#### Overview

The analysis module provides three main trajectory/energy classes:

- **`EnergyAnalyzer`** - Parse and plot NAMD energy data with full customization
- **`TrajectoryAnalyzer`** - Calculate and plot RMSD, RMSF, distances, and radius of gyration with complete control

- `BilayerTrajectoryAnalyzer` - Calculate area per lipid (**FATSLiM** CLI default; EVAPL experimental; official lipyphilic `AreaPerLipid` via `apl_method='lipyphilic'`) and membrane thickness (lipyphilic)

**Key Features:**

- Simple 2-line API for quick analysis
- Full customization with all GUI options
- Multi-file support with proper time scaling
- Unit conversions (Å/nm, ps/ns/µs, kcal/kJ)
- Publication-quality plots (300 DPI)
- **PlotSpec** — shared JSON plot contract for API matplotlib export and GUI publication PNG

**PlotSpec (energetic plots)**

All engines share a **PlotSpec** description (`gatewizard.utils.plot_spec`) consumed by `matplotlib_renderer.render_energetic()` for publication PNGs:

```
from gatewizard.utils.plot_spec import build_plot_spec_from_series
from gatewizard.utils.matplotlib_renderer import render_energetic_to_bytes

data = run_energetic_analysis(log_files, properties=["TEMP", "PRESSURE"])
spec = build_plot_spec_from_series(data["series"], layout="grid", cols=2)
png_bytes = render_energetic_to_bytes(data, spec)
```

| Field | Values | Notes |
|---|---|---|
| `layout` | `overlay` / `grid` | Overlay = one axis; grid = one property per panel |
| `sync_x` | bool | Multi-panel shared time window |
| `global` | colors, units, `xlim`, `dpi`, … | Dark-theme defaults match GUI |
| `panels[]` | `key`, `line_color`, `xlim`, `ylim`, … | Per-series overrides |

GUI **Analysis** → **Pub PNG** calls the same renderer via `POST /analysis-render-plot`. Amber and GROMACS: `plot_amber_properties()` / `plot_gromacs_properties()`.

```
from gatewizard.utils.namd_analysis import EnergyAnalyzer, get_equilibration_progress
from gatewizard.utils.trajectory_analysis import TrajectoryAnalyzer
from gatewizard.utils.lipid_bilayer_analysis import (
    BilayerTrajectoryAnalyzer,
    run_bilayer_analysis,
)
```

---

**Using Custom Time Input in the GUI**

**For Trajectory Files (.dcd, .xtc, .trr)**

When analyzing multiple trajectory files in the GUI **Structural Analysis** tab:

1. **Add Files**: Click "Add Files" or "Add Folder" to load trajectory files
2. **Input Time**: Each file will appear in the list with a time input field showing "Time (ns)"
3. **Enter Duration**: Enter the simulation duration for each file in **nanoseconds**
4. Example: For a 50 ps simulation, enter `0.05` (50 ps = 0.05 ns)
5. **Auto-Detect**: Click "Auto-Detect Time" to automatically extract times from DCD headers
6. **Reorder**: Drag files using the `:::` handle to reorder if needed

**Example File List:**

```
:::  step1_equilibration.dcd    [0.05] ns
:::  step2_equilibration.dcd    [0.05] ns
:::  step3_equilibration.dcd    [0.05] ns
```

Total time: 0.15 ns (150 ps) continuous

#### For Energy Log Files (.log)

When analyzing NAMD log files in the GUI **Energetic Analysis** tab:

1. **Add Files**: Click "Add Files" or "Add Folder" to load NAMD log files
2. **Input Time**: Each file will appear with a time input field
3. **Enter Duration**: Enter the simulation duration for each file in **nanoseconds**
4. Example: For step1_equilibration.log (50 ps), enter `0.05`
5. **Auto-Detect**: Click "Auto-Detect Time" to extract from log file timestamps
6. **Plot**: The time axis will scale correctly across all files

**Example File List:**

```
:::  step1_equilibration.log    [0.05] ns
:::  step2_equilibration.log    [0.05] ns
:::  step7_production.log       [0.10] ns
```

**Important Notes:**

- Time input is the **duration** of each file, not cumulative
- Files are processed in order - time is sequential (file1: 0-50ps, file2: 50-100ps, etc.)
- Leave empty or 0 to use default timestep-based calculation
- Time unit is always **nanoseconds** in the input fields
- You can change display units (ps/ns/μs) in Plot Settings

---

**Class: EnergyAnalyzer**

Easy-to-use wrapper for NAMD energy analysis with built-in plotting and customization.

#### Constructor

```
EnergyAnalyzer(
    log_file: Union[Path, str, List[Union[Path, str]]],
    file_times: Optional[Dict[str, float]] = None
)
```

Initialize energy analyzer with NAMD log file(s).

**Parameters:**

`log_file` (Path, str, or List): Path to NAMD log file, or list of paths for multi-file analysis

- Single file: `"step1_equilibration.log"`
- Multiple files: `["step1.log", "step2.log", "step3.log"]`

`file_times` (Dict[str, float], optional): Mapping of log filenames to their simulation durations in **nanoseconds**

- Example: `{"step1.log": 0.05, "step2.log": 0.05, "step3.log": 0.05}`
- **Important:** Use just the filename (not full path) as the key
- Time is the **duration** of each file, not cumulative
- **Note:** In the GUI, you can input times for each file directly in the file list

**Examples:**

```
# Single file
analyzer = EnergyAnalyzer("step1_equilibration.log")

# Multiple files with custom time
analyzer = EnergyAnalyzer(
    ["step1.log", "step2.log", "step3.log"],
    file_times={
        "step1.log": 0.05,  # 50 ps = 0.05 ns
        "step2.log": 0.05,
        "step3.log": 0.05
    }
)
```

---

#### plot_energy

Create a 4-panel energy analysis plot with full customization.

```
plot_energy(
    energy_units: str = "kcal/mol",
    time_units: str = "ns",
    target_temperature: Optional[float] = None,
    target_pressure: Optional[float] = None,
    bg_color: str = "#2b2b2b",
    fig_bg_color: str = "#212121",
    text_color: str = "Auto",
    show_grid: bool = True,
    title: Optional[str] = None,
    save: Optional[str] = None,
    show: bool = False,
    figsize: tuple = (12, 10),
    dpi: int = 300
)
```

**Parameters:**

| Parameter | Type | Default | Description |
|---|---|---|---|
| `energy_units` | str | `"kcal/mol"` | Energy units: `"kcal/mol"` or `"kJ/mol"` |
| `time_units` | str | `"ns"` | Time units: `"ps"`, `"ns"`, or `"μs"` |
| `target_temperature` | float or None | `None` | Target temperature for reference line (K). Defaults to 300.0 K if not provided. |
| `target_pressure` | float or None | `None` | Target pressure for reference line (atm). Defaults to 1.0 atm if not provided. |
| `bg_color` | str | `"#2b2b2b"` | Plot area background color (hex or `"none"`) |
| `fig_bg_color` | str | `"#212121"` | Figure border background color (hex or `"none"`) |
| `text_color` | str | `"Auto"` | Text/axes color (`"Auto"`, color name, or hex) |
| `show_grid` | bool | `True` | Show grid lines on plots |
| `title` | str or None | `None` | Custom main title (auto-generated if None) |
| `save` | str or None | `None` | Filename to save plot |
| `show` | bool | `False` | Display plot interactively |
| `figsize` | tuple | `(12, 10)` | Figure size (width, height) in inches |
| `dpi` | int | `300` | Resolution for saved figure |

**Plots Generated:**

1. **Total Energy** - Convergence over time
2. **Potential & Kinetic** - Energy components
3. **Temperature** - Stability with target line

4. **Pressure** - Fluctuations (or energy components if no pressure)

---

#### plot_properties

Create custom plots of selected energy/thermodynamic properties with full control over visualization.

This method provides complete API access to all GUI plotting capabilities, allowing users to:

- Analyze multiple log files with custom time assignments
- Select specific properties to plot
- Combine or separate data from multiple files
- Apply full customization (colors, units, grid, limits, etc.)

```
plot_properties(
    properties: Union[str, List[str]],
    separate_plots: bool = False,
    energy_units: str = "kcal/mol",
    time_units: str = "ns",
    line_colors: Optional[Union[str, List[str]]] = None,
    bg_color: str = "#2b2b2b",
    fig_bg_color: str = "#212121",
    text_color: str = "Auto",
    grid_color: str = "#444444",
    show_grid: bool = True,
    xlim: Optional[tuple] = None,
    ylim: Optional[tuple] = None,
    title: Optional[str] = None,
    save_prefix: Optional[str] = None,
    show: bool = False,
    figsize: tuple = (10, 6),
    dpi: int = 300
)
```

**Parameters:**

| Parameter | Type | Default | Description |
|---|---|---|---|
| `properties` | str or List[str] | - | Property name(s) to plot. See available properties below. |
| `separate_plots` | bool | `False` | If True, create individual plots per file; if False, combine all files on same plot |
| `energy_units` | str | `"kcal/mol"` | Energy units: `"kcal/mol"` or `"kJ/mol"` |
| `time_units` | str | `"ns"` | Time units: `"ps"`, `"ns"`, or `"µs"` |
| `line_colors` | str or List[str] | `None` | Line color(s). Single color or list for multiple files. Auto-assigned if None. |
| `bg_color` | str | `"#2b2b2b"` | Plot area background color (hex, color name, or `"none"`) |
| `fig_bg_color` | str | `"#212121"` | Figure border background color (hex, color name, or `"none"`) |
| `text_color` | str | `"Auto"` | Text/axes color. `"Auto"` auto-detects from bg luminance. |
| `grid_color` | str | `"#444444"` | Grid line color (hex or color name) |
| `show_grid` | bool | `True` | Show grid lines |
| `xlim` | tuple or None | `None` | X-axis limits: `(min, max)` |
| `ylim` | tuple or None | `None` | Y-axis limits: `(min, max)` |
| `title` | str or None | `None` | Custom plot title (auto-generated if None) |
| `save_prefix` | str or None | `None` | Prefix for saved filenames. Files saved as `{prefix}_{property}.png` |
| `show` | bool | `False` | Display plot interactively |
| `figsize` | tuple | `(10, 6)` | Figure size (width, height) in inches |
| `dpi` | int | `300` | Resolution for saved figures |

**Available Properties:**

Use `get_available_properties()` method to see all properties in your log file(s).

**Note:** Property names are **case-insensitive** and support multiple formats. For example, to plot temperature, you can use:

- `"TEMP"`, `"temp"`, `"Temp"` (short names)
- `"Temperature"`, `"TEMPERATURE"`, `"temperature"` (full names)
- Any mixed case variation like `"TeMpErAtUrE"` will also work!

The same applies to all properties - use whatever format is most convenient for you.

| Property | Alternative Names | Description | Units |
|---|---|---|---|
| `"TOTAL"` | `"Total"`, `"total"`, `"Total Energy"`, etc. | Total energy | kcal/mol or kJ/mol |
| `"KINETIC"` | `"Kinetic"`, `"kinetic"`, `"kin"`, `"Kinetic Energy"` | Kinetic energy | kcal/mol or kJ/mol |
| `"POTENTIAL"` | `"Potential"`, `"potential"`, `"pot"`, `"Potential Energy"` | Potential energy | kcal/mol or kJ/mol |
| `"ELECT"` | `"Elect"`, `"elect"`, `"elec"`, `"Electrostatic Energy"` | Electrostatic energy | kcal/mol or kJ/mol |
| `"VDW"` | `"vdw"`, `"Vdw"`, `"Van der Waals Energy"` | van der Waals energy | kcal/mol or kJ/mol |
| `"TEMP"` | `"temp"`, `"Temp"`, `"Temperature"` | Temperature | K |
| `"PRESSURE"` | `"pressure"`, `"Pressure"`, `"press"`, `"Press"` | Pressure | bar |
| `"VOLUME"` | `"volume"`, `"Volume"`, `"vol"`, `"Vol"` | System volume | Å² |
| `"BOND"` | `"bond"`, `"Bond"`, `"Bond Energy"` | Bond energy | kcal/mol or kJ/mol |
| `"ANGLE"` | `"angle"`, `"Angle"`, `"Angle Energy"` | Angle energy | kcal/mol or kJ/mol |
| `"DIHEDRAL"` | `"dihedral"`, `"Dihedral"`, `"Dihedral Energy"` | Dihedral energy | kcal/mol or kJ/mol |
| `"IMPROPER"` | `"improper"`, `"Improper"`, `"Improper Energy"` | Improper energy | kcal/mol or kJ/mol |

#### Example 1: Basic Energy Analysis

Simplest energy analysis using `plot_energy()` - 4-panel plot with default settings.

```python
from pathlib import Path
from gatewizard.utils.namd_analysis import EnergyAnalyzer

# Get the directory where this script is located
script_dir = Path(__file__).parent
data_dir = script_dir / "equilibration_folder"

# Multiple log files from equilibration_folder
log_files = [
    data_dir / "step1_equilibration.log",
    data_dir / "step2_equilibration.log",
    data_dir / "step3_equilibration.log",
]

# Initialize analyzer with custom time for each file (in nanoseconds)
analyzer = EnergyAnalyzer(
    [str(f) for f in log_files],
    file_times={
        "step1_equilibration.log": 0.1,  # 100 ps
        "step2_equilibration.log": 0.1,  # 100 ps
        "step3_equilibration.log": 0.1,  # 100 ps
    },
)

# Generate 4-panel energy plot
analyzer.plot_energy(
    target_temperature=300,  # 300 K
    target_pressure=1.01325,  # 1.01325 atm (1 bar)
    time_units="ps",
    save="energy_analysis_example_01.png",
    dpi=300,
)

print(f"Energy analysis complete!")
print(f"Plot saved: energy_analysis_example_01.png")
```

**Output figure:**

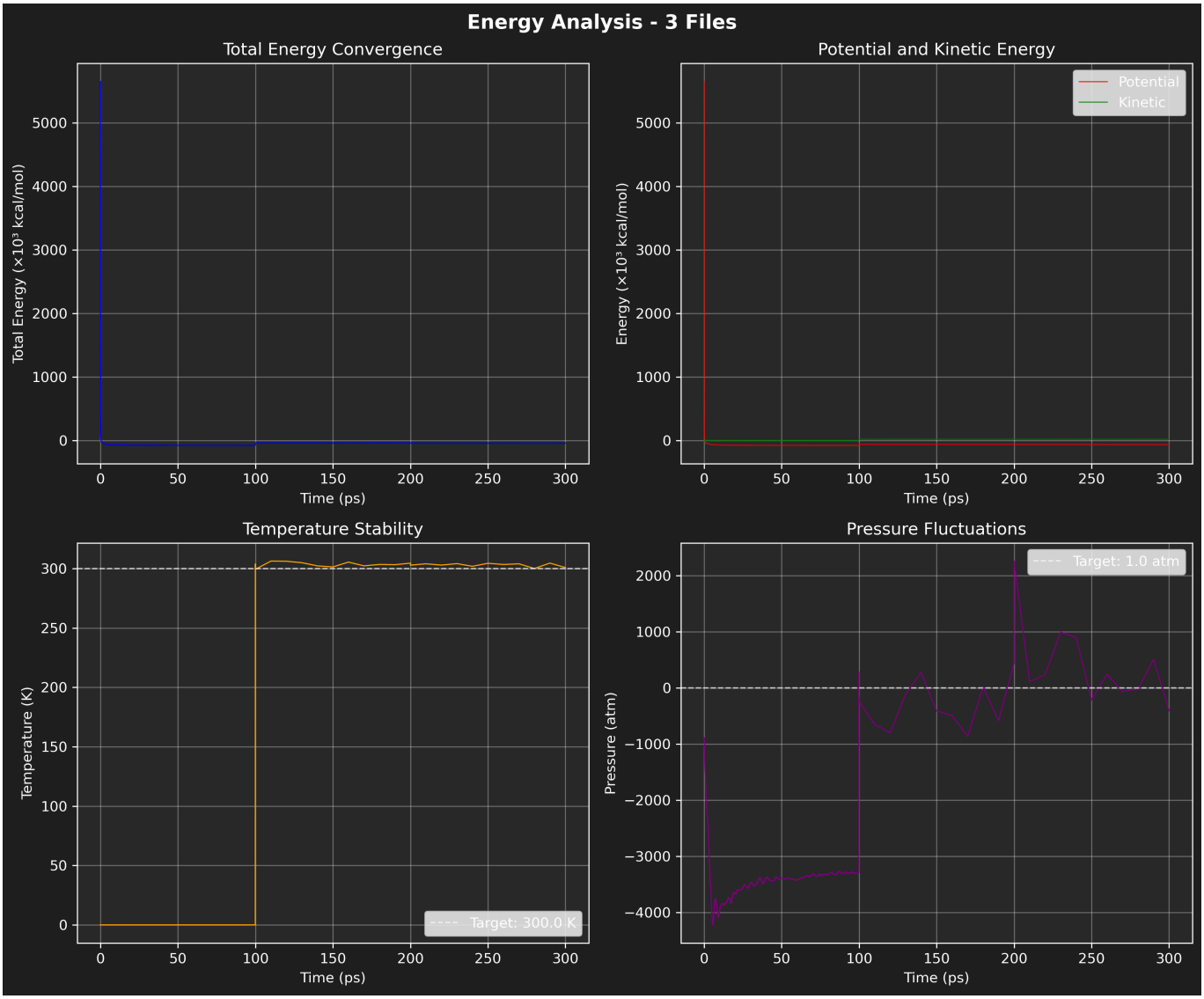



*Figure: Default energy plot using the specified NAMD .log files.*

---

**Multi-File Color Assignment:**

When plotting multiple files on the same plot, colors are automatically assigned:

- If `line_colors` is a single color: all files use that color
- If `line_colors` is a list: each file gets corresponding color
- If `line_colors=None`: auto-assigned from matplotlib color cycle

**Time Scaling with `file_times`:**

The `file_times` parameter in the constructor allows mapping data points to correct time ranges:

```
# Example: 3 files with different simulation times
# File 1: 0-50 ns (5000 frames)
# File 2: 50-125 ns (7500 frames)
# File 3: 125-200 ns (7500 frames)

analyzer = EnergyAnalyzer(
    log_file=["file1.log", "file2.log", "file3.log"],
    file_times=[(0, 50), (50, 125), (125, 200)],
    # or
    #file_times={
    #    "file1.log": 0.05,  # 50 ps = 0.05 ns
    #    "file2.log": 0.05,
    #    "file3.log": 0.05
    #}
)

# Now time axis will correctly show 0-200 ns with proper frame distribution
analyzer.plot_properties(properties="TEMP", time_units="ns")
```

---

#### get_available_properties

Get list of all plottable properties available in the loaded log file(s).

```
get_available_properties() -> List[str]
```

**Returns:** List of property names that can be passed to `plot_properties()`

##### Example 2: Get available properties

```
from pathlib import Path
from gatewizard.utils.namd_analysis import EnergyAnalyzer

# Get the directory where this script is located
script_dir = Path(__file__).parent

# Path to log file in equilibration_folder
log_file = script_dir / "equilibration_folder" / "step1_equilibration.log"

# Initialize energy analyzer
analyzer = EnergyAnalyzer(log_file)
props = analyzer.get_available_properties()
print(f"Can plot: {', '.join(props)}")
# Output: Can plot: Total Energy, Potential Energy, Kinetic Energy, Electrostatic Energy, Van der Waals
Energy, Bond Energy, Angle Energy, Dihedral Energy, Improper Energy, Temperature, Pressure, Volume
```

---

##### Example 3: Individual energy plots

```python
from pathlib import Path
from gatewizard.utils.namd_analysis import EnergyAnalyzer

# Get the directory where this script is located
script_dir = Path(__file__).parent
data_dir = script_dir / "equilibration_folder"

# Multiple log files from equilibration_folder
log_files = [
    data_dir / "step1_equilibration.log",
    data_dir / "step2_equilibration.log",
    data_dir / "step3_equilibration.log",
    data_dir / "step4_equilibration.log",
    data_dir / "step5_equilibration.log",
    data_dir / "step6_equilibration.log",
    data_dir / "step7_production.log",
]

# Initialize analyzer with custom time for each file (in nanoseconds)
analyzer = EnergyAnalyzer(
    [str(f) for f in log_files],
    file_times={
        "step1_equilibration.log": 0.1,  # 100 ps
        "step2_equilibration.log": 0.1,  # 100 ps
        "step3_equilibration.log": 0.1,  # 100 ps
        "step4_equilibration.log": 0.1,  # 100 ps
        "step5_equilibration.log": 0.1,  # 100 ps
        "step6_equilibration.log": 0.1,  # 100 ps
        "step7_production.log": 0.1,  # 100 ps
    },
)

# Plot specific energy properties
analyzer.plot_properties(
    properties=["bond energy", "angle energy", "dihedral energy"],
    energy_units="kcal/mol",
    time_units="ps",
    bg_color="#ffffff",
    fig_bg_color="#FFFFFF",
    save="energy_analysis_example_03.png",
    dpi=300,
)

print(f"Energy properties plot complete!")
print(f"Plot saved: energy_analysis_example_03.png")
```

**Example Output:**

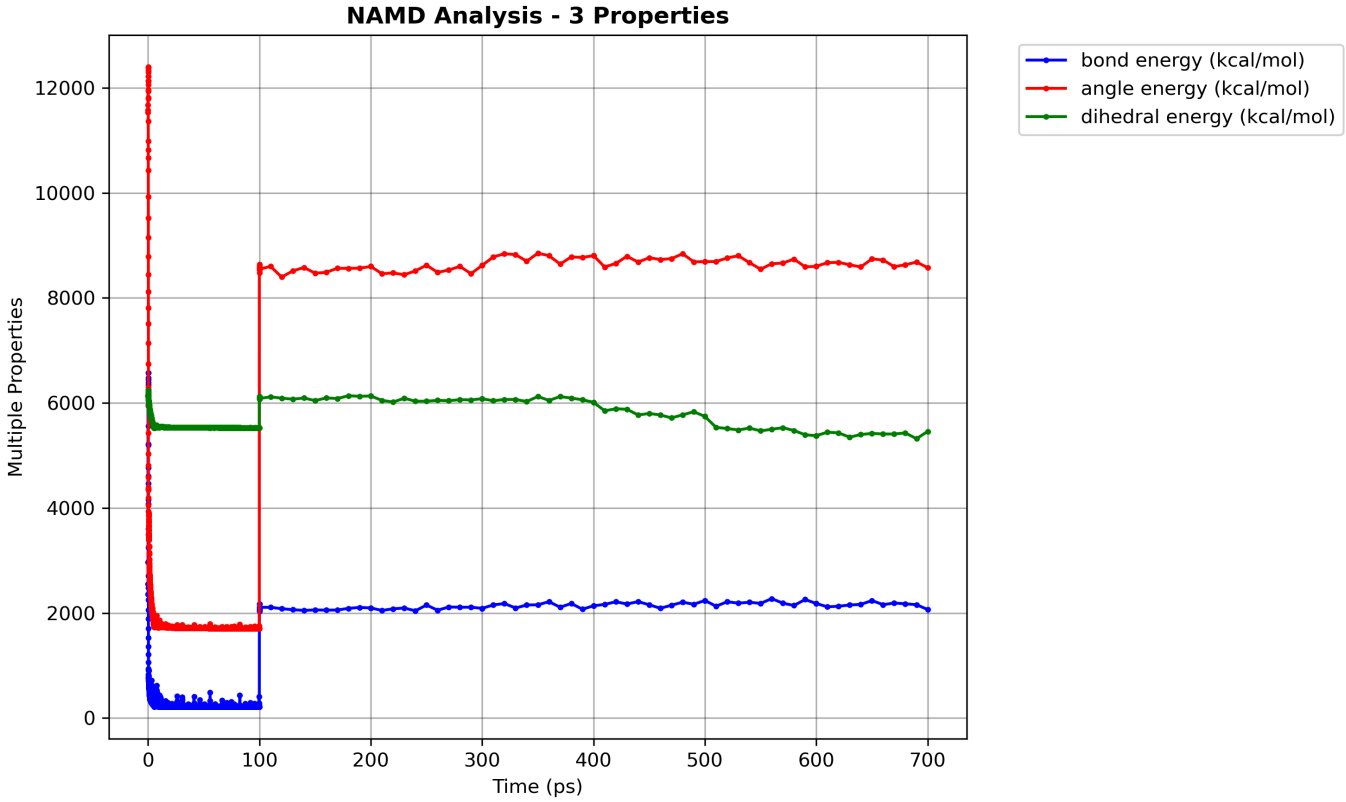



*Figure: Multiple energy properties plot.*

---

#### get_statistics

Get statistical summary of all energy components.

```
get_statistics() -> Dict[str, Dict[str, float]]
```

**Returns:** Dictionary with statistics for each energy term

**Statistics Provided:**

- `mean` - Average value
- `std` - Standard deviation
- `min` - Minimum value
- `max` - Maximum value
- `initial` - First value
- `final` - Last value

**Available Energy Terms:**

- `timestep`, `total`, `kinetic`, `potential`
- `elect`, `vdw`, `boundary`, `misc`
- `temp`, `pressure`, `volume`

#### Example 4: Individual energy plots

```python
from pathlib import Path
from gatewizard.utils.namd_analysis import EnergyAnalyzer

# Get the directory where this script is located
script_dir = Path(__file__).parent

# Path to log file in equilibration_folder
log_file = script_dir / "equilibration_folder" / "step2_equilibration.log"

# Initialize energy analyzer
analyzer = EnergyAnalyzer(log_file)

stats = analyzer.get_statistics()

# Temperature
temp = stats["temp"]
print(f"Temperature: {temp['mean']:.1f} ± {temp['std']:.1f} K")
print(f"  Range: {temp['min']:.1f} - {temp['max']:.1f} K")

# Energy
energy = stats["bond"]
print(f"Bond Energy: {energy['mean']:.0f} kcal/mol")
print(f"  Final: {energy['final']:.0f} kcal/mol")
print(f"  Convergence: {abs(energy['final'] - energy['mean']):.0f} kcal/mol")

# Output:
# Temperature: 303.6 ± 2.1 K
#   Range: 299.4 - 306.3 K
# Bond Energy: 2080 kcal/mol
#   Final: 2097 kcal/mol
#   Convergence: 17 kcal/mol
```

---

**Class: TrajectoryAnalyzer**

Easy-to-use wrapper for MD trajectory analysis with built-in plotting and full customization.

**Constructor**

```python
TrajectoryAnalyzer(
    topology: Path,
    trajectory: Path | List[Path],
    file_times: Optional[Dict[str, float]] = None
)
```

Initialize trajectory analyzer with topology and trajectory file(s).

**Parameters:**

- `topology` (Path or str): Path to topology file (PSF, PDB, IMPCRD, etc.)
- `trajectory` (Path, str, or List): Path(s) to trajectory file(s) (DCD, XTC, TRR, etc.)
- Single file: `"trajectory.dcd"`
- Multiple files: `["eq1.dcd", "eq2.dcd", "prod.dcd"]`
- `file_times` (Dict[str, float], optional): Mapping of trajectory filenames to their simulation durations in **nanoseconds**

Example: `{"eq1.dcd": 0.05, "eq2.dcd": 0.05, "prod.dcd": 0.05}`

- **Important:** Use just the filename (not full path) as the key
- Time is the **duration** of that file, not cumulative
- **Note:** In the GUI, you can input times for each file directly in the file list

**Examples:**

**Single Trajectory:**

```
analyzer = TrajectoryAnalyzer("system.pdb", "trajectory.dcd")
```

**Multiple Trajectories with Time Scaling:**

```
analyzer = TrajectoryAnalyzer(
    "system.pdb",
    ["step1_equilibration.dcd", "step2_equilibration.dcd", "step7_production.dcd"],
    file_times={
        "step1_equilibration.dcd": 0.05,  # 50 ps
        "step2_equilibration.dcd": 0.05,  # 50 ps
        "step7_production.dcd": 0.05      # 50 ps
    }
)
# Time axis will be continuous: [0...0.05] [0.05...0.10] [0.10...0.15] ns
```

---

#### calculate_rmsd

Calculate RMSD for selected atoms.

```
calculate_rmsd(
    selection: str = "protein and backbone",
    reference_frame: int = 0,
    align: bool = True
) -> Dict[str, np.ndarray]
```

**Parameters:**

- `selection` (str): MDTraj selection string
- `reference_frame` (int): Frame to use as reference (0 = first frame)
- `align` (bool): If True, perform structural alignment (rotation + translation) before RMSD. If False, calculate raw coordinate RMSD without alignment.

**Returns:** Dictionary with:

- `'time'`: Time array in **nanoseconds**
- `'rmsd'`: RMSD values in **Angstroms**

**Example:**

```
data = analyzer.calculate_rmsd("protein and backbone", align=True)
print(f"Time range: {data['time'][0]:.3f} - {data['time'][-1]:.3f} ns")
print(f"RMSD range: {data['rmsd'].min():.2f} - {data['rmsd'].max():.2f} Å")
```

---

#### plot_rmsd

Plot RMSD with full customization options including line styling, threshold highlighting, and convergence line control.

```
plot_rmsd(
    selection: str = "protein and backbone",
    reference_frame: int = 0,
    align: bool = True,
    distance_units: str = "Å",
    time_units: str = "ns",
    line_color: str = "blue",
    line_width: float = 1.2,
    line_style: str = "-",
    bg_color: str = "#2b2b2b",
    fig_bg_color: str = "#212121",
    text_color: str = "Auto",
    show_grid: bool = True,
    xlim: Optional[tuple] = None,
    ylim: Optional[tuple] = None,
    title: Optional[str] = None,
    xlabel: Optional[str] = None,
    ylabel: Optional[str] = None,
    highlight_threshold: Optional[float] = None,
    highlight_color: str = "orange",
    highlight_alpha: float = 0.2,
    show_convergence: bool = True,
    convergence_color: str = "red",
    convergence_style: str = "--",
    convergence_width: float = 1.5,
    hlines: Optional[List[float]] = None,
    hline_colors: Optional[List[str]] = None,
    hline_styles: Optional[List[str]] = None,
    hline_widths: Optional[List[float]] = None,
    vlines: Optional[List[float]] = None,
    vline_colors: Optional[List[str]] = None,
    vline_styles: Optional[List[str]] = None,
    vline_widths: Optional[List[float]] = None,
    save: Optional[str] = None,
    show: bool = False,
    figsize: tuple = (10, 6),
    dpi: int = 300
)
```

**Parameters:**

| Parameter | Type | Default | Description |
|---|---|---|---|
| `selection` | str | `"protein and backbone"` | MDTraj selection string |
| `reference_frame` | int | `0` | Reference frame index |
| `align` | bool | `True` | Perform structural alignment before RMSD |
| `distance_units` | str | `"Å"` | Distance units: `"Å"` or `"nm"` |
| `time_units` | str | `"ns"` | Time units: `"ps"`, `"ns"`, or `"µs"` |
| `line_color` | str | `"blue"` | Plot line color (matplotlib color or hex) |
| `line_width` | float | `1.2` | Line thickness/width |
| `line_style` | str | `"-"` | Line style: `"-"` (solid), `"--"` (dashed), `"-."` (dash-dot), `":"` (dotted) |
| `bg_color` | str | `"#2b2b2b"` | Plot area background (hex or `"none"`) |
| `fig_bg_color` | str | `"#212121"` | Figure border background (hex or `"none"`) |
| `text_color` | str | `"Auto"` | Text/axes color (`"Auto"`, color, or hex) |
| `show_grid` | bool | `True` | Show grid lines |
| `xlim` | tuple or None | `None` | X-axis limits (min, max) |
| `ylim` | tuple or None | `None` | Y-axis limits (min, max) |
| `title` | str or None | `None` | Custom title (auto-generated if None) |

| Parameter | Type | Default | Description |
|---|---|---|---|
| `xlabel` | str or None | `None` | Custom X-label (auto with units if None) |
| `ylabel` | str or None | `None` | Custom Y-label (auto with units if None) |
| `highlight_threshold` | float or None | `None` | Highlight regions above this RMSD value |
| `highlight_color` | str | `"orange"` | Color for highlight region and line |
| `highlight_alpha` | float | `0.2` | Alpha transparency for highlight fill |
| `show_convergence` | bool | `True` | Show convergence line (mean of last 20% of trajectory) |
| `convergence_color` | str | `"red"` | Color for convergence line |
| `convergence_style` | str | `"--"` | Line style for convergence line |
| `convergence_width` | float | `1.5` | Width of convergence line |
| `hlines` | list or None | `None` | List of Y values for horizontal reference lines |
| `hline_colors` | list or None | `None` | List of colors for horizontal lines |
| `hline_styles` | list or None | `None` | List of line styles for horizontal lines (default: `"--"`) |
| `hline_widths` | list or None | `None` | List of line widths for horizontal lines (default: `1.0`) |
| `vlines` | list or None | `None` | List of X values for vertical reference lines |
| `vline_colors` | list or None | `None` | List of colors for vertical lines |
| `vline_styles` | list or None | `None` | List of line styles for vertical lines (default: `"--"`) |
| `vline_widths` | list or None | `None` | List of line widths for vertical lines (default: `1.0`) |
| `save` | str or None | `None` | Filename to save plot |
| `show` | bool | `False` | Display plot interactively |
| `figsize` | tuple | `(10, 6)` | Figure size (width, height) |
| `dpi` | int | `300` | Resolution for saved figure |

**Example 5: RMSD Plot**

```python
from pathlib import Path
from gatewizard.utils.trajectory_analysis import TrajectoryAnalyzer

# Get the directory where this script is located
script_dir = Path(__file__).parent
data_dir = script_dir / "equilibration_folder"

# Path to topology in equilibration_folder
topology_file = data_dir / "system.pdb"

# Multiple trajectory files from equilibration_folder
trajectory_files = [
    data_dir / "step1_equilibration.dcd",
    data_dir / "step2_equilibration.dcd",
    data_dir / "step3_equilibration.dcd",
]

# Initialize analyzer with custom time for each file (in nanoseconds)
analyzer = TrajectoryAnalyzer(
    topology_file,
    trajectory_files,
    file_times={
        "step1_equilibration.dcd": 0.1,  # 100 ps
        "step2_equilibration.dcd": 0.1,  # 100 ps
        "step3_equilibration.dcd": 0.1,  # 100 ps
    },
)

# Plot RMSD across all trajectories
analyzer.plot_rmsd(
    selection="protein and backbone",
    time_units="ns",
    bg_color="white",
    fig_bg_color="white",
    text_color="black",
    show_grid=False,
    line_color="#1f77b4",
    line_width=2,
    # title=" ",
    save="trajectory_analysis_example_05.png",
    dpi=300,
    # other settings...
)

print(f"Multi-file trajectory analysis complete!")
print(f"Plot saved: trajectory_analysis_example_05.png")
print(f"Total simulation time: 300 ps")
```

---

#### calculate_rmsf

Calculate RMSF (Root Mean Square Fluctuation) for selected atoms.

```python
calculate_rmsf(
    selection: str = "protein and name CA"
) -> Dict[str, np.ndarray]
```

**Parameters:**

- `selection` (str): MDTraj selection string

**Returns:** Dictionary with:

- `'resids'`: Residue numbers
- `'rmsf'`: RMSF values in **Angstroms**
- `'resnames'`: Residue names (e.g., ALA, GLY, VAL)
- `'atom_indices'`: Atom indices

**Example 6: RMSF Calculation**

```python
from pathlib import Path
from gatewizard.utils.trajectory_analysis import TrajectoryAnalyzer

# Get the directory where this script is located
script_dir = Path(__file__).parent
data_dir = script_dir / "equilibration_folder"

# Path to topology in equilibration_folder
topology_file = data_dir / "system.pdb"

# Multiple trajectory files from equilibration_folder
trajectory_files = [
    data_dir / "step1_equilibration.dcd",
    data_dir / "step2_equilibration.dcd",
    data_dir / "step3_equilibration.dcd",
]

# Initialize analyzer with custom time for each file (in nanoseconds)
analyzer = TrajectoryAnalyzer(
    topology_file,
    trajectory_files,
    file_times={
        "step1_equilibration.dcd": 0.1,  # 100 ps
        "step2_equilibration.dcd": 0.1,  # 100 ps
        "step3_equilibration.dcd": 0.1,  # 100 ps
    },
)

# Calculate RMSF for alpha carbon atoms
data = analyzer.calculate_rmsf("protein and name CA")
print(f"Residues analyzed: {len(data['resids'])}")
print(f"RMSF range: {data['rmsf'].min():.2f} - {data['rmsf'].max():.2f} Å")
print(f"Mean RMSF: {data['rmsf'].mean():.2f} Å")
```

---

**plot_rmsf**

Plot RMSF with full customization including line styling, residue labeling, and flexibility highlighting.

```
plot_rmsf(
    selection: str = "protein and name CA",
    xaxis_type: str = "residue_number",
    show_residue_labels: bool = True,
    residue_name_format: str = "single",
    label_frequency: str = "auto",
    distance_units: str = "Å",
    line_color: str = "blue",
    line_width: float = 1.2,
    line_style: str = "-",
    bg_color: str = "#2b2b2b",
    fig_bg_color: str = "#212121",
    text_color: str = "Auto",
    show_grid: bool = True,
    xlim: Optional[tuple] = None,
    ylim: Optional[tuple] = None,
    title: Optional[str] = None,
    xlabel: Optional[str] = None,
    ylabel: Optional[str] = None,
    highlight_threshold: Optional[float] = None,
    highlight_color: str = "orange",
    highlight_alpha: float = 0.2,
    hlines: Optional[List[float]] = None,
    hline_colors: Optional[List[str]] = None,
    hline_styles: Optional[List[str]] = None,
    hline_widths: Optional[List[float]] = None,
    vlines: Optional[List[float]] = None,
    vline_colors: Optional[List[str]] = None,
    vline_styles: Optional[List[str]] = None,
    vline_widths: Optional[List[float]] = None,
    save: Optional[str] = None,
    show: bool = False,
    figsize: tuple = (12, 6),
    dpi: int = 300
)
```

**Parameters:**

| Parameter | Type | Default | Description |
|---|---|---|---|
| `selection` | str | `"protein and name CA"` | MDTraj selection string |
| `xaxis_type` | str | `"residue_number"` | X-axis type: `"residue_number"`, `"residue_type_number"`, `"atom_index"` |
| `show_residue_labels` | bool | `True` | Show residue labels on X-axis |
| `residue_name_format` | str | `"single"` | Residue format: `"single"` (A, G, V) or `"triple"` (ALA, GLY, VAL) |
| `label_frequency` | str | `"auto"` | Label frequency: `"all"`, `"auto"`, `"every_2"`, `"every_5"`, `"every_10"`, `"every_20"` |
| `distance_units` | str | `"Å"` | Distance units: `"Å"` or `"nm"` |
| `line_color` | str | `"blue"` | Plot line color (matplotlib color or hex) |
| `line_width` | float | `1.2` | Line thickness/width |
| `line_style` | str | `"-"` | Line style: `"-"` (solid), `"--"` (dashed), `"-."` (dash-dot), `":"` (dotted) |
| `highlight_threshold` | float or None | `None` | If set, highlight residues above this RMSF value |
| `highlight_color` | str | `"orange"` | Color for highlight region and threshold line |
| `highlight_alpha` | float | `0.2` | Alpha transparency for highlight fill region |
| `hlines` | list or None | `None` | List of Y values for horizontal reference lines |
| `hline_colors` | list or None | `None` | List of colors for horizontal lines (cycles if fewer than lines) |

| Parameter | Type | Default | Description |
|---|---|---|---|
| `hline_styles` | list or None | `None` | List of line styles for horizontal lines (default: `"--"`) |
| `hline_widths` | list or None | `None` | List of line widths for horizontal lines (default: 1.0) |
| `vlines` | list or None | `None` | List of X values (residue numbers) for vertical reference lines |
| `vline_colors` | list or None | `None` | List of colors for vertical lines (cycles if fewer than lines) |
| `vline_styles` | list or None | `None` | List of line styles for vertical lines (default: `"--"`) |
| `vline_widths` | list or None | `None` | List of line widths for vertical lines (default: 1.0) |
| *...other params* | | | Same as `plot_rmsd()` for colors, grid, limits, etc. |

**X-Axis Types:**

- `"residue_number"` - Simple numbers: 1, 2, 3, ...
- `"residue_type_number"` - Type + number: ALA1, GLY2, VAL3, ...
- `"atom_index"` - Atom indices: 0, 15, 30, ...

**Label Frequencies:**

- `"all"` - Every residue labeled
- `"auto"` - Smart frequency based on count (<20: all, <50: every_2, <100: every_5, <200: every_10, else: every_20)
- `"every_2"`, `"every_5"`, `"every_10"`, `"every_20"` - Specific intervals

#### Example 7: RMSF Plotting

```python
from pathlib import Path
from gatewizard.utils.trajectory_analysis import TrajectoryAnalyzer

# Get the directory where this script is located
script_dir = Path(__file__).parent
data_dir = script_dir / "equilibration_folder"

# Path to topology in equilibration_folder
topology_file = data_dir / "system.pdb"

# Multiple trajectory files from equilibration_folder
trajectory_files = [
    data_dir / "step1_equilibration.dcd",
    data_dir / "step2_equilibration.dcd",
    data_dir / "step3_equilibration.dcd",
    data_dir / "step4_equilibration.dcd",
    data_dir / "step5_equilibration.dcd",
    data_dir / "step6_equilibration.dcd",
    data_dir / "step7_production.dcd",
]

# Initialize analyzer with custom time for each file (in nanoseconds)
analyzer = TrajectoryAnalyzer(
    topology_file,
    trajectory_files,
    file_times={
        "step1_equilibration.dcd": 0.1,  # 100 ps
        "step2_equilibration.dcd": 0.1,  # 100 ps
        "step3_equilibration.dcd": 0.1,  # 100 ps
        "step4_equilibration.dcd": 0.1,  # 100 ps
        "step5_equilibration.dcd": 0.1,  # 100 ps
        "step6_equilibration.dcd": 0.1,  # 100 ps
        "step7_production.dcd": 0.1,  # 100 ps
    },
)

# Plot RMSF across all trajectories
analyzer.plot_rmsf(
    selection="protein and name C",
    xaxis_type="residue_type_number",
    residue_name_format="triple",
    label_frequency="all",
    highlight_threshold=0.6,
    highlight_color="red",
    bg_color="white",
    fig_bg_color="white",
    text_color="black",
    show_grid=False,
    line_color="#1f77b4",
    line_width=2,
    # title=" ",
    save="trajectory_analysis_example_07.png",
    dpi=300,
    # other settings...
)
```

**Output figure:**

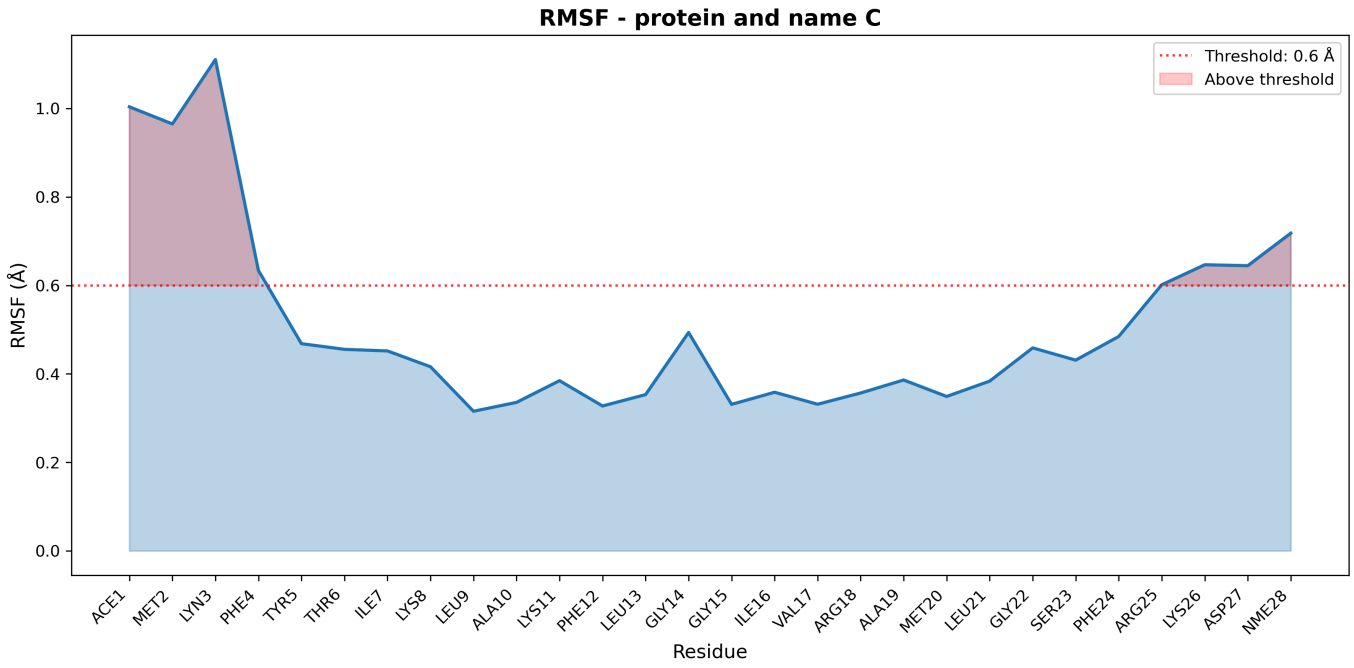



*Figure: RMSF plot for multiple with threshold highlight.*

---

#### calculate_distances

Calculate distances between atom selections over time.

```
calculate_distances(
    selections: Dict[str, tuple]
) -> Dict[str, Dict[str, np.ndarray]]
```

**Parameters:**

- `selections` (dict): Dictionary of `{name: (selection1, selection2)}`

**Returns:** Dictionary with distance data for each named selection:

- `'time'`: Time array in **nanoseconds**
- `'distance'`: Distance array in **Angstroms**

**Example:**

```
results = analyzer.calculate_distances({
    "gate": ("resid 1-3 and name CA", "resid 18-20 and name CA"),
    "binding_site": ("resid 100-110", "resname LIG")
})

for name, data in results.items():
    print(f"{name}:")
    print(f"  Time: {data['time'][0]:.3f} - {data['time'][-1]:.3f} ns")
    print(f"  Distance: {data['distance'].mean():.2f} ± {data['distance'].std():.2f} Å")
```

---

#### plot_distances

Plot distances with full customization including line styling.

```
plot_distances(
    selections: Dict[str, tuple],
    distance_units: str = "Å",
    time_units: str = "ns",
    line_colors: Optional[List[str]] = None,
    line_width: float = 1.2,
    line_style: str = "-",
    bg_color: str = "#2b2b2b",
    fig_bg_color: str = "#212121",
    text_color: str = "Auto",
    show_grid: bool = True,
    xlim: Optional[tuple] = None,
    ylim: Optional[tuple] = None,
    title: Optional[str] = None,
    xlabel: Optional[str] = None,
    ylabel: Optional[str] = None,
    show_mean_lines: bool = True,
    hlines: Optional[List[float]] = None,
    hline_colors: Optional[List[str]] = None,
    hline_styles: Optional[List[str]] = None,
    hline_widths: Optional[List[float]] = None,
    vlines: Optional[List[float]] = None,
    vline_colors: Optional[List[str]] = None,
    vline_styles: Optional[List[str]] = None,
    vline_widths: Optional[List[float]] = None,
    save: Optional[str] = None,
    show: bool = False,
    figsize: tuple = (10, 6),
    dpi: int = 300
)
```

**Parameters:**

| Parameter | Type | Default | Description |
|---|---|---|---|
| `selections` | dict | - | Dictionary of `{name: (sel1, sel2)}` |
| `distance_units` | str | `"Å"` | Distance units: `"Å"` or `"nm"` |
| `time_units` | str | `"ns"` | Time units: `"ps"`, `"ns"`, or `"µs"` |
| `line_colors` | list or None | `None` | List of colors for each distance pair |
| `line_width` | float | `1.2` | Line thickness/width |
| `line_style` | str | `"-"` | Line style: `"-"` (solid), `"--"` (dashed), `"-."` (dash-dot), `":"` (dotted) |
| `show_mean_lines` | bool | `True` | Show dashed mean distance lines |
| `hlines` | list or None | `None` | List of Y values for horizontal reference lines |
| `hline_colors` | list or None | `None` | List of colors for horizontal lines (cycles if fewer than lines) |
| `hline_styles` | list or None | `None` | List of line styles for horizontal lines (default: `"--"`) |
| `hline_widths` | list or None | `None` | List of line widths for horizontal lines (default: 1.0) |
| `vlines` | list or None | `None` | List of X values (time points) for vertical reference lines |
| `vline_colors` | list or None | `None` | List of colors for vertical lines (cycles if fewer than lines) |
| `vline_styles` | list or None | `None` | List of line styles for vertical lines (default: `"--"`) |
| `vline_widths` | list or None | `None` | List of line widths for vertical lines (default: 1.0) |
| *...other params* | | | Same as `plot_rmsd()` |

**Example 8: Distances**

```python
from pathlib import Path
from gatewizard.utils.trajectory_analysis import TrajectoryAnalyzer

# Get the directory where this script is located
script_dir = Path(__file__).parent
data_dir = script_dir / "equilibration_folder"

# Path to topology in equilibration_folder
topology_file = data_dir / "system.pdb"

# Multiple trajectory files from equilibration_folder
trajectory_files = [
    data_dir / "step1_equilibration.dcd",
    data_dir / "step2_equilibration.dcd",
    data_dir / "step3_equilibration.dcd",
    data_dir / "step4_equilibration.dcd",
    data_dir / "step5_equilibration.dcd",
    data_dir / "step6_equilibration.dcd",
    data_dir / "step7_production.dcd",
]

# Initialize analyzer with custom time for each file (in nanoseconds)
analyzer = TrajectoryAnalyzer(
    topology_file,
    trajectory_files,
    file_times={
        "step1_equilibration.dcd": 0.1,  # 100 ps
        "step2_equilibration.dcd": 0.1,  # 100 ps
        "step3_equilibration.dcd": 0.1,  # 100 ps
        "step4_equilibration.dcd": 0.1,  # 100 ps
        "step5_equilibration.dcd": 0.1,  # 100 ps
        "step6_equilibration.dcd": 0.1,  # 100 ps
        "step7_production.dcd": 0.1,  # 100 ps
    },
)
# Calculate and plot distances between selections
analyzer.plot_distances(
    selections={
        "gate_distance": ("resid 1 and name C", "resid 28 and name C"),
        "domain_distance": ("resid 1-2 and name C", "resid 9-10 and name C"),
    },
    bg_color="white",
    fig_bg_color="white",
    text_color="black",
    line_width=3,
    dpi=300,
    show_mean_lines=False,
    figsize=(5, 4),
    save="trajectory_analysis_example_08_distances.png",
)

print(f"Distances plot saved: trajectory_analysis_example_08_distances.png")
```

**Output figure:**

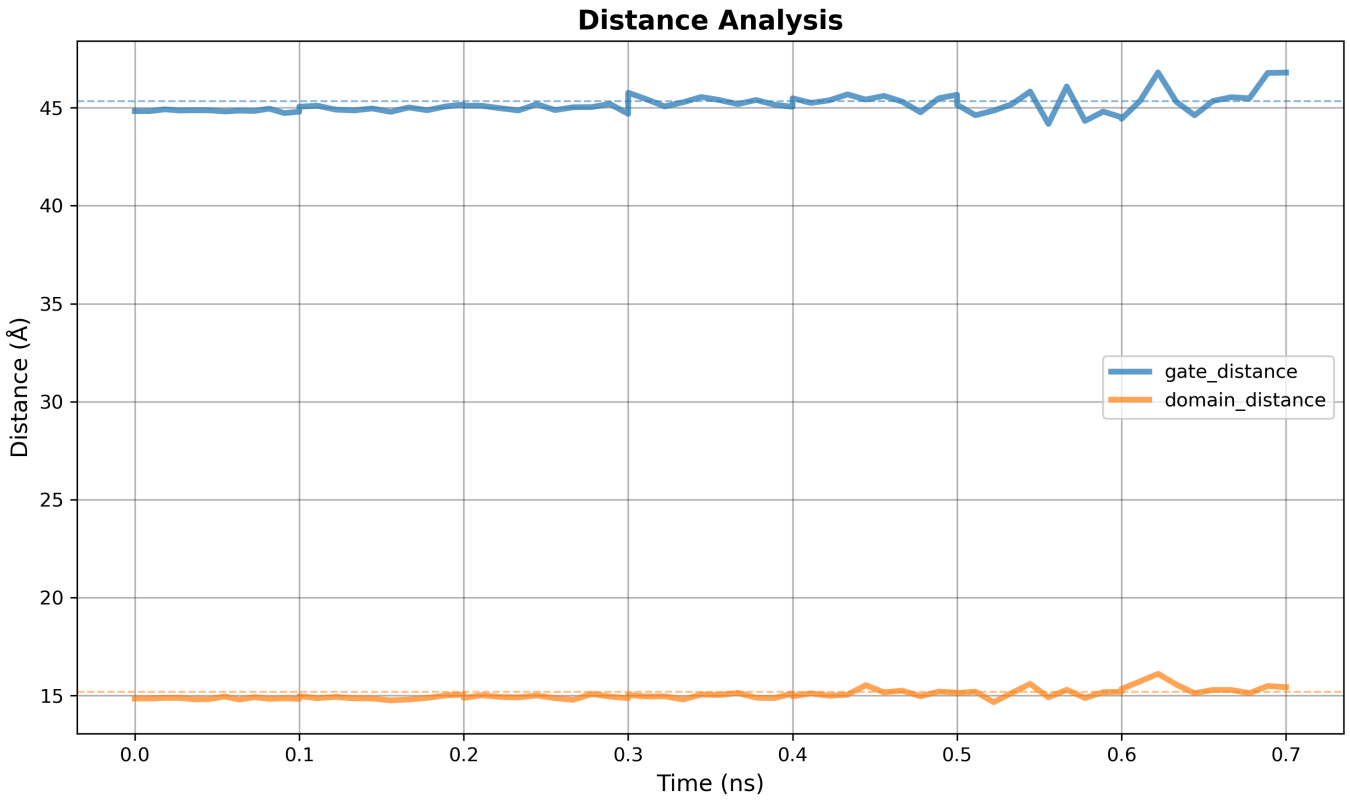



*Figure: Distances plot for multiple defined regions on the protein.*

---

#### calculate_radius_of_gyration

Calculate radius of gyration over trajectory.

```
calculate_radius_of_gyration(
    selection: str = "protein"
) -> Dict[str, np.ndarray]
```

**Parameters:**

- `selection` (str): MDTraj selection string

**Returns:** Dictionary with:

- `'time'`: Time array in **nanoseconds**
- `'rg'`: Radius of gyration in **Angstroms**

**Example:**

```
data = analyzer.calculate_radius_of_gyration("protein")
print(f"Rg range: {data['rg'].min():.2f} - {data['rg'].max():.2f} Å")
print(f"Mean Rg: {data['rg'].mean():.2f} Å")
```

---

#### plot_radius_of_gyration

Plot radius of gyration with full customization including line styling and convergence control.

```
plot_radius_of_gyration(
    selection: str = "protein",
    distance_units: str = "Å",
    time_units: str = "ns",
    line_color: str = "purple",
    line_width: float = 1.2,
    line_style: str = "-",
    bg_color: str = "#2b2b2b",
    fig_bg_color: str = "#212121",
    text_color: str = "Auto",
    show_grid: bool = True,
    xlim: Optional[tuple] = None,
    ylim: Optional[tuple] = None,
    title: Optional[str] = None,
    xlabel: Optional[str] = None,
    ylabel: Optional[str] = None,
    show_convergence: bool = True,
    convergence_color: str = "red",
    convergence_style: str = "--",
    convergence_width: float = 1.5,
    hlines: Optional[List[float]] = None,
    hline_colors: Optional[List[str]] = None,
    hline_styles: Optional[List[str]] = None,
    hline_widths: Optional[List[float]] = None,
    vlines: Optional[List[float]] = None,
    vline_colors: Optional[List[str]] = None,
    vline_styles: Optional[List[str]] = None,
    vline_widths: Optional[List[float]] = None,
    save: Optional[str] = None,
    show: bool = False,
    figsize: tuple = (10, 6),
    dpi: int = 300
)
```

**Parameters:**

| Parameter | Type | Default | Description |
|---|---|---|---|
| `selection` | str | `"protein"` | MDTraj selection string |
| `distance_units` | str | `"Å"` | Distance units: `"Å"` or `"nm"` |
| `time_units` | str | `"ns"` | Time units: `"ps"`, `"ns"`, or `"µs"` |
| `line_color` | str | `"purple"` | Plot line color (matplotlib color or hex) |
| `line_width` | float | `1.2` | Line thickness/width |
| `line_style` | str | `"-"` | Line style: `"-"` (solid), `"--"` (dashed), `"-."` (dash-dot), `":"` (dotted) |
| `show_convergence` | bool | `True` | Show convergence line (mean of last 20% of trajectory) |
| `convergence_color` | str | `"red"` | Color for convergence line |
| `convergence_style` | str | `"--"` | Line style for convergence line |
| `convergence_width` | float | `1.5` | Width of convergence line |
| `hlines` | list or None | `None` | List of Y values for horizontal reference lines |
| `hline_colors` | list or None | `None` | List of colors for horizontal lines (cycles if fewer than lines) |
| `hline_styles` | list or None | `None` | List of line styles for horizontal lines (default: `"--"`) |
| `hline_widths` | list or None | `None` | List of line widths for horizontal lines (default: 1.0) |
| `vlines` | list or None | `None` | List of X values (time points) for vertical reference lines |
| `vline_colors` | list or None | `None` | List of colors for vertical lines (cycles if fewer than lines) |
| `vline_styles` | list or None | `None` | List of line styles for vertical lines (default: `"--"`) |
| `vline_widths` | list or None | `None` | List of line widths for vertical lines (default: 1.0) |
| *...other params* | | | Same as `plot_rmsd()` for colors, grid, limits, etc. |

**Example 9: Radius of gyration**

```python
from pathlib import Path
from gatewizard.utils.trajectory_analysis import TrajectoryAnalyzer

# Get the directory where this script is located
script_dir = Path(__file__).parent
data_dir = script_dir / "equilibration_folder"

# Path to topology in equilibration_folder
topology_file = data_dir / "system.pdb"

# Multiple trajectory files from equilibration_folder
trajectory_files = [
    data_dir / "step1_equilibration.dcd",
    data_dir / "step2_equilibration.dcd",
    data_dir / "step3_equilibration.dcd",
    data_dir / "step4_equilibration.dcd",
    data_dir / "step5_equilibration.dcd",
    data_dir / "step6_equilibration.dcd",
    data_dir / "step7_production.dcd",
]

# Initialize analyzer with custom time for each file (in nanoseconds)
analyzer = TrajectoryAnalyzer(
    topology_file,
    trajectory_files,
    file_times={
        "step1_equilibration.dcd": 0.1,  # 100 ps
        "step2_equilibration.dcd": 0.1,  # 100 ps
        "step3_equilibration.dcd": 0.1,  # 100 ps
        "step4_equilibration.dcd": 0.1,  # 100 ps
        "step5_equilibration.dcd": 0.1,  # 100 ps
        "step6_equilibration.dcd": 0.1,  # 100 ps
        "step7_production.dcd": 0.1,  # 100 ps
    },
)
# Calculate and plot radius of gyration of a selection
analyzer.plot_radius_of_gyration(
    selection="protein",
    bg_color="white",
    fig_bg_color="white",
    text_color="black",
    line_width=3,
    save="trajectory_analysis_example_09_rdgyr.png",
    dpi=300,
)
```

---

#### plot_summary

Create a 4-panel analysis plot with RMSD, RMSF, Rg, and statistics.

```python
plot_summary(
    save: Optional[str] = None,
    show: bool = False,
    figsize: tuple = (14, 10),
    dpi: int = 300
)
```

**Parameters:**

- `save` (str, optional): Filename to save plot
- `show` (bool): Display plot interactively
- `figsize` (tuple): Figure size
- `dpi` (int): Resolution

**Example:**

```python
analyzer.plot_summary(save="comprehensive_analysis.png")
```

---

### Tips & Best Practices

#### 1. Always Provide file_times for Multiple Files

```
# ✅ GOOD: Proper time scaling
file_times = {
    "step1_equilibration.dcd": 0.05,  # 50 ps
    "step2_equilibration.dcd": 0.05,
    "step3_equilibration.dcd": 0.05
}

# ❌ BAD: Frame-based time (incorrect)
file_times = None  # Will use frame indices
```

#### 2. Match Units to Your Analysis

```
# Short simulations (< 1 ns)
time_units="ps", distance_units="Å"

# Medium simulations (1-100 ns)
time_units="ns", distance_units="Å"

# Long simulations (> 100 ns)
time_units="ns", distance_units="nm"
```

#### 3. RMSF Labeling for Different Protein Sizes

```
# Small protein (< 100 residues)
label_frequency="every_2"

# Medium protein (100-300 residues)
label_frequency="every_5"

# Large protein (> 300 residues)
label_frequency="every_10" or "every_20"
```

#### 4. Color Schemes

```
# Dark theme (presentations)
bg_color="#2b2b2b", text_color="Auto"

# Light theme (publications)
bg_color="#ffffff", text_color="black"

# Transparent (overlays)
bg_color="none", fig_bg_color="none"
```

---

### Function: get_equilibration_progress()

Get progress information from NAMD equilibration logs.

```
get_equilibration_progress(
    log_directory: Path,
    output_file: Optional[Path] = None
) -> Dict[str, NAMDProgress]
```

**Parameters:**

- `log_directory` (Path): Directory containing NAMD log files
- `output_file` (Path, optional): Save progress summary to JSON

**Returns:** Dictionary of progress information for each stage

**Example:**

```
from gatewizard.utils.namd_analysis import get_equilibration_progress

progress = get_equilibration_progress("equilibration/")
for stage, info in progress.items():
    print(f"{stage}: {info.progress_percent:.1f}% complete")
```

---

### Line Styling and Customization

All trajectory analysis plotting methods (`plot_rmsd()`, `plot_rmsf()`, `plot_distances()`, `plot_radius_of_gyration()`) support line styling options for full control over plot appearance.

#### Common Line Parameters

Available in all plotting methods:

| Parameter | Type | Default | Description |
|---|---|---|---|
| `line_color` | str | varies | Color name (e.g., `"blue"`) or hex code (e.g., `"#1f77b4"`) |
| `line_width` | float | `1.2` | Line thickness (e.g., `0.8`, `1.5`, `2.5`) |
| `line_style` | str | `"-"` | Line style (see options below) |

#### Line Style Options

| Style | Value | Visual | Use Case |
|---|---|---|---|
| Solid | `"-"` | ———— | Default, standard plots |
| Dashed | `"--"` | — — — | Highlighting, predictions |
| Dash-dot | `"-."` | — · — · | Alternative highlighting |
| Dotted | `":"` | · · · · · | Subtle lines, backgrounds |

#### RMSD & Radius of Gyration Specific

Additional convergence line control for `plot_rmsd()` and `plot_radius_of_gyration()`:

| Parameter | Type | Default | Description |
|---|---|---|---|
| `show_convergence` | bool | `True` | Toggle convergence line on/off |
| `convergence_color` | str | `"red"` | Color for convergence line |
| `convergence_style` | str | `"--"` | Line style for convergence |
| `convergence_width` | float | `1.5` | Width of convergence line |

#### RMSD Specific

Threshold highlighting for `plot_rmsd()`:

| Parameter | Type | Default | Description |
|---|---|---|---|
| `highlight_threshold` | float or None | `None` | Highlight regions above this RMSD value (in Å or nm) |

#### RMSF Specific

Flexibility highlighting for `plot_rmsf()`:

| Parameter | Type | Default | Description |
|---|---|---|---|
| `highlight_threshold` | float or None | `None` | Highlight residues above this RMSF value (in Å or nm) |

#### Quick Reference Examples

### Thick Dashed Line:

```
analyzer.plot_rmsd(line_width=2.5, line_style="--")
```

### Thin Dotted Line:

```
analyzer.plot_rmsf(line_width=0.8, line_style=":")
```

### Custom Convergence (Green Dash-dot):

```
analyzer.plot_rmsd(
    convergence_color="green",
    convergence_style="-.",
    convergence_width=2.0
)
```

### No Convergence Line:

```
analyzer.plot_radius_of_gyration(show_convergence=False)
```

### RMSD with Threshold:

```
analyzer.plot_rmsd(highlight_threshold=3.0)  # Highlight > 3.0 Å
```

### RMSF with Flexibility Threshold:

```
analyzer.plot_rmsf(highlight_threshold=2.0)  # Highlight > 2.0 Å
```

---

## MDTraj Selection Syntax

#### Common Selections

```
"protein"                          # All protein atoms
"protein and backbone"             # Backbone atoms (N, CA, C, O)
"protein and name CA"              # C-alpha atoms only
"resid 50-150"                     # Residues 50 to 150
"resid 50-70 and name CA"          # C-alpha of residues 50-70
"resname LIG"                      # Ligand with residue name LIG
"protein and not name H*"          # Protein without hydrogens
"within 5 of resname LIG"          # Atoms within 5Å of ligand
```

#### Combining Selections

```
"(resid 1-50) or (resid 100-150)"    # Two regions
"protein and (name CA or name N)"    # Multiple atom names
"resid 25 and name NH1 NH2"          # Multiple atoms in residue
```

---

### Class: BilayerTrajectoryAnalyzer

Lipid bilayer analysis built on lipyphilic and MDAnalysis.

#### Constructor

```
BilayerTrajectoryAnalyzer(
    topology: Path,
    trajectory: Path | List[Path],
    file_times: Optional[Dict[str, float]] = None
)
```

Same topology/trajectory interface as `TrajectoryAnalyzer`. Leaflet assignment is performed automatically before each analysis.

#### calculate_area_per_lipid

Calculate the area per lipid. Leaflet assignment uses lipyphilic `AssignLeaflets`.

- **`fatslim` / `auto` (default):** original archived FATSLiM CLI via subprocess. Requires a companion conda env — run `scripts/install_fatslim_env.sh` from an API **source checkout** (not from a GUI `/usr/bin` install or a plain `pip install gatewizard`), then set `GATEWIZARD_FATSLIM`. Full GUI-vs-API notes: analysis.md. `exclude_sel` → FATSLiM interacting group. GateWizard remains MIT (no FATSLiM import).
- **`evapl`:** GateWizard **EVAPL** via freud — periodic XY Voronoi, then `exclude_sel` atoms in the leaflet headgroup Z-range shrink lipid cells with successive half-plane clips. **Experimental / not yet validated.** Default `exclude_sel` is `"protein"`.
- **`lipyphilic`:** official `lipyphilic.AreaPerLipid`. Empty `exclude_sel` works on PyPI 0.12.x. Non-empty `exclude_sel` requires upstream exclude support (PR #164); install `pip install -r requirements-lipyphilic-git.txt` or use `fatslim` / `evapl`.
- **`gridmat`:** GateWizard **GW experimental** GridMAT-style grid (closer to GridMAT-MD.pl, not bit-identical).
- **`gridmat_md`:** original GridMAT-MD.pl via Perl (external).
- **`vtmc`:** GateWizard **GW experimental** Voronoi + Monte Carlo (Mori et al.).

```
calculate_area_per_lipid(
    lipid_sel: str = "name PO4",
    leaflet_lipid_sel: Optional[str] = None,
    exclude_sel: Optional[str] = "protein",
    exclude_cutoff: float = 30.0,
    exclude_dim: int = 3,
    apl_method: Optional[str] = "fatslim",
    fatslim_nthreads: int = 1,
    fatslim_jobs: int = 1,
    start: Optional[int] = None,
    stop: Optional[int] = None,
    step: Optional[int] = None,
    verbose: bool = False,
) -> Dict[str, np.ndarray]
```

**Parameters:**

| Parameter | Description |
|---|---|
| `lipid_sel` | Atoms used for Voronoi tessellation / FATSLiM headgroups. MARTINI: `"name GL1 GL2 ROH"`. All-atom: `"name PO4"` or phosphate selections. |
| `leaflet_lipid_sel` | Selection for leaflet assignment. Defaults to `lipid_sel`. |
| `exclude_sel` | Non-lipid occupant atoms (e.g. `"protein"`, a peptide, DNA). Empty/`None` disables exclusion. |
| `exclude_cutoff` | Å cutoff for exclude atoms near the leaflet (`0` = all exclude atoms). Default `30` Å (3.0 nm). Used by lipyphilic AreaPerLipid (when supported), GridMAT, VTMC. **Ignored by FATSLiM and EVAPL.** |
| `exclude_dim` | `3` = 3D distance to leaflet atoms; `1` = \|z − leaflet midplane\|. |

| Parameter | Description |
|---|---|
| `apl_method` | `fatslim` / `auto` (default; external FATSLiM CLI), `evapl` (experimental), `lipyphilic` (official `AreaPerLipid`), `gridmat` (GW experimental), `gridmat_md` (external `.pl`), or `vtmc` (GW experimental). |
| `fatslim_nthreads` | FATSLiM `--nthreads` per process (`-1` = all CPUs). Default `1`. Prefer 1–4 when `fatslim_jobs` > 1. |
| `fatslim_jobs` | Parallel frame chunks (`--begin-frame` / `--end-frame`, half-open). Default `1`. Prefer several jobs with small nthreads over one job with all CPUs. |
| `start`, `stop`, `step` | Trajectory frame range. |

**Returns:** `time` (ns), `areas` (n_lipids × n_frames, Å$^2$), `mean_area_per_lipid`, `mean_upper_leaflet`, `mean_lower_leaflet`, `resids`, `resnames`.

See Example 14: Area per Lipid.

#### calculate_membrane_thickness

Calculate bilayer thickness from interleaflet headgroup distances (lipyphilic `MembThickness`).

```
calculate_membrane_thickness(
    lipid_sel: str = "name PO4",
    leaflet_lipid_sel: Optional[str] = None,
    leaflet_filter_sel: Optional[str] = None,
    n_bins: int = 1,
    interpolate: bool = False,
    start: Optional[int] = None,
    stop: Optional[int] = None,
    step: Optional[int] = None,
    verbose: bool = False,
) -> Dict[str, np.ndarray]
```

**Parameters:**

| Parameter | Description |
|---|---|
| `lipid_sel` | Headgroup atoms for the thickness calculation. |
| `leaflet_lipid_sel` | Selection for leaflet assignment. Defaults to `lipid_sel`. |
| `leaflet_filter_sel` | Optional filter passed to `AssignLeaflets.filter_leaflets()` to exclude species (e.g. cholesterol). |
| `n_bins` | Grid resolution for intrinsic surface construction (`1` = global mean height). |
| `interpolate` | Fill missing grid values when `n_bins` is large (slower). |

**Returns:** `time` (ns), `thickness` (Å).

See Example 15: Membrane Thickness.

#### plot_area_per_lipid / plot_membrane_thickness()

Generate time-series plots with the same styling options as `TrajectoryAnalyzer` plots. See Examples 14 and 15.

#### run_bilayer_analysis

JSON-serializable API for GUI and downstream consumers.

```
run_bilayer_analysis(
    topology_file: Union[str, Path],
    trajectory_files: List[Union[str, Path]],
    analysis_type: str,
    lipid_sel: str = "name PO4",
    leaflet_lipid_sel: Optional[str] = None,
    leaflet_filter_sel: Optional[str] = None,
    n_bins: int = 1,
    interpolate: bool = False,
    file_times: Optional[Dict[str, float]] = None,
    start: Optional[int] = None,
    stop: Optional[int] = None,
    step: Optional[int] = None,
    verbose: bool = False,
) -> Dict[str, Any]
```

**Supported `analysis_type` values:** `area_per_lipid`, `membrane_thickness` (aliases: `apl`, `thickness`, `memb_thickness`).

**Area per lipid return shape:**

```
{
    "analysis_type": "area_per_lipid",
    "x": [...],           # time (ns)
    "y": [...],           # mean area per lipid (Å²)
    "mean_upper_leaflet": [...],
    "mean_lower_leaflet": [...],
    "lipid_resids": [...],
    "lipid_resnames": [...],
    "per_lipid_areas": [[...], ...],
    "stats": {"mean", "std", "min", "max"},
}
```

**Membrane thickness return shape:**

```
{
    "analysis_type": "membrane_thickness",
    "x": [...],
    "y": [...],           # thickness (Å)
    "n_bins": 1,
    "stats": {"mean", "std", "min", "max"},
}
```

---

### Examples

Complete working examples are available in Analysis examples. Each example demonstrates specific analysis capabilities and can be run directly.

#### Example 10: Dark Theme RMSD Analysis - Full Customization

Complete RMSD customization with dark themes showing **all available plot options**.

```python
from pathlib import Path
from gatewizard.utils.trajectory_analysis import TrajectoryAnalyzer

# Get the directory where this script is located
script_dir = Path(__file__).parent
data_dir = script_dir / "equilibration_folder"

# ============================================================================
# Dark Theme RMSD Analysis - Full Customization
# ============================================================================

topology_file = data_dir / "system.pdb"
# Multiple trajectory files from equilibration_folder
trajectory_files = [
    data_dir / "step1_equilibration.dcd",
    data_dir / "step2_equilibration.dcd",
    data_dir / "step3_equilibration.dcd",
]

# Initialize analyzer with custom time for each file (in nanoseconds)
analyzer = TrajectoryAnalyzer(
    topology_file,
    trajectory_files,
    file_times={
        "step1_equilibration.dcd": 0.1,  # 0.1 ns
        "step2_equilibration.dcd": 0.1,  # 0.1 ns
        "step3_equilibration.dcd": 0.1,  # 0.1 ns
    },
)

# RMSD plot with full dark theme customization
analyzer.plot_rmsd(
    selection="protein and backbone",
    reference_frame=0,
    align=True,
    distance_units="Å",
    time_units="ps",
    line_color="#00d9ff",  # Bright cyan
    line_width=2.0,
    line_style="-",
    bg_color="#1a1a2e",  # Dark blue-black
    fig_bg_color="#16213e",  # Darker border
    text_color="#eee",  # Light gray
    show_grid=True,
    xlim=None,
    ylim=None,
    title="Dark Theme RMSD - Full Customization",
    xlabel="Time (ps)",
    ylabel="RMSD (Å)",
    highlight_threshold=None,
    highlight_color="orange",
    highlight_alpha=0.2,
    show_convergence=True,
    convergence_color="#ff006e",  # Magenta
    convergence_style="--",
    convergence_width=2.0,
    hlines=None,
    hline_colors=None,
    hline_styles=None,
    hline_widths=None,
    vlines=None,
    vline_colors=None,
    vline_styles=None,
    vline_widths=None,
    save="dark_theme_rmsd_example_10.png",
    show=False,
    figsize=(12, 7),
    dpi=300,
)

print(f"Dark theme RMSD plot saved: dark_theme_rmsd_example_10.png")

# ============================================================================
# Dark Theme RMSD - With Threshold and Reference Lines
# ============================================================================
```

```
analyzer.plot_rmsd(
    selection="protein and backbone",
    align=True,
    distance_units="Å",
    time_units="ps",
    line_color="#7fff00",  # Chartreuse
    line_width=1.8,
    line_style="-",
    bg_color="#0d1117",  # GitHub dark
    fig_bg_color="#010409",
    text_color="#c9d1d9",
    show_grid=True,
    title="Dark Theme RMSD - With Threshold and Reference Lines",
    xlabel="Time (ps)",
    ylabel="RMSD (Å)",
    highlight_threshold=0.25,  # Highlight regions > 0.25 Å
    highlight_color="#ff6b6b",  # Red highlight
    highlight_alpha=0.5,
    show_convergence=True,
    convergence_color="#ffd700",  # Gold
    convergence_style="-.",
    convergence_width=1.5,
    hlines=[0.1, 0.2, 0.25],  # Horizontal reference lines
    hline_colors=["#4ecdc4", "#95e1d3", "#ff6b6b"],  # Teal to red gradient
    hline_styles=[":", ":", ":"],
    hline_widths=[1.0, 1.0, 1.0],
    save="dark_theme_rmsd_with_lines_example_10.png",
    figsize=(12, 7),
    dpi=300,
)

print(f"Dark theme RMSD with lines saved: dark_theme_rmsd_with_lines_example_10.png")

# ============================================================================
# Dark Theme RMSD - Minimal Style
# ============================================================================

analyzer.plot_rmsd(
    selection="protein and backbone",
    align=True,
    distance_units="Å",
    time_units="ps",
    line_color="#ffffff",  # Pure white line
    line_width=1.5,
    line_style="-",
    bg_color="#000000",  # Pure black
    fig_bg_color="#000000",
    text_color="#ffffff",
    show_grid=False,  # No grid for minimal look
    title="",  # No title
    show_convergence=False,  # No convergence line
    save="dark_theme_rmsd_minimal_example_10.png",
    figsize=(10, 6),
    dpi=300,
)

print(f"Dark theme RMSD minimal saved: dark_theme_rmsd_minimal_example_10.png")

print(f"\nAll dark theme RMSD figures created with:")
print(f"  - Custom dark backgrounds")
print(f"  - Full customization options")
print(f"  - High resolution (300 DPI)")
```

**Output figures** Three dark-themed RMSD plots:

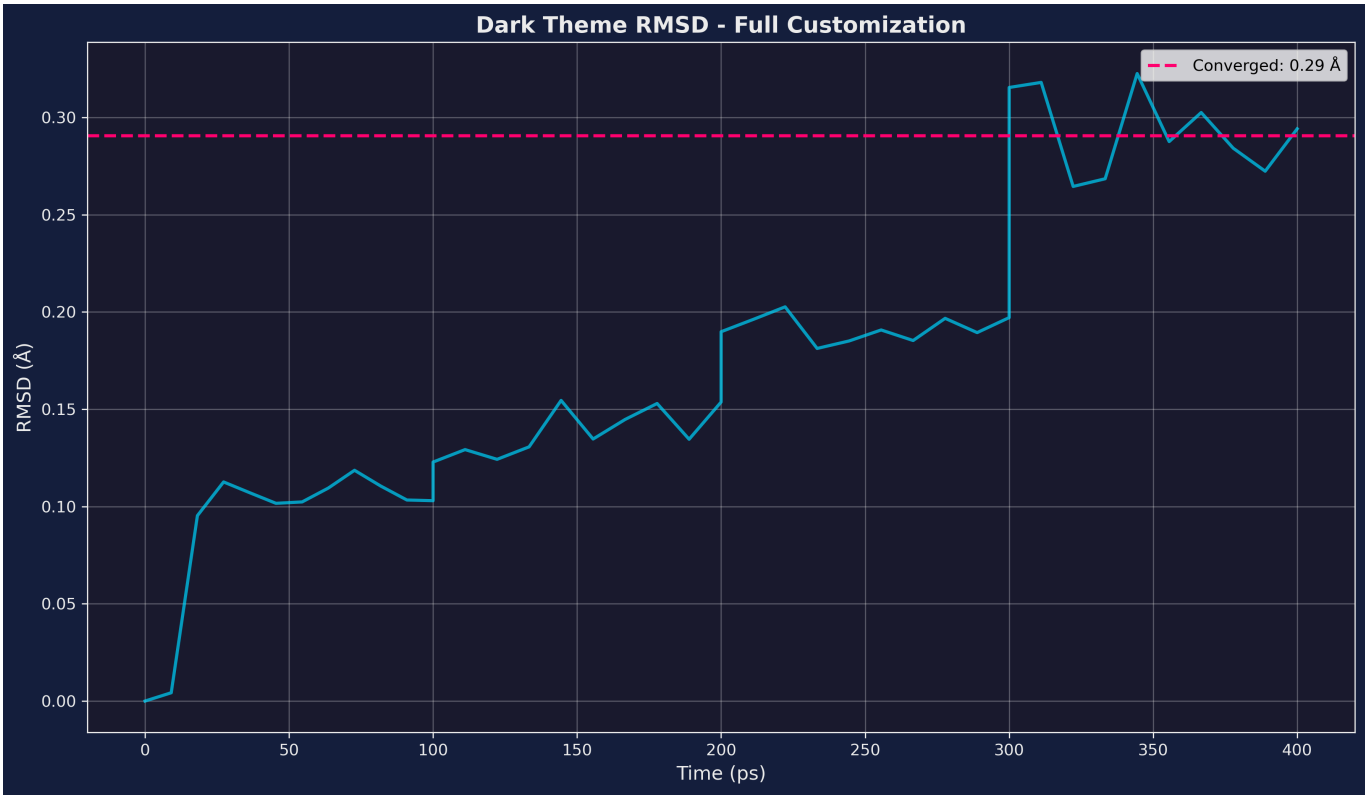



*Figure: Dark theme RMSD plot*

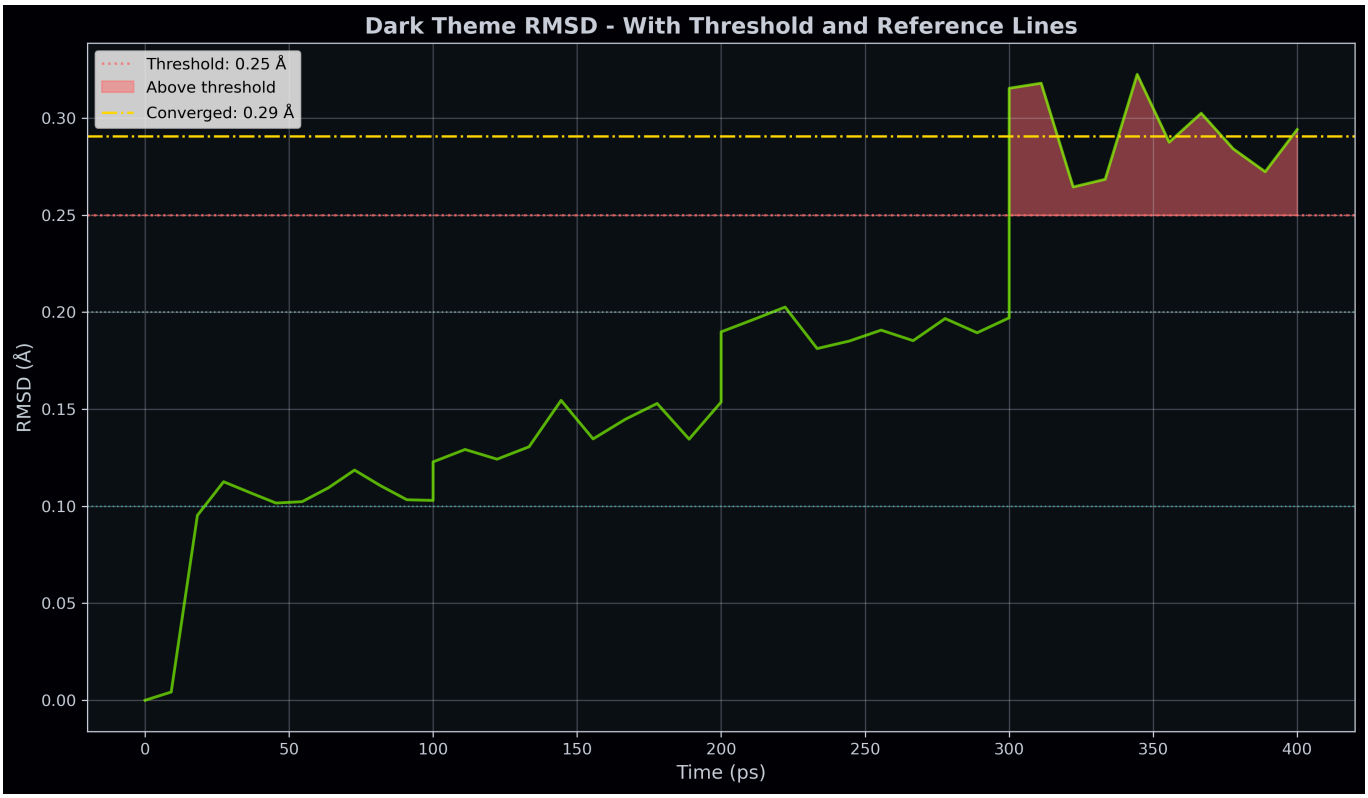



*Figure: Dark theme RMSD with lines*

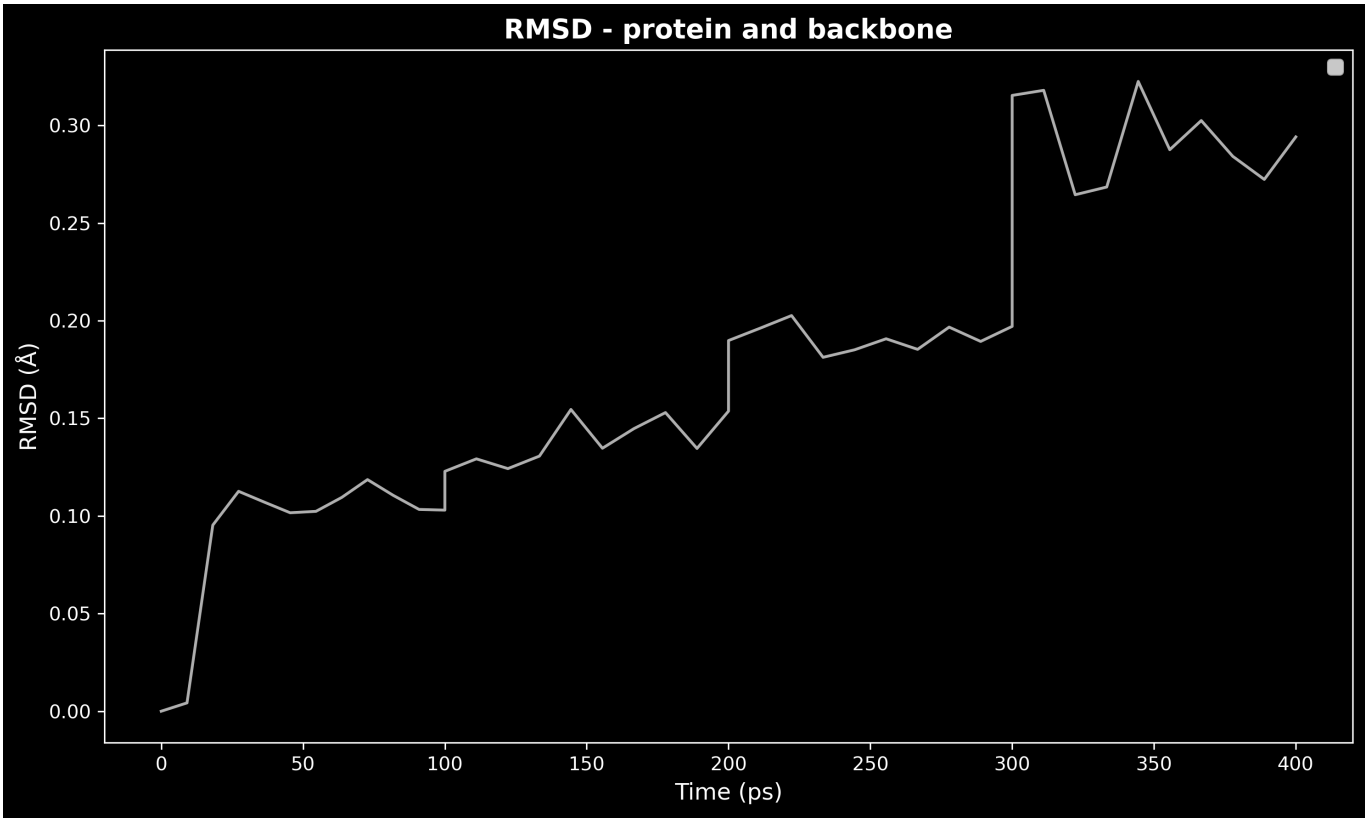



*Figure: Dark theme RMSD minimal*

---

### Class: OpenMMLogAnalyzer

Parse and analyze **OpenMM StateDataReporter** log files. The interface mirrors EnergyAnalyzer so the two can be used interchangeably in analysis workflows.

```
from gatewizard.utils.openmm_analysis import OpenMMLogAnalyzer
```

### Constructor

```
OpenMMLogAnalyzer(
    log_file: Union[Path, str, List[Union[Path, str]]],
    file_times: Optional[Dict[str, float]] = None,
)
```

| Parameter | Type | Description |
|---|---|---|
| `log_file` | `str \| Path \| list` | Path to a single OpenMM log file, or a list of paths for multi-stage analysis (times are concatenated). |
| `file_times` | `dict` | Optional `{filename: duration_ns}` mapping that overrides the time axis from the log's `"Time (ps)"` column. Useful when logs lack the time column or have restarted counters. |

**Key differences from EnergyAnalyzer:**

| Feature | `EnergyAnalyzer` (NAMD) | `OpenMMLogAnalyzer` (OpenMM) |
|---|---|---|
| Native energy units | kcal/mol | **kJ/mol** |
| Color kwarg in `plot_properties` | `line_colors` | `colors` |
| Log format | space-delimited `.log` | tab-delimited (StateDataReporter) |

**Available property keys** (depend on what StateDataReporter reports):

| Key | Description |
|---|---|
| `potential` | Potential energy (kJ/mol) |
| `kinetic` | Kinetic energy (kJ/mol) |
| `total` | Total energy (kJ/mol) |
| `temp` | Temperature (K) |
| `volume` | Box volume (nm³) |
| `density` | Density (g/mL) |

---

#### get_statistics

```
analyzer.get_statistics() -> Dict[str, Dict[str, float]]
```

Returns mean / std / min / max / initial / final for every numeric column that contains data.

**Returns** `{key: {"mean", "std", "min", "max", "initial", "final"}}`

---

#### plot_energy

```
analyzer.plot_energy(
    energy_units: str = "kJ/mol",
    time_units: str = "ns",
    bg_color: str = "#2b2b2b",
    fig_bg_color: str = "#212121",
    text_color: str = "Auto",
    show_grid: bool = True,
    title: Optional[str] = None,
    target_temperature: Optional[float] = None,
    target_pressure: Optional[float] = None,
    save: Optional[str] = None,
    show: bool = False,
    figsize: tuple = (12, 10),
    dpi: int = 300,
)
```

2×2 energy summary plot — total energy, potential + kinetic, temperature, volume / density.

| Parameter | Default | Description |
|---|---|---|
| `energy_units` | `'kJ/mol'` | `'kJ/mol'` or `'kcal/mol'` |
| `time_units` | `'ns'` | `'ps'`, `'ns'`, or `'µs'` |
| `target_temperature` | `None` | Reference temperature (K) drawn as a horizontal dashed line |
| `save` | `None` | Filename for saved figure |
| `show` | `False` | Display interactively |

---

#### plot_properties

```
analyzer.plot_properties(
    properties: Optional[List[str]] = None,
    energy_units: str = "kJ/mol",
    time_units: str = "ns",
    bg_color: str = "#2b2b2b",
    fig_bg_color: str = "#212121",
    text_color: str = "Auto",
    show_grid: bool = True,
    separate_plots: bool = False,
    save: Optional[str] = None,
    show: bool = False,
    figsize: Optional[tuple] = None,
    dpi: int = 300,
    xlim: Optional[tuple] = None,
    ylim: Optional[tuple] = None,
    colors: Optional[List[str]] = None,
)
```

Plot selected properties vs time. If `properties` is not provided, all non-empty numeric columns are plotted automatically.

| Parameter | Default | Description |
|---|---|---|
| `properties` | `None` (all) | List of keys to plot, e.g. `["potential", "temp", "volume"]` |
| `energy_units` | `'kJ/mol'` | `'kJ/mol'` or `'kcal/mol'` |
| `separate_plots` | `False` | Save/show each property as its own figure |
| `colors` | `None` | Line color list (one per property). Unlike `EnergyAnalyzer`, the kwarg is `colors`, not `line_colors`. |
| `save` | `None` | Filename (or prefix when `separate_plots=True`) |
| `show` | `False` | Display interactively |

---

**Example 11: Basic Energy Analysis**

```
"""
analysis_example_11.py
======================
Basic OpenMM log analysis — single log file.

Prerequisites
-------------
- ``step7_production.log`` output from a StateDataReporter run
- matplotlib / numpy (``conda install matplotlib numpy``)

"""

from pathlib import Path
from gatewizard.utils.openmm_analysis import OpenMMLogAnalyzer

# Get the directory where this script is located
script_dir = Path(__file__).parent
data_dir = script_dir / "equilibration_folder"

# Single OpenMM log file (StateDataReporter output)
log_file = data_dir / "step7_production.log"

# Initialize the analyzer
analyzer = OpenMMLogAnalyzer(log_file)

# ------------------------------------------------------------------
# 1. Get statistics for all properties
# ------------------------------------------------------------------
stats = analyzer.get_statistics()

for key, s in stats.items():
    print(
        f"  {key:20s}  mean={s['mean']:12.3f}  std={s['std']:10.3f}"
        f"  min={s['min']:12.3f}  max={s['max']:12.3f}"
    )
# → All values in kJ/mol (energies) or native units (temperature in K,
#   volume in nm³, density in g/mL)

# ------------------------------------------------------------------
# 2. 2×2 summary plot: total energy, potential + kinetic, temperature,
#    volume / density
# ------------------------------------------------------------------
analyzer.plot_energy(
    save="energy_summary_example_11.png",
    show=False,
    target_temperature=303.15,
)

print("Energy summary saved: energy_summary_example_11.png")

# ------------------------------------------------------------------
# 3. Same plot converted to kcal/mol
# ------------------------------------------------------------------
analyzer.plot_energy(
    energy_units="kcal/mol",
    save="energy_summary_kcal_example_11.png",
    show=False,
)

print("Energy summary (kcal/mol) saved: energy_summary_kcal_example_11.png")
```

---

**Example 12: Multi-Stage Analysis**

```
"""
analysis_example_12.py
======================
Multi-stage OpenMM analysis — concatenate several log files.

Prerequisites
-------------
- Log files ``step1_equilibration.log`` … ``step7_production.log``
  (StateDataReporter output, one per equilibration stage)
- matplotlib / numpy

"""

from pathlib import Path
from gatewizard.utils.openmm_analysis import OpenMMLogAnalyzer

script_dir = Path(__file__).parent
data_dir = script_dir / "equilibration_folder"

# Pass a list of log files — time axis is concatenated automatically
log_files = [
    data_dir / "step1_equilibration.log",
    data_dir / "step2_equilibration.log",
    data_dir / "step3_equilibration.log",
    data_dir / "step4_equilibration.log",
    data_dir / "step5_equilibration.log",
    data_dir / "step6_equilibration.log",
    data_dir / "step7_production.log",
]

# Provide real stage durations (ns) to override the "Time (ps)" column
# — useful when logs lack the time column or have restarted counters.
file_times = {
    "step1_equilibration.log": 0.125,
    "step2_equilibration.log": 0.125,
    "step3_equilibration.log": 0.125,
    "step4_equilibration.log": 0.25,
    "step5_equilibration.log": 0.25,
    "step6_equilibration.log": 0.5,
    "step7_production.log": 50.0,
}

analyzer = OpenMMLogAnalyzer(log_files, file_times=file_times)

# ------------------------------------------------------------------
# 1. Print key statistics
# ------------------------------------------------------------------
stats = analyzer.get_statistics()

print("=== Multi-stage OpenMM analysis ===")
for key in ("potential", "kinetic", "total", "temp", "volume", "density"):
    if key not in stats:
        continue
    s = stats[key]
    print(
        f"  {key:12s}  mean={s['mean']:12.3f}"
        f"  initial={s['initial']:12.3f}  final={s['final']:12.3f}"
    )

# ------------------------------------------------------------------
# 2. Energy summary over the full trajectory
# ------------------------------------------------------------------
analyzer.plot_energy(
    save="energy_multistage_example_12.png",
    show=False,
    title="Full equilibration + production (51.375 ns)",
    target_temperature=303.15,
)

print("Saved: energy_multistage_example_12.png")
```

---

**Example 13: Custom Property Plots**

```
"""
analysis_example_13.py
======================
OpenMM plot_properties – select specific columns, custom colours.

Prerequisites
-------------
- Log file ``step7_production.log`` from a StateDataReporter run
- matplotlib / numpy

"""

from pathlib import Path
from gatewizard.utils.openmm_analysis import OpenMMLogAnalyzer

script_dir = Path(__file__).parent
data_dir = script_dir / "equilibration_folder"

log_file = data_dir / "step7_production.log"
analyzer = OpenMMLogAnalyzer(log_file)

# ---------------------------------------------------------------------
# 1. Plot all available properties (auto-detected)
# ---------------------------------------------------------------------
analyzer.plot_properties(
    save="all_properties_example_13.png",
    show=False,
)
print("All-properties plot saved: all_properties_example_13.png")

# ---------------------------------------------------------------------
# 2. Plot only energies, converted to kcal/mol
# ---------------------------------------------------------------------
analyzer.plot_properties(
    properties=["potential", "kinetic", "total"],
    energy_units="kcal/mol",
    save="energies_kcal_example_13.png",
    show=False,
    colors=["#61afef", "#98c379", "#e06c75"],
)
print("Energy plots (kcal/mol) saved: energies_kcal_example_13.png")

# ---------------------------------------------------------------------
# 3. Temperature and density as separate figures
# ---------------------------------------------------------------------
analyzer.plot_properties(
    properties=["temp", "density"],
    separate_plots=True,
    save="temp_density_example_13",
    show=False,
)
print(
    "Separate figures saved: temp_density_example_13_temp.png, temp_density_example_13_density.png"
)
```

---

#### Example 14: Area per Lipid

Calculate and plot the area per lipid using the default **FATSLiM** CLI (`exclude_sel="protein"` → interacting group). Requires companion `fatslim` — see `scripts/install_fatslim_env.sh`. For the experimental freud path use `apl_method="evapl"`.

```
"""
analysis_example_14.py
======================
Area per lipid analysis using lipyphilic (BilayerTrajectoryAnalyzer).

Prerequisites
-------------
- ``equilibration_folder/system.pdb`` and equilibration DCD trajectories
- lipyphilic (core gatewizard dependency; installed via ``pip install -e .``)
"""

from pathlib import Path

from gatewizard.utils.fatslim_apl import is_available as fatslim_is_available
from gatewizard.utils.lipid_bilayer_analysis import BilayerTrajectoryAnalyzer, run_bilayer_analysis

script_dir = Path(__file__).parent
data_dir = script_dir / "equilibration_folder"

topology_file = data_dir / "system.pdb"
trajectory_files = [
    data_dir / "step1_equilibration.dcd",
    data_dir / "step2_equilibration.dcd",
    data_dir / "step3_equilibration.dcd",
]

# POPC headgroup phosphorus atoms in the AMBER lipid parametrization
LIPID_SEL = "resname PC and name P31"
# FATSLiM is optional; EVAPL stays in-process for a clean clone.
APL_METHOD = "fatslim" if fatslim_is_available() else "evapl"

file_times = {
    "step1_equilibration.dcd": 0.1,  # 0.1 ns
    "step2_equilibration.dcd": 0.1,  # 0.1 ns
    "step3_equilibration.dcd": 0.1,  # 0.1 ns
}

# ---------------------------------------------------------------------
# 1. Class API — calculate area per lipid
# ---------------------------------------------------------------------
analyzer = BilayerTrajectoryAnalyzer(
    topology_file,
    trajectory_files,
    file_times=file_times,
)
data = analyzer.calculate_area_per_lipid(lipid_sel=LIPID_SEL, apl_method=APL_METHOD)

mean_area = float(data["mean_area_per_lipid"].mean())
print(f"Mean area per lipid: {mean_area:.1f} Å²")
print(f"Lipids analysed: {len(data['resids'])}")
print(f"Frames analysed: {data['areas'].shape[1]}")

# ---------------------------------------------------------------------
# 2. Plot mean area per lipid time series
# ---------------------------------------------------------------------
analyzer.plot_area_per_lipid(
    lipid_sel=LIPID_SEL,
    series="mean",
    time_units="ns",
    apl_method=APL_METHOD,
    save="area_per_lipid_example_14.png",
    show=False,
)
print("Plot saved: area_per_lipid_example_14.png")

# ---------------------------------------------------------------------
# 3. JSON API — run_bilayer_analysis()
# ---------------------------------------------------------------------
result = run_bilayer_analysis(
    topology_file,
    trajectory_files,
    analysis_type="area_per_lipid",
    lipid_sel=LIPID_SEL,
    file_times=file_times,
    apl_method=APL_METHOD,
```

```
)
print(f"JSON API mean area: {result['stats']['mean']:.1f} Å²")
```

For other force fields, adjust `lipid_sel` to match your headgroup atoms, e.g. `"name GL1 GL2 ROH"` (MARTINI) or `"name PO4"` (all-atom CHARMM).

---

#### Example 15: Membrane Thickness

Calculate and plot bilayer thickness as the mean interleaflet headgroup distance (lipyphilic `MembThickness`) on the same equilibration trajectories.

```
"""
analysis_example_15.py
======================
Membrane thickness analysis using lipyphilic (BilayerTrajectoryAnalyzer).

Prerequisites
-------------
- ``equilibration_folder/system.pdb`` and equilibration DCD trajectories
- lipyphilic (core gatewizard dependency; installed via ``pip install -e .``)
"""

from pathlib import Path

from gatewizard.utils.lipid_bilayer_analysis import BilayerTrajectoryAnalyzer, run_bilayer_analysis

script_dir = Path(__file__).parent
data_dir = script_dir / "equilibration_folder"

topology_file = data_dir / "system.pdb"
trajectory_files = [
    data_dir / "step1_equilibration.dcd",
    data_dir / "step2_equilibration.dcd",
    data_dir / "step3_equilibration.dcd",
]

LIPID_SEL = "resname PC and name P31"

file_times = {
    "step1_equilibration.dcd": 0.1,  # 0.1 ns
    "step2_equilibration.dcd": 0.1,  # 0.1 ns
    "step3_equilibration.dcd": 0.1,  # 0.1 ns
}

# ---------------------------------------------------------------------
# 1. Class API — calculate membrane thickness
# ---------------------------------------------------------------------
analyzer = BilayerTrajectoryAnalyzer(
    topology_file,
    trajectory_files,
    file_times=file_times,
)
data = analyzer.calculate_membrane_thickness(lipid_sel=LIPID_SEL)

mean_thickness = float(data["thickness"].mean())
print(f"Mean membrane thickness: {mean_thickness:.1f} Å")
print(f"Frames analysed: {len(data['thickness'])}")

# ---------------------------------------------------------------------
# 2. Plot membrane thickness time series
# ---------------------------------------------------------------------
analyzer.plot_membrane_thickness(
    lipid_sel=LIPID_SEL,
    time_units="ns",
    save="membrane_thickness_example_15.png",
    show=False,
)
print("Plot saved: membrane_thickness_example_15.png")

# ---------------------------------------------------------------------
# 3. JSON API — run_bilayer_analysis()
# ---------------------------------------------------------------------
result = run_bilayer_analysis(
    topology_file,
    trajectory_files,
    analysis_type="membrane_thickness",
    lipid_sel=LIPID_SEL,
    file_times=file_times,
)
print(f"JSON API mean thickness: {result['stats']['mean']:.1f} Å")
```

To exclude cholesterol from a mixed bilayer thickness calculation, pass `leaflet_filter_sel="resname DPPC DOPC"` and set `lipid_sel` to the headgroup atoms of the lipids included in the calculation.

---

#### Class: GROMACSLogEnergyAnalyzer

```
from gatewizard.utils.gromacs_analysis import GROMACSLogEnergyAnalyzer

analyzer = GROMACSLogEnergyAnalyzer(log_files, file_times=None, file_strides=None)
```

Parse GROMACS `.log` energy blocks. Same plotting surface as `EnergyAnalyzer`: `get_statistics()`, `plot_energy()`, `plot_properties()`.

#### Class: AmberLogEnergyAnalyzer

```
from gatewizard.utils.amber_analysis import AmberLogEnergyAnalyzer

analyzer = AmberLogEnergyAnalyzer(mdout_files, file_times=None, file_strides=None)
```

Parse Amber `mdout` energy frames. Use `run_amber_energetic_analysis` for the one-shot helper.

##### run_energetic_analysis

```
from gatewizard.utils.namd_analysis import run_energetic_analysis

data = run_energetic_analysis(log_files, properties=["TEMP", "PRESSURE"])
```

One-shot NAMD energetic analysis used by the GUI and scripts. Returns series data for `plot_spec` rendering.

##### run_openmm_energetic_analysis

```
from gatewizard.utils.openmm_analysis import run_openmm_energetic_analysis
```

Same role as `run_energetic_analysis` for OpenMM StateDataReporter logs.

##### run_amber_energetic_analysis

```
from gatewizard.utils.amber_analysis import run_amber_energetic_analysis
```

One-shot Amber mdout energetic analysis.

##### parse_namd_log

```
from gatewizard.utils.namd_analysis import parse_namd_log
```

Low-level NAMD log parser used by `EnergyAnalyzer`.

##### prepare_structural_inputs

```
from gatewizard.utils.trajectory_analysis import prepare_structural_inputs
```

Normalize topology/trajectory lists (split PDB-as-traj, fill missing box dimensions) before `TrajectoryAnalyzer`.

##### run_structural_analysis

```
from gatewizard.utils.trajectory_analysis import run_structural_analysis
```

One-shot RMSD/RMSF/distance/Rg runner used by the Analysis page.

#### clear_analysis_cache

```
analyzer.clear_analysis_cache()
```

Drop cached MDAnalysis universes / prepared frames on a `TrajectoryAnalyzer` after changing stride or inputs.

#### run_fatslim_apl

```
from gatewizard.utils.fatslim_apl import run_fatslim_apl
```

Call the archived FATSLiM CLI (GPLv3 companion env). Discovery: `GATEWIZARD_FATSLIM` → `PATH` → conda env `fatslim-py38`. See Analysis features.

#### run_gridmat_md_apl

```
from gatewizard.utils.gridmat_md_apl import run_gridmat_md_apl
```

Run original GridMAT-MD.pl (Perl). Requires `GATEWIZARD_GRIDMAT` or a clone on `PATH`.

### See Also

- User Guide - Desktop Analysis page
- Examples - Working code examples
- Troubleshooting - Common issues